

ABRA — roadmap

2026-08-01

The bar for the completed list is one thing: **would I defend this sentence for twenty hours against someone trying to break it.** Anything that fails that test is in section 2 or section 3, however much work went into it.

A result is only here if it is (a) measured rather than argued, (b) measured on data that passes `engine/quality.js` , and (c) still true when re-run today. Several things that were on this list a week ago have been removed for failing (c).

1. DONE — and defensible

1.1 The data is trustworthy, and we can prove which parts are not

87% of the ladder store is unusable and we know exactly why. 15,973 stored games, ~2,057 clean. Bots, forfeits, partial brings and stubs, each counted separately in a funnel that anyone can re-run. 27 assertions in `tests/test-quality.js` .

Behavioural bot detection works on accounts that do not announce themselves. Team invariance: an account playing 50+ games without ever changing a slot is a script. It caught six accounts that appeared in **52.2%** of the games a name-only filter called clean. Carchdraw84172 played 459 games with one team, 367 of them in a single day.

Defensible because: the rule is stated, the evidence is per-account, and the false-positive check is published — no account with ≥50 games in the store has more than one distinct team, so the rule separates cleanly.

Known limit, stated: a bot playing under 50 games or varying its team is invisible. The right phrase is always "no bot detected", never "human".

1.2 The damage engine is correct

31 of 31 scenarios within 2% of `@smogon/calculator` , through the official Showdown engine pinned at commit `20ad99f` . Golden-master, re-run before every engine change.

Defensible because: it is validated against an independent implementation, not against itself, and the harness caught two of its own wiring bugs on first run.

1.3 MAGNEMITE reads the board, and the improvement is measured out of sample

600 seed-matched battles per policy — same teams, same seeds, same engine — against 1,905 clean ladder games:

	before	after	real humans
super effective	9.71%	14.91%	21.37%
moves that outright failed	9.68%	6.34%	2.47%
moves that hit an immune target	4.30%	2.92%	1.91%

Defensible because: the fit never saw the realism report — it was held back as the out-of-sample check — and the comparison is paired on seeds so team difficulty cancels.

What this does NOT claim: that MAG is good. See 2.1.

1.4 Imitation is a ceiling, and we have the number

A challenger optimised for **winning** rather than for resembling humans, drawn from MAG's own 17 features, beats MAG **63.2%** — and beats MAG's predecessor **68.2%** where MAG itself manages **60.2%**.

Defensible because: the 68.2% is an independent evaluation the search never optimised against, on an opponent it was not tuned for. It is transitively better, not a counter.

RETRACTED 2026-08-04 — all three figures describe a 17-feature vector we no longer ship. They were computed on an engine 25 wire-fixes old and **before the quality filter existed**; `provenance.js` carried `data/exploitability.json` as its only `UNSAFE` artifact on exactly that ground. The re-run on the shipped 58 features is **void** — `data/policy-weights.json` was refitted at 22:15:24 UTC while the search was running and `engine/medicham2-browser.js` moved twice more afterwards — and its hill-climb accepted **1 of 24** steps, so it was uninformative regardless. See `docs/SEARCH.md §R8`. **The claim of this section may still be true; the evidence for it no longer exists.** The general point that imitation and winning are different objectives stands on its own separate evidence (see MAGNEMITE in `docs/MODELS.md`), not on these three numbers.

And it is interpretable: the winning weights barely care about type effectiveness and care enormously about not wasting a turn and finishing what is already hurt.

1.5 The open-sheet corpus is safe to learn from, and we checked rather than assumed

Teams differ enormously between open- and closed-sheet play — **551.9 points** of total absolute species difference, Garchomp on 81.6% of open-sheet teams against 47.7% of closed. Behaviour given a board differs by **at most 1.49 points** on every measure.

Defensible because: both sides were measured with the same code, and the policy is conditional on the board — it never learns what to bring, and MEW draws teams from the ladder store regardless. `engine/fit_policy.js` re-checks it on every refit by re-estimating on a reweighted sample.

1.6 Collection is complete and automatic

The Bo3 **open-team-sheet** ladder is now pulled hourly into its own store. That format carries `Force Open Team Sheets` , so every game publishes all six sets — the only continuously-collected corpus where the choice set of a decision is known rather than guessed.

Defensible because: the format's ruleset was read from the simulator, the pull was tested live, and the store is separate — Bo3 is a different information regime and pooling it would corrupt every behavioural statistic.

1.7 Facts the model can see are derived, never typed

Ability blocking (Levitate/Ground, Flash Fire/Fire, Good as Gold/status, Armor Tail/priority) is **read out of 14,933 recorded battles**, not written down — including inferring *which rule* each ability follows by testing candidate hypotheses and preferring the narrower on ties.

Defensible because: it was verified against clean-only data and every rule is identical; and because probing Showdown's handlers directly was tried first and **failed silently**, which is why the measured route was taken.

1.8 The project can audit itself

- engine/provenance.js — every artifact checked for staleness against the filter, its inputs, and the clean-game count. Found **28 unsafe to quote**.
- engine/conformance.js — S1–S13 as executable checks over 132 files. **S12 is now clean**.
- engine/selftest.js — 24 assertions on failures that are silent by nature.
- build/build_pdfs.js — every document has a current PDF, list derived.

Defensible because: each found real defects on its first run, including in the tool that found them.

2. NOT DONE — and specifically why

2.1 MAG is not good

It is **exploitable** (1.4), it still repeats Protect at roughly double the human rate, it wastes 6.34% of moves against a human 2.47%, and its one head-to-head win was against a baseline with a flaw it was purpose-built to punish. **It is a starting position, not a player**.

2.2 The self-play ladder has produced no evidence

One confirmed promotion. The round robin that would establish whether the ladder climbs or goes in circles has never had enough games to resolve anything. Reported as a null.

2.3 ~Twenty-eight artifacts are unsafe to quote~ — CLOSED 2026-07-31

o UNSAFE, 45 ok. Twenty-five were regenerated by re-running their own generators; the last was fixed by addressing what the gate was actually complaining about. provenance.js --strict is wired into the suite and green. This was named "the blocker on everything in section 3" and it is no longer blocking anything.

Two real bugs were underneath the staleness, neither of which waiving by name would have found: illusion.json carried a sound raw-store justification in its generator that never travelled to the artifact; archetypes.json reported **team sides** (2 per game) under the name n_games, so every share and silhouette in it had the wrong sample size on its label.

What this does NOT establish. provenance.js says so itself: it checks what artifacts DECLARE. o UNSAFE means nothing is misdeclared, not that every number is right. **14 are still flagged "possibly stale"**, and only re-running a generator can settle whether its output is correct.

2.4 Models that are retracted, null, or without usable input

Recorded so they are not quoted again:

model	state
PORY	its comparison artifact was trained without the filter; standing unknown , not weak
SLOWKING	mathematics sound, but the clean matrix has o decisive matchups — no usable input
GURU	o decisive matchups on clean data; descriptive only
JOLTEON	ties a coin; recommended for retirement
WAR	withdrawn — the signal was four bots playing one team 1,446 times
CHOMP-EV	honest null; brings do not separate from a coin
the 199,524-game self-play corpus	unusable — generated before megas worked, with duplicate Protects on one set
"the meta is rock-paper-scissors"	withdrawn; computed pre-filter, deleted

2.5 Nothing here handles hidden information

Self-play has none — we run both sides. Nothing in the current work teaches the bot to play against an unknown team, which is the defining difficulty of the closed-sheet ladder.

2.6 Closed since this list was written (2026-07-29 → 30)

Items 3, 4 and 5 below have shipped. Recorded rather than deleted, because a roadmap that quietly drops its own entries cannot be audited.

- **"Give MAG a damage calculation, and a KO check both ways"** — DONE. board.js calls the real damage engine throughout; koTarget, killIsRoll, diesBeforeMoving and the switch-survival features all read it.
- **"Score both Pokémon together"** — BUILT AND MEASURED, and it **lost**: 42.0% [39.9, 44.3] over 1,934 paired games, 28.4% of decisive pairs, against its own zeroed control. The imitation fit is refuted; the idea is not. Now item 1 below.
- **"Switching"** — no longer a coin flip. Voluntary switches score through switchFeatures, the post-KO replacement is scored rather than rolled, and entryHits = forced ? 0 : 1 separates the two cases.

A process failure that caused real damage, recorded so it is not repeated. On 2026-07-30 the model family was repeatedly mischaracterised in conversation — DODUO described as unbuilt when it had been built, wired, controlled and measured; MEDICHAM audited in its graveyard version; top-K pruning "proposed" when fit_joint.js already had it. Cause: two full days and ~40 commits of work existed in NO document. MODELS.md was last written 2026-07-28; every other living doc 2026-07-26. The only reason DODUO's verdict was findable at all is that its experiment happened on the one day MODELS.md was touched. **CLAUDE.md requires the docs to move in the same pass as the code. They did not**.

2.7 Closed 2026-07-31

- **"Choice lock in the candidate set"** (was item 2) — DONE. The logit denominator no longer contains moves the human could not click. After the refit, six of eight switch features clear zero, and **switches now win the argmax** where --switching previously changed nothing.
- **The tag artifact reaches the feature vector** — speedSwing +0.983, screenValue +1.128, healValue +2.220, all clearing zero by wide margins. 53 → 56 features. This is the first feature addition since 3.28.0's four nulls to measure as real, and the difference is that these are CONDITIONS on mechanics with no prior representation rather than restatements.
- **ALAKAZAM job 2 (guess what they will do)** — built, off by default, needs a refit. The max in incomingThreat is now an expectation over P(their action).
- **A gate against wasted runs** — engine/preflight.js, 24 games, refuses a training run whose feature blocks receive no gradient. Two 1.5-hour runs were lost to exactly that before it existed.

2.8 Closed 2026-07-31 (evening)

- **DODUO is trainable for winning** (was item 1, the top of section 3). Wiring only, as the item predicted. `--joint` trains the 74-length vector on the game result; `--joint-only` freezes the 56 singles so the whole trust-region step reaches the 18 pair terms. Four separate faults would each have produced a plausible, wrong learning curve, and every one was caught by a check rather than by reading. **The head-to-head is running; no win rate is claimed.**
- **The trust region starved the pair block, structurally.** The first run moved it 4.7% while the singles moved 21.7% — single features appear in nearly every choice set, pair features are sparse, so they lose a shared budget by construction and more iterations scale both equally. `--joint-only` took it to **40.0%**, with three sign flips: `overkill -0.951 → +0.980`, `focusFireKills -0.107 → +1.094`, `partnerCoversMe -0.004 → +0.639`, and `bothSameTarget 0.031 → 2.249`. The imitation fit penalised focus fire; training to win reverses it.
- `mew_farm` **exited 0 with every worker dead.** A farm that wrote zero games was indistinguishable, to anything checking the exit code, from a clean run. Found because the DODUO gate refused to start.
- **The speed-multiplier divergence is cross-checked** (systems audit R2, which the audit could not finish because `board.js` was off-limits mid-run). `tests/test-speed-multipliers.js`, 15 passed, compares behaviour as ratios and compares the SETS, so a new weather-speed ability in the dex cannot hide behind four passing constants.
- **A test that asserted a count.** `sanity_check.py` required Sun > 1000 teams; Sun is the third largest of eight styles. Now relative and mutation-tested. **96 passed, 0 failed.**
- **All twelve recommendations from the three 2026-07-31 reviews are closed.** Including the two that had been built but never reached a reader: the multiplicity correction (on disk since 14:04, in no reader-facing document) and the retired NMF justification, still live in `PUBLICATION.md` — the document that exists to be published — while the honest version sat in `SUMMARY.md`.
- **There is a rollback point.** `v3.31.0`, tagged and pushed, with the one known failure named in the tag message. Both reviews had recorded that nothing existed to roll back to.

2.9 Closed 2026-08-01 (live play against a human)

Will played MAG in the real Showdown client for the first time — six games. **Eleven defects in about two hours**, more than the 47 test files had surfaced. Full evidence: `docs/FINDINGS-2026-08-01-live-play.md`. Handoff: `docs/HANDOFF-2026-08-01.md`.

- **The bot was not delivering the model to the board.** `makeScoringPlayer(opts)` accepted an options object and never used it — the class reads its own second constructor argument — so `{greedy: true, switching: true}` configured nothing. MAG played its first real games *sampling* its moves instead of taking its best one. Fixed by merging factory options into constructor options.
- `joint: true` **silently disabled mega evolution.** `_withMega` had exactly one call site and the joint path returned around it. Both joint paths now go through it, and `test-wiring.js` re-asserts mega, aiming and open sheets **under** `--joint` — the broken cell was the intersection of two capabilities each tested alone. Fourth mega defect in this project.
- **Voluntary switching was reverted to off.** The bot had requested it since its first version, against `mew.js:135` 's measured verdict of a **10-point loss** — **RETRACTED 2026-08-06 as unattributable and confounded: it ran through medicham2 playouts predating WIRES 123-128, its artifact carries no `engine_release` stamp, and `bringIn()` selects `live(bench)[0]` so the experiment measured switching to an ARBITRARY body, which the engine itself calls "evaluating LEAVE" rather than evaluating a switch. See #63.** It only took effect once the options bug was fixed — an unmeasured change against a measured result.
- **Item Clause repair could never succeed.** It resampled while telling the sampler the answer had to be the colliding item, so the "rare" no-item fallback fired every time.
- Open team sheets are now accepted; failed team draws redraw instead of rejecting the challenge.

Nothing about MAG's playing strength was established. MAG went 1–2 in the three games where the policy was actually driving. Six games is not evidence in either direction.

3. NEXT — in order, with the reason each is next

o. REFIT — and it may rewrite item 1 below. `board.js` changed a feature's *meaning* under an unchanged *name*, so every shipped weight was fitted against the old definition. `board.js` is required by 40 files; the vector is read by 24. Nothing in the project detects this: `magemite.js:297` compares feature names and `:299` compares vector length, and both pass when semantics drift. A guard — a stored hash of each feature's values over a fixed fixture board — is the cheapest high-leverage item on this list, and the only one that prevents a repeat rather than fixing an instance.

Why this comes before item 1, and may dissolve it. Item 1 rests on `spreadFreeBesideAlly` being priced at −5.054, read as "humans rarely click this, not this is bad." That feature fires only when `allyHit === 0`, and `allyHit` used `getEffectiveness >= 0`, which **returns 0 for an immunity**. So the feature's coverage was inverted at its most important case:

ally beside a spread move	did <code>spreadFreeBesideAlly</code> fire?	is it actually free?
Flying partner + Earthquake	no	yes — fully immune
Levitate partner + Earthquake	yes (ability path)	yes
merely <i>resisting</i> partner	yes	no — it still takes damage

The clearest free-spread case in VGC never fired once, while a case that costs real HP did. A weight fitted on that is a weight fitted on the wrong question, and **−5.054 may be an artifact rather than a preference**. Stated as a hypothesis; the refit is the test. If it survives the refit, item 1 stands exactly as written.

RESOLVED 2026-08-01 — it WAS an artifact, and of something else entirely **The immunity hypothesis above is refuted, and the conclusion it was reaching for is confirmed by a different cause.** Both fits were re-run. **The refit alone changed almost nothing.** `allyHit` moved from −0.0187 to −0.0126 (unweighted −0.0937 → −0.0875) — about **0.2 standard errors**, i.e. noise. Measured rather than argued: the immunity fix changes **0.198% of candidates**, and `allyHit` fires on only 0.63% of them to begin with. So the honest reading of `allyHit` is the plain one — **humans do not avoid hitting their own partner much** — and it should be reported that way rather than re-explained. **What produced −4.986 was the FITTER, not the feature.** `fit_joint.js` matched a human's click by requiring the candidate's target to match, and `board.js` builds a spread candidate with `targetMon: null` because Earthquake is not aimed. **No spread click could ever match.** Measured over 400 corpus games: spread moves are **14.94% of all human move clicks**, 99.71% carry a recorded target, and **1,393 of 1,397 failed to match**. Because a joint turn needs both slots, the loss compounded — the fit discarded **57,486 of 82,483 joint turns** and estimated 74 weights from 30% of the data. The discarded 70% was not random: it was exactly the turns containing a spread move. `spreadFreeBesideAlly` therefore fired on 8.0% of the enumerated alternatives and **0 of 1,461 pairs a human actually chose**, and a conditional logit reads

"always available, never chosen" as a large negative. `fit_policy.js:432` had this right all along, so the **56-feature vector was never affected**. Corrected, on 63,305 usable turns, **nine of eighteen pair weights flip sign**: | pair term | old | refit + matcher fix | |---|---|---| | `spreadFreeBesideAlly` | -4.986 | **+0.863** | | `terrainSetupHelpsPartner` | -4.125 | **+2.005** | | `screenWhileThreatened` | -2.982 | **+0.110** | | `speedSetupHelpsPartner` | -1.595 | **+0.540** | | `weatherSetupHelpsPartner` | -0.549 | **+1.292** | Every "one slot sets up, the partner benefits" term was negative and is now positive. Held out, the joint layer takes top-1 from 9.7% to 11.5%. **And it wins**. Refit+fix against the shipped vector, seed-paired, greedy and joint on both arms, on three disjoint seed blocks: | run | games | decisive pairs to the refit | SPRT decided at | |---|---|---|---| | first | 17,350 | 66.7% | 78 decisive pairs | | replication | 1,934 | 65.9% | 69 | | challenger in arm 1, as convention | 1,932 | **70.9%** [66.0, 75.4] | 71 | The third is the one to quote: `sprt.js` and `paired_h2h.js` were run over it independently and agree to the decimal. All three are far past the 55% ship threshold, and every one was decided inside ~420 games — the fixed 200,000-game run this started as was 99.8% waste. **Item 1's stated premise is void**. It reads -5.054 as evidence that the imitation objective prices good play badly. That number was a fitter defect. Item 1 may still be worth doing — the objective argument has independent support from greedy — but **this is no longer evidence for it**, and DODUO's 42.0% was measured with the contaminated vector and no longer describes the current one. **A correction about the analysers, recorded because it was nearly shipped as a fix**. The first reading of this H2H looked inverted, and `sprt.js` was changed to treat arm 2 as the challenger on the strength of `mew.js:215` calling `--weights2` "the challenger". That comment is about the EXPLOITABILITY search, not the standard A/B. **Arm 1 is the challenger** — `paired_h2h.js:183` builds its NEW label from arm 1, and the run that measured greedy at 79.7% put `--greedy` on arm 1. The change was reverted; making it would have put the two analysers at odds and inverted every run already analysed. What WAS wrong is narrower: `sprt's` name-based fallback hardcoded 'score' as the new arm, so on `--policy prior --policy2 score` it disagreed with its own arm-based path. Fixed, and `tests/test-sprt-arm-sign.js` now asserts the two analysers use the same rule so they cannot drift.

1. DODUO trained for WINNING, not for resemblance. The most precise open question in the project. DODUO is fully built: `fit_joint.js` fits the pair block, `magnemite.js` plays it (`--joint`), `--joint-zero` is a true control running the whole pair path with only the coordination weights zeroed, top-K is capped, and fallbacks are counted in `stats.jointFellBack`. It lost at 42.0%, and the fit itself says why — it prices `spreadFreeBesideAlly` at **-5.054**, which means humans rarely click this, not this is bad. A bot told to avoid a free spread move beside its own ally by -5 will decline its best plays.

2026-07-30 established the pattern that makes this item one: **four knowledge additions produced four nulls; two changes to the OBJECTIVE produced two large wins** (greedy +12 points / 79.7% of decisive pairs; self-play 55.9%). The objective is the binding constraint, and DODUO has only ever been fitted to the losing one.

RECONCILED 2026-07-31. That 55.9% was measured on the **53-feature vector with switching OFF**. Repeating the experiment on the **56-feature vector with switching ON** gives **48.1%** [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (which used to be supported by the older 10-point switching loss — **that figure is RETRACTED 2026-08-06 as unattributable and confounded**: `medicham2` payouts predating WIRES 123-128, no `engine_release` stamp, and `bringIn()` selects `live(bench)[0]`, so it measured switching to an ARBITRARY body rather than to a chosen one. The candidate cause stands; its supporting evidence does not. See #63), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.

THE WIRING IS DONE, 2026-07-31 (commit `ce5367c`). It was wiring, as this item predicted, and the arithmetic in the sentence below was wrong in its parts while right in its total: the vector is **56 singles + 18 pair terms = 74**, not 53 + 21. The pair softmax is $P(q) = \exp(s_q) / \sum \exp(s_k)$ with $s_q = w_s \cdot x_a + w_S \cdot x_b + w_J \cdot jf$, so the feature vector attached to pair q is the concatenation $[x_a + x_b, jf]$ — the two single vectors **summed**, because both are scored by the same single block. `accumulateLogitGrad` was already generic over length, so no new mathematics was needed. **Four separate faults would each have produced a plausible and wrong learning curve**, and every one was caught by a check rather than by reading the code: | fault | what it would have looked like | |---|---| | `mew.js` sized the run gradient from the **single** weight file and summed with `k < GRAD.length` | 74 truncated to 56 — every pair weight silently dropped, curve printed as normal | | `magnemite.js` allocated `learnGrad` ~50 lines **before** `this.wj` loaded, so `this.joint` was still false | every joint player returns 56 entries; caught by the above on the first 6-game probe | | `magnemite.js` read the joint vector from a **hardcoded** path | every iteration replays the frozen shipped fit; pair terms never move; flat curve reads as convergence | | `preflight.js` indexed only `B.FEATURES` | the gate certifies a joint run **without checking the block that run exists to train** | `--joint` on one arm is the A/B experiment, not a training configuration. `mew.js` now refuses `--learn` with the layer on one side (the run gradient sums both sides, and index k is a different feature in each); `train_policy.js` adds the mirror flag rather than leaving it to be remembered. **Measured**: `preflight` at 24 games with `--joint --joint2 --switching --switching2` — every block live, pair block 15 of 18 live, | gradient | 9.12. Two real iterations at 40 games each move the pair terms on their own merit: `bothSameTarget` **+0.164** (third-largest change in the whole 74-vector), `overkill` **+0.120**, `focusFireKills` **+0.094**. Suite 40 passed / 2 failed, the same two known failures. **TRAINED 2026-07-31 (evening)**. 12 iterations x 5,000 games with `--joint-only`, which freezes the 56 singles so the whole step reaches the pair block (the first run, unfrozen, moved it 4.7% -- see 2.8). Pair block moved **40.0%** of its own length; singles moved **0.000**, verified, so the head-to-head differs in the 18 coordination weights and nothing else. | pair term | resemble | win | |---|---|---|---| | `bothSameTarget` | 0.031 | **2.249** | | `overkill` | -0.951 | **+0.980** | **SIGN FLIP** | | `focusFireKills` | -0.107 | **+1.094** | **SIGN FLIP** | | `partnerCoversMe` | -0.004 | **+0.639** | **SIGN FLIP** | | `spreadFreeBesideAlly` | -4.986 | -4.815 | barely moved | Three of the four largest changes are sign flips and they tell one story: **the imitation fit penalised focus fire, and training to win reverses it**. Humans spread damage; winning concentrates it. `spreadFreeBesideAlly` **is the exception and it is this item own headline weight**. The argument above is that a bot told to avoid a free spread move beside its ally by -5 will decline its best plays. After training it is -4.8. Whatever moved the other three did not move this one, and that is unexplained. **WHY IT DID NOT MOVE — ANSWERED 2026-07-31 (Will)**. Not cancellation and not a dead feature: the gradient was **positive in all 12 iterations, never once negative**, so the evidence is unanimous that -4.99 is too negative. It is ~13x smaller per round than `bothSameTarget` because the feature UNDER-FIRES, and the reason is a detector that is too narrow: ``js const freeSpread = (c, x) => c &= c.move && c.spread && c.spread.length > 1 && F(x, 'allyHit') === 0;`` `allyHit` is a SINGLE-SLOT feature computed from the partner's TYPE and ABILITY — Levitate, Flash Fire, Volt Absorb, Earth Eater, or resisting the type. ****A partner who clicks PROTECT is spared exactly as completely as a Flying partner, and the feature cannot see it****, because Protect is not a property of the partner; it is a choice the partner makes that turn. That is structural, and it is the whole argument for the pair layer: `spreadFreeBesideAlly` RECEIVES both candidates and reads the `allyHit` of its own slot only. The partner's Protect is already in hand and is never consulted. ****Measured cost:**** Earthquake is on ****69 of 308 species (22.4%)***, by far the commonest spread move, and it is `allAdjacent` — so today it qualifies only when the partner is immune. 40 species (13.0%) carry an `allAdjacentFoes` move that spares the ally automatically; 83 (26.9%) carry an `allAdjacent` move that does not. The commonest real line in VGC — Earthquake plus partner Protect — is in the second group and is invisible. ****FIX, QUEUED** (`board.js` was in use by the head-to-head, per the no-edit-during-a-run rule): ****extend `freeSpread` to treat the partner candidate as sparing the ally when it is a protect-family move. PROTECTMOVES and `tgtMayProtect` already exist, so nothing is typed in. **WIDE GUARD: RESOLVED, and it is BOTH cases depending on whose side it is on.**** On MY side it spares my partner from my own spread move exactly as Protect does. On the FOE side it blocks my spread move outright, which is an anti-synergy and belongs in the dead-move tests, not here. ****THE FULL SET IS DERIVED, NOT TYPED**** — probe each candidate move's volatile/side condition by calling its real `onTryHit` against the actual spread move, the same technique `entryEffects` and `speedStub` already use. Measured 2026-07-31: | spares the ally (10) | excluded, and why | |---|---| | Protect, Detect, Spiky Shield, Baneful Bunker, Burning Bulwark, Silk Trap, King's Shield, Obstruct, Wide Guard, Mat Block | ****Quick Guard**** — priority only; ****Crafty Shield**** — status only; ****Endure**** — no `onTryHit` at all, it prevents FAINTING, not damage | Three flag-based shortcuts were tried and are all WRONG, recorded so they are not retried: `stallingMove` admits ****Endure****; the

presence of `onTryHit` admits `**Quick Guard and Crafty Shield**`; `target=== 'allySide'` admits both of those and misses every self-targeting Protect. Only running the handler against the move separates them. `**This weakens the interpretation of the head-to-head now running.**` Both arms share the same detector, so the comparison is still fair, but neither arm can express the coordination pattern the feature is named for. `**MEASURED 2026-07-31. IT LOST.**` 194,514 paired games, 97,257 complete pairs. | | | |---|---| | decisive pairs | `**26,405**` (27.1% of all pairs) | | NEW (trained to win) takes | `**45.7%**`, 95% CI `**[45.1, 46.3]**` | | verdict | `**worse`, and the interval clears 50`**` | Training the 18 coordination weights on the game result made the policy WORSE than the same weights fitted to imitate a human click. The three sign flips that looked like a coherent story about focus fire do not pay. `**The likely reason is in the same table: 72.9% of pairs were 1-1 splits**`, i.e. the TEAM decided the game and not the policy. That contamination is in the TRAINING gradient too, so REINFORCE on win/loss was mostly fitting a team lottery. The objective was right; the ESTIMATOR of it was not. Three fixes follow, and none of them is "go back to imitation": 1. `**Pair the TRAINING runs**` as the tests already are, so team luck cancels out of the gradient. 2. `**Score decisions against a win-probability baseline**` (PORY) rather than the final result, so a move is credited with what it changed instead of what happened six turns later. 3. `**Train against a pool, not only against self**` – a self-play policy beats its training partner, and this one had never faced the imitation policy until the head-to-head it lost. That is VGC-Bench's own finding, already recorded in docs/COMPETITORS.md. `**CAVEAT, AND IT IS LARGE.**` The whole-repo review found `engine/fit_policy.js:170` computes the reweighting factors from ONE store and applies them across all three, so the shipped weight number was fitted against the wrong population. Both arms here share those singles, so this comparison is internally fair – but every ABSOLUTE number in this project is provisional until that refit. `**engine/sprt.js` reached this verdict after 3,516 games instead of 194,514 – 98.2% of the run saved. `**--verify` confirms it agrees with `paired_h2h.js` exactly (12,073 and 14,332 on both readers). A 136-minute run becomes about two and a half minutes. What remains: `` node engine/train_policy.js --joint --switching --switching2 --procs 6 --iters <N> --games <G> `` then the head-to-head the trainer now prints, in which `**both arms run the pair path with identical singles and only the 18 pair weights differ**` – so it measures `*trained to win*` against `*fitted to resemble*`, and nothing else. The previous printout passed a 74-length file to `--weights``, which would have been rejected outright or, worse, run an experiment whose arms differed in the wrong block.

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost 31.2% on decisive pairs and would have been reported as "coordination does not help."

2. Re-run WOBUFFET against the current vector. (Will, 2026-07-30: put this back on the list.) **ATTEMPTED 2026-08-04 AND VOID — still open, and now there is no number at all rather than a stale one.** `~~`Its result is still the most important number in the repo and it is three feature-generations stale: a counter found in forty minutes beat MAG **63.2%** [56.6, 69.3] on the 17-feature vector, mirror control 47.5%.`~~` The re-run's defender was refitted mid-run and the simulator moved twice more; separately its hill-climb accepted 1 of 24 steps, so `exploit.js` needs a stamp and a dimension-aware step rule before the next attempt. `docs/SEARCH.md` §R8 has the prepared command and its three preconditions. That challenger was not rock-paper-scissors — it was a better player drawn from the same features and optimised for wins rather than resemblance, which is item 1's lesson in another form. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, so the EXPLOITABILITY argument for coordination — the one the 42.0% result explicitly does not settle — can finally be tested instead of argued.

3. MACHAMP, re-run on the current vector. (Will, 2026-07-30: keep it.) Half-run and stale: the 2026-07-26 run completed 2 of 6 generations on a **17-feature** vector and recorded no verdict; the vector is now 53. Its METHOD is alive — champion/challenger promotion behind a Wilson interval is the same win-objective idea `train_policy.js` now implements by policy gradient. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, so that "gen 5 beats gen 4" is not read as progress by itself. Kept as insurance rather than as evidence — the "this metagame is cyclic" justification is withdrawn (2026-08-02); see `docs/MODELS.md` under MACHAMP.

4. Branch scoring with a mixed strategy — the research is done, so this is no longer vague. Measured on a real mid-game board: **9 × 8 = 72 joint actions per side, 72 × 72 = 5,184 matrix cells.** The 10¹² figure from VGC-Bench is the game TREE, not one turn.

- Only **28 of 72** pairs have a non-zero joint vector. The other **44 are free** — their score is exactly the sum of two singles already computed.
- With two Pokémon the coordination graph is a SINGLE EDGE, so Variable Elimination reduces to enumerating the 72 pairs and Max-Plus is unnecessary. The MARL factorisation literature (QMIX, QPLEX, Weighted QMIX) exists to avoid enumeration at many agents, which is not our problem.
- What that literature DOES tell us: QMIX's monotonicity constraint cannot represent non-monotonic coordination — exactly "Protect while my partner removes the threat." Independent per-slot scoring cannot express it either. Not "scores it badly": cannot express it.
- Pruning order matters. Structural prunes (choice lock, spread moves with no target, immunities) are exact and happen BEFORE scoring. Score-based top-K is circular for the 72 and only pays for the 5,184.
- Pokémon is SIMULTANEOUS-move, so the answer at a node is a mixed strategy, not an argmax. Minimax is the wrong algorithm; this is poker-shaped. SLOWKING's regret matching already does this mathematics at team preview and nobody has pointed it at a turn.

5. A leaf evaluator, so depth > 1 is affordable. `position_features.js` (16 features) plus PORY retrained on clean data. Without it every node rolls to the end; with it, depth 2 is 625 cheap evaluations.

6. Retire the second scorer in `app/index.html`. (Now the only failing test in the suite, and the only one at the `v3.31.0` tag.) It assigns **21 of 56 features** — 35 are never written and silently read 0, and 8 of the 21 disagree with the engine outright. This is the "facts are global" rule broken inside the UI. `app/scoreboard.html` already does it correctly by rendering engine-computed values. The real fix is to ship the actual engine to the browser — `medicham2-browser.js` already runs in both, and `board.js` has no top-level requires; the only blocker is one `process.env` read at module scope.

7. Bring the five living docs current, and keep them current. Whitepaper, deck, technical docs, SUMMARY and MODELS drifted to 3.21.0 against a CHANGELOG at 3.28.0 once already — see the process failure recorded in 2.6. SUMMARY, MODELS, the white paper, PUBLICATION and this file were brought to 3.31.0 on 2026-07-31; the deck and technical docs have NOT been checked since.

8. Measure exploitability again, against the mixed-strategy version. The test that says whether the project has moved from *plays okay* onto the branch that can approach *solved*.

9. External validation. Our bots grading each other cannot establish competence. The two real scoreboards are the Bo3 open-sheet ladder — where the hidden-information problem does not exist and we already collect the games — and VGC-Bench's published agent results on that same format.

4. THE SEARCH REDESIGN — established 2026-08-06

This section exists because a conversation about *why the bot picks badly* turned up four things that were true, written down, and unacted on. Every figure below was read out of a file or measured in this session; two of them correct claims I made earlier in the same session from memory.

4.1 The structural fact everything else follows from — and it was already written down

Real games are six turns. Measured over 53,059 stored games:

median	mean	p90	p99	over 30 turns	over 60 turns
6	6.5	10	16	0.05%	0.01%

The rollout horizon is capped at **60**. That is ~10× the real game, and reasoning from the cap rather than the distribution is how the cost model came to be 10× too big: a leaf evaluation is about **5,600** move-decisions, not 48,000.

docs/POKER-TO-POKEMON.md §4b had this number and its consequence before tonight:

"the horizon is short: the median game is 6 turns... the binding constraint is **breadth, not depth** — action abstraction (pruning dominated moves, bucketing targets) is where the real work is, **more so than the value net**."

Two things follow, and the second is the uncomfortable one:

- **A leaf matters less here than in poker or Go.** A leaf evaluator exists because you cannot reach the end of the game. At six turns the rollout *does* reach the end — it returns a real win or loss. There is nothing to approximate. **The problem is not the judge at the bottom of the tree; it is that the six turns leading to it are played by a coin.**
- Which means the PORYGON2 separation gate (#23), dispatched earlier the same evening, is answering a question of smaller consequence than the dispatch brief claimed. It is still worth having. It is not the thing that unblocks anything.

4.2 GARY — the opponent inside the search, now named, and built-and-switched-off

A capability that cannot prove it ran is assumed broken. GARY could not, because it had no name.

```
engine/mag_bot.js:173      const MILTANK_FOE = arg('miltank-foe', 'uniform');
engine/miltank.js:455      DEFAULTS = { ... foePolicy: 'uniform', ... }
engine/rollout_leaf.js:289 const mv = (foePolicy === 'prior' && pickByPrior(mon, rng)) || mvs[random];
```

'prior' samples data/move-priors.json — **128,548 recorded clicks over 295 species** — so the imagined opponent clicks Protect and Fake Out at the rate real players do instead of 25% each. It is wired end to end and **the default is the coin, in the library and in the live bot.**

Four separate defects, each recorded as its own item:

1. foePolicy **steers both sides.** rollout_leaf.js:302-303 applies the same pick to S.actA and S.actB. One flag governs the whole imagined game, so you cannot give the opponent a brain without also changing how the search models itself. (#34)
2. **Targeting is random in both settings.** rollout_leaf.js:290 draws the target uniformly whether or not prior is on. prior fixes *which move* and never *who it hits* — and in doubles the target is half the decision. board.js:377 already records humans double-targeting 23.4% of the time against ~50% for independent choice. (#35)
3. **GARY has two seats and they disagree.** From rolloutAfterActions' s own comment: *"The opponent is NOT modelled. It plays chooseAction during the stepped turn."* Deterministic greedy on the turn being ranked; a coin for every turn after. (#36)
4. **No artifact records which GARY ran.** data/rollout-r1.json and data/rollout-r1-explore-sweep.json carry no foePolicy key, so R1 and R4 cannot say whether their opponent was a person or a coin. (#33)

Correction to something asserted earlier this session: MAG is *not* deterministic and the "sampling would collapse the variance" objection was wrong. greedy=false already draws from a softmax, and magmeme.js:217 calls it *"the single biggest measured lever in the project."*

4.3 The pruning is done by the coin

miltank.js:1204 screens every candidate pair by running a **cheap rollout** on it and keeps the top FINAL_K. The rollout is the 51%-accurate leaf. **So the coin is not merely misjudging the finalists — it is selecting them.**

MAG is not used for this, and the file says why:

"The first version pruned to the best K per slot — except _candsFor returns no scores, so it was taking the first three in array order and calling that the best three. That is worse than not pruning at all: an arbitrary shortlist that LOOKS principled."

The candidate list arrives with no scores attached. Fixing that is §4b's "action abstraction", and it costs *one* MAG pass per turn rather than one per imagined turn — the difference between affordable and impossible. Measured branching, from 7,976 real brought-teams: ~76 action combos per side per turn, ~5,738 joint. MILTANK samples 200 rollouts against that. (#37)

4.4 The equilibrium solver is unplugged, and the reason is the language boundary

How the comparable projects handle the opponent: they do not pick one. CFR → DeepStack → Libratus → ReBeL never assume an opponent policy; they run regret matching for both players jointly and output a *mixed strategy*, whose guarantee is that it cannot be exploited whatever the opponent does. AlphaZero uses self-play with a policy prior and deletes rollouts entirely. AlphaStar — the closest structural analogue, being simultaneous, hidden-information and wide — uses a *league* including deliberate exploiters, which is what WOBBUFFET is here.

The poker answer is already implemented in this repository, verified, and unreachable. engine/slowking/nash.py and ismcts.py do simultaneous-move regret matching and recover exact Nash on RPS and an asymmetric 2×2. rollout_leaf.js states the gap plainly: *"a best response to a fixed opponent rather than an equilibrium: weaker than the design's matrix game."*

```
127 JavaScript files  everything that must PLAY a battle — Showdown is TypeScript
40 Python files      everything that does MATH — numpy/scipy live there
7 in slowking/       the solver
```

Neither language choice was wrong. **The consequence is that the thinking cannot play.** Three routes out, and the third is the one to scope first:

route	cost
port the solver to JS	a second implementation of verified math — the failure this repo keeps repeating
call Python per turn	a subprocess inside a Showdown turn timer
precompute in Python, ship a lookup table	what chess does with Syzygy — and it is DUSK

4.5 DUSK is a tablebase, and open sheets are what make it exact

Will, 2026-08-06: *"at the end game, we can have a repository of scenarios in dusk that can show which mons beat which in a straight up battle and solve for those endings."*

That is an endgame tablebase, and it is the right shape for this game specifically. At 1v1 under **open team sheets there is nothing hidden left** — species, set, item, ability and nature are all declared. The only unknown is which of four moves they pick this turn: a small matrix game with known payoffs, which is exactly what `nash.py` already solves exactly. **Closed sheets would make the table a guess; open sheets make it a fact**, which is the payoff of the open-sheet-only directive. The store supports reconstructing these positions — turn records carry `tgthp`, boosts and megas, and `sets` carries `declared:true`.

And DUSK is simultaneously the bridge in 4.4: solve offline in Python, emit a table, let the JavaScript bot read it. No port, no subprocess.

The open question is size, and it decides everything: enumerate all of $(\text{mon} + \text{set} + \text{HP} + \text{status} + \text{boosts})^2 \times \text{field}$ and it is astronomical; restrict to positions that actually occur on the ladder and it may ship as a JSON file. Not yet measured. (#40)

4.6 A coverage number that overstates itself ~3×

`data/mechanics-census.json` : **probed 219, live 216, missing 3 — armed 74, unarmed 145**. An unarmed probe runs, reports a result, and would report the same result if the mechanic were deleted. So "216/219 mechanics live" counts probes that executed, not mechanics that work.

The instrument that does *not* depend on anyone thinking of the right probe — random games diffed against real Showdown — finds **1 disagreement in 150**, and that is the honest signal. It is good. The remedy is arming the 145, not writing more. (#42)

4.6b Speed is revealed for free by resolution order, and nothing reads it

Will, 2026-08-06: *"if two incins come out, which intim goes first indicates speed."*

Entry abilities resolve in **speed order**, so two Intimidates on the same switch reveal a strict speed inequality **before anybody clicks a move**. It costs no turn and no risk. The same channel has several sources: any two entry abilities (Drizzle against Drought), end-of-turn residuals (Leftovers, burn, Rough Skin), and same-bracket move order (two Fake Outs, two Protects).

This matters more than it sounds, because of what open sheets leave hidden. `web/models.html` 's own mode note: *"Hidden information is exactly two things: what's in the back and the exact EV spread."* Species, item, ability, nature and moves are all declared. **So Speed is essentially the hidden variable in this format**, and every resolution-order observation is a hard constraint on it. Accumulated across a battle it narrows to an interval, which is exactly the *"do I outspeed this"* question that decided three of the five replays reviewed the same evening.

What exists is not this. `engine/dynamics.js` → `data/dynamics.json` already derives speed from *who moved first* — but it reads **move order only**, aggregates **per species over the whole corpus** into an empirical rank rather than per-battle per-opponent, and is read by `ditto.js` and `kadabra.js` and by **nothing** in `board.js`, `miltank.js`, `mag_bot.js`, `magnemite.js` or `medicham2-browser.js`. It is a population prior, not a live belief.

And entry-ability order is modelled nowhere at all — `entryOrder|switchInOrder|abilityOrder| speedOrder` returns no match across all 127 JavaScript files. `medicham2-browser.js:2444` applies entry drops and carries no notion of which resolved first.

Three pieces, and **the first is an ENGINE correctness question rather than a feature**: does MEDICHAM resolve entry abilities in speed order at all? The official engine does, so if MEDICHAM does not, that is a differential-test miss to fix before anything is built on top of it. Then the extraction (protocol order → a qualified inequality), then the belief (inequalities → a Speed interval a decision can read). ****XATU** is currently drawn as *"mostly idle — little left to guess"* under open sheets. This is the thing it should be doing instead. ****** (#43)

4.7 The order, and why each is where it is

#	item	why here	cost
#33	record which GARY ran	nothing below is interpretable without it	trivial
#32	flip <code>uniform</code> → <code>prior</code> , measured	free, and it gates whether a better GARY is worth building	a flag
#38	set the horizon from the measured 6	every cost estimate downstream depends on it	trivial
#37	prune with MAG's scores, not the coin	\$4b's "the real work"; one MAG pass per turn	small
#34 #35 #36	split the policy, model targeting, unify the seats	make GARY one declared thing	medium
#39	measure the board↔MEDICHAM translation	decides whether MAG-as-GARY is possible at all	measurement
#40	DUSK size gate	decides whether the tablebase is a weekend or a year	measurement
#41	reach the solver from JS, via #40	the equilibrium answer exists and cannot play	scoping
#42	arm the census	stops an inflated number being quoted again	ongoing
#43	speed from resolution order — and Foul Play goes further, running the damage calc BACKWARDS : it simulates rolls against the damage actually observed to infer the opponent's real stats, and reads items from behaviour (outspeeds you repeatedly with the same move → Choice Scarf). We infer nothing from either channel (#74)	free information on the one variable open sheets still hide	ENGINE check, then small

The through-line: four of the six findings in this section are capabilities that were *built, correct, and switched off* — `prior`, MAG's scores at the candidate list, MAG's softmax, and the Nash solver. That is the same failure mode as 2026-07-28, arriving through a door the rules had not covered, and the countermeasure is the same one: **a capability that cannot prove it ran is assumed broken**. GARY is named in this pass for exactly that reason.

5. THE REGISTER — everything we intend to build, and why it is not built yet

Will, 2026-08-06: *"I WANT THE ROADMAP TO BE COMPREHENSIVE AND UNABLE TO LOSE TRACK OF THE THINGS WE WANT TO BUILD LATER."*

The rules, because a list that is not checked is a wish-list.

1. An item leaves this register in exactly two ways: **it is done**, or **Will kills it by name**. Nothing leaves by being forgotten, superseded in conversation, or quietly reprioritised.
2. Every item names **what unblocks it**. "Later" is not a status; "blocked by #45" is.

3. Every item carries a **task number**, and the task holds the detail. The register is the index — it exists so that nothing is only in someone's head, or only in a conversation, or only in a division ledger nobody opens.
4. **A deferred item stays here with its reason.** HYPNO is deferred by decision, not dropped, and it says so.

5.0 THE FOUR PHASES — the ordering this register is arranged in (3.59.0, ADR-003)

This section is new and it re-orders everything under it. It exists because ADR-003 changed what the project is trying to prove. The headline metric is **exploitability, not win rate**; VGC-Bench (AAMAS 2026) beat a professional with a compiled policy and measured all of its agents at approximately **100% exploitable**; and the thesis under test is that a **re-solving** agent is harder to exploit than a compiled one. That thesis makes MEDICHAM's existence conditional, and the condition is a gate that was already in this register.

1	finish MEDICHAM	search needs an engine that is fast AND correct
2	GATE #62	does compute buy anything: untimed vs on-the-clock
3	if yes ->	search, and measure EXPLOITABILITY against their ~100%
4	if no ->	adopt their recipe: BC + PPO self-play/FP/DO, open source, reproducible

phase	what it is	where it lives in this register
1	MEDICHAM is fast and correct	§5.1, and #68 as the whole-engine gate
2	#62 — the project gate , below	this section
3	the search, then the exploitability measurement	§5.2, §5.3, §5.4, and #6 in §5.3
4	BC + PPO with self-play / fictitious play / double oracle	§5.8, as the paper's fallback result

#62 IS PROMOTED TO THE FRONT OF THIS REGISTER. It is no longer a MILTANK tuning question; it is the project's gate, and it was moved here from §5.3 in this pass.

#	item	blocked by
#62	GATE, AND IT COMES FIRST: play MILTANK untimed against MILTANK on the clock. R6 measured 31.6% of move decisions deferred — handed back to MAG unsearched, consuming 30.5% of the spend. <code>defer</code> fires both when the clock runs out <i>and</i> when the search cannot separate its options, and only the first is fixable with compute . This decides whether #25/#37/#61 are worth doing at all — and, since 3.59.0, whether the whole engine programme was worth doing. VGC-Bench used real Showdown through poke-env and carried no engine-correctness debt , because behaviour cloning and PPO do not need a fast simulator. We wrote one so per-turn re-solving is affordable, so the engine work is justified if and only if search pays	#45

Why this is the right gate rather than a proxy for one. Every project in `docs/PRIOR-ART.md` that searches hits the engine-speed wall, and the pattern is clean: VGC-Bench does not search and used real Showdown; Future Sight AI searches, modified Showdown, and reaches about 3 turns of lookahead in 15 seconds on 16 cores; Foul Play searches, **built** poke-engine, and reaches about 10+ turns. Building an engine is what searching costs. #62 asks whether the thing it buys exists.

On compute, because it is the obvious response to #62 failing. Cores help the search — it is CPU-bound and root-parallelisable. GPUs help behaviour cloning and PPO. MILTANK needs **26 s against a 20 s budget on one core of sixteen**, so sixteen cores fixes the clock today. But root parallelisation scales **sublinearly**, so cores convert a *failed budget into a met one*, not a *shallow search into a deep one*. If #62 says compute buys nothing at depth, more cores does not answer it.

Phase 4 is approved in advance and is a result, not a defeat. Will, 2026-08-06: *"IM OKAY TRYING THINGS OUT LIKE SEARCH TO SEE IF OUR ATTEMPT WORKS BUT IF THAT FAILS IM OKAY ALSO TRYING THEIR OTHER METHODS."* Their recipe is published, open-source and reproducible, so the fallback is cheap precisely because they made it so.

5.1 MEDICHAM completeness — PHASE 1, and everything below waits on it

Will's bar, 2026-08-06: *"i still want medicham to be fully wired and tested on every move and ability and item in the regulation (with any usage at all) before we start taking its output and using them."* Target agreed the same day: **99% of usage — 484 of the 819 things that carry real usage** — plus a carve-out for anything that turns a certainty into a failure regardless of usage.

#	item	status
#45	the gate itself: 99%-of-usage coverage, enforced not remembered	being built
#42	ratchet on DIRECT-CALL probes, not on <code>unarmed</code> — 37 left of 47	in progress
#51	23 tags with real usage and no probe at all ; accuracy modification is the top	queued behind 42
—	3 declared-missing mechanics: Avalanche, No Guard, Sand Veil	queued behind 42
—	1 live differential: <code>chesnaught woodhammer -> mimikyu</code> (Disguise, unimplemented)	queued behind 42
—	6 interaction-matrix disagreements: <code>stone axe ×4, giga impact, supercell slam</code>	queued behind 42
#27	the represented-clicks number — what % of real clicks the engine can even perform. Last measured 15.3% failing when it handled 4 action kinds; it now handles 12 MEASURED 2026-08-24 BY MEASURE, AND THIS ROW'S OWN PROSE WAS THE STALEST PART OF IT. <code>data/medicham-represented-clicks.json</code> (untracked, generated by <code>engine/medicham_coverage.js</code> at 2026-08-23T19:47Z) is a FULL run and not a scoped one — it takes <code>fit_policy.loadCorpus()</code> , the standard clean open-sheet filter, with no thinning flag, no sample cap and no <code>--games</code> : 12,806 clean games, 298,910 human clicks, 298,888 representable and 22 not — 99.9926% covered. The kind table holds 42 kinds , not the 12 this row's text claims, and the whole unmodelled remainder is two moves: Role Play (19 clicks) and Reflect Type (3). IT IS STILL NOT PUBLISHABLE, AND THE REASON IS THE STAMP RATHER THAN THE METHOD. It pins itself to ONE build (<code>engine/medicham2-browser.js digest d35956ad9967</code>) and says in its own note that a coverage figure transfers to no other build — and the simulator has moved through six releases since, one of which FIXED Role Play, i.e. 19 of its 22 remaining pass-clicks. Its denominator moves too: OPS appends to the store hourly, so 12,806 is a photograph. It was NOT committed. WHAT THIS ROW WANTS IS UNCHANGED — the same run on a settled engine and a fresh release — and it is now a short re-run rather than an unknown.	open — needs a settled engine + fresh release, unchanged. MEASURED 2026-08-24: a FULL run (12,806 clean games / 298,910 clicks / 99.9926% representable / 42 kinds), but stamped to a superseded build and stale by construction — Role Play, 19 of its 22 remaining pass-clicks, was fixed after it was written. Artifact left untracked and uncommitted. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: needs a settled engine + fresh release
#49	delete the 77 hand-typed mega abilities; the derived path exists and is proven equal	queued behind 42
#9	harden <code>pranksterBlocked</code> to check the move's target	latent, not live
#71	seven moves share one tag signature — Pain Split, Recycle, Copycat resolve identically	—
#72	hazard SETTING by an attacking move, and ALL hazard removal	CLOSED 2026-08-11 — setting landed at 3.99.0; REMOVAL did not exist at all. <code>medicham2-browser.js</code> contained the strings <code>defog</code> , <code>rapidspin</code> , <code>mortalspin</code> and <code>tidypup</code> zero times , so a laid hazard was permanent and the engine over-valued setting them while valuing Rapid Spin at nothing. New derived tag <code>removesHazards</code> ,

#	item	status
		membership printed before wiring — exactly 4 members of the 500 legal moves — with one consumer, <code>sweepField()</code> , called from three sites. Three probes, each RED first with the control arm already correct: Rapid Spin sweeps MY side and leaves theirs standing (<code>0/1/0</code> against Iron Head's <code>1/1/40</code>); Defog clears both sides' rocks, THEIR screen and the terrain and NOT my screen (the surviving own-Reflect is the over-match guard); Tidy Up tidies both sides and every Substitute, and Mortal Spin pulls its own Leech Seed. COURT CHANGE IS NOT IN THIS FORMAT — <code>isNonstandard: 'Past'</code> — so this row named an illegal move, and it SWAPS rather than removes in any case
#98	THE INERT BUCKET — DONE, 3.8o.o. 124 abilities covering 72,609 uses fell through <code>ability/generic</code> , which stages a plain attack, so the condition each one needs was never created: Showdown's own board came out identical with and without them and the roster reported INERT. That reads as "nothing to test" when the truth is "never tested" — a coverage hole wearing a completed run's clothes. Fifteen new shape rules took it to 59 abilities / 4,261 uses , nothing left above 500 uses. Its first real finding is the pinch family, #112	done — kept here because <code>docs/ENGINE.md</code> cites it
#107	PER-STAGE ROSTER ARTIFACTS — DONE, 3.8o.o. <code>--write</code> wrote <code>data/roster.json</code> whatever stage ran, so one file could not carry three stages and a moves run silently destroyed the abilities results — twice in one day. It now writes <code>data/roster.<stage>.json</code> , <code>--stage</code> <code>all</code> writes <code>data/roster.all.json</code> , and an overwrite is ANNOUNCED with the outgoing artifact's stage, release and counts, its bytes kept at <code>.prev.json</code> . This is what let the quarantine gate (#99) read measurement instead of absence on two of its four clauses	done — kept here because <code>docs/ENGINE.md</code> cites it
#109	THE PHOTOGRAPH FREEZES THE SUBJECT BUT NOT THE CAMERA — GUARDED, 3.84.o, BUT THE BACKLOG IS NOT RECOVERED. <code>engine/game_differential.js</code> is not in <code>engine_release.js</code> 's <code>SOURCES</code> , so the ENGINE is frozen and the DRIVER that reads it is live. Re-measured over 65 release directories against the 13 symbols the driver actually reads: 56 SYMBOL-ABSENT (no <code>natureL50</code>), 1 FILE-ABSENT (the oldest predates <code>mc_key.js</code> joining <code>SOURCES</code> and dies inside <code>engine_release.js</code> before the symbol check is reached), 4 pruned, 5 usable . Fix (c) chosen: <code>REL.require(file, {need, want})</code> refuses a missing export BY NAME with the cut date and a <code>compat</code> command, instead of a <code>TypeError</code> 1,280 lines deep. (a) was rejected on the driver's own argument that freezing the instrument would mean "each rung was scored by its own contemporaneous reader, which is the one thing a ladder must not do"; (b) rejected because the exported surface IS the version and is derivable. THE 56 DO NOT COME BACK — those bytes never held the function. Said plainly here because the +6 and +5 <code>SOURCES</code> growths did not repair anything cut before them either and neither of them said so . Consequences: <code>engine/wire_ladder.js</code> cannot run at all (all 14 rungs lack the symbol, so the published ladder stands but is no longer replayable); #57 is unaffected (it re-runs against the LIVE engine and never re-opens a release); #99's lift condition runs through the differential, which now has 5 usable releases of 65	guarded; backlog unrecoverable; <code>wire_ladder</code> needs a decision
#103	THE MULTI-HIT COUNT WAS THE MEAN — DONE, 3.9o.o. Eleven roster rows on one rule, -1,743 uses, and the differences went BOTH WAYS by small margins, which is what said it was a COUNT and not a missing mechanic. <code>sim/battle-actions.ts:869</code> samples a twenty-element table and <code>PRNG.sample</code> is <code>items[random(items.length)]</code> , so the pin selects element 19 or element 0 and never a middle : Icicle Spear lands 5 at one corner and 2 at the other. This engine answered 3.1 — the mean — to every question , including the one a real turn asks. Too few against a 5, too many against a 2, on the same move. <code>rollHitsOf()</code> now draws the count once per move use, beside <code>expectedHitsOf()</code> which stays the PRICE for the search. Per-hit flooring was ruled out with arithmetic rather than preference: <code>roll()</code> already returns an integer, so <code>Math.floor(v*n)</code> equals <code>n*v</code> for every value — that line did not change. Newly REACHABLE and filed : Skill Link rewrites the hit count (forces 5), and until now there was no count to rewrite — 46 uses, legal only on Heracross-Mega and Toucannon. (<i>This first also named Loaded Dice; Will caught it — Loaded Dice is isNonstandard: 'Past' and does not exist in this format. Read the ban from the format, not from memory.</i>)	done — 8 of 11 rows closed
#101	<code>buffsHolderOnHit</code> NOW READS ITS CONDITION — DONE, 3.89.o. The consumer applied <code>_buff.boosts</code> on EVERY connecting hit, so Anger Point maxed Attack off any hit rather than a critical one, Justified fired on non-Dark and Weak Armor on special hits. Stamina — 2,773 uses, the only member with real usage — is unconditional and therefore correct, which is exactly why nothing noticed. <code>condHolds(w, self)</code> widened to <code>condHolds(w, self, hit)</code> , the cost WIRE 2 predicted; four shapes readable (<code>crit</code> , <code>moveType</code> , <code>moveCategory</code> , <code>moveFlag</code>) and <code>buffOnHitUnknownCond</code> reads o , which is how we know every condition in the artifact is readable. <code>+6, not +12</code> : Showdown's handler is <code>setBoost({atk:12})</code> = "max it from wherever you are"; this engine clamps stages to ±6 twice over, so the effective Attack is identical and a literal 12 would be a number nothing here can read. HALF-FIXED AND SAID SO : volatile-payload members (<code>electromorphosis</code> 98 uses, <code>windpower</code> , <code>perishbody</code>) are still not granted — no consumer multiplies an Electric move by a banked consumer Charge — but the debt is now COUNTED at <code>MEDFAILS.buffOnHitVolatileUnwired = 2</code> rather than silently dropped	done; volatile payloads still owed
#102	THE PROCEDURAL HEAL FAMILY — DONE, 3.89.o. Synthesis, Moonlight and Morning Sun resolved to <code>{kind: 'pass'}</code> — a wasted turn, and in sand the click was strictly worse than passing because the residual chipped the body that had just healed o. On a 155 HP body: clear o → 77, sun o → 103 , rain o → 39 . <code>healParam</code> returns a RECIPE rather than an amount, because <code>playerAction</code> sorts the turn before the sky is final; fractions go through <code>md4096</code> because the handler is <code>this.heal(this.modify(maxhp, factor))</code> and <code>maxhp * 2/3</code> is a different number. MY DIAGNOSIS WAS WRONG ON STRENGTH SAP (693 of the 1,024 uses): it resolved to <code>affect</code> , not <code>pass</code> — WIRE 79's Attack drop had already landed and only the HEAL was missing, so it was deliberately NOT reclassified as a heal, which would have traded one half for the other	done
#111	THE VOLATILE DURATION FAMILY — DONE, 3.82.o. 9,092 uses and the top of the moves queue: Encore 5,599, Taunt 1,714, Infestation 971, Disable 808, all on one rule and one mechanism. Showdown decrements inside the Residual event (<code>battle.js:342</code> , ordered by <code>onResidualOrder</code>), so a volatile applied on turn N has already spent a turn by the end of it — the Perish Song bug a second time , documented at <code>medicham2-browser.js:6588</code> , fixed for that one volatile and left standing for every other. Three sub-rules were missing entirely: re-application FAILS (a second Taunt was refreshing the counter), the counter is adjusted by whether the target already moved, and Encore/Disable need the target's last move — which Disable never had. Taunt and Disable now MATCH; whole-game agreement 76.9% → 78.9% paired; roster moves 52 → 50 DIFFER. Encore remains DIFFER on HP only — <code>onOverrideAction</code> replaces the chosen	done — Encore's HP row and infestation remain

#	item	status
	move and medicham2 honours the lock only inside <code>_chooseAction</code> , so a scripted action walks past it. Its own row. <code>infestation</code> also still parts, one LOWER and with no plain duration	
#112	THE PINCH FAMILY HAS NEVER FIRED — 8,524 uses. Blaze 5,903, Torrent 1,924, Overgrow 651 and Swarm 46 carry <code>onlyWhen</code> : <code>"only below 1/3 HP"</code> as PROSE, and the <code>damageBoost</code> consumer at <code>medicham2-browser.js:2761</code> requires <code>!_db.onlyWhen</code> . The refusal is correct under #92 — a guessed condition is worse than none — and the defect is that <code>onlyWhen</code> was never made machine-readable. The consumer's own comment names its live members and says <i>"All five are o corpus uses"</i> : armed for what nobody runs, failing closed on what everybody runs. Same class as #101. DONE, 3.83.0. <code>engine/tag_dex.js</code> derives the gate by SHAPE out of Showdown's own <code>attacker.hp <= attacker.maxhp / 3</code> into <code>{cond:'hpFraction', of:'self', cmp:'<=', num:1, den:3}</code> , and <code>condHolds</code> in <code>medicham2</code> evaluates it in INTEGER arithmetic — <code>maxhp * (1/3)</code> is not <code>maxhp / 3</code> and would refuse a body at exactly one third. The fail-closed rule is untouched: an unreadable <code>onlyWhen</code> still returns null, still refuses, and is now counted. Roster abilities 1 DID-NOT-FIRE → 0 , with exactly four verdicts changed (Overgrow out of DID-NOT-FIRE; Blaze/Swarm/Torrent out of CONTROL-NOT-QUIET) and nothing else moved. Census 324 → 325 . Gate: <code>tests/test-pinch-family.js</code> , RED at 31/61 before the fix, plus two halves of the reversal in <code>tests/probe_red_demo.js</code> and the five o-use members as an explicit positive control	done
#81	THE WIRE QUEUE. The whole-game differential's divergences, ranked and verified against Showdown's source. ONE at a time, re-run between each (Will, 2026-08-06). WIRE 1 (Protect is not an immunity, and a shielded High Jump Kick still crashes) landed in 3.61.1	in progress
#84	"my move didn't happen" is TWO states in Showdown — <code>false</code> counts as a failure for Stomping Tantrum, <code>null</code> does not, and flinch returns <code>false</code> . Get Faked Out, and your Stomping Tantrum doubles	CLOSED 2026-08-11 — STRUCK on measurement, not on reading the code. The engine already implemented both states; the row read open because no probe covered the MISS case . New probe reads <code>MEDSEEN.powerDoubledAfterFailure</code> through real turns: landed-first 101, delta 0 ; missed-first 200, delta 2 . Will identified the miss case unprompted and was right — verified in the authority: <code>hitStepAccuracy</code> sets <code>hitResults[i]=false</code> (<code>sim/battle-actions.ts:748</code>), <code>atLeastOneFailure</code> goes true (<code>:610</code>), the null branch is skipped (<code>:618</code>), <code>useMove</code> writes false (<code>:374</code>), and the callback tests <code>moveLastTurnResult === false</code> (<code>data/moves.ts:18047</code>). Doubling still matches two move names — counted, with a stated reason
#87	CACHE THE TEAM POOL — DONE, 3.69.0 (<code>b168490</code>). 41s of every invocation was spent reading a 311MB store to pick 87 teams, and the pool was read LIVE from a file OPS appends to, so a divergence seed stored in an artifact NO LONGER RESOLVED — its pair was not in the next day's pool. Startup 45.1s → 0.8s, and the pool is now digested, so a seed plus a pool digest reproduces the exact game for the first time. That is what made <code>turn1_queue_at_the_top_rung</code> replayable	done — kept here because <code>docs/ENGINE.md</code> cites it
#88	ONE PIN IS ONE CORNER. Every die is pinned one way, so a game runs once and is deterministic — no noise, but the tie always resolves to input order, every sub-100 move misses on BOTH sides, and damage is always the max roll. Agreement is therefore BY CONSTRUCTION on the ~91% of matchups that are speed ties (compounds with #86). The crit's wrong position, found by the stage audit, is the first confirmed defect this hid: 46.5% wrong at the bottom roll, invisible at the top one we always use. Fix is four pinned arms, not repetition-and-average, which would throw away the tolerance-zero this instrument exists for	CLOSED 2026-08-11 — AND THE PIN WAS NOT WHERE THIS ROW SAID IT WAS. The gate's <code>game_differential</code> clause does not read <code>engine/game_differential.js</code> (which already ran both arms); it reads <code>data/engine-diff.json</code> from <code>tests/test-engine-diff.js</code> , and THAT was the pinned instrument — in a way nobody had written down. It computed Showdown's damage at roll o AND roll 15, AVERAGED THEM , averaged MEDICHAM's min/max, and compared the two midpoints in a 12% band. A RANGE WRONG BY THE SAME AMOUNT AT BOTH ENDS HAS AN IDENTICAL MIDPOINT AND CANNOT MOVE THAT NUMBER. Shown red by a plant the old number is structurally incapable of seeing — <code>--plant</code> spread widens our range symmetrically, and at n=300 reads midpoint o of 300, top 196 of 300, bottom 218 of 300 . The measurement after: top o/6000, bottom o/6000 , seed 20260804 — both corners clean, so the CLAIM got stronger while the answer did not change. The clause now REQUIRES both, an artifact with no <code>arms</code> block FAILS rather than passing by absence, and one carrying <code>plant</code> fails too; a planted run writes <code>data/engine-diff-PLANTED-spread.json</code> and never the gate's artifact. Seven selftest rows drive the SHIPPING function on whole synthetic artifacts
#89	the engine justified its Disguise model against <code>battle.update()</code> , a Showdown method that does not exist . The model is CORRECT (both engines end at 114/130) — a right answer resting on a false reason, which is worse than a wrong one, because the next reader re-derives from the reason	CLOSED 2026-08-11 — <code>battle.update()</code> confirmed ABSENT by walking <code>Battle.prototype</code> . The engine had already been corrected; the fiction still lived in <code>tests/test-engine-diff.js:503</code> and is now cited line by line (<code>abilities.ts:962-966</code> and <code>:996</code>). AND THE CORRECTED PARAGRAPH WAS ITSELF READING MAINLINE — Champions DOES override Disguise (<code>data/mods/champions/abilities.ts:14</code> , <code>onEffectiveness</code> only). A right answer resting on a false reason, corrected by a second false reason, caught only because the rule says cite the line you read it on
#90	THE PERISH SONG KO IS PROVEN — CLOSED 2026-08-11, 5.4.0. Will, 2026-08-07: <i>"the counter being right is not evidence the faint happens"</i> . He was right and it is now demonstrated, not argued: with the KO deleted and the clock left intact , <code>tests/test-volatile-duration.js</code> is still GREEN — <code>target.faint()</code> had never once been executed by anything in this repo and 1,141 corpus uses rested on it. <code>tests/test-perish-song.js</code> asserts all three of his clauses — the faint fires on turn 4 with turns 1-3 as the negative; it fires on BOTH SIDES at once because <code>perishsong.target</code> is <code>all</code> , so the singer's own partner dies too; and a body that leaves the field survives. Shown RED on <code>--break-the-faint</code> first. The switch clause was staged with a PIVOT rather than declared unstageable — <code>staged_board.js</code> exclusion D says the script language has no voluntary switch (#122), and its own text says every switch there is driven by a pivot move. A fourth scenario for FAINT ORDER was written and REMOVED: it passed against a deliberately reversed residual loop, because four simultaneous deaths look identical to a board comparator. That became #115. VERIFIED BY: <code>node tests/test-perish-song.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	closed 2026-08-11
#91	A CLICK IS NOT A TEST. <code>creditClick</code> credits the CLICK, so Haze into a board with no boosts on it marks Haze covered and the steering stops selecting it. The census's own PROBES are behavioural and sound (o hollow, o direct-call); this is the differential's coverage steering only. Credit must move to the observed EFFECT, with the precondition derived from the tag's params — not a hand-written scenario per mechanic	after #92
#160	ILLUSION IS IN THE CLOSET, AND THE CLOSET IS LABELLED — DECLARED 2026-08-11. Will: <i>"at some point we are going to have to have zoroark in our engine but to start we can just put those in the closet."</i> Zoroark enters disguised as the LAST living body on its bench and the protocol names the DISGUISE, not the mover — so every click it makes is filed against a Pokemon	open — engine

#	item	status
	that never moved, manufacturing false set evidence for the impersonated body and hiding the real one. Measured: 386/12,314 b03 raw (3.13%), 1,731/52,840 ladder (3.28%), and 384 games rejected out of the clean fit corpus (8,543 -> 8,159). <code>fit_policy.js</code> counts and NAMES the rejection beside <code>partial_bring</code> , because a filter that cannot say how much it removed is indistinguishable from one that is not running. TEMPORARY BY DECISION . The real fix is to model Illusion; then re-admit with <code>ILLUSION_IN=1</code> and REFIT , because a corpus that changes size changes answers RE-RULED 2026-08-28, SAME SHELF, NEW WORDS . Will: <i>"we put illusion and zoroark into the closet cause its too ahrd to deal with"</i> , and <i>"bitter malice and night daze are only learned by zoroark i believe, which we put in the closet"</i> . Both derivations CONFIRMED against <code>TeamValidator</code> over the 347 legal species of <code>gen9championsvgc2026regmb</code> : Bitter Malice -> Zoroark-Hisui, Night Daze -> Zoroark, one legal learner each , and both carriers hold <code>{"0": "Illusion"}</code> — one ability, no second slot — and are the ONLY two legal Illusion carriers in the regulation, so neither move can be staged on a body without Illusion. NO NEW EXEMPTION WAS WRITTEN: see #520 . The reason is cost, not absence of a defect, and a cost ruling is permission to stop working on it rather than evidence about the engine.	
#161	THE FIVE LIVE MATRIX DISAGREEMENTS — ALL CLOSED 2026-08-11, 5.7.0. The matrix is 1,640 / 1,640, 100.0%, parting 0 . Each wire landed alone and was measured alone (#81), and <code>live</code> stayed 1,640 with <code>ran</code> at 2,253 across all five — the rate moved because the engine moved, not because the instrument shrank , which is the failure that hid here four days earlier. Root causes: Throat Chop's sound lock sat BELOW the secondary loop and so outside its Shield Dust gate (its volatile is a <code>CLOSURE</code> in <code>secondary.onHit</code> , which is why no field-reading derivation saw it); Psychic Noise wrote <code>_vol.healblock</code> on the gated road and <code>_healBlock</code> on an ungated one — two implementations of one fact ; Instruct, the only road to a second action in a turn, asked none of the ordinary refusals; Psyshield Bash's whole effects step was discarded by <code>if(!R.hit)return;</code> because <code>R.hit</code> means <i>the target survived</i> ; and the recharge was armed for a move that reached no body. TWO OF MY DIAGNOSES WERE WRONG AND ONE ENGINE COMMENT WAS FICTION — WIRE 43 asserted a blocked Hyper Beam still recharges, and a staged control shows Hyper Beam into a GHOST parts (<code>[]</code> vs <code>["mustrecharge"]</code>), a far commoner board than the 8-use row that found it	closed 2026-08-11
#162	**SPLIT THE COLLAPSED TAGS — WILL'S CALL 2026-08-11: "we need to split the tags, they require very different things".** #59 (the protection counter has three behaviours and our tags carry two) and #60 (Upper Hand and Sucker Punch share one signature: Sucker Punch needs the target ATTACKING, Upper Hand needs the target using PRIORITY) are the same defect — one tag standing for two rules, so one of them is always wrong. #71 and #127 are the same shape at larger scale (105,709 uses). The shape decision was the blocker and it is now made; deriving the conditions is mechanical	open — engine. Families 1 and 2 SPLIT and landed 2026-08-11 (#59, #60, both closed). Families 4 and 5 were found ALREADY SPLIT by earlier work and were measured rather than claimed — <code>wish / rest</code> carry <code>healDescriptor</code> , Pain Split -> <code>sharehp</code> , Copycat -> <code>callmove</code> . Recycle is still untagged and is registered separately. Gate OPEN 6/6 after every family; differential 0/20,000 throughout; matrix 1,640 -> 1,641 at 100.0% with <code>part 0</code>
#115	RESIDUALS RUN IN SPEED ORDER — CLOSED 2026-08-11, 5.7.0 . Will: <i>"pokemon faint in speed order"</i> , <i>"in trick room do they faint in reverse speed order?"</i> , <i>"sandstorm damage, etc"</i> . <code>medicham2-browser.js:14890</code> iterated <code>[...actA,...actB]</code> in <code>SLOT</code> order and now sorts through <code>compareTurnOrder</code> , which reads the field so Trick Room's inversion comes free. Scoped by measurement: the mega step (<code>:9313</code>) and the switch-in step (<code>:8827</code>) ALREADY sorted, so Intimidate and mega ordering were correct all along — the two obvious suspects. The whole residual family rides it: sand and hail chip, burn, poison, Leftovers, Salt Cure, Yawn, Heal Block. NOT cosmetic — <code>battle.ts:2604</code> awards a simultaneous double-KO to <code>faintData.target.side</code> , the side of the LAST faint off the queue, not a tie	closed 2026-08-11
#92	THE DAMAGE-STAGE CLASS — DONE, 3.72.0 . We applied multipliers at the wrong STAGE and often unchained; Black Glasses on final damage where Showdown does base power was 108 against 109. LANDED into ONE <code>onBasePower</code> relay: the 18 type items, Muscle Band, Wise Glasses, Technician, Tough Claws, Sharpness, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Sheer Force, Supreme Overlord, Expanding Force / Rising Voltage, Dry Skin and the -ate x1.2. Into the STAT relay: Thick Fat (73%), Heatproof, Purifying Salt, Water Bubble (77%). Off the hit site and into the chains they belong to: Helping Hand (5/5 rows) and Friend Guard (21.4%). Sniper out of the crit's plain multiply and into <code>onModifyDamage</code> . The crit's POSITION into <code>dmgRange</code> , before the randomizer. The four field terrains, which were absent entirely (161 uses — Will corrected my "most games"; the reason to land them is the magnitude, a Grassy Earthquake was priced at DOUBLE). <code>damageBoost</code> is wired only for the narrowed self-describing shape and the reason the other 39 members are NOT is recorded in <code>docs/ENGINE.md</code> . New gate <code>tests/test-damage-stages.js</code> — 1,728/1,728 exact against the authority across all 16 rolls and both crit states, shown RED on two deliberate reversions before being trusted. Census 293/294 → 298/299. Full audit in <code>docs/DAMAGE-STAGES.md</code> . VERIFIED BY: node tests/test-damage-stages.js — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	done — kept here because <code>docs/ENGINE.md</code> cites it
#96	TWO TYPE AUTHORITIES, TWO DIFFERENT SKIES — DONE, 3.87.0 . <code>effMoveType</code> — the battle loop's authority for the stage-5 immunity gate, the absorb check, the Lightning Rod draw, Protean's retype and the Fire thaw — resolved the weather branch off <code>field.weather</code> RAW, while <code>dmgRange</code> resolved it off <code>effWeatherOf</code> , which applies the PRIVATE sky (the <code>privateWeather</code> tag; Mega Sol). Under a private sun with a clear field the damage calc priced Weather Ball as Fire, 128-151 and the loop refused it as Normal , so a Meganium-Mega's headline click dealt LITERALLY ZERO to a Ghost. This is WIRE 126's own hazard inside WIRE 126's own function: the thing built to be the single answer to "what type is this move" then disagreed about a different input. Fixed by a CALL to <code>effWeatherOf</code> , never a copy. WIRE 126's declared hold in <code>ClickFragility</code> is lifted with its reason kept — that function's own <code>base</code> already saw the private sun, so it contradicted itself; benchRisk moves and a refit is owed at the next release cut (MEASURE) . New probe <code>ability/privateWeatherMoveType</code> is the CROSS neither half's probe could reach — a private sky AND a move the sky retypes AND a Ghost — and asserts private sun === public sun, not merely <code>> 0</code> . Three rows in <code>probe_red_demo.js</code> , one of them the public-weather positive control. Census 325 → 326 live ; roster and the 1/150 differential unmoved	done — kept here because <code>docs/ENGINE.md</code> cites it
#83	the mirror-symmetry gate false-alarms ~1 run in 20 by construction — n=300 gives a ±5.6 point half-width	—
#82	the differential's BRING is a run parameter, not the accident of sheet order	—
#80	Knock Off records the wrong DISPOSITION for a self-eaten berry — <code>[eat]</code> vs move-attribution, which Harvest/Recycle/Belch/Cud Chew/Unburden read. The damage claim originally filed here was measured FALSE and is retracted; Colbur fires correctly for us	found by the differential

#	item	status
#20	**RETRACTED AS WRITTEN, 2026-08-10 — A TYPE IS A REACTOR AND THE HARNESS WAS THE LIMITATION.** The item said Grass-blocks-powder was untestable because a TYPE cannot be a reactor. Will: "ALL TYPES HAVE A SORTA ABILITY, DARK IMMUNE TO PRANKSTER, GRASS TO POWDER MOVES, GHOSTS CANNOT BE TRAPPED, ETC." He is right, and it is measurable: asked of a live battle in this format, Fire refuses burn, Steel refuses poison, Electric refuses paralysis, Ice refuses freeze ; the type chart carries seven more (Dark/Psychic, Fairy/Dragon, Flying/Ground, Ghost/Fighting+Normal, Ground/Electric, Normal/Ghost, Steel/Poison); Prankster names Dark in its own text; the <code>trapped</code> volatile refuses Ghost; and powder is gated in the battle rules. The reason this looked impossible is that type reactions live in FIVE different places — the type chart, <code>runStatusImmunity</code> , an ability text, a volatile condition, and the battle rules — and only one of them resembles a reactor if you are searching for an ability slot. The fix is that the harness reactor may be a TYPE , which is CHEAPER than an ability reactor, not harder: there is no ability to swap, you pick a body of that type and the negative control is a body that is not. This unblocks the ~400 pairs the interaction matrix drops	open — <code>docs/ENGINE.md</code>
#21	AXIS 4 — DENIAL: a move that costs a turn also denies a field effect	—
#28	resist berries: the one staged pair is physically impossible	—
#7	Coverage Layer 2 — the mutation harness	—
#59	the protection counter has three behaviours and our tags carry two. Protect <i>checks</i> the shared 1/X counter; Wide Guard <i>feeds it and never checks</i> ; Ally Switch has its own private one and carries neither tag. The engine implements none of it — <code>grep allyswitch</code> and <code>grep stalling</code> both come back empty	CLOSED 2026-08-11 — split three ways and the engine now implements all of it. Six derived tags, every membership printed over all 500 legal moves before wiring, zero over-match: <code>stallCounterChecks</code> 6, <code>shieldsUser</code> 5, <code>stallCounterFeeds</code> 2, <code>failsIfMovesLast</code> 8, <code>privateStallCounter</code> 1. The real defect: Wide Guard CLEARED the shared stall counter where Showdown TRIPLES it (<code>onHitSide</code> -> <code>stall.onRestart</code>), so a Wide Guard was a free way to refresh a decaying Protect. Probes red first at <code>stallCounterFeeds</code> 182 vs 0 and <code>failsIfMovesLast</code> 182 vs 0. Ally Switch's private counter was implemented and had NEVER been probed; it is probed now
#60	Upper Hand and Sucker Punch share a tag and have different conditions — Sucker Punch needs the target attacking, Upper Hand needs it attacking <i>with priority</i> . Modelled the broad way, the bot thinks Upper Hand beats an ordinary Earthquake	CLOSED 2026-08-11 — <code>failsIfTargetMoveNotPriority</code> derived, 1 member, shown RED at 44/44 before the fix. The bot no longer believes Upper Hand beats an ordinary Earthquake. Visible in the matrix too: off-gate disagreements 30 -> 24, <i>upperhand</i> -> <i>roughskin</i> among them. Sucker Punch's SECOND refusal is still unmodelled and is registered separately

Both came from Will reading the rules, not from any instrument, and both are the same shape: a tag derived from Showdown's flags is only as complete as the flag. Ally Switch's stalling lives in `onPrepareHit` with no flag on it, so the deriver never saw it. **Showdown does not have tags — it has code**, and every mechanic here is a re-implementation of a handler, which is the entire reason `engine/tag_dex.js`, the differential test and the arming work all exist.

5.2 GARY — the opponent inside the search, named 2026-08-06

Four defects in one place, all found by reading the source. See §4.2.

#	item	blocked by
#33	every artifact records which GARY ran — <code>rollout_r1.js</code> done, 3 callers left	—
#32	<code>flip uniform</code> → <code>prior</code> and measure it	33, and a fresh release
#34	the flag steers my side too — split it	—
#35	targeting is drawn uniformly in both modes. Foul Play weights its chance nodes by likelihood — a 90/10 accuracy move becomes two children selected by probability. It does correctly what #32/#35 have open as a defect (#74)	—
#36	GARY has two seats and they disagree	—
#55	GARY does not coordinate either — the imagined opponent picks its two Pokémon independently, so it never focus-fires, never Protects while its partner kills, never redirects. **DODUO is GARY** , pointed at the other side	#39, #45

The fifth defect is the largest and it collapses two workstreams into one. MAG choosing independently costs us our own quality — a symmetric loss. **GARY choosing independently makes the imagined opponent structurally incapable of the plays that actually beat us**, which is bias with a direction. And the objection that imitation is a ceiling *does not apply to the opponent model*: we **want** the imagined foe to behave like a human, because humans are who we play.

5.3 The search redesign — PHASE 3, and none of it starts until #62 answers

Reordered 3.59.0. This whole table used to be the front of the project. It is now downstream of the gate in §5.0: if compute buys nothing, none of these items is worth building, and the project takes phase 4 instead. **#61 keeps its place here and gains a second reading** — the 3,401 battles/sec it is measured against is itself corrected in 3.59.0 to 13,041 turns/sec against `champions_sim`'s 523, a ratio of 24.9x rather than the recorded 117x. Three readings of one quantity, no ratchet on any of them; see ADR-001's correction note.

#	item	note
#37	prune with MAG's scores, not with the coin	Will approved with a condition: <i>"make sure it doesnt toss moves a VGC pro would make"</i>
#25	prune by PAIR score, not single-move	the truncation curve says single-move ranking cannot meet that condition at any affordable K
#53	DODUO — the pair model is fitted, better, and the search never reads it	the fix is wiring, not building
#56	extract what MAG actually prunes that DODUO wants — the truncation misses <i>are</i> the catalogue, and reading them out beats reasoning from examples	needs a frozen release
#24	replace MILTANK's leaf with PORYGON2 + a MAG-sampled opponent, and add Foul Play's damage-roll grouping as a third arm — group outcomes by whether they cause a FAINT, average the damage inside each group, sum the likelihoods. Keeps the only distinction the decision turns on and collapses the rest; cheaper AND lower-variance without being dumber, which is the exact pair of properties this task is short of (#74)	gate passed (#23)
#38	the rollout cap is 60 turns; real games end at 6	trivial
#39	measure the <code>board.js</code> ↔ MEDICHAM translation cost	decides whether MAG-as-GARY is possible at all
#61	MEDICHAM measures 1,606 battles/sec against the 3,401 on record — 47% . On the measured figure the 200-rollout x 64-pair search needs 26 s against a 20 s budget and does not fit. Nothing ratchets speed, which is why a 2x regression went unseen	—

#	item	note
#62	MOVED TO §5.0 IN 3.59.0 — it is the project gate, not a search-redesign item. Everything in this table is phase 3 and does not start until #62 answers	see §5.0
#63	the rollout never switches. The engine <i>can</i> , and the search can offer a switch as a root candidate — but the payout has no switch branch at all, so every imagined future is a game where nobody ever leaves. Misprices preserving a Pokémon, punishing a switch, and the <i>switch out of a predicted attack</i> pattern the engine models priority order for	#45, #61

Four different states get quoted for each other here and only one is broken. The engine can switch; the search can offer it; **the rollout never does**; and the live bot has it **off** against a measured **10-point loss**. That loss is real — but `bringIn()` picks `live(bench)[0]` , the first healthy body, and its own comment calls that *"a real limitation, not a detail"*. ****A** switch to an arbitrary Pokémon is not a switch, it is a *leave*** — and a 10-point loss is precisely what evaluating "leave" would produce. Test that before concluding switching is bad.

5.4 Solving — DUSK, and the retrograde idea

#	item	note
#47	solve exactly whenever switching is off the table — 1v1 is 16 cells, 2v1 is 96, 2v2 is ~1,296	2v1 alone reaches 45.9% of games
#163	THE BLUEPRINT + LIVE RE-SOLVE ARCHITECTURE — WILL, 2026-08-11. <i>"you know how poker solvers have charts for every scenario that people try and memorize, i want that for us, so we can try searching for the most similar scenario from a repository as well as live rollout"</i> , alongside <i>"i dont want to behavior clone first i want to try our searching and poker like simulation"</i> . This is the architecture that beat humans at NLHE — Libratus and Pluribus do not play off charts, they compute a coarse BLUEPRINT offline and do REAL-TIME SUBGAME RE-SOLVING at the table. Blueprint + live search is exactly what he described, and it is a far better fit here than behaviour cloning: VGC-Bench trains offline so inference is cheap, whereas search pays per turn on a clock. THE ENDGAME HALF IS ALREADY SCOPED AND SMALL — #47 measures 1v1 at 16 cells, 2v1 at 96, 2v2 at ~1,296, with 2v1 alone reachable in 45.9% of games; retrograde analysis builds up from there (#30, #40, #41). THE EARLY GAME IS THE RESEARCH PROBLEM AND IT IS THE ABSTRACTION, NOT THE RETRIEVAL. Poker charts work on isomorphism — A(s)K(s) and A(h)K(h) are one hand — and Pokemon has no isomorphism that clean, because WHICH species is on the field is the information content. So the repository needs a learned similarity metric. WE HAVE A CANDIDATE AND IT HAS BEEN TESTED IN THE WRONG DIRECTION: #23 gated PORYGON2 on whether it can SEPARATE positions two turns apart; retrieval needs the inverse, that genuinely similar positions land CLOSE. Same encoder, opposite test, never run. THE HONEST DEPENDENCY: the re-solve half needs a fast simulator, which loops back to #62 and to the poke-env benchmark nobody has ever taken	open — search
#164	**THE END GOAL, STATED SO PRIORITIES CAN BE ARGUED AGAINST IT — WILL, 2026-08-11: <i>"the end goal is to beat closed team sheets and get to number 1 on the ladder but we are far off"</i> . It had never been written down, so every ordering argument in this register was being made against a target nobody had stated. IT IS A CURRICULUM, NOT A MISMATCH — WILL CORRECTED THIS THE SAME DAY**: <i>"once we solve open team sheets, we can think about the more difficult closed team sheets and adapt our models accordingly"</i> . I first filed the open-sheet corpus against a closed-sheet target as a violation of this file's <i>fitting environment must match playing environment</i> rule. That was an over-application. The rule governs fitting and playing in the SAME deployment; deliberately solving the fully-observable version first and adapting afterwards is staging, and staging is how you get a measurable baseline at all. WHAT SURVIVES IS NARROWER AND STILL WORTH HOLDING: a model fitted where every set is KNOWN must be able to accept UNCERTAINTY about sets later, and that is an architecture question better answered before everything is built assuming certainty — the same lesson as the Focus Sash drag, where replacing a hedge with a certainty inherited responsibility for every event that staled it. AND IT FLIPS THE POKER ARGUMENT THE RIGHT WAY. Open sheets strip out most hidden information and make our game EASIER than poker in the dimension poker is hardest; closed sheets put it back — unknown items, spreads, the fourth move, which four of six they brought — so information sets become the game and #163's blueprint+re-solve becomes the right primitive rather than a flourish. TWO THINGS ALREADY BUILT FOR OTHER REASONS ARE THE CLOSED-SHEET MACHINERY: #52 HYPNO (rate the opponent, decide how hard to exploit — deferred by Will) and the PREFER OBSERVED OVER DECLARED rule with its Knock Off / Trick / consumed-berry tracking, which is exactly the inference a closed-sheet bot needs. Will's own Farigiraf/Cud Chew read is the same idea from the player side: a bluff that exists ONLY because sheets are closed, and that open sheets delete. "Far off" is accurate — #62 unanswered, ~26 open rows in the MEDICHAM block, no search redesign started	open — direction
#165	ARGMAX IS UNSOUND IN A SIMULTANEOUS-MOVE GAME, AND OPEN TEAM SHEETS DO NOT RESCUE IT — CITED, 2026-08-11. Bosansky, Lisy, Lanctot, Cermak & Winands, <i>Algorithms for computing strategies in two-player simultaneous move games</i> , Artificial Intelligence 237 (2016) 1-40: agents <i>"may need to use randomized (or mixed) strategies in order to play optimally ... even though the only information that is hidden is each player's</i>	open — search

#	item	note	
	<i>action as they play it</i>". SIMULTANEITY ALONE FORCES MIXING. I had told Will the opposite — that open sheets make our game easier than poker in poker's hardest dimension. On this axis they do not. WHAT MEDICHAM IS: <code>miltank.js:1315 is if (v > bestVal) { bestVal = v; bestPair = [ia, ib]; } and :1384 clicks it — a plain argmax, deterministic by construction, answerable for free by anyone who knows the scorer</code>. MAGNEMITE underneath it DOES sample (<code>magnemite.js:954 , :1154 ,</code> a softmax its own header calls "<i>the single biggest measured lever</i>" and which argues "<i>Taking the argmax would be the obvious thing and it is wrong here</i>"), so the bot gets LESS mixed as it gets smarter — the 31.6% of turns MILTANK defers are the only ones that come out mixed, which inverts #62: more compute means fewer deferrals means more predictable. DUCT IS NOT THE FIX — I had it in the fix list and the paper reports it "<i>also might not converge to a Nash equilibrium</i>", even the mixed variant; UCT's failure is shown on biased Rock-Paper-Scissors (Shafiei, Sturtevant & Schaeffer, GIGA 2009). WHAT IS SOUND, three tiers from that paper: exact backward induction solving the matrix game per node, which "<i>outputs a subgame perfect NE</i>"; SM-OOS, which "<i>guarantee[s] to closely approximate the optimal strategies given enough time</i>"; and SM-MCTS-RM (regret matching at simultaneous nodes) as the practical middle. SO ROADMAP #41's PYTHON EQUILIBRIUM SOLVER IS THE TEXTBOOK-CORRECT PRIMITIVE FOR THIS GAME CLASS, currently filed as "<i>the equilibrium answer exists and cannot play</i>". And unlike Pluribus we are <code>rv1</code> zero-sum, so the "<i>equilibrium is not the right goal</i>" retreat does not apply to us — equilibrium IS the target here. PHASE 3, NOT NOW: MEDICHAM is Phase 1 and a correct simulator is a prerequisite under every version of this. Registered so it is not rediscovered		
#166	FOCUS SASH DOES NOT SAVE, AND STURDY DOES NOT EITHER. <code>test-tag-wire : "full-HP sash body survives the lethal hit at exactly 1, sash spent"</code> reads HP 0, item <code>' '</code>; "<i>Sturdy survives the same way and is not consumed</i>" fails beside it. The ARTIFACT is right and the WIRE is missing — the same test passes "<i>Focus Sash: full-HP gate, Move-only, leaves 1, SPENT — all from the handler (was a name check)</i>" and "<i>Sturdy: the same idiom minus the consumption</i>", and the full-HP gate itself is proven live ("<i>the SAME hit kills at 90% HP</i>"). So the tag carries the fact and nothing consumes it. Focus Sash is legal in this format and Sturdy is a common lead ability; both change whether a KO happens, which is the class of error that changes a game rather than a rating	CLOSED 2026-08-11 — THE ENGINE WAS NEVER BROKEN AND THIS ROW MISDIAGNOSED IT. One defect explains all four: <code>_chooseAction</code> 's last fallback clicked Struggle past a legal button (the #152 block, whose own comment stated the bug in place — "<i>a body with legal status moves left is not on Struggle</i>" — and filed it as the lesser evil). Struggle is a 50 BP attack with 1/4-max recoil, so every fixture body holding a single Protect or Roost was hitting the thing under test and paying for it. Heat Wave NEVER touched the partner: the trace reads <code>`</code>	move
#167	SITRUS BERRY FIRES ON THE WRONG SIDE OF ITS OWN THRESHOLD, BOTH DIRECTIONS. <code>test-tag-wire : "at 40% it eats the berry and heals exactly 1/4"</code> fails at 18/170 with item <code>' '</code>, and "<i>at 80% the berry stays in the pocket — the threshold is the tag's, not a vibe</i>" fails too. Failing in BOTH directions means the threshold is not being read at all rather than being read wrongly. The artifact is proven correct in the same run ("<i>Sitrus declares its own threshold and restore (was a name check in the residual loop)</i>"), so this is a consumer gap. NOTE FOR WHOEVER TAKES IT: the berry work closed earlier in this sprint reported <code>Sitrus 118</code>, so a live counter may be firing while this fixture is not — establish which is wrong BEFORE editing the engine, because a broken probe has already cost this project a session (<code>docs/LESSONS.md §5</code>)	CLOSED 2026-08-11 — THE ENGINE WAS NEVER BROKEN AND THIS ROW MISDIAGNOSED IT. One defect explains all four: <code>_chooseAction</code> 's last fallback clicked Struggle past a legal button (the #152 block, whose own comment stated the bug in place — "<i>a body with legal status moves left is not on Struggle</i>" — and filed it as the lesser evil). Struggle is a 50 BP attack with 1/4-max recoil, so every fixture body holding a single Protect or Roost was hitting the thing under test and paying for it. Heat Wave NEVER touched the partner: the trace reads a <code>move</code> line from <code>p2b: whimsicott</code> clicking <code>struggle</code> at <code>p1b: kingambit</code>, twice, for 21 of the 21 HP. The Sash DID save at 1 and Struggle recoil then killed the survivor. Fallback fixed, <code>test-tag-wire 10 FAIL -> 0 FAIL</code> over 104 checks **NOTATION REPAIRED 2026-09-04 BY MEASURE — pipe characters inside inline code were cutting this status cell, so the register read the row OPEN against its own closure. Only the notation changed; no claim was altered or removed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: cheap check from committed artifacts only — <code>data/mechanics-census.json</code> carries <code>item/healsAtThreshold "Sitrus Berry heals when it drops below half"</code> live, and <code>tests/test-tag-wire.js</code> is on disk; the row's 10 FAIL to 0 FAIL was not re-run here.**	
#168	AFTERMATH CHARGES A SURVIVOR NOTHING, AND THE KILL PATH IS FINE. <code>test-tag-wire : "contact into a SURVIVING Aftermath body costs nothing"</code> (HP 109/173) fails while "<i>KILLING it costs 1/4 of max HP (paid 43)</i>" passes. That split is the whole diagnosis — the tag now declares <code>contact + onFaintOnly</code> (asserted green in the same run) and the consumer is applying the faint-only branch to every contact hit. Ordinary contact into a living Aftermath body must cost nothing, so confirm the assertion is the correct expectation before changing the engine: this may be a wrong fixture rather than a wrong engine	CLOSED 2026-08-11 — FIXTURE, NOT ENGINE, AND THE ROW SAID TO CHECK THAT FIRST. The fixture predates PP being a resource (#144): the battle runs to the 20-turn cap, the moves empty, <code>mustStruggle</code> fires CORRECTLY, and the recoil contaminates the number the assertion reads. Measured before anything was edited — with PP restored the Aftermath attacker ends 173/173 and <code>_seededBy</code> on the Grass victim is false. The shared <code>one()</code> helper is why the neighbouring rows were red too	
#169	A BASE-FORME GULP MISSILE BODY PUNISHES NOBODY. <code>test-tag-wire : "a base-forme Gulp Missile body punishes nobody"</code> fails, immediately after "<i>Gulp Missile carries its forme gate, so the engine skips it whole</i>" passes. Read together those two say the forme gate is doing more than gating — it is skipping the mechanic even in the state where it should apply. Cramorant only; low usage, and it is on the list because a forme gate that over-fires is a SHAPE this engine has been bitten by elsewhere, not because Cramorant matters	CLOSED 2026-08-11 — FIXTURE, NOT ENGINE, AND THE ROW SAID TO CHECK THAT FIRST. The fixture predates PP being a resource (#144): the battle runs to the 20-turn cap, the moves empty, <code>mustStruggle</code> fires CORRECTLY, and the recoil contaminates the number the assertion reads. Measured before anything was edited — with PP restored the Aftermath attacker ends 173/173 and <code>_seededBy</code> on the Grass victim is false. The shared <code>one()</code> helper is why the neighbouring rows were red too	
#170	STORM DRAIN ABSORBS AND DOES NOT BANK THE BOOST. <code>test-tag-wire : "Storm Drain banks +1 SpA off the absorbed Hydro Pump"</code> fails at <code>sa 1</code>, untouched HP — the	CLOSED 2026-08-11 — THE ENGINE WAS NEVER BROKEN AND THIS ROW MISDIAGNOSED IT. One defect explains all four: <code>_chooseAction</code> 's last fallback clicked Struggle past a legal button (the #152 block, whose own comment stated the	

#	item	note
	immunity half works (HP untouched) and the Special Attack half does not. The neighbouring absorb assertions all pass, including <i>"the absorbed hit HEALS the absorber 1/4"</i> , so the absorb family's healing branch is wired and its boosting branch is not. Directly relevant to redirection reads: a Storm Drain body the bot does not know gains SpA is a body it will keep attacking into	bug in place — <i>"a body with legal status moves left is not on Struggle"</i> — and filed it as the lesser evil). Struggle is a 50 BP attack with 1/4-max recoil, so every fixture body holding a single Protect or Roost was hitting the thing under test and paying for it. Heat Wave NEVER touched the partner: the trace reads a <code>move</code> line from <code>p2b: whimsicott</code> clicking <code>struggle</code> at <code>p1b: kingambit</code> , twice, for 21 of the 21 HP. The Sash DID save at 1 and Struggle recoil then killed the survivor. Fallback fixed, <code>test-tag-wire</code> 10 FAIL -> 0 FAIL over 104 checks NOTATION REPAIRED 2026-09-04 BY MEASURE — the same cut cell as #167 and #172. This row already read closed through its title, so NO VERDICT MOVED; only the notation changed.
#171	LEECH SEED LANDS ON A GRASS TYPE. <code>test-tag-wire</code> : <i>"a Grass-type victim is immune, from the move's own onTryImmunity"</i> fails, losing 25000. The immunity is written on the move itself in the authority, and the rest of the Leech Seed wire is proven in the same run — the drain, the fraction, the direction, and <i>"the seed alone kills a victim that was never hit by an attack"</i> . So the engine seeds Grass types, which no legal game allows, and every rollout in which it does is scoring a click that cannot happen	CLOSED 2026-08-11 — FIXTURE, NOT ENGINE, AND THE ROW SAID TO CHECK THAT FIRST. The fixture predates PP being a resource (#144): the battle runs to the 20-turn cap, the moves empty, <code>mustStruggle</code> fires CORRECTLY, and the recoil contaminates the number the assertion reads. Measured before anything was edited — with PP restored the Aftermath attacker ends 173/173 and <code>_seededBy</code> on the Grass victim is false . The shared <code>one()</code> helper is why the neighbouring rows were red too
#172	HEAT WAVE HITS YOUR OWN PARTNER. <code>test-tag-wire</code> : <i>"Heat Wave leaves the partner untouched — the ally-safe family stays safe"</i> fails, while both membership assertions pass — <i>"spreadFoes (22) is the ally-safe family — Heat Wave in, Earthquake out"</i> and <i>"spreadAll (16) hits the partner"</i> . The artifact separates the two families correctly and the consumer is collapsing them. This is the one on the list with the largest effect on PLAY: the whole point of the ally-safe spread moves is that you can click them beside your own body, and an engine that charges you for it will refuse a line that is actually free	CLOSED 2026-08-11 — THE ENGINE WAS NEVER BROKEN AND THIS ROW MISDIAGNOSED IT. One defect explains all four: <code>_chooseAction</code> 's last fallback clicked Struggle past a legal button (the #152 block, whose own comment stated the bug in place — <i>"a body with legal status moves left is not on Struggle"</i> — and filed it as the lesser evil). Struggle is a 50 BP attack with 1/4-max recoil, so every fixture body holding a single Protect or Roost was hitting the thing under test and paying for it. Heat Wave NEVER touched the partner: the trace reads a <code>move</code> line from <code>p2b: whimsicott</code> clicking <code>struggle</code> at <code>p1b: kingambit</code> , twice, for 21 of the 21 HP. The Sash DID save at 1 and Struggle recoil then killed the survivor. Fallback fixed, <code>test-tag-wire</code> 10 FAIL -> 0 FAIL over 104 checks **NOTATION REPAIRED 2026-09-04 BY MEASURE — pipe characters inside inline code were cutting this status cell, so the register read the row OPEN against its own closure. Only the notation changed; no claim was altered or removed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: cheap check from committed artifacts only — the census carries <code>move/spreadAll</code> "Earthquake hits your own partner too" and <code>move/spreadFoes</code> "Rock Slide hits both foes and Stone Edge hits one" live; nothing was re-run here.**
#173	FAKE OUT IS STILL CLICKABLE ON TURN TWO. <code>test-tag-wire</code> : <i>"turn 2 the illegal Fake Out is refused at pick time (attack) — not a fake click"</i> fails. Turn 1 is proven legal in the line above it. The assertion's own wording is the requirement — the refusal must happen at PICK TIME, so the chooser never offers it, rather than the move being offered and then failing. An illegal action in the candidate set is worse than a wrong one: it wastes rollout budget on a branch that cannot exist, and Fake Out is one of the highest-usage leads in the format	CLOSED 2026-08-11 — REAL WIRE, and the cause is the rule this repo keeps relearning: <code>firstTurnOnlyRefused</code> was <code>id==='fakeout'</code> , justified by a comment READ OFF MAINLINE. <code>data/mods/champions/moves.ts:331</code> adds <code>onDisableMove</code> , so in Champions it is a MENU disable. New derived tag <code>firstTurnOnly</code> , 2 members printed before wiring — and First Impression is legal here and the name check never covered it. <code>firstTurnOnlyByNameOnly</code> now reads 0
#174	<code>test-assert-mode</code> IS RED ON PURPOSE AND THE SCENARIOS ARE THE DEFECT. All twelve rows pass — Good as Gold refuses Corrosive Gas / Dragon Cheer / Electrify / Encore / Howl / Toxic Thread, Levitate refuses Bulldoze / Earth Power / Earthquake / Headlong Rush / Mud-Slap / Scorching Sands — and they pass IDENTICALLY with both abilities deleted from the engine (<code>--break</code>). The test says so itself: <i>"ALL 12 ROWS AGREE — AND THAT CURRENTLY PROVES NOTHING."</i> The fix is scenarios in which the refusal has an observable consequence, not more rows. Left green it would have published 5,769 teams of false coverage	CLOSED 2026-08-11 — <code>--break</code> now makes it RED, all 3 rows FAIL , verified explicitly. Three real causes, none of them 'add more rows': (1) <code>game_differential.js</code> 's <code>scripted()</code> answered <code>pass</code> SILENTLY for a click not on Showdown's request, so six of the twelve moves were never clicked at all — now counted, exported and asserted; (2) the Levitate anchor patched <code>AIRBORNE_ABIL</code> , the GROUNDLED set, not the damage immunity; (3) Levitate has two independent gates (<code>absorbedBy / typeImmunity</code> and <code>isGrounded / AIRBORNE_ABIL</code>) so breaking either alone proved nothing — measured 0/85 in every single-gate arm. The demonstration is no longer behind a flag: every row plays both arms on every run , and a row whose mutant arm agrees is reported <code>HOLLOW</code> and fails. Flaked once in six runs (3 <code>HOLLOW</code> , five subsequent runs 3/3 <code>PASS</code>); cause not established, contention with a concurrent agent's writes the leading candidate — reported rather than claimed as a stable green
#175	TWENTY-THREE TAGS HAVE NO CONSUMER — WE DERIVED THE FACT AND NEVER TAUGHT THE ENGINE TO USE IT. <code>test-tag-consumed</code> names them: <code>stealsItem</code> , <code>announcesOnEntry</code> , <code>reflectsStatusToSource</code> , <code>refusesIndirectDamage</code> , <code>clearsScreensOnEntry</code> , <code>nameImplementedBySim</code> , <code>passesItemToAlly</code> , <code>multiHitAlwaysMax</code> , <code>ignoresRedirection</code> , <code>boostsAtHPThreshold</code> , <code>curesStatusResidual</code> , <code>healsFromOwnStatus</code> , <code>addsOwnSecondary</code> , <code>formeFollowsWeather</code> , <code>secondaryChanceMult</code> , <code>typeFollowsTerrain</code> , <code>removesOwnMoveFlag</code> , <code>inheritsAllyAbility</code> , <code>boostsNotVeryEffective</code> , <code>boostsAlliesWithAbility</code> , <code>allyBasePowerBoost</code> , <code>suppressesOwnItem</code> , <code>guaranteesNextMove</code> . The test also prints a longer DEAD list headed by <code>ignoresRedirection</code> (47 uses, Propeller Tail / Stalwart). EACH ONE IS A DECISION, not a batch: wire it, or delete the tag and say why. A derived tag nothing reads is indistinguishable from a tag that is wrong. Expect this count to MOVE when the tag split lands — re-measure before working it	open — engine DEFECT. 23 -> 22 on 2026-08-11: <code>ignoresRedirection</code> wired and closed, 21 rows untouched because EACH IS A DECISION and batching them is the failure mode. That one fix found a live bug of #181's exact shape one function over: <code>tracksTargetOf</code> looked for an ability-side <code>tracksTarget</code> tag THAT DOES NOT EXIST IN THE ARTIFACT, so <code>ab==='stalwart'</code> — <i>(the original expression is truncated at this point in the row, and the two pipe characters that ended it were being read as a column break; removing them is all this pass changed here)</i> . CLOSED 2026-08-11 — seven wired, one tossed, and the wiring found five artifact defects that review had missed. <code>guaranteesNextMove</code> (Lock-On), <code>typeFollowsTerrain</code> (Mimicry), <code>formeFollowsWeather</code> (Forecast), <code>passesItemToAlly</code> (Symbiosis), <code>inheritsAllyAbility</code> (Receiver), <code>announcesOnEntry</code> (Frisk), <code>boostsAlliesWithAbility</code> (Magnetic Flux) — each shown RED first, re-run before the next, gate checked between. THREE OF THE SEVEN RESOLVED TO <code>{kind:'pass'}</code> — a whole spent turn doing nothing. <code>allyBasePowerBoost</code> tossed under Will's rule: all three members carrier-less. The five defects were each a PARAMETER NOTHING HAD EVER READ: <code>revertsWithoutTerrain</code> : <code>false</code> against a handler that reverts; no <code>forme->type</code> map at all; <code>announcesOnEntry</code> . <code>reveals</code> being prose shared by two members that emit different events; <code>boostsAlliesWithAbility</code> . <code>boots</code> : <code>null</code> ; and a regex matching <code>function</code> against a method shorthand — caught by PRINTING the rows before anything consumed them. THE CARRIER-LESS TOSS COST FIVE PROBES, NOT TWO (<code>tag_dex</code> 294 rows -> 201, 263 tags -> 258): the two known probe deaths plus eight more that staged a carrier-less ability as their SECOND ARM and went MISSING — five repaired by swapping to a legal member of the same tag (Storm Drain->Lightning Rod, Transistor->Water Bubble, Air Lock->Cloud Nine, Galvanize->Pixilate, Ice Scales->Multiscale), two lost an arm, one removed. Collateral caught: <code>test-damage-stages</code> read 64 disagreements, ALL of them the TEST staging a carrier-less ability, now 1696/1696 exact. The roster denominator did not move — <code>tests/roster.js</code> derives carrier legality from the dex rather than from <code>data/tags.json</code> , so the two instruments already agreed. Census 470 -> 474:

#	item	note
		arrived 7, broke 0, REMOVED-AS-UNCARRIABLE 3, net +4, missing o throughout NOTATION REPAIRED 2026-09-04 BY MEASURE, AND THIS ROW WAS READING CLOSED AGAINST ITS OWN CELL. Two pipe characters inside an inline code span split this cell in two, so the register read the appended CLOSED-2026-08-11 narrative as the whole verdict. The authored status is the open – engine DEFECT this cell begins with, and the row now reads OPEN and asserting breakage again. No claim was added or removed and no engine byte moved; the register's open-and-asserting count rises by one because of this row alone.
#176	test-effective-identity 's BASELINE IS STALE AND THE RATCHET IS UNREADABLE. It counts raw reads of a field that changes on mega evolution — declared ability versus the one on the field — against a baseline dated 2026-08-02 holding 234. It now counts 1,195, and thirty files are already declared as legitimate with written reasons. Most of the gap is files written AFTER the baseline, which count as new by default. It went red in part because a measuring file was written while an engine agent held the tree, which CLAUDE.md already names as a routing failure rather than a code one. DO NOT RESTAMP IT — a re-stamp makes the number look fine without anyone having read a line, which is the exact move this project has been burned by. Walk the undeclared reads, route the ones that want the live value and declare the ones that legitimately want the declaration, then re-baseline with the walk cited A DECISION IS NOW ON THE TABLE FOR WILL — SEE #391 , which walks all 130 new matches (zero stale-identity reads), records that 45.2% of what this ratchet counts is prose or assignments, and offers three named options. This row's own diagnosis — <i>"most of the gap is files written AFTER the baseline"</i> — was re-measured 2026-08-23 and is 62% of the red : 24 of the 30 over-baseline files did not exist on 2026-08-11. CLOSED 2026-08-23 — AND THE ROW'S OWN INSTRUCTION WAS HONOURED: IT WAS NOT RESTAMPED. <i>"DO NOT RESTAMP IT — a re-stamp makes the number look fine without anyone having read a line"</i> is exactly what did NOT happen. The unreadable ratchet was RETIRED and replaced by a runtime assertion (#411): a booby-trapped board on which the declared ability is wrong for all four actives, driven through the live decision path, with every read recorded and exactly one call site permitted. The stale 2026-08-02 baseline is kept under <code>last_count_baseline</code> and cannot be written over. THE WALK THIS ROW ASKED FOR WAS DONE FIRST AND IS WHY THE RETIREMENT IS SAFE RATHER THAN CONVENIENT — all 130 new matches were read (#391): 44 prose, 12 writes, 122 structurally impossible <code>.baseStats</code> reads, and zero stale-identity reads . What the retired count could never have seen is caught by name now: three read shapes the old header conceded were undetectable by any text scan are planted and caught. WHAT THIS CLOSE DOES NOT CLAIM: the new gate is EXECUTION coverage on the driven path only, and it prints that limit on every run. The remainder is #183. VERIFIED BY: <code>node tests/test-effective-identity.js</code>	closed 2026-08-23 — the ratchet was retired and replaced, never restamped; the remainder of the unreviewed adoption is #183
#177	TWENTY ARTIFACTS WERE NEITHER CLEARED NOR WITHHELD, AND THE SENTENCE EXPLAINING WHY HAD BEEN FALSE FOR TWO DAYS — DONE (discovery), OPEN (re-run). <code>engine/quarantine.js</code> printed <i>"provenance.js finds a writer only in engine/ and build/"</i> beside twenty files. <code>GEN_DIRS</code> has held <code>tests/</code> since 2026-08-09 (#105) and the stated reason applied to none of them. It survived because the set was computed by SUBTRACTION — list <code>data/</code> , remove what has a row — and a subtraction returns the same list whatever the cause, so nothing could contradict the caption. An unknown is a ROW now: <code>provenance.js</code> emits <code>unknown: true</code> , by: <code>null</code> and a DERIVED reason for every file in <code>data/</code> , <code>--graph --json</code> carries them, and <code>quarantine.js</code> reads that set instead of growing its own. One of the twenty was a real miss and it was the RELEASE POINTER — <code>data/engine-release.json</code> , the file every frozen measurement resolves through, written via a same-file helper and one hop of object-property indirection. 181 -> 182 rows, path variable 64 -> 65, no existing attribution moved; 19 explicit unknowns from 5 derived causes. Shown RED three times: against the old code (3 of 5 assertions fail), with the scratch-tree guard disabled (the pointer misattributes to <code>tests/test-miltank-release.js</code> , which writes that basename into a temp dir as a fixture), and with <code>provenance.js</code> blinded to one file (<code>quarantine.js</code> still names it, with NO REASON RECORDED). THE FIX WOULD HAVE INTRODUCED THE SAME CLASS OF FAILURE: unknown rows fed into the classification loop come out <code>quarantined: false</code> because no clause fires on a null writer, and <code>web/build-quarantine.js</code> asks only whether a row EXISTS — twenty artifacts would have moved from <code>unclassified</code> to <code>clear</code> on the site, in the permissive direction, through a change to the one thing that exists to refuse to default. Split out before classification. STILL OPEN — NO STALENESS VERDICT WAS PUBLISHED: ENGINE was rewriting <code>tag_dex.js</code> , <code>tags.json</code> and <code>medicham2-browser.js</code> throughout and a release was cut mid-run, so every provenance verdict beside this must be re-run on a settled tree; <code>data/quarantine-stamp.json</code> is UNSAFE by construction (it digests both edited files); and <code>node engine/quarantine.js --check</code> was deliberately NOT run, because with the gate transiently	open — engine

#	item	note
	open it rewrites <code>citation_sites</code> to <code>[]</code> and a gate that closes again would then flag every existing citation as new	
#178	A GREEN CENSUS COUNT CANNOT DETECT AN OVER-MATCH, AND THE WORKING RULE WOULD HAVE PASSED ONE — 2026-08-11. While splitting the protection counter the first shield dispatch matched on <code>stallCounterChecks</code> , and Endure is a stallingMove that rolls the same die and blocks nothing . It became a shield and broke both Endure probes. <code>live read 444 before and 444 after</code> — two probes broke and two arrived, and the count is a NET. Only <code>missing moving 0 -> 2</code> caught it. The working practice "confirm <code>live went up</code> " is therefore not a check at all against this failure mode, and it is the practice most often used, because it is the number printed largest. Fixed with a sixth tag, <code>shieldsUser</code> , derived from <code>checkMoveBypassesProtect</code> on the volatile's own condition — 5 members, Endure correctly absent. The row is the RULE CHANGE, not the bug: a census movement must be reported as arrived/broke/net, never as a net alone, and <code>missing</code> must be read beside <code>live</code> every time	open — engine
#179	RECYCLE CARRIES NO MECHANIC TAG AND RESOLVES TO A SPENT TURN. Found while closing <code>#71</code> 's seven-move signature 2026-08-11. Pain Split reached <code>sharehp</code> and Copycat reached <code>callmove</code> ; Recycle (3 uses) reaches <code>{kind:'pass'}</code> , which is the terminal do-nothing branch. Low usage, and registered anyway because a move that provably does nothing is a WEIGHTED PREFERENCE rather than a rule — the same shape as <code>#44</code> — and because the seven-move signature cannot be called closed while a member of it is unmodelled	CLOSED 2026-08-11 — REAL WIRE. New tag <code>restoresOwnLastItem</code> (1 member, all three handler clauses derived) and a new action kind <code>itemback</code> . No new state was needed; <code>_lastItem</code> already existed
#180	SUCKER PUNCH HAS A SECOND REFUSAL AND WE MODEL ONE. <code>data/moves.ts:18400</code> — the move also fails when the target carries <code>volatiles['mustrecharge']</code> , i.e. the turn after a Hyper Beam class move. <code>#60</code> closed the priority-versus-attacking half and this is the other clause of the same handler, named here rather than absorbed into a row that is now marked closed	CLOSED 2026-08-11 — REAL WIRE. The third clause of one <code>if at data/moves.ts:18400</code> . Derived <code>refusesRechargingTarget</code> discriminates correctly: <code>suckerpunch</code> yes, <code>upperhand</code> no. Shown red by disabling the clause — both arms collapsed to 86, which is <code>#178</code> 's signature
#181	<code>playerAction</code> STILL CARRIES A NAME FALLBACK FOR ALLY SWITCH, BESIDE THE DERIVED TEST. <code>`</code>	<code>id === 'allyswitch'</code> <code>sits</code> uncounted, so if the tag ever the derived path failed — a <code>fr</code> is the mechanism <code>#127</code> indic
#182	THE GATE READ 'NO OPEN ENGINE DEFECT' WITH ELEVEN OF THEM IN THE REGISTER, BECAUSE THE CLAUSE MATCHED VOCABULARY — FIXED 2026-08-11. <code>roadmapRowSaysBroken</code> tested a fixed phrase list (<code>NEVER FIRED</code> , <code>DOES NOT WORK</code> , <code>IS ABSENT</code> , <code>does not exist</code> , ...). The ten defects registered tonight off <code>test-tag-wire</code> 's own assertions say <i>does not save, lands on a Grass type, hits your own partner, punishes nobody, charges a survivor nothing, still clickable on turn two</i> — every one a mechanic behaving wrongly in a real game, and the clause matched ZERO of them . The gate printed <code>clean: the roadmap registers no open row describing a live engine defect with Focus Sash failing to save at 1 HP two screens above it in the same file. This is #148's own lesson unspent on half the problem: "a defect register whose enforcement depends on word choice is a structural weakness"</code> was cashed in for <code>roadmapRowIsClosed</code> (which reads the status cell) and left in place for the BROKEN half. The claim is DECLARED now — the token <code>DEFECT</code> in the status cell — and the prose scan is kept beside it so no existing row needed editing. Shown inert first (clause added, nothing marked, gate unchanged at OPEN 6/6), then eleven rows marked and the gate went OPEN -> CLOSED, 1 of 6, naming exactly those eleven. THE CONSEQUENCE IS THE POINT: the gate opening on 2026-08-11 was in part an artifact of defects that had no register row. <code>test-tag-wire</code> was red with those same ten failures BEFORE that gate opened	DONE 2026-08-11
#183	THE EFFECTIVE-IDENTITY RATCHET WAS RESTAMPED WHOLESALE, ON WILL'S INSTRUCTION, AND 964 READS WENT IN UNREVIEWED — 2026-08-11. Will, asked whether to walk the reads or reset the reference point: <i>"restamp i guess"</i> . Recorded as a row rather than done quietly, because this is the exact move the repo has been burned by — <i>"the gate went green after somebody re-stamped it"</i> — and the difference between a legitimate re-baseline and laundering is whether anybody can later see what was swallowed. Baseline 2026-08-02, count 234 -> 2026-08-11, count 1,198 : 964 reads adopted across 46 files. Largest: <code>tests/roster.js</code> 247 (new), <code>tests/test-mechanics.js</code> 226 (new), <code>engine/medicham2-browser.js</code> 59 -> 215, <code>tests/probe_red_demo.js</code> 123 (new). THE SIMULATOR ENTRY IS THE ONE THAT MATTERS — the other large entries are harnesses that legitimately construct bodies, but 156 new raw reads inside the engine itself are exactly where a declared-versus-effective ability confusion would do damage, and NOBODY READ THEM. The restamp was timed AFTER the tree settled (doing it mid-run is what put this red in the first place, per CLAUDE.md on a measuring agent beside a writing one) and the adoption was diffed rather than assumed. What is owed: a walk of the 156 simulator reads specifically, routing the ones that want the live value and declaring the ones that legitimately want the declaration. The other 808 can stay adopted RE-SCOPED 2026-08-23 — THE	open — MEASURE, re-scoped 2026-08-24. The 964-read adoption is history (the count is retired); what remains owed is the coverage question the runtime gate declares about itself. INSTRUMENT OWED

#	item	note
	COUNT THIS ROW IS DENOMINATED IN NO LONGER EXISTS, SO THE DEBT HAD TO BE RESTATED OR IT WOULD HAVE GONE QUIET. The raw-count ratchet was RETIRED (#411) and the 2026-08-11 baseline of 1,198 is kept under <code>last_count_baseline</code> as history. "964 reads adopted" is therefore no longer a live quantity, and this row must not be read as though 808 of them are still sitting in a floor somewhere. WHAT IS STILL OWED IS NARROWER AND IS STATED IN THE NEW GATE'S OWN TERMS: the replacement asserts a RUNTIME property and is EXECUTION coverage only — it is blind to a sheet value copied off a body before a <code>Board</code> ever existed, and it says so on every run. The 156 simulator reads this row named are covered exactly to the extent that the driven path reaches them, and nothing measures which of them it does not reach. WHAT WOULD DECIDE IT: a coverage read of which of those 156 sites the gate's driven board actually executes, so the uncovered remainder is a list rather than a worry. Account: <code>docs/_reports/2026-08-23-identity-gate-permanent.md</code>. INSTRUMENT OWED	
#184	test-tag-consumed IS RED AND IT IS THE INSTRUMENT, NOT THE ENGINE — 2026-08-11. Its ratchet diffs a NAME LIST, so ten tags that ARRIVED without a consumer read as ten that LOST one. Verified against <code>23661cc:data/tags.json</code>, where all ten are absent. The count is therefore not a regression and never was. It is in NEITHER <code>run-all.js</code> NOR the pre-commit hook, which is why a broken ratchet could sit red across a whole sprint being quoted as an engine figure — including by me, to Will, twice tonight. Fix the diff to compare CONSUMPTION rather than membership, then register it as a gate	CLOSED 2026-08-11 — and it changed the DIAGNOSIS, not the arithmetic. The ratchet diffed a NAME LIST and printed <i>"8 tag(s) newly have NO consumer"</i>, which is the wording for an engine regression. A tag missing from the old dead list is missing for two OPPOSITE reasons, and the artifact had grown 194 -> 263 tags, so all eight were ARRIVALS, not regressions. <code>data/tag-consumption.json</code> now carries <code>by_tag</code> with a status per tag — REGRESSED / ARRIVED / STILL DEAD / UNCLASSIFIABLE. First run after the fix: o REGRESSED, 8 UNCLASSIFIABLE, and it says out loud that a pre- <code>by_tag</code> baseline cannot separate the two, for one run only. A SECOND BUG WENT WITH IT: the old write gate (<code>dead.length <= prevDead.length</code>) froze the baseline the instant DEAD grew, so it re-reported the same eight rows forever — which is exactly how this instrument sat red across a whole sprint being quoted as an engine figure, including by me to Will twice. Now 7 passed, o failed. STILL NOT REGISTERED in <code>run-all.js</code> or the pre-commit hook — that remains open and is the reason it could rot unnoticed
#185	SINGLE-TARGET SCRIPTED CLICKS DO NOT REACH EITHER FOE — the open half of #174. In <code>staged_board / game_differential</code>, spread clicks diverge correctly under a mutant engine but <code>normal</code>-target clicks are IDENTICAL at t:0 AND t:1, with <code>moveNotOnRequest</code> at zero — so the click is accepted and lands on nobody. The ENGINE is fine: the same Charm through <code>battleTurn</code> reads -2 with the ability off. It is the scripted path. Until this is fixed, any single-target assertion in that harness is vacuous in the same way #174's twelve rows were, and #174 was only caught because somebody ran it with the mechanic deleted	CLOSED 2026-08-11 — MISDIAGNOSED BY ME, AND THE ENGINE WAS NEVER TOUCHED. The row claimed single-target scripted clicks reach neither foe, citing <code>normal</code>-target clicks reading IDENTICAL at t:0 and t:1 under a mutant engine. Staged against the same mutant <code>test-assert-mode.js</code> builds: <code>charm</code> (<code>normal</code>, t:0) clean IDENTICAL / broken DIFFERS (<code>boosts.atk -2</code> vs 0); <code>earthpower</code> (<code>normal</code>, t:0) clean IDENTICAL / broken DIFFERS (<code>party.hp 93</code> vs 150); <code>earthquake</code> (<code>allAdjacent</code>) the same. <code>moveNotOnRequest</code> o throughout and <code>aimCounters()</code> reads <code>foe: 3</code> on a <code>normal</code> t:0 click. The t:1 rows are identical because t:1 IS THE PARTNER — Corviknight beside the Gholdengo, Snorlax beside the Chimecho — and the partner never carried the ability the mutant deletes. That is the harness being CORRECT. The row's t:0 claim does not reproduce. Filed from a report I did not stage myself
#186	LEVITATE IS GATED TWICE AND THE TWO GATES AGREE ONLY BY LUCK — 2026-08-11. <code>absorbedBy / typeImmunity</code> and <code>isGrounded / AIRBORNE_ABIL</code> are independent implementations of one fact, which is FACTS-ARE-GLOBAL broken inside the engine. Found because breaking either one alone left the assert-mode probe green at o/85 — the duplication is what made the test unfalsifiable. They agree today. One implementation, everyone calls it	CLOSED 2026-08-11 — THE SECOND GATE WAS REAL AND WILL'S BOARD FOUND IT IN ONE RUN. <code>absorbedBy()</code> zeroed a Ground hit off a STATIC <code>typeImmunity</code> tag while <code>typeEffAgainst</code> consulted <code>isGrounded()</code> — two readers of one fact, one clause apart. Every grounding probe in the census stages CORVIKNIGHT, a FLYING body, so Gravity, Smack Down, Ingrain and Iron Ball had all been proven on a clause the ability never touches, and did NOTHING AT ALL to a Levitate body. Measured against the authority: Showdown's Hydreigon took the click under Gravity (turn 4) and Smack Down (turn 11); ours took neither. The fix defers only the airborne abilities — <code>eartheter</code> carries the same tag and is Ground-immune standing on the floor, so it is the over-match control and must stay o with Gravity up. New census probe <code>typeImmunity</code>, five arms, shown RED first. This is the row that recorded the two gates agreeing 'only by luck' and measured o/85 with each broken singly — the luck had run out and nothing could see it
#187	FREEZE-DRY ROLLS A 10% FREEZE THAT THIS FORMAT DELETED — 1,656 corpus uses. <code>data/move-effects.js</code> carries MAINLINE's secondary; <code>pokemon-showdown/data/mods/champions/moves.ts:394-399</code> sets secondary: undefined, // no inherit. No tag covers a REMOVED secondary, so the rulebook fallback is unchallenged and the engine freezes people with a move that cannot freeze in Champions. Found by <code>million_run.js --declaration-only</code> — the ONE real disagreement of 118 pairs. The pair is refused for scoring so the published run is unaffected. This is the shape the whole mod-override rule exists for: reading <code>/data/moves.ts</code> is reading mainline, and here the mainline value reached an artifact	CLOSED 2026-08-11 — and it is a REPRESENTATION fix, not a value fix, which is why it closes a class rather than a move. New derived tag <code>formatSecondaryCount</code> over all 500 legal moves, param <code>{count}</code>. <code>count: 0</code> is a POSITIVE statement that the format removed the secondaries; absence of the row now means "no format reading at all" and is counted in <code>MEDFAILS.secondaryProfileUnknown</code> (reads o). Before that distinction existed, a REMOVED secondary and an UNDERIVED one were the same thing and the rulebook fallback fired on both. Printed before wiring: 500 tagged, 380 carry <code>count: 0</code>, of which exactly ONE has a generic secondary to refuse — <code>freezedry</code> — and zero go the other way, so nothing else in the format shares its shape. Probe shown RED first: Freeze-Dry 45/400 freezes against control Ice Beam 45/400, identical; after, Freeze-Dry o and Ice Beam 45/400 unchanged, refusal fired 600 times. THE EXPOSURE NUMBER IS THE POINT: chances coming from the rulebook alone with nothing format-derived guarding them went 13 -> o, and declaration disagreements 1 -> o. A second wire landed in the same spirit: <code>_fmtChance</code> never read <code>statChange.user[]</code>, so twelve pairs rolled gen-9's number unguarded
#188	THE --strict CONFORMANCE RATCHET HAS NO REGRESSION-vs-DISCOVERY SPLIT, SO A CHECKER THAT GETS BETTER BREAKS THE GATE. <code>node engine/conformance.js --strict</code> is RED: 19 new findings at HEAD before anything was touched (verified by running <code>git show HEAD:engine/conformance.js</code>), 27 after S13 was improved to ask <code>provenance.js --graph --json</code> instead of hand-rolling <code>allSrc.includes(file)</code>. The +8 are TRUE POSITIVES the substring search concealed, and rewriting the baseline would have buried the other 19. <code>data/provenance-stamp.json</code> already has exactly the split this needs — copy it. A near-miss is recorded with it: the first version of the S13 change put provenance's reason into the	open — measure

#	item	note	
	finding's <code>what</code> field, which retired 20 baselined findings and re-raised them as new — the exact laundering the ratchet exists to stop. Caught, moved to <code>detail</code>		
#189	THE RUNNER'S LOUDEST NUMBER WAS ITS OWN BUG, AND THAT IS THE ROW. The OHKO arm read 34.40% against a declared 30 on 7,005 trials, z = +8.0 — Bonferroni-surviving, and a clean FALSE ACCUSATION against the engine. A raw probe over 25,000 games counting only `	-miss	and
#190	FIVE ABILITIES RECORD CORPUS USES THAT THE CARRIER DERIVATION SAYS ARE IMPOSSIBLE — AND THE CAUSE IS ALREADY KNOWN: THE CORPUS IS CONTAMINATED. <code>serenegrace 5, stormdrain 3, psychicsurge 2, tintedlens 2, steelyspirit 1</code> , all sitting in the 93 abilities derived as having NO legal carrier in Reg M-B. **Will, 2026-08-11: "i think the corpus is contaminated and people uploaded games with custom rules, weve been over this bro in the last session". The row was filed as an open either/or — contaminated corpus OR incomplete legality read — and that framing was the defect: it had been established in a prior session and was recorded NOWHERE a later session would find it. Not in <code>docs/</code>, not in this register, not in memory. So it was re-derived from scratch, at the owner's expense, and he had to say so. The answer: a replay carrying the format name is NOT proof the game was played under the regulation's rules. The legality derivation is fine; the corpus is the suspect. CONSEQUENCE, and it is wider than these five: every usage-weighted figure inherits this, INCLUDING the 99%-of-usage coverage bar the MEDICHAM gate is defined against. ** The fix is a corpus filter that validates each stored game against the regulation — not a loosening of the legality read, and not a per-ability exception. Until it exists, a usage count is a lower bound on legal usage and an unknown mixture otherwise	open — ops	
#191	OUR MOVE RULEBOOK IS GENERATED FROM MAINLINE, NOT FROM THE CHAMPIONS MOD — and 22 values differ. THE 22 ARE MOOT; THE GENERATOR IS NOT. <code>data/move-effects.js</code> is generated from <code>CHOMP/data/move-effects.json</code> , built by <code>CHOMP/build/build_move_effects.js</code> , which contains ZERO references to <code>forFormat</code> , <code>mods/champions</code> or the Champions dex. Its own header claims the values come <i>"straight from Showdown's moves.json - the same server data that runs the format"</i> — that sentence is false: <code>moves.json</code> is mainline, the format runs the mod. <code>tests/test-mod-conformance.js</code> (new) asks the two dexes WHICH FIELDS DIFFER rather than comparing a chosen list, because the hand-written first version compared six fields, found 24, and missed 42 PP differences, 6 flag changes and Salt Cure's HALVED residual — the exact failure <code>mod_audit.js</code> 's own header warns about, read on the same night it was repeated. The 22 were then FALSIFIED rather than assumed live: driving the rulebook's <code>bp</code> to 999 moves a real turn by nothing (56 -> 56) while <code>engine-data.js</code> moves it (56 -> 647); forcing its <code>type</code> moves nothing (155 -> 155) while forcing engine-data's does (155 -> 78). Base power, type and accuracy all come from format-derived surfaces; the only field the engine reads out of that file is the secondary, and after #187 none is unguarded. So this row is about the GENERATOR, not the values. Repoint it at the mod. Cross-checked independently against Bulbapedia's Champions section, which confirms Dire Claw 50% -> 30% + slicing, and Freeze-Dry losing freeze	open — engine	
#192	THE SPEED CLAIM THE WHOLE ENGINE PROGRAMME RESTS ON, MEASURED FOR THE FIRST TIME: C/B = 7.82x. <code>engine/speed_vs_pokeenv.js</code> had NEVER been executed. 200 games, same teams, same random policy, quiet machine: Showdown in-process 28.7 games/sec; MEDICHAM 224.5 games/sec.	open — measure	

#	item	note
	ADR-003's feared null result — <i>"if C/B is near 1 the programme's justification is transport avoidance, which driving BattleStream in-process already gave us for free"</i> — did not happen. Three defects were found BY RUNNING IT, each invisible to inspection: (1) it loaded the engine wrong, copying <code>game_differential.js:147</code> and not <code>:128</code>, so the data tables were never loaded and the first call died on <code>MC is not defined</code>; (2) it handed Showdown-shaped sets to a MEDICHAM-shaped builder and got 0 usable bodies from 6 — fixed with the repo's existing <code>buildPair</code>, as its own header instructed, rather than a third implementation; (3) an uncaught throw in one arm destroyed the other arm's measurement, because the artifact is written after both — arm B, the honest baseline, was computed and discarded every time. Arms are isolated now. Taken under <code>-- anyway</code> with the gate closed and 5 uncommitted engine files, both stamped into the artifact. Arm A (poke-env) remains unwritten and Will has declared it out of scope — it measures a network round trip	
#193	A RED DEMONSTRATION WENT INTERMITTENTLY GREEN AND NOBODY CAN EXPLAIN IT. <code>staged_board.js</code> <code>--only nuzzle-paralysis</code> <code>--reds</code> printed <code>NOT CAUGHT</code> once at ~10:40 on 2026-08-11 and <code>CAUGHT AND LOCALISED</code> on four consecutive runs after, with no file edited between and the same census digest both times. A demonstration that a probe CAN fail is the only thing that makes it worth trusting; one that is intermittently green is worse than the run it appeared in, because it makes every future red look like noise. Not reproduced, not called fixed, deliberately not buried in a passing run	open — engine
#194	MILTANK AND MAG ARE BOTH OWNED BY WILL AND ARE BEING REWORKED BY HIM — DECLARED 2026-08-11. Will: <i>"im gonna rework miltank so dont spend too much time on it"</i>, then <i>"same with mag"</i>. This extends #26, which already put the MAG weights in his hands, to the search itself. CONSEQUENCES, so nobody spends a night on a number that is about to be replaced: do not refit the weights, do not restamp the feature fixture, and do not re-run the MILTANK timing — <code>data/miltank-timing-r6.json</code> is 7 days old and describes a superseded engine build (<code>medicham2-browser.js</code> <code>b1b3ea94d5c3</code>), and re-running it costs ~8 minutes of quiet machine to characterise something being rewritten. The invocation is recorded in <code>docs/SEARCH.md</code> if it is ever wanted: <pre>MILTANK_TIMING=... engine/mew.js --miltank --miltank-n 200 --miltank-preview-n 40 --seed 90001, then engine/miltank.js --reduce</pre> What is still worth knowing and is NOT his: #62's gate — MILTANK untimed against MILTANK on the clock — because <code>clock_observations</code> reads o in that artifact, so the clocked arm has never once been observed, and that is a fact about the harness rather than about any particular MILTANK	open — search
#195	STALL IS IN THE CLOSET — DECLARED 2026-08-11, TEMPORARY BY DECISION. Will: <i>"bro put stall in the quarantine closet aint nobody using dat"</i>. MEASURED BEFORE CLOSETING, not asserted: <code>stall</code> records ZERO uses in the corpus, and its only legal carrier in Reg M-B is Sableye. It sat COULD-NOT-STAGE with the reason <code>no handler</code> — the effect (always move last inside the priority bracket) lives in an engine table, so nothing in the ability text suggests a board, and the harness separately reports that a priority bracket is only observable when the ORDER DECIDES THE FINAL BOARD, which is the same wall that holds Extreme Speed, Ice Shard and Jet Punch. Closeted the same way ILLUSION was (#160): declared, counted, named, and reversible — not quietly dropped. It is excluded from the staging backlog and must NOT be counted as a coverage gap while it is in here. Re-admit when either its usage stops being zero or the priority-bracket harness learns to build a board where going last changes the outcome — that second one is shared with the three moves above, so this row reopens for free when they are solved. Zero usage is the reason and the reason is measured, so if the corpus ever shows a Sableye this row is wrong and should be reopened on that evidence alone. ADDENDUM 2026-09-04 BY MEASURE — THIS ROW IS THE ABILITY <code>stall</code>. IT IS NOT THE VOLATILE <code>stall</code>, AND THE TWO SHARE A NAME AND NOTHING ELSE. Nothing above is retracted or reworded: the ability Stall is legal here, its only legal carrier is Sableye, its corpus usage is zero, and it stays closeted on Will's ruling. The <code>stall</code> VOLATILE is a different entity that happens to be spelled the same — Showdown's <code>data/conditions.ts:439</code>, the 1/3, 1/9, 1/27 DENOMINATOR a stalling move rolls against — and it is applied by <code>addVolatile('stall')</code> from eight moves that are legal in this regulation: <code>protect</code>, <code>detect</code>, <code>endure</code>, <code>banefulbunker</code>, <code>spikyshield</code>, <code>kingsshield</code>, <code>quickguard</code>, <code>wideguard</code>. (<code>burningbulwark</code>, <code>obstruct</code>, <code>silktrap</code> and <code>maxguard</code> call it too and are <code>Past</code> here, which is why the collision has to be derived rather than eyeballed.) <code>Protect</code> alone is one of the most-clicked moves in this format, so the volatile is live on the ladder while the ability's usage is zero, and a closet decision taken on the ability's zero usage says nothing whatever about the volatile. DERIVED on the regulation rather than recalled: <code>engine/effect_kind.js</code>'s computed condition set holds 96 names in play, four of which also name a LEGAL ABILITY — <code>stall</code>, <code>primordialsea</code>,	closet — engine, the ABILITY only; the <code>stall</code> VOLATILE counter is #463 and is NOT closeted (scope stated by MEASURE 2026-09-04)

#	item	note
	desolateland , deltastream — and three of which name a MOVE this format does not contain (confusion , hail , healblock). This repository has already published three divergence causes as <code>cannot_occur_in_format: true</code> off the second of those two collision shapes, two of them board-material and neither ever filed (CHANGELOG 5.234.0); being labelled impossible is what kept them out of every work queue, and a closet row over the wrong entity is the same failure with a different door. THE COUNTER HAS ITS OWN ROW AND IT IS #463, closed 2026-08-26 at board-material 2 of 961. On release <code>8ad06030e129</code> (arm <code>middle</code> , <code>--end-state</code> , census <code>9446a684709d</code> , pool <code>0d103fb9fa87</code>) the <code>active[]</code>.stall leaf parts a board in 6 games of 961: four where an Encore'd Protect holds on one engine and not the other — <code>stall</code> reading 0 against 9 in games 3 and 33, 9 against 0 in game 43, and game 30 cascading into an 8-leaf board on which a mega Dragonite is alive at 7 HP for the authority and fainted for us; and two where it reads 0 against 3 with the protocol never parting at all, games 25 and 29. Whether that is a regression of #463 or a distinct Encore clause is NOT decided here and is ENGINE's to answer. What this addendum settles is only that #195 does not cover it and must never be read as excusing it. Account: <code>docs/_reports/2026-09-04-the-77-by-mechanism.md</code> sections M7 and M9.	
#196	THE RATE RUNNER MUST PLAY THE STAGED SCENARIOS, NOT RANDOM SELF-PLAY — WILL'S DESIGN, 2026-08-11. Will: <i>"the million game run will be all these staging scenarios right, just with normal accuracy, proc chances, crit chances, etc"</i>, then <i>"so those will have to be in the million game run"</i>. ONE SET OF FIXTURES AT TWO DICE SETTINGS: the roster plays a staged board with the dice PINNED and asks <i>does it do the right thing when it fires</i>; the rate runner plays THE SAME BOARD with the dice FREE and asks <i>does it fire at the right rate</i>. THE RUNNER ALREADY NAMES THIS AS ITS MISSING PIECE — <code>engine/million_run.js</code> marks the whole <code>proc</code> family observable: <code>false</code> because <i>"an ability/item proc needs the trigger to be REACHED ... the trace does not say whether the trigger was reached, only whether it fired, so the denominator would be invented. Needs a staged arm, not a bigger corpus."</i> THAT IS THE WHOLE ARGUMENT: on a staged board you KNOW the trigger was reached, so the denominator is REAL rather than invented, and more random games never fixes an invented denominator — it only makes it bigger. Today the runner is self-play over random teams (211 rows) and coverage is whatever the games happen to produce. WHAT THIS RECLASSIFIES, and it is most of the remaining backlog: the 17 pinned-dice rows (Static, Effect Spore, Flame Body, Poison Point, Poison Touch, Cute Charm, Shed Skin, Stench, Harvest, Healer, Cursed Body, Quick Draw, King's Rock, Quick Claw, Focus Band, Aspear Berry, Rawst Berry), Early Bird as a sleep-DURATION row, and the modifier abilities that need a real roll — Sniper and Merciless, because NO Sniper or Merciless carrier learns Flower Trick (verified: Beedrill, Ariados, Barbaracle, Toxapex; only Meowscarada learns it), plus Compound Eyes, No Guard and Tangled Feet as accuracy rows. None of these is unstageable. Every one is in the WRONG INSTRUMENT	CLOSED 2026-08-11 — THE STAGED ARM SHIPPED AND THE PROC SAMPLER IS RIGHT. <code>engine/million_run.js --staged -> data/million-run-staged.json</code> , release <code>eafb0abb9736</code> , free-running dice. 17 fixtures covering all 12 <code>proc</code> target rows plus 5 the target list does not carry. Pooled over every row that fires: 58,738 of 176,209 against 58,613.76 expected, z = +0.64, every row powered to ONE POINT. Denominator known BY CONSTRUCTION and proved so by a trigger-control arm that must record zero trials — <code>MILLIONRUN_SABOTAGE=staged-precondition</code> shows it red (13 of 17 controls went 0 -> 60 trials, refused). WILL'S STOPPING RULE, and it answers the question nobody could: 241,525 staged battles / 63 SECONDS for 1-point resolution; 78,494 / 21 s for 2 points; 19,758 / 6 s for 5. A staged battle is one constructed four-body board played for ONE TURN at 0.26 ms — not comparable to a self-play game, which is why 'a million games' was always the wrong unit. THE RULE EARNED ITSELF IMMEDIATELY: at 2 points Effect Spore read 31.90% and diverged at 95%, with a split-half spread of 1.97 against an effect of 1.90 — not an effect (LESSONS §9) — and at the powered n it reads 30.34%. A row stopped early is not a smaller answer, it is a different one. AND THE INSTRUMENT'S OWN THREE BUGS WERE FOUND BEFORE PUBLICATION: the first version matched the tag's status word against the engine's and read Flame Body 0/40 and Poison Point 0/40 — a headline that would have been filed against ENGINE — because the tag speaks <code>burn / poison / sleep</code> and the engine speaks <code>brn / psn / slp</code> , and Static passed by pure accident on three shared letters; the filler pool was <code>ATTACKERS.slice()</code> over a three-body pool, so six fixtures reported zero trials; and one refusal rate conflated by-design refusals with construction failures, silently dropping Harvest and both berries
#197	THREE ABILITIES DO NOTHING AT ALL, PROVEN BY CONSTRUCTED TRIALS RATHER THAN INFERRED FROM CODE — 2026-08-11. The staged arm built boards that GUARANTEE each trigger is reached, then counted. Cute Charm: 0 fires in 16,575 trials, 95% upper bound 0.023% against a declared 30% — its tag <code>inflictsVolatile {attract}</code> has no consumer, and the engine's own comment says so. Stench: 0 in 7,248, upper bound 0.053% against 10% — <code>addsOwnSecondary</code> appears ZERO times in <code>medicham2-browser.js</code> , verified by grep. Quick Draw: 0 in 16,575, upper bound 0.023% against 30% — <code>fractionalPriority</code> is read at exactly ONE site (<code>medicham2-browser.js:10259</code>) and off <code>it.mon.item</code> only, so the ITEM path fires correctly at 20.19% (Quick Claw) while the ABILITY path cannot fire at all. Focus Band is a fourth, tallied and deliberately NOT scored: its frozen tag carries only flingable , so there is nothing to roll. These are not sampling errors and no amount of extra games would have distinguished them from a cold sampler — pooled WITH them the arm reads $z = -49.8$, which is three abilities doing nothing rather than a broken die, and that is exactly why they are split out. ONLY A STAGED BOARD COULD SAY WHICH: random self-play cannot tell 'never triggered' from 'triggered and refused'	CLOSED 2026-08-11 — all three wired and proven by census probes, each shown RED first. <code>ability/fractionalPriority</code> (Quick Draw): MISSING at <code>draw</code> wins the roll 79 -> LIVE at 0. One reader now asks ITEM and ABILITY rather than a second ability-shaped branch — the fact is <code>fractionalPriority</code> and it had one carrier too few. <code>ability/addsOwnSecondary</code> (Stench): wired as an ADDITION to <code>move.secondaries</code> , so Shield Dust filters it — arms 4 and 5 of the probe are exactly the arms a bolt-on implementation fails. <code>ability/punishesAttacker</code> (Cute Charm): needed a whole <code>attract</code> volatile plus GENDER, which this engine has never had. BUT CUTE CHARM STILL CANNOT FIRE IN ANY EXISTING FIXTURE, AND THAT IS CORRECT: every harness in the repo declares <code>gender: 'N'</code> to Showdown (<code>game_differential.js:3762</code> says so in words) and Showdown's <code>attract</code> refuses genderless, so the engine now refuses in the same place. The staged rate arm's declared 30% is unreachable on its own board until the fixtures carry genders — registered separately. Census 476 -> 480, arrived 4, broke 0, missing 0 throughout. Zero roster rows moved and that is expected: all three read COULD-NOT-STAGE because their effect is a sub-100% chance and the driver's pin sends every such roll one way. The roster is structurally blind to them; the CENSUS is the instrument
#198	EXPORT CODE_OF_STATUS FROM engine/medicham2-browser.js — ONE LINE, AND THE ALTERNATIVE IS A FACTS-ARE-GLOBAL BREACH. The staged arm cannot score Effect Spore's 11/10/9 ladder band by band, because the tag artifact speaks Showdown's status vocabulary (<code>burn</code> , <code>poison</code> , <code>sleep</code>) and the engine speaks its own (<code>brn</code> , <code>psn</code> , <code>slp</code>). Writing a second copy of that map in the instrument is precisely the duplication CLAUDE.md forbids — and the first version of the staged arm DID hand-match the words, read Flame Body 0/40 and Poison Point 0/40, and would have filed a false headline against ENGINE. Printed side by side today: declared 11/10/9, observed <code>slp 1,823 / par 1,678 / psn 1,528</code> of 16,575	CLOSED 2026-08-11 — <code>CODE_OF_STATUS</code> is exported from <code>engine/medicham2-browser.js</code> , verified loadable. MEASURE asked for it by name so the staged arm can score Effect Spore's 11/10/9 ladder band by band without keeping a second copy of the status-vocabulary map

#	item	note
#199	<code>million_targets.js</code> 's <code>PROC_WHAT</code> IS A HAND-TYPED LIST OF TWELVE AND THE PROC FAMILY HAS SEVENTEEN MEMBERS . Static, Stench, Shed Skin, Rawst Berry and Aspear Berry have no target row, so five staged rates rest on ONE declaration surface with no independent corroboration — the frozen <code>data/tags.json</code> alone, with nothing derived from the Showdown handler to cross-check them. The other twelve are corroborated by two surfaces and agree 10-for-10 with zero disagreements, which is the standard the five cannot meet. Derive <code>PROC_WHAT</code> from the tag artifact instead of typing it — a hand-maintained list of twelve went stale exactly the way the ban list of four did, and this repo has a rule about that	open — measure
#200	THE PRIVATE SKY HAD THREE READERS AND ONLY TWO WERE RIGHT — CLOSED 2026-08-11 . <code>chargeSkip</code> asked <code>field.weather</code> where every other Mega Sol path asks <code>effWeatherOf</code> , so a Meganium-Mega's Solar Beam spent a charge turn Showdown does not spend — the target sat at 53 HP upstream on the click turn against 122 here. WIRE 99 did the DAMAGE path, WIRE 126 the TYPE path, and the CHARGE path was never asked — three readers of one fact, wired one at a time across three separate passes, which is the FACTS-ARE-GLOBAL failure arriving in instalments. Found by Will's board: <i>"have it use solar beam against mega swampert and see if it fires in one turn"</i> . New census probe <code>chargeSkippedByWeather</code> with a real public sun as the positive control	closed — engine
#201	A PROBE THAT PROVED NOTHING, CAUGHT AND NAMED RATHER THAN COUNTED — 2026-08-11 . The Shield Dust board's SECOND arm measured nothing in either arm , and the row still read FIRED-AND-BOARDS-MATCH off the first. Cause: the cheapest 100%-chance stat-drop move in this format is Mud-Slap, which is GROUND, and Vivillon is Bug/FLYING — immune, so neither arm could move. The drop click is now chosen per carrier against the type chart. A second, smaller correction is recorded because saying it matters: <code>learnsMove</code> also offered Blastoise/Mud-Slap, which the validator refuses — that was NOT the cause , and <code>CAST</code> 's own header states staged moves are not learnset-checked; the legality check is kept as conservatism and described as such rather than credited with the fix. This is the exact failure assert-mode had at twelve rows and the misdiagnosed seven had tonight: an arm that cannot move looks identical to an arm that agrees	closed — engine
#202	THE recycle DEFERRAL IS STALE — TAKE THE SHELF DOWN . It came off the owner's shelf reading FIRED-AND-BOARDS-MATCH on 2026-08-11 after <code>restoresOwnLastItem</code> and the <code>itemback</code> action kind landed. Moves stage 427-> 428. A deferral that now passes on its own is exactly what <code>staleShelf</code> exists to catch, and leaving it shelved understates coverage while pretending a decision is still live	CLOSED 2026-08-11 — measured rather than assumed: the moves stage's DEFERRED-BY-OWNER list holds no Recycle at all (it is Axe Kick, Copycat, Corrosive Gas, Electrify, Flying Press, Stuff Cheeks, Syrup Bomb, Teatime) and the stage stands at 428. The shelf this row asked to take down was already down
#203	THE FORME ASSERT-MODE IS BUILT AND IT FOUND A FOURTH ENGINE DEFECT ON ITS FIRST RUN — 2026-08-11 . <code>tests/test-forme-assert.js</code> + <code>data/forme-assert.json</code> , 3 of 3 rows agreeing, each of the three assertions shown RED by its own plant under <code>--reds</code> . Will asked for it by name. THE DEFECT IS THE THIRD ASSERTION'S WHOLE REASON FOR EXISTING: <code>formeSwap</code> rebased a MID-BATTLE forme change on UNNATURED anchors while <code>megaEvolveNow</code> had already been corrected to natured ones — one fact, two implementations, only one fixed. Measured: Adamant Aegislash -> Blade reads atk 167 / spa 153 against the authority's 176 / 144; Adamant Palafin -> Hero reads 189 / 118 against 198 / 113. It was invisible for its entire life because a NEUTRAL body is byte-identical either way and every fixture in this repo was Serious — which is precisely why the spec said the assertion had to be <i>the body's own SP spread</i> survived rather than <i>the stats are the new forme's</i> . Fixed at <code>medicham2-browser.js</code> <code>formeSwap: 150(_old.bs, null, mon._nature).test-nature-differential.js</code> PART 7 (75 un-natured mega swaps, pinned release <code>6b5447db1738</code>) still green. VERIFIED BY: <code>node tests/test-forme-assert.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	closed — engine
#204	THREE FORMES HAVE NO BODY TO BECOME, AND IT IS A DATA GAP RATHER THAN AN ENGINE ONE . <code>data/engine-data.js</code> carries no row for <code>mimikyu-busted</code> , <code>morpeko-hangry</code> or <code>castform-sunny/rainy/snowy</code> , so Disguise, Hunger Switch and Forecast cannot be asserted at all — there is no target forme to compare against. The new forme instrument PRINTS this on every run and never as a pass, which is the correct handling of an absence (an unprinted gap reads exactly like an agreeing one). <code>engine/engine-data.js</code> was NOT regenerated — a previous agent hit the same wall wiring <code>formeFollowsWeather</code> and it is explicitly not to be rebuilt. AND THE SPRINT NOTES' TABLE OF NINE FORME-CHANGERS IS WRONG FOR THIS REGULATION: derived from <code>Dex.forFormat</code> , exactly FIVE <code>formeChange()</code> -calling abilities have a legal carrier. Ice Face, Shields Down, Schooling, Power Construct, Zen Mode, Gulp Missile, Flower Gift and Tera Shift have NONE, and	rows changed 0
		rows added 5. **THE BUI REASON ITS OWN COMMENT merge_mega_into_engine.js (STATS only) while its <code>types**</code> match its base differ in TYPE and the ARE INERT TO THE SIMUL against a control <code>rele formeRenamedNoRow</code> → <code>first</code> , deliberately) a <code>formeSwap</code>) are byte-ic <code>LINE**</code> was added becau are blind to it; shown A1 for <code>Forecast**</code> — sy

#	item	note
	Mimicry changes TYPE, not forme. <code>refusesCopy</code> holds TEN members, not the 34/37 the spec quotes — that count predates the carrier toss and is stale. THE FIVE ROWS LANDED 2026-08-22 (ENGINE) and the instrument moved from 5 UNCOVERABLE to 0: disguise and hungerswitch now AGREE with the authority on all four assertions, forecast is DECLARED KNOWN-OPEN on A1 alone. Every field DERIVED from <code>Dex.forFormat</code> and spliced in beside its base row by brace-matching — not a regeneration; the field-by-field diff reads `rows lost o`	
#205	EVERY FIXTURE IN THE REPO IS GENDERLESS, AND ONE ABILITY NEEDS GENDER TO EXIST. All harnesses declare <code>gender: 'N'</code> to Showdown — <code>game_differential.js:3762</code> states it in words — and Showdown's <code>attract</code> refuses a genderless target. So Cute Charm is correctly wired and can still never fire on any board we own, and the staged rate arm's declared 30% is unreachable rather than wrong. This is not a defect in either engine; it is a property of every fixture, and it means any mechanic keyed on gender is currently untestable by construction — Rivalry and Attract are in the same position. Give the fixture builders a gender before scoring any gender-keyed row, and until then report those rows UNREACHABLE rather than SHORT	open — engine
#206	PP IS MATERIALLY WRONG IN THIS ENGINE — FOUR DEFECT FAMILIES, AND NOTHING IN THE REPOSITORY COULD SEE ANY OF THEM. <code>board_state.js</code> published PP under <code>NOT_COMPARED</code> on the reason <i>"medicham2 does not track PP at all"</i> — dead since #144, and four other files still quote it. Comparing it for the first time: showdown/ours reads Trick Room 3/1, Stealth Rock 3/1, Haze 2/1, Charm 1/2, Crunch 1/2, Roar 2/4, Perish Song 2/1, Parting Shot 1/0, Triple Axel 3/2, Electro Shot 1/2. On the roster's moves stage the leaf takes COULD-NOT-STAGE 64 -> 11 (forty-four 'inert' moves now move a board) and FIRED-AND-BOARDS-DIFFER 0 -> 26; abilities MATCH 107 -> 117. The mapping is declared, not collapsed: <code>pp-is-what-has-been-spent</code> — our <code>_pp</code> is lazy (absent = untouched), Showdown's <code>moveSlots</code> are eager (present = full), and SPENT is 0 on both; our side reads the engine's own new <code>ppSpentMap</code> export and nothing is filled in from Showdown . Two plants both CAUGHT+LOCALISED, one of them on a slot NOTHING has touched — the blind spot the lazy table makes possible. (<i>The first run said caught-but-NOT-localised because the assertion expected <code>pp</code>, and the comparator emits <code>pp[0].blizzard</code> — the probe was wrong before the engine was.</i>) ONE CAUSE IS REPRODUCED WITH A ONE-SUBSTITUTION CONTROL : Showdown resolves <code>target: all</code> to allies AND foes and prices Pressure off that list (<code>sim/pokemon.ts:794-860</code>); ours hands <code>ppPressureExtra</code> an empty list whenever the action names no explicit target — the same staged Haze fixture reads 1/1 against Scrappy and 1/2 against Pressure. The fix needs a move TARGET CLASS that <code>data/tags.json</code> does not carry	CLOSED 2026-08-11 — all four families fixed, plus a fifth the fix itself exposed, each with a probe shown RED first. (1) Pressure was priced off an EMPTY LIST for any move naming no target; new <code>targetClass</code> move tag derived in <code>tag_dex</code> from <code>target + mustpressure</code> by replaying Showdown's own <code>getMoveTargets</code> — membership printed over 500/500 legal moves before a line was wired: foes 59, aimed 370, none 71 — and Haze, Trick Room, Stealth Rock go 1 -> 3, Imprison 2 -> 3. (2) A rampage lock charged every turn because the gate tested <code>_lock</code> , which is the CHOICE/ENCORE field, while its own comment called it 'the rampage lock' — the rampage is <code>_mtLock</code> ; two-turn Outrage 2 -> 1. (3) A charge move billed twice: <code>twoturnmove</code> carries <code>onLockMove</code> , so the RELEASE turn is a locked turn; Electro Shot 2 -> 1, still 2 in rain. (4) Eight moves paid NOTHING AT ALL — the gate excluded the action KIND NAME (<code>switch</code> , <code>pass</code>) instead of asking whether the action carried a move id: <code>chillyreception</code> , <code>partingshot</code> , <code>fairyluck</code> , <code>healbell</code> , <code>roleplay</code> , <code>spite</code> , <code>teatime</code> , <code>transform</code> all 0 -> 1. (5) Found by lifting the hold, not predicted: Pressure charged for the body the click NAMED rather than the one standing in the slot when the move RAN — the engine already re-aimed the EFFECT through <code>reaimToSlot</code> and priced the PP off the raw request; Charm, Crunch 2 -> 1, Roar 4 -> 3. Families 2 and 3 do share a cause (Showdown's <code>getLockedMove()</code>) and that was ESTABLISHED PER FAMILY rather than assumed — the first Outrage measurement said the authority charges every turn, but the target had fainted and taken the lock with it; against a body that survives the trace reads 12 -> 11, 11 -> 11. Loud fallback measured silent: all 500 moves through a real <code>battleTurn</code> gives <code>ppTargetClassUnknown</code> = 0, <code>ppUnknownMove</code> = 0. Census 480 -> 485 live / 0 missing / 0 broke
#207	PP IS HELD OUT OF EVERY PASS/FAIL VERDICT, DELIBERATELY AND VISIBLY — 2026-08-11. Comparing PP flips a passing gate clause (26 FIRED-AND-BOARDS-DIFFER), and the standing instruction is to STOP rather than trade an open gate for a coverage gain. So PP is compared in <code>engine/game_differential.js</code> , which reports rates and is not gated, and held out of the verdicts via <code>ppHold: true</code> at four call sites, each printing its reason and its numbers on every run. <code>board_state.js</code> stamps <code>pp_comparable.held_by_the_caller</code> , so a run that never asked can never read as a run where PP agreed — which is the whole difference between this and the absence it replaces. Lifting it is deleting one line per site, and it is worth 44 newly-stageable moves once #206 closes. This row exists so the hold cannot become permanent by being forgotten	open — engine. QUANTIFIED 2026-08-11, and two of the three hold sites are LIFTED. <code>staged_board.js</code> is 24/24 clean and <code>staged_status_counters.js</code> clean with PP compared — both holds deleted. <code>tests/roster.js</code> keeps its hold, measured both ways: hold ON 428 MATCH / 0 DIFFER / 64 COULD-NOT-STAGE; hold OFF 472 / 7 / 11. So 26 predicted differs came down to 7 and 44 moves became stageable — and it still flips the clause, so it went back. THE BAD NEWS, AND IT IS THE INTERESTING PART: six of those seven are NOT PP defects, and putting the hold back RE-HIDES them. They are registered separately as #210. That cost is now printed at the call site on every run rather than living in a comment
#208	ABILITY TRAPPING WAS ALREADY COMPARED AND THE DECLARATION WAS THE STALE PART — CLOSED 2026-08-11. <code>NOT_COMPARED</code> claimed a comparator would have to reimplement <code>medicham2</code> 's rule. It does not: <code>tests/roster.js</code> 's <code>switchVerdict</code> asks both engines the same question at the same moment — Showdown by rejecting the choice string, ours by whether the slot changed hands — with an in-game pre-mega control and exception arms. Shadow Tag reads FIRED-AND-BOARDS-MATCH. Arena Trap and Magnet Pull have NO legal carrier in this regulation, so Shadow Tag is the entire population. The entry now carries <code>measured_by</code> and <code>population</code> instead of implying a gap that was not there. Move trapping was already compared and all seven pass	closed — engine
#209	THREE PRE-EXISTING REDS, MEASURED RATHER THAN INHERITED SILENTLY. <code>tests/staged_status_counters.js</code> : all 11 rows read <code>release THREW</code> — its BEFORE arm is pinned to release <code>6b5447db1738</code> , frozen before <code>natureL50</code> was exported. The LIVE arm is IDENTICAL on all 11, so the engine is fine and the BASELINE is stale; it needs a newer pinned release. <code>tests/staged_board.js</code> : <code>allowProof()</code> 's <code>fakeout-flinch</code> anchor matches ZERO times because the engine block is now split across two lines — all 24 scenarios are clean and board-identical, so this is an anchor that has drifted off its source, not a failure. <code>tests/test-no-silent-failure.js</code> : 64 new catch blocks since baseline across	open — engine

rows exist, which is a account: docs/_reports/

#	item	note
	~40 files, none added tonight. All three verified as pre-existing rather than assumed to be somebody else's	
#210	COMPARING PP EXPOSED SIX MECHANICS WHERE WE RESOLVE A MOVE THE AUTHORITY REFUSES OR DELAYS — 2026-08-11. Found by lifting the <code>ppHold</code> on <code>tests/roster.js</code> , and the mechanism of the hiding is the finding : a row reads <code>COULD-NOT-STAGE</code> when the <code>REFERENCE</code> board is identical with and without the entity, and a move the authority REFUSES moves no HP — so it read inert and was never compared at all. PP gives the refused click an observable, the row stages, and our engine is seen on it for the first time. Burn Up 1240/1240 authority vs 1195 ours — we resolve it off a non-Fire user, which the authority refuses. Last Resort 1240 vs 1209 . Poltergeist 1240 vs 1198 — we resolve it against a target with no item. Future Sight 1240 vs 1219 — the authority lands it <code>TWO TURNS LATER</code> and we do not. Safeguard refreshes a clock with <code>no onSideRestart</code> , reading 4 turns here. Transform is the weak generic arm. Only Mirror Coat (2 PP vs 1) is a PP row, and it is not the re-aim defect. Six separate mechanics, six probes — batching them makes a bad result unattributable (LESSONS, and the seven misdiagnosed defects of this same night). Until they close, <code>tests/roster.js</code> keeps its hold and 44 moves stay unstageable	CLOSED 2026-08-11 — all six closed, one probe each, every one shown RED before the engine moved and the gate re-run between each. Poltergeist refused against a bare target (<code>move/readsTargetItem</code> , 46/46 -> 0/46); Burn Up spends the user's Fire type then refuses (<code>move/spendsOwnType</code>); Last Resort refused until every other slot is used (<code>move/failsUnlessOtherMovesUsed</code> , 136/136/136 -> 0/0/136); Transform copied nothing (<code>move/transformsIntoTarget</code> , 61 Atk and an unchanged movelist -> 200 Atk and Garchomp's four moves, max HP unmoved); Mirror Coat answered the last hitter rather than the special one (<code>move/fixedDamage</code> -> Counter 122/0, Mirror Coat 0/184). FUTURE SIGHT NEEDED NO NEW MACHINERY, WHICH IS THE MOST USEFUL SENTENCE IN THE REPORT : WIRE 154 already built a delayed-effect queue for Wish — <code>sf.slot</code> , a residual tick, a <code>due</code> flag — and Future Sight is that primitive with a different payout, so it REUSES it rather than growing a second one. And WHEN it lands was measured on a real battle rather than read off <code>endingTurn = (this.turn - 1) + 2</code> , which invites 'one turn later' and is wrong: nothing after turn 1, nothing after turn 2, <code>-end + -damage</code> at the end of turn 3. THE SEVENTH ROW, SAFEGUARD, WAS NOT AN ENGINE DEFECT AT ALL — THE COMPARATOR WAS ACCUSING AN ENGINE THAT HAD BEEN RIGHT SINCE WIRE 133. <code>board_state.js</code> 's <code>mediScreens</code> walked EVERY KEY OUR SIDE WROTE while <code>sdScreens</code> walked a FIXED THREE-KEY LIST, so anything our engine can hold that the list did not name read <code>present-in-one-engine-only</code> whatever the authority had done; staged against the official simulator, Showdown's p2 carried <code>safeguard {duration: 4}</code> throughout. <code>SCREEN_KEYS</code> now names the four keys <code>sf.sc</code> can hold, derived from the eleven side conditions this format's 500 moves declare — the other seven live in <code>field.twA/twB</code> , <code>field.sgA/sgB</code> and the hazard store, so naming one of those would make the mirror-image bug. Census 485 -> 491 live / 0 missing , six arrived, none broke
#211	THE ppHold IS LIFTED AND THE ROSTER IS AT ZERO DIFFERS — CLOSED 2026-08-11. Measured across three states: hold ON 428 MATCH / 0 DIFFER / 64 COULD-NOT-STAGE ; hold OFF before #210 472 / 7 / 11 ; hold OFF after 479 / 0 / 11 . Worth 51 newly-stageable moves and 11 newly-stageable abilities at ZERO differs — abilities 107 -> 117 tested, items unchanged at 139. This closes the loop opened when PP was first compared: the hold existed so an open gate would not be traded for a coverage gain, it was printed at the call site on every run so it could not become permanent by being forgotten, and it came off the moment the six defects behind it closed. Differential re-run at n=6000 seed 20260804: 0/6000 midpoint, 0/6000 top, 0/6000 bottom	closed — engine
#212	WIRING TRANSFORM LET THE INTERACTION MATRIX SCORE A PAIR IT NEVER COULD, AND IT PARTED — CLOSED 2026-08-11. <code>transform -> goodasgold</code> : <code>.A.active[0].species read medi="gholdengo"</code> against <code>sd="ditto"</code> . Good as Gold refuses any status move from another body and Transform is one , so a move that used to do nothing now does something — and the moment it did, a second engine disagreed about it. Fixed through the shared <code>refusesStatusMoves</code> reader rather than a Transform-shaped branch. Matrix 1641/1641 -> 1642/1642 at 100.0%, and the DENOMINATOR moved — which is the signature of coverage genuinely widening rather than a ratio being talked up. A second instrument catching a defect the first one's fix created is the cheapest bug this project ever finds , and it only happens because both are re-run after every change	closed — engine
#213	TWO PROBES WERE WRONG BEFORE THE ENGINE WAS, BOTH CAUGHT BY ARMS AGREEING FOR THE WRONG REASON — 2026-08-11. The Transform probe asserted Garchomp's Attack is 150 — that is the o-EV Serious set, and <code>bare()</code> builds a Champions body which returns 200 . The Mirror Coat probe first AIMED its click, and <code>playerAction</code> only sets <code>rescript</code> when the caller names nobody, so both arms came back identical ; its first special hitter also clicked Surf, which is <code>allAdjacent</code> and splashed the physical attacker. <code>engine/fixture_preflight.js</code> separately refused Milotic as the retaliator — it cannot learn Counter — and exactly two legal bodies learn both Counter and Mirror Coat . Recorded because 'the probe was wrong before the engine was' is now the fourth time in one night, and the tell was the same every time: two arms agreeing when they had no way to differ	closed — engine
#214	<code>mcKey</code> COULD NAME A ROW BUT NOT HAND IT OVER, SO EVERY CALLER WENT BACK TO THE ONE LINE THE RATCHET BANS — CLOSED 2026-08-11. <code>tests/test-mc-key.js</code> was RED on two clauses: three files hand-rolling the species lookup (<code>million_run.js</code> 3, <code>replay_differential.js</code> 6, <code>test-mechanics.js</code> 1) and <code>merge_mega_into_engine.js</code> grown 14 -> 23. THE THREE WERE NOT CARELESSNESS AND THE FIX IS NOT A BASELINE ENTRY. All three already called <code>mcKey</code> as their PRIMARY doorway — what they then did was index <code>MC.mons</code> with the answer, because <code>mcKey</code> returns a KEY and the file exposed no way to get the ROW. <code>mcKey.all</code> had been added for exactly this reason on the enumeration side, with the finding written in its own comment: <i>the accessor was missing a verb</i> . It was missing a second one. <code>mcKey.row(name, opts)</code> added , returning the entry and declaring a miss through <code>miss()</code> rather than throwing, because 'this table has no row for that species' is a legitimate answer — <code>MC.mons</code> does not cover the format, which is what lets a fixture read <code>COULD-NOT-STAGE</code>	closed — engine

#	item	note
	with a reason instead of a zero. Membership is now <code>!!mcKey.row(x, {mayMiss})</code> and nobody indexes anything. The regex was right to flag <code>MC.mons[mcKey(x)]</code> and must not be taught to allow it — a pattern clever enough to permit that is clever enough to admit <code>MC.mons[norm(x)]</code>, which is the 8.17%-of-usage silent miss this ratchet exists for. Files hand-rolling: 19 -> 16, three off the list entirely. <code>replay_differential.js:581</code> was a genuine second doorway rather than a membership test — <code>MC.mons[id(row)]</code> used this file's own normaliser to answer the question <code>mcKey</code> is the single answer to; it is now a counted <code>mcKey.row</code> fallback with its purpose unchanged. <code>million_run.js</code> reads the accessor through <code>REL.require</code> like its other frozen deps, so a measurement still reads the photograph's bytes. Verified: ratchet 15/15, census 503 live / 0 missing / 0 probes threw	
#215	A BASELINED-AS-LEGITIMATE FILE GREW AND THE RATCHET COUNTS LINES, SO IT FIRED ON GROWTH THAT CARRIES NO RISK — 2026-08-11, RAISED WITH THE AUDIT ON RECORD. <code>engine/merge_mega_into_engine.js</code> went 14 -> 23 hits. Its exemption is CATEGORICAL, not budgetary: the file BUILDS <code>MC.mons</code>, so it cannot ask an index of a table it has not finished writing — and <code>mcKey</code> caches its index against the table's object identity, which the merge mutates in place, so the index would go stale mid-run even if it could. All 23 sites were read individually before the number moved: membership during construction, base-row reads during construction, the <code>byNorm</code> map the ratchet's own header names as legitimate, and the post-merge audit walk. None resolves an external name. The re-baseline was taken and DIFFED to prove it laundered nothing else: exactly one number changed, <code>14 -> 23</code>, with the other fifteen files byte-identical. Recorded rather than done quietly, because 'we re-baselined the ratchet' is indistinguishable from 'we loosened the gate to make it green' unless the audit is written down. The residual weakness is real and stated: for a file whose every hit is table construction the COUNT carries no information, so this clause will fire again on the next honest growth. The structural answer — a categorical exemption for the two files whose reason is categorical (<code>medicham2-browser.js</code> cannot <code>require</code>; this one builds the table) — is deliberately NOT taken tonight, because it would also stop catching a genuine hand-rolled resolution inside those two, and that trade needs measuring rather than assuming. AND THIS ROW CLOSED THE GATE ON ITS FIRST RUN, WHICH IS A SECOND FINDING AND IS KEPT HERE RATHER THAN QUIETLY EDITED AWAY. <code>quarantine.js</code>'s defect clause is a <code>DEFECT</code> token in the status cell PLUS a backstop list of phrases for rows written before the token existed — and one of those phrases appeared inside a sentence explaining why a file is EXEMPT, in a row that describes a test's design and asserts nothing about the simulator at all. The wording was changed to a synonym and the gate reopened. That is the cheap fix, not the right one: a vocabulary backstop cannot tell a claim from a quotation, and an over-firing gate is the one people learn to ignore (#148). The row is left open on that ground	open — measure
#216	FOCUS BAND CANNOT SAVE ANYTHING — ITS WHOLE TAG IS <code>flingable</code> — 2026-08-11, FOUND BY WILL ASKING WHAT ITS RATE WAS. The rate runner reported <code>proc:focusband</code> UNREACHABLE with the reason 'the FROZEN tag artifact carries no such param', which reads like a missing DECLARATION and is a missing MECHANIC. The target row is right: <code>DERIVED:randomChance(1,10) = 10%</code>, and Champions does not override the item. The engine's <code>data/tags.json</code> entry carries <code>tags: ['flingable']</code> and nothing else, so a body holding Focus Band dies exactly as if it held nothing, on 15 sheet teams. IT MUST NOT BE FOLDED INTO <code>survivesFromFull</code>, WHICH IS THE COLLAPSED-LABEL TRAP THIS REPO HAS ALREADY PAID FOR. That tag's params are <code>{leavesHP:1, onlyFromFullHP:true, movesOnly:true, consumesItem:true}</code> and Focus Sash and Sturdy both fit it. Focus Band fits NONE of the last three: it fires at any HP, at a 10% chance rather than a certainty, and does not consume the item (<code>return target.hp - 1</code> with no <code>useItem</code>). Three of four params differ, so a shared label would be one name standing for two rules — the split #162 already had to undo. The parameterisation that holds both is <code>chance + onlyFromFullHP</code> derived from the handler, matched on SHAPE so a third member is picked up without editing the engine. Related: #166 (Focus Sash and Sturdy do not save either) — if that one is still open this is one mechanism and not two, and the fix must be shown red on all three bodies	CLOSED 2026-08-11 — Focus Band's whole tag was <code>flingable</code>, so a lethal hit met no code path that asks whether the item saves. <code>survivesFromFull</code> now carries a DERIVED <code>chance</code> and a DERIVED <code>onlyFromFullHP</code>; membership printed over the whole dex first and returns exactly three (focussash, sturdy, focusband), no over-match, and no ability or item name appears in the engine so a fourth member needs no edit. Probe <code>item/survivesFromFull</code>, five arms — the load-bearing one is that inside the 10% it survives at 1 AND KEEPS THE ITEM, because the handler has no <code>useItem</code> and an engine that copied the Sash's consumption would pass the survival and silently disarm the next hit. Staged arm went UNREACHABLE -> 195/2000 = 9.75% against 10%, the only row in that arm that had never produced a number. <code>million_run.js:2237</code> passed a null tagPath, so the caption still said 'carries no such param' after the param existed — closed by reading <code>survivesFromFull.chance</code> out of the frozen artifact, which corroborates against <code>million-targets.json</code>'s independent <code>DERIVED:randomChance(1,10)</code>
#217	FOUR STALE REASONS IN ONE PASS, ALL THE SAME SHAPE: A SENTENCE TRUE WHEN WRITTEN, QUOTED AFTERWARDS AS A MEASUREMENT — CLOSED 2026-08-11. Will found three of them inside twenty minutes and the fourth fell out of the sweep. (1) Tangled Feet sat switched off in <code>ACCMOD</code> behind <code>off:'no confusion volatile exists in this engine'</code> — false for weeks: <code>applyConfusion</code> writes <code>_vol.confusion</code> with the self-hit roll, expiry, Safeguard's refusal and Persim's cure. I repeated that string to Will as fact and he closeted the ability on it, then reversed the closeting once it was checked. Probe is a 2x2 where three	closed — engine

#	item	note
	cells must be EQUAL — ability alone 176, confusion alone 176, neither 176, only the pair 0. (2) Every other ACCMOD reason re-derived rather than re-read: skilllink's holds (it has no accuracy handler at all — a genuine tag false positive), victorystar's was corrected to the real blocker, which had never been stated: Victini is not in this format, zero carriers; laxincense and wonderskin are DEAD-BUT-ON and now named. (3) Plus and Minus were ONE DEAD MECHANIC FILED TWO WAYS — minus DEFERRED-BY-OWNER, plus COULD-NOT-STAGE printing <i>THE STAGING IS INERT... this is the honest coverage limit, not a pass</i>'. It was not a coverage limit: the rule reads <i>'onModifySpA with NO type and NO HP gate'</i> and Plus satisfies both while being gated on the ALLY, so the fixture never placed a partner. And the mechanic was dead underneath — all five cells read 121 because the consumer refused it for a third, uncounted reason. Fixed at the RULE, not the row; onlywhen:{cond:'allyHasAbility'} derived; membership over the whole dex is exactly three (Friend Guard, Plus, Minus); hasAbility(['minus','plus']) is SYMMETRIC so Plus beside Plus counts, which the shelf reason also had wrong. Minus is off the closet and was never a rarity call. (4) Flying Press off the usage shelf on Will's domain call — 21 clicks against a floor of 25, and <i>'hawtucha gets some play'</i> beats a threshold. Sibling checked first: only two legal moves carry onEffectiveness, so addsType became a second PARAM on Freeze-Dry's tag rather than a new one — fifth time this sprint the answer was a param. Targets DERIVED from the chart, and the Aggron arm is the one that matters: Fighting x4 times Flying x0.25 = x1, so the 100 BP move does LESS than a 75 BP Brick Break, 46 against 136 — an engine ignoring the added type or hardcoding a flat x2 fails it. Moves 479 -> 480 tested, closet 10 -> 9. The floor was NOT lowered — fitting a threshold to one example is worse than keeping it; a shelf entry is now recorded as a prompt to ask somebody, not a verdict. Census 511 -> 514 live, 0 missing	
#218	THE WHOLE-GAME DIFFERENTIAL SAYS 39.6% OF GAMES DIVERGE, IT GATES NOTHING, AND I QUOTED THE DAMAGE NUMBER ALL SESSION AS IF IT COVERED THIS , CLOSED 2026-08-12. engine/game_differential.js plays one team pair through BOTH engines with real sheets and real natures, pins every die identically on both sides so any difference is a rule bug, and records the first divergence. Re-run on release 5a557b07821c: 1,213 games, 480 diverged (39.6%), 0 threw, planted proof caught. The previous run was 570 of 1,556 (36.6%) — a full evening of engine work did not move it, which is itself the finding. The gate's game differential clause reads data/engine-diff.json , which is DAMAGE ONLY and says 0 of 6,000 — its own scope line reads <i>'no items or abilities. Turn order, status duration and switching need a different harness and are not attempted here rather than attempted badly.'</i> Both numbers are true; they answer different questions; I conflated them in the project's favour repeatedly and Will caught it by asking whether we had ever played the same game on both engines. New engine/divergence_report.js rolls the causes up — the artifact has recorded every one since it was built and nothing had ever read them. 307 distinct causes, 251 appearing in exactly ONE game. Ranked by CORPUS USAGE rather than by count, because the swarm is coverage-steered so its game counts describe the instrument while the corpus counts describe what people click. The two rankings disagree and both are needed: by games the head is residual weather ORDER, ~80 games — six causes identical in shape, differing only in which body is chipped first, one rule; by usage the head is PROTECT, 32 games but 94,313 clicks, where Showdown says the move worked and we say it failed, scattered across four of the differential's classes at once, which is why the class view hid it RE-MEASURED 2026-08-18 BY MEASURE, AND THE GATING HALF FINALLY HAS A COMMAND. The gate landed 2026-08-12 as clause 7 of 7 of a four-minute report, which is not a thing engine/register_reality.js can run — so this row still had no instrument, and #297's closed-detector repair reopened it. node engine/quarantine.js --whole-game is that same clause with a process exit attached and nothing else: it CALLS wholeGameClause rather than restating it, and both it and --order-probe map their verdict through one clauseExit , so a withheld figure cannot exit o through one command and 2 through the other. 0 = the two engines agree on every game, 1 = they do not, 2 = the artifact describes other bytes (#298). Measured on a differential re-run of the current tree — 995 games, arm A/middle, release 978ca8fe72c9 , census 2e3953f1f882 , team pool 631d4ea60a80 — it is 696 of 995 = 69.9% (700 raw, less 4 declared, 0 cleared on decision impact) and it EXITS 1. THE 39.6% IN THIS TITLE AND THE 33.4% IN THIS CELL ARE NOT A TREND WITH IT. Different releases, different pins and different samples: the driver steers off a census that is regenerated and picks pairs from a team pool read live from the store, so two runs a day apart do not play the same games. RE-MEASURED 2026-08-23 BY MEASURE — THE HEADLINE IS 8.0% UNDECLARED / 8.5% RAW, AND THE 39.6% IN THIS TITLE IS NOT ITS BEFORE. On release c36782953dee (data/game-differential.json , generated 2026-08-23To6:10:58Z, mode A/middle/pins:1fd77b835ee2/credit:observed-effect/v1/nature:real , team store pinned to data/team-pool-	open — engine DEFECT. NOT MEASURED 2026-08-24 (instrument banned that session); artifact fields only: 57/961 RAW middle arm and DIFFERENT-END-STATE 18/961 on release 3929459bb195. 33.4% as of 2026-08-12 (328 of 983, release 2fa7c11fd0e6), down from 41.5% on the Regenerator emission fix and from 36.6% at first measurement. Now reported as TWO numbers rather than one, per #223: EMISSION 96 causes / 135 games, RULE 50 / 85, UNPARSED 65 / 68, ORDERING 26 / 35, FIELD 5 / 5. CLOSED 2026-08-12 — the gating half landed: wholeGameClause() is clause 7 of 7 in engine/quarantine.js and prints the absolute figure on every run with the words NOT ZERO, AND THE RATCHET IS NOT A PASS. The 39.6% in this title is superseded — the same instrument on release 225ddao4a2a8, with pinned spreads and a frozen team pool, reads 260/983 = 26.4% top-tie-first and 284/983 = 28.9% bottom-tie-first. The quoting half of this row is fixed by the clause existing; the divergences it counts are #81's queue, not this row. 2026-08-18: 696 of 995 = 69.9% on release 978ca8fe72c9, and the row now has a runnable instrument that exits 1. Every earlier figure in this cell was measured on another sample and another pin and none of them subtract.

#	item	note
	frozen digest 0d103fb9fa87 , census digest 8c778268919e / 634 rows read LIVE and NOT pinned) the clause reads 77 of 961 = 8.0% — 82 raw, less 5 declared and 0 cleared on decision impact. All five declared are one entry, Supreme Overlord fallenundefined , where matching the authority would make this engine less correct. THE DENOMINATORS DIFFER AND SO DOES THE QUANTITY, SO NO DELTA IS CLAIMED. The title figure is 480 of 1,213 on release 5a557b07821c , 2026-08-12, and it is a RAW rate — the declared subtraction did not exist then. Today RAW is 82 of 961 = 8.5%, and that is the only pair of numbers measuring the same quantity. Even those two are not a trend: different release, different pin corner, and the census SELECTS the sample (steering.rule says so in as many words), so two runs on different censuses do not play the same games. The clause agrees — its own baseline_comparable reads false and it withholds direction of travel for exactly this reason. THE RUN ASKED FOR 1200 AND PLAYED 961, AND THE SHORTFALL IS STRUCTURAL RATHER THAN A TRUNCATION: the swarm splits nine ways and two configs have almost no eligible pairs in the frozen pool — omit-protect 99 available so 48 games, omit-priority 7 available so 3 games — so the sample is real and it underweights those two corners, which has to be said before anyone reads 8.0% as covering them. AND THE HEADLINE IS THE UNDECLARED RATE, NOT THE RAW ONE, WHICH IS THE HALF THAT GETS MISREAD. 8.0% = 77/961 undeclared; 8.5% = 82/961 raw. The three arms of this same run read middle 82, top-tie-first 68, bottom-tie-first 77 of 961, and pins.primary is middle , whose 82 is what the top-level diverged field carries. THE BOTTOM-TIE-FIRST ARM ALSO READS 77 AND THAT IS A COINCIDENCE, PROVEN RATHER THAN ASSERTED — it was read as the gate quoting a corner by omission, which would be a serious defect and is not what happens. wholeGameClause takes j.diverged and j.games at the TOP LEVEL and indexes no arm at all. Driven through the shipping function on perturbed copies of this artifact: setting the bottom-tie-first arm to 999 leaves the headline at 77, setting the middle arm to 999 leaves it at 77, setting the TOP-LEVEL diverged to 100 moves it to 95, and deleting the declared cause moves it to 82. The 77 is 82 minus 5, and nothing else. WHAT IS GENUINELY WRONG IS THAT NOTHING LABELS THE QUANTITY — see #387. AND THIS RUN MEASURED PROTOCOL ONLY: state_mode is false and end_state is null , so no count in it supports a claim about board-materiality, and none is made here. Shape of the 82: emission 41, rule 17, ordering 13, field 11 (computed from this artifact's own classes[].causes[] , so the composition and the headline describe the same run). AND THE 8.0% WAS ALREADY WITHHELD BY 02:46 LOCAL THE SAME MORNING: ENGINE cut a further release while this was being written, so wholeGameClause returns MEASURED AGAINST A DIFFERENT ENGINE and publishes nothing at all. The figure is true of c36782953dee and of no other bytes. VERIFIED BY: node engine/quarantine.js --whole-game RE-READ 2026-08-24 BY MEASURE, AND THE INSTRUMENT WAS NOT RUN — THE HEADLINE IS OWED, NOT MEASURED. This session was under a no-play clause and node engine/quarantine.js --whole-game was on its ban list, so NO verdict for this row was taken and none is published here. What can be read out of the artifact WITHOUT the clause, labelled as an artifact FIELD rather than as the gate's answer: data/game-differential.json generated 2026-08-24T00:54:34Z on release 3929459bb195 , 961 games, team store pinned to data/team-pool-frozen digest 0d103fb9fa87 , census pinned at digest 9446a684709d / 643 rows (matches_live: false , and the census SELECTS the sample by the artifact's own steering.rule). Top-level diverged reads 57 of 961 RAW on the primary middle arm; the other two arms read top-tie-first 52 and bottom-tie-first 58. THE UNDECLARED RATE IS NOT IN THE ARTIFACT. The declared subtraction lives inside wholeGameClause and nothing on disk carries it, so no undeclared figure is stated here rather than a second implementation of the clause being written to produce one. THE 39.6% IN THIS ROW'S TITLE IS 480 OF 1,213 RAW ON RELEASE 5a557b07821c , 2026-08-12. Raw against raw is the same QUANTITY and it is still not a trend: different release, different pin corner, different census, different pool, and two runs on different censuses do not play the same games. NO DELTA IS CLAIMED. AND THIS ROW'S SENTENCE THAT THE RUN MEASURED PROTOCOL ONLY IS NOW STALE. This artifact carries state_mode: true and end_state_mode: true , so it publishes the board-material quantity Will's 2026-08-22 call named: middle arm DIFFERENT-END-STATE 18 of 961, of which 17 are among the 57 protocol-parted games and 1 is a game whose protocol never parted at all, split by shape EMISSION 7, FIELD 5, RULE 3, ORDERING 2. Those are the artifact's own published fields and not a derivation taken here. OWED: this row's own clause verdict, node engine/quarantine.js --whole-game , the first session that is allowed to run it. THE MARKER ON THIS ROW NOW DECIDES A DIFFERENT QUANTITY — 2026-09-04, RECORDED BY ENGINE, AND NO FIGURE IN THIS CELL IS RE-MEASURED HERE. node engine/quarantine.js --whole-game was SPLIT tonight (commit 9dca646b): wholeGameClause is now named	

#	item	note
	"whole-game differential / BOARD-MATERIAL — games whose boards part" and is computed as <code>state.games less state.games_board_never_diverged</code>, which is Will's 2026-08-22 bar. The PROTOCOL first-divergence quantity that EVERY figure in this cell reports moved to a second clause with its own command, <code>node engine/quarantine.js --narration</code>, which reports and does NOT gate (<code>gates: false</code> applied by a wrapper to every return path, refusals included). So the VERIFIED BY above still names a runnable command, and that command no longer answers the question these numbers are about. THE FIGURES, ATTRIBUTED RATHER THAN RE-MEASURED: ENGINE's account <code>docs/_reports/2026-09-04-board-material-clause.md</code> reads, on <code>data/game-differential.json</code> at release <code>8ad06030e129</code>, 961 games, arm A/middle — BOARD-MATERIAL 77 of 961 (8.0%) and NARRATION 167 of 961 (17.4%), 168 raw less 1 declared, with an UNCAUSED set of 11 games whose board parts while the protocol never diverges at all. AND THE COINCIDENCE HAS TO BE SAID OUT LOUD: this cell already contains <code>77 of 961 = 8.0%</code> as a PROTOCOL undeclared figure from 2026-08-23 on release <code>c36782953dee</code>, and the new BOARD-MATERIAL figure is also 77 of 961 = 8.0% on release <code>8ad06030e129</code>. Different releases, different samples, DIFFERENT QUANTITIES, same digits. No delta, no trend, and no relation between the two — a reader who lines them up is reading a coincidence. THE CLAUSE WAS NOT RUN TONIGHT AND THAT IS DELIBERATE: another agent is mapping the board-partings and may be rewriting <code>data/game-differential.json</code>, and a torn read of that file has already produced a plausible, well-formed, fictitious class table once. <code>data/register-reality.json</code> IS GENERATED AND IS NOT HAND-EDITED HERE: it still carries this row from 2026-08-27 with <code>EXIT CODE UNDECLARED</code> (exit 2, 58 ms) and quotes only the row's title, not a protocol figure; it will re-state itself on the next <code>engine/register_reality.js</code> run.	
#219	A DATA-ONLY COMMIT CAN ORPHAN EVERY FIGURE IN A LIVING DOC, AND THE HOOK THAT WATCHES FOR THAT SKIPS DATA-ONLY COMMITS — 2026-08-11. <code>tests/test-docs-current.js</code> calls a figure traceable when SOME artifact contains it. <code>docs/MEDICHAM-SPRINT-NOTES.md</code> recorded the first rate run — 16,575 trials, 30.42%, the confidence intervals — and every one was traceable because <code>data/million-run.json</code> held it. Re-running the same instrument overwrote the artifact with new numbers and orphaned nineteen figures at a stroke. Nothing was typed and nothing became false; the doc still describes what was measured, and the artifact that proved it no longer exists. Isolated rather than assumed: at HEAD with none of the evening's doc edits applied the count already read 47 against a baseline of 28, so only ONE of the twenty came from the pass being committed. The reason it went unseen is the hole: <code>.github/hooks/pre-commit</code> skips data-only commits, so the run that rewrote the artifact was exactly the commit the gate did not inspect. Baseline raised 28 -> 48 with the diff shown (two lines, the count and the timestamp, nothing else laundered). The structural weakness is real and is NOT fixed by this row: the check cannot tell 'a figure nobody measured' from 'a figure whose instrument has since been re-run', and it will fire again on every re-run. The candidate repair is to make a re-run STAMP the figures it supersedes — a doc figure would then cite a RUN rather than a value, which is the same lesson as the frozen release and is a bigger job than a baseline bump. Recorded now because the alternative is that the next person raises the number again and the ratchet quietly becomes decoration	open — measure
#220	THE WEATHER RESIDUAL RAN IN SLOT ORDER WHILE THE CLOCK RESIDUAL HAD BEEN SPEED-SORTED SINCE #115 — CLOSED 2026-08-11. The largest single family in the whole-game differential: 80 games across 6 causes, every one identical in shape and differing only in WHICH BODY IS CHIPPED FIRST. Read from the authority rather than inferred from the traces: the sand chip is not in the clock pass at all — <code>sandstorm.onFieldResidual</code> calls <code>this.eachEvent('Weather')</code> and <code>battle.ts:465</code> speed-sorts descending. This engine had it RIGHT in one pass and wrong in the other, which is why nobody saw it. #115 speed-sorted the CLOCK loop — perish, yawn, status chip, Leftovers — when Will asked whether bodies faint in speed order; the WEATHER loop one screen above kept iterating <code>[...actA, ...actB]</code>. NOTHING COULD HAVE CAUGHT IT: every body takes the same 1/16, so every TOTAL agreed and a damage differential is structurally blind to an ordering defect. Fixed as <code>one_residualOrder(actA, actB, field)</code> that both passes ask, with #115's three inline lines DELETED rather than left beside it — re-sorting the weather loop in place would have written the fact twice with the same chance of drifting apart again. Probe <code>move/weatherSetter</code>, four bodies at Speed 205/101/80, and the Trick Room arm is the discriminator: <code>getActionSpeed</code> returns <code>10000 - speed</code>, so the inversion is already inside the number both passes sort on and no second branch exists — an engine sorting by raw speed passes arm 1 and fails arm 2, slot order fails both. Garchomp on the board takes no chip, the control that says this is the sand. Verified independently of the agent's	REOPENED 2026-08-12 — THE EXPLANATION WAS WRONG AND THE 32 GAMES ARE UNEXPLAINED AGAIN. This row said Protect's divergences were an instrument artefact: our stall counter shares the scalar die, so mode A's corner forces every consecutive Protect to fail. The coupling is real and the first Protect is clean under both pins — but that CANNOT be the cause, because Showdown's stall check is pinned to refuse at the same corner (#222, measured). Both engines refuse; they were agreeing for that reason. **So a trace showing the authority emit <code>`-singleturn</code>

#	item	note
	report: the both-sides family reads 3 causes / 4 games against 6 / 8o	
#221	RESIDUALS ARE RUN PER BODY AND INTERLEAVED; SHOWDOWN RUNS ONE HANDLER LIST — 2026-08-11, NAMED AND DELIBERATELY NOT STARTED. Surfaced by the #220 fix rather than predicted. Showdown sorts a single handler list by (residualOrder, residualSubOrder, speed) — sandstorm is onFieldResidualOrder: 1, Leftovers is onResidualOrder: 5 / subOrder 4 — so it chips EVERY body, then heals EVERY body. This engine runs one loop per body and interleaves the two. 17 games in the current rollout. Not a branch and not a sort: it is a restructure of every end-of-turn effect at once, and it could not be gated inside the pass that found it. Recorded now because the alternative is that it is rediscovered as six unrelated causes next time. CLOSED 2026-08-12. The walk is one pass per ORDER GROUP over the bodies, the groups read out of data/residual-order.json (derived from the format by engine/residual_order.js) so no order number is written in the engine, and residualOrder() is re-asked per group because Speed Boost is itself a step at order 28. THE KEY IN THE ROW ABOVE IS WRONG AND IS CORRECTED RATHER THAN EDITED: Battle#comparePriority is order ASC -> priority DESC -> SPEED DESC -> subOrder ASC , so SPEED SITS BETWEEN order AND subOrder — a restructure written from (residualOrder, residualSubOrder, speed) would have sorted Leftovers against Shed Skin by category when the authority sorts them by who is faster. FOUR CHUNKS HAD NO STEP IN THE FILED PLAN AND AN UNWRAPPED CHUNK IS THE CATASTROPHIC CASE (it runs once per group — Cud Chew would decrement sixteen times a turn): Future Sight (futuremove , order 3, and it was ABSENT FROM THE DERIVED TABLE because Future Sight carries no move.condition — the generator now derives slot conditions from slotCondition and addSlotCondition(...)), Wish (4), Salt Cure (13, filed as sharing Aqua Ring's order 6 and eleven orders away from it), and Cud Chew / Harvest / Pickup (28). The onUpdate berries are NOT residual steps and now close every group rather than sitting at one fixed point, so a body dropped under half by Leech Seed eats the same turn. Census 537 -> 541 live / 0 missing. Four probes, each shown red against MEDI_RESIDUAL_COLLAPSE=1 — a deliberate break that collapses the table to one group, IS the body-major walk, stamps MEDFAILS.residualCollapsed , and is refused the census file. Controlled 200-game differential, same command both ways: top-tie-first 92/179 -> 88/179, bottom-tie-first 77/179 -> 73/179, planted_divergence_proof_ok true in both, and the attributable number is the ordering class falling 24 games / 20 causes -> 18 / 15 with the sandstorm-chip <> leftovers-heal pair #218 named as unfixed now gone from it	closed 2026-08-12 — engine
#222	PROTECT IS NOT AN ENGINE DEFECT — IT IS THE INSTRUMENT'S PIN COUPLING, AND I SENT THE DIVISION AFTER IT WRONGLY — 2026-08-11. I ranked it the top target on 94,313 corpus clicks: Showdown says the move worked, we say it failed, scattered across four of the differential's classes at once. The instrument had already written the answer down, in a PINS block nobody had read: medicham2_has_one_scalar_die — 'accuracy, the crit, every secondary, THE STALL COUNTER and the damage roll all read the same rng() , so there are TWO corners and not four independent knobs.' Mode A's top corner pins that die HIGH so every sub-100 move misses, which necessarily forces rng() < 1/counter to fail; Showdown draws its stall check separately. Measured at both corners over three consecutive turns: pinned HIGH gives true/false/true, pinned LOW gives true/true/true. THE FIRST PROTECT SUCCEEDS UNDER BOTH PINS — the counter short-circuits at _ctr <= 1 and draws no die — and that is the arm that would have exposed a real bug. Only the consecutive roll moves with the pin. The repair belongs to the instrument: give the stall counter its own stream, or record the family as a known coupling and exclude it from mode A. Changing the stall probability to turn this green would be a RATE explanation for a RULE-mode divergence and would break the mechanic on every unpinned run — the division was briefed against exactly that and refused it, correctly. The `	cant
		nopp

#	item	note	
#223	THE 408 IS TWO DIFFERENT QUESTIONS ADDED TOGETHER, AND THE BIGGER HALF IS THE TRACE EMITTER RATHER THAN THE SIMULATOR — 2026-08-12. <code>divergence_report.js</code> now clusters by what the two protocol lines actually disagree ABOUT instead of by the differential's own bucket names, which had two proofs of being the wrong grouping: the weather residual was 6 causes reading as separate rows and one rule, and Protect is 15 causes spread across FOUR classes at once. 300 causes collapse to 125 families; the 249 single-game causes collapse to 115. The split: EMISSION 130 causes / 183 games (one side has a line the other does not), RULE 81 / 122, ORDERING 35 / 45, UNPARSED 43 / 45, FIELD 11 / 13. So 183 of 408 games are the engine NARRATING the game differently rather than PLAYING it differently, and a single blended 41.5% has been reported for both — the identical merged-number failure as quoting the damage differential for whole games. THE LARGEST FAMILY IS REGENERATOR: 36 games, 31 causes, 1,257 uses, every one of them naming it. Showdown emits only `	switch	p1a

#	item	note	
#224	THE ?? : SLOT PLACEHOLDER REACHES THE PROTOCOL STREAM, AND IT IS NOW THE LARGEST SINGLE FAMILY , CLOSED 2026-08-12. `	-start	p1a

#	item	note
#225	**THE WHOLE-GAME COMPARISON IS NOW A GATE CLAUSE — WILL, 2026-08-12: <i>"add the whole game comparison to the medicham its part of it and we need to focus on getting that lining up"</i>. It was measuring and gating nothing, which is how a 39.6% sat beside a 0 of 6,000 for months without anybody noticing they were different questions. The gate is six clauses to seven. The damage clause is DAMAGE ONLY and says so in its own scope line; is the other harness — one team pair, real sheets, real natures, both engines, every die pinned identically, first divergence recorded. THE BAR IS A RATCHET AND NOT ZERO, AND THAT IS A CHOICE AGAINST THE OBVIOUS ONE.** Zero is correct in principle: mode A makes the two engines deterministic functions of one input, so any difference is a bug. But a clause that reads red for weeks is one people learn to skip, and this repo has the receipt — a docs gate sat red for two days as <i>"one of the two known failures"</i> until the rule it guarded broke. So it fails on a RISE, the rate may only go down, and the clause prints the absolute figure on every run with the words NOT ZERO, AND THE RATCHET IS NOT A PASS so nobody can mistake one for the other. The tolerance is DERIVED rather than picked: the swarm re-selects teams between runs so two raw counts are not comparable — 408/982 and 570/1556 are one instrument saying different things — and the rate is a binomial proportion, so the band is 2 SE computed from the runs own n (± 3.0 points at n=983). It prints EMISSION and RULE separately (#223), because a blended percentage is the same merged-number failure as the midpoint residual that hid a range wrong at both ends. SHOWN RED THREE WAYS BEFORE BEING TRUSTED: no baseline FAILS by design rather than seeding itself from whatever the last run produced; a planted better baseline of 25.0% against the real 33.4% FAILS on the rise; and REFUSES a worse number without , because a baseline may only ratchet down and raising one is how a gate becomes decoration. A run whose planted-divergence proof did not fire is refused outright — an instrument blind to a divergence it planted cannot be believed about the ones it did not	closed — engine
#226	THE DIFFERENTIAL RAN BLANK SPREADS AND A MOVING TEAM POOL, SO IT WAS TESTING TURN ORDER IN THE ONE CONFIGURATION WHERE TURN ORDER CANNOT BE GOT WRONG — WILL, 2026-08-12. He asked <i>"how much of the divergence is us not knowing the sps"</i>, then <i>"i thought the whole point was we invent teams that we know every stat of so nothing can drift"</i>,	open — measure

#	item	note
	then <i>"we can yolk moves, we just need to give them our stat spreads"</i>. builds every body with — not a guessed spread, NO spread. An open team sheet reveals species, item, ability, moves, nature, gender and level and NOT the spread; every stored sheet reads <code>evs: null , 173,784</code> of 173,784 bodies. NEITHER THE BLANK SPREAD NOR THE SPEED TIES CAUSE A SINGLE DIVERGENCE — both are applied identically to both engines — THEY HIDE ERRORS INSTEAD, and the file already said so in a comment nobody had acted on: <i>"with every body flat AND Serious, 91.4% of legal species share a base Speed with some other species, so THE RIG MANUFACTURED SPEED TIES"</i>. The prediction is recorded before the fact: adding spreads should make the number WORSE and ordering should GROW; an unmoved rate is the suspicious outcome. Second unfrozen input named the same day: the team pool is read LIVE from stores OPS appends to continuously, which is why three runs of one instrument reported 1,556, 1,213 and 983 — a measurement that freezes the engine and pins every die took its SAMPLE from a moving file. Frozen on Will's instruction to <code>data/team-pool-frozen (bo3 13,214 lines sha 5e10d7ba991f, ots 4,167 lines sha cd21077a4578)</code> and the first pinned baseline is 327/983 (33.3%) top-tie-first and 377/983 (38.4%) bottom-tie-first — the first whole-game figure in this project whose counts are comparable run to run. That five-point arm gap also weakens a claim I made to Will off the unpinned run — I said the two corners agreed to within half a point and therefore the divergences were rules rather than dice; on a pinned pool they are five apart and the higher one is the tie-sensitive arm	
#227	NATURAL CURE READS COULD-NOT-STAGE WHILE REGENERATOR STAGES FINE UNDER THE IDENTICAL RULE — AND IT WAS RAISED, AGREED AND THEN DROPPED — 2026-08-12. Will asked <i>"did we fix natural cure"</i>. It had been flagged hours earlier as worth checking, and <code>grep naturalcure docs/ROADMAP.md</code> returned <code>o</code> — it was never registered. That is the exact failure this register exists to make impossible, committed by me in the session I spent enforcing it on everyone else. The verdict text is <i>"THE STAGING IS INERT. Showdown's own board is identical with and without it, over 419 compared leaves — this is the honest coverage limit, not a pass."</i> Three reasons to distrust that sentence. (1) It is the SAME sentence <code>plus</code> printed, and <code>plus</code> turned out to be a misclassified rule with a DEAD mechanic underneath — see #217. (2) Its tag set is <code>["switchOutTrigger"]</code> and nothing else, while Regenerator under the same <code>ability/switch-out</code> rule carries <code>healsOnSwitchOut</code> <code>PLUS switchOutTrigger</code> and stages cleanly: Natural Cure has the tag for WHEN it fires and no tag for WHAT it does, which is Focus Band's shape exactly (<code>flingable</code> and nothing else, #216). (3) 419 compared leaves proves less than it sounds — <code>status</code> IS in the board, so a cure must show, unless the fixture never gave the body a status to cure. The likely answer is that the board switches out a HEALTHY Pokemon and the ability correctly does nothing — a claim about the FIXTURE, never about the mechanic, which is the rule Will has taught me twice and which is in my memory file. NOT asserted without staging it. Derived: 114 corpus uses, handlers <code>onCheckShow, onSwitchOut</code> , four legal carriers each with a quiet alternative for the control arm — Starmie {Illuminate, Analytic}, Altaria {Cloud Nine}, Roserade {Poison Point, Technician}, Trevenant {Frisk, Harvest}	CLOSED 2026-08-12 — STAGED, AND THE MECHANIC IS FINE: THE ROSTER'S VERDICT IS ABOUT ITS OWN FIXTURE. Three arms on a real switch — Natural Cure + burn comes back <code>""</code> CURED; Illuminate + the same burn comes back <code>"brn"</code> , so the control holds; Natural Cure with NO status changes nothing. That third arm is what the roster is looking at — a healthy body switching out, where the ability correctly does nothing and both engines produce identical boards. 419 leaves agreed because there was never anything to cure. A claim about the FIXTURE, never about the mechanic, exactly as #217 turned out for <code>plus</code> . THE PROCESS IS THE PART WORTH RECORDING. The first run of this probe printed "THE MECHANIC IS DEAD — 114 corpus uses" and would have gone into this register as an engine defect against a mechanic that works. Three probe bugs in sequence: the switch action never fired so three identical arms read as evidence; the switch DETECTOR's regex lost its escapes through the shell and matched everything; and the real cause — <code>battleInit</code> SLICES ITS OWN BENCH at <code>teamA.slice(2)</code> , so a two-body team has an EMPTY bench and the intended 'bench' Pokemon was standing on the field as the partner. There was nothing to switch into. The only reason it did not become a filed defect is a guard added mid-way: refuse to adjudicate unless the trigger provably fired — it turned a confident false verdict into NOT ADJUDICATED. Sixth time this sprint the probe was wrong before the engine was, and the reason it keeps happening is structural: a broken fixture and a broken mechanic produce identical output. The roster row stays COULD-NOT-STAGE and that is now a MEASURED verdict rather than an assumed one; giving its fixture a status to cure is the fix, and it is a fixture change
#228	THE BOARD COMPARISON DID NOT COMPARE TYPING OR ABILITY, AND THE MEASUREMENT ABOUT TO USE IT WOULD HAVE CALLED #224 COSMETIC — CLOSED 2026-08-12. Will, mid-plan: <i>"our board state analysis needs to be comprehensive"</i>. <code>board_state.js</code> walked <code>species</code>, <code>hp</code>, <code>maxhp</code>, <code>fainted</code>, <code>status</code>, <code>status_counter</code>, <code>item</code>, <code>boosts</code>, <code>eight</code> <code>volatiles</code>, <code>screens</code>, <code>tailwind</code>, <code>hazards</code>, <code>party</code> and <code>PP</code> — and neither typing nor ability. So a Soak that changes the WRONG Pokemon's type leaves every compared leaf identical, and so do Skill Swap, Trace, Mummy, Simple Beam and an ability overwritten by a mega. The end-of-battle measurement had already been briefed on that comparator and would have returned a confident, clean, WRONG answer about the most serious defect in the register. The omission was undeclared as well as absent: <code>NOT_COMPARED</code> names three deliberate gaps with reasons — ability trapping, item disposition, the stall counter — and typing was not among them, so it read as <i>compared and equal</i> rather than <i>not looked at</i>. Both leaves added to both walkers, and PROVEN on a live board three ways rather than read: both emit them; they AGREE on an untouched board (or every game parts on line one and the leaf is worse than useless); and a forced Soak-style change IS reported — <code>types</code> <code>medicham</code> "water" vs showdown "dragon/ground" , <code>ability</code> "levitate" vs "roughskin" . A leaf that is present and never differs is decoration, which is why the third arm exists. Showdown reads <code>getTypes()</code> , the METHOD rather than the species default, so it reports what the body is NOW after a mega, Protean or Soak; both sides normalised and sorted, because a pure ordering difference on a dual-typed body is not a rule disagreement. BOTH FAILURES ON THE WAY THERE WERE THE PROBE: the first run never took the	closed — engine

#	item	note	
	Showdown battle through team preview so it read an EMPTY board and every leaf came back null — indistinguishable from the leaves being broken — and the second gave the two sides different abilities and read <code>.diffs</code> off an array that has no such property, which reads as <i>nothing differed</i> . Fifth time this sprint the probe was wrong before the engine was		
#229	**THE PREFLIGHT ASKED "IS THIS LEGAL" AND NEVER "CAN THE TRIGGER FIRE" — WILL, 2026-08-12: "so fix the instrument" — CLOSED. Six times in one day the probe was wrong before the engine was, and three of those were a board that was entirely LEGAL and structurally incapable of firing its own trigger. NATURAL CURE** printed <i>"THE MECHANIC IS DEAD — 114 corpus uses"</i> because <code>battleInit</code> slices its bench at <code>teamA.slice(2)</code> , so a two-body team had an EMPTY bench, the intended bench Pokemon was standing on the field as the partner, and the switch silently did nothing — every arm identical, which is exactly what a dead mechanic looks like. CUTE CHARM read 0 fires in 4,166 trials and was filed ABSENT because every fixture declared <code>gender: 'N'</code> and the authority gates it on two real genders. RIVALRY , same cause. A LEGAL BOARD THAT CANNOT FIRE ITS TRIGGER IS INDISTINGUISHABLE FROM A DEAD MECHANIC — the inverse this file's own header says has no name. Four clauses added, all DERIVED from the handler rather than from a list: a switch trigger against <code>teamSize</code> , a gender-gated mechanic against two declared genders, a status-reading mechanic against a declared status, a weather-gated one against a declared sky. SHOWN RED ON THE THREE BOARDS THAT ACTUALLY FOOLED SOMEBODY, plus the fixed version of each, plus two regressions — eight cases, all behaving. AND RUNNING IT RED CAUGHT TWO OF MY OWN ERRORS THAT WOULD OTHERWISE HAVE SHIPPED. (1) The gender clause tested whether the ability's own handler mentions <code>.gender</code> — Rivalry does, so it passed its worked example, while Cute Charm does not read gender at all : it applies the <code>attract</code> volatile and ATTRACT'S condition is the gate. The clause now follows the volatile one hop down. (2) That chase matched single quotes only , and the compiled <code>dist/sim</code> uses double — the IDENTICAL bug that made the whole refusal family come out empty in <code>million_targets.js</code> earlier the same day , repeated within the hour. A guard that had only ever been green would have passed Rivalry, missed Cute Charm and looked finished	closed — engine	
#230	THE OHKO CLASS TOOK EVERY ACCURACY MODIFIER AND SHOULD TAKE NONE — WILL, 2026-08-12, CLOSED 2026-08-12. Will: <i>"i dont think the ohko moves can ever be boosted by accuracy, only no guard"</i> . He is right, and <code>hitStepAccuracy</code> says so STRUCTURALLY rather than as an exception: <code>if (move.ohko)</code> sets the accuracy outright and the ModifyAccuracy event AND both stage adjustments live in the <code>else</code> , so nothing in the modifier pipeline is reachable for the class. The <code>Accuracy</code> event sits BELOW the branch and does still fire — which is exactly why No Guard and Lock-On work on Fissure and nothing else does. <code>hitChance</code> had no <code>ohko</code> branch at all: every <code>ohko</code> mention in the simulator was in the DAMAGE path (<code>_fd.source==='ohko'</code> returns the target's HP), none in the accuracy path. HALF OF THIS WAS A REGRESSION FROM THE SAME DAY — the evasion-stage half had been wrong for the life of the function, and adding Gravity to <code>ACCMOD</code> hours earlier gave a standing hole a second way to be wrong on the same 141 corpus uses (Fissure 70, Sheer Cold 64, Guillotine 5, Horn Drill 2); fixing only the half I introduced would have left the older one behind. <code>tag_dex</code> 's <code>ohko</code> param was <code>{ohko: true}</code> , which FLATTENED a value Showdown reads twice: it stores <code>ohko: 'Ice'</code> for Sheer Cold and uses that string both for the accuracy (20 rather than 30 when a non-Ice body clicks it) and for an outright immunity (<code>!target.hasType(move.ohko)</code> emits <code>-immune [ohko]</code> and never rolls). Both rules were underivable and the engine had neither. The param now carries <code>m.ohko</code> itself plus <code>typeGate</code> , <code>acc</code> and <code>accIfNotGateType</code> , all derived; the level term is omitted because Champions is Level 50 throughout, so <code>pokemon.level - target.level</code> is identically zero — a format fact, not an approximation. THE WHOLE-GAME DIFFERENTIAL CANNOT SEE ANY OF THIS AND NEVER WILL : both arms pin accuracy to a corner, so a computed accuracy never decides an outcome, which is why this class needs its own probe rather than a bigger sample. <code>tests/test-ohko-accuracy.js</code> derives every expected number from the format on each run and was shown RED on a deliberate break first — with the branch disabled it fails 8 of 11 checks, and <code>MEDSEEN.ohkoAccuracy</code> and <code>MEDSEEN.ohkoImmune</code> are counted separately because the immunity and the 20-vs-30 split are two different reads of one string and one can be wired while the other is not	closed — engine DEFECT	
#231	THE COMPARATOR COULD NOT SEE ITS OWN LARGEST CLASS: OUR ENGINE GOING QUIET READ AS OUR ENGINE BEING RIGHT — 2026-08-12, CLOSED 2026-08-12. The aligner walked <code>Math.min(a.length, b.length)</code> and stopped, so a <code>medicham2</code> stream that is a strict PREFIX of Showdown's agreed all the way to its own end. The instrument said so itself and was not read : <code>planted_divergence_proof_ok: false</code> sat in <code>data/game-differential.json</code> beside the divergence counts, with the MISSING-event plant recorded <code>applied: 1, caught: false</code> —	turn	13 — the authority anr playing. Shipping that rule as an engine bug, streams are trimmed to then compared, EXCEPT did not — there the ex would re-hide it. endl classes, deliberately we halted, <i>missing</i> means wc

#	item	note	
	placed and genuinely undetected, not a mis-aimed plant. The file's own code prints <i>THE COMPARATOR FAILED ITS OWN PROOF</i> — <i>everything below is worthless</i> , and I quoted 260/983 = 26.4% out of that run all evening, into the CHANGELOG and three commit messages, having read the counts and not the line above them. That is the caption-is-not-a-quarantine rule broken by the session that had just re-read it. THE BLIND SPOT WAS AIMED AT THE BIGGEST TARGET: event missing from medicham2 is the largest class in BOTH arms (96 and 88 games), and the end-state work hit the same wall from the other side — at a 40-turn cap, 937 of 983 games came back <i>ONLY medicham2 ended the battle</i> . THE FIRST FIX WAS WRONG AND MEASURING IS WHAT CAUGHT IT: a bare length test fired on 9 of 9 games, and the unmatched line was `		and the first is worse because different. All three plants no
#232	PROTECT IS THE LARGEST SINGLE DIVERGENCE IN THE ENGINE AND willAct() IS POSITIONAL HERE AND BY ACTION KIND THERE — 2026-08-12, CLOSED 2026-08-12. engine/divergence_report.js ranks by corpus usage rather than by how often the swarm staged it, and Protect tops the list at 100,806 clicks across 12 distinct causes , with the worst reading `	-singleturn	p2a

#	item	note
#233	MEDICHAM HAS NO GENDER AT ALL, SO TWO ABILITIES CANNOT BE TESTED AND ONE OF THEM IS A LIVE BLUFF — 2026-08-12. <code>game_differential.js</code> hard-codes <code>gender: 'N'</code> on every body of both sides and its own comment gives the reason: <i>"medicham2 has no gender at all, so a declared gender would part the streams on line one of every game."</i> That is a correct decision for the differential and a dead end for the mechanics runner: Cute Charm and Rivalry can never fire in any fixture this harness builds, and no amount of preflight repair changes it — the preflight now labels them CANNOT-FIRE-IN-THIS-FIXTURE with the clause named, which is the honest state rather than a fix. Cute Charm was separately measured at 30.92% once a staged fixture DID carry genders, against 0 of 4,166 read as "absent" before — so the mechanic works and only this harness cannot reach it. THE COST IS NOT THEORETICAL: <code>rivalry</code> is one of the 13 roster rows whose control arm is itself a live ability, and it came back FIRED on a genderless board, which its own guard makes impossible — an over-match that turned out to be CONTROL-NOT-QUIET arriving through a new door rather than a broken clause. Fixing this needs gender in <code>medicham2-browser.js</code> FIRST and then in <code>game_differential.js</code>; doing the second without the first is what the hard-coded <code>N</code> is protecting against	open — engine
#234	THE [from] ATTRIBUTION IS 28% OF ALL DIVERGING MECHANIC USAGE AND IT IS ONE FIX — CLOSED 2026-08-13. Engine-complete: one derivation (<code>ATTR</code> in <code>engine/medicham2-browser.js</code> , five shaped readers off the authority's own emit sites), ten probes, census 542 -> 552 live / 0 missing. On release <code>084c86beb8d2</code> : moves 76 -> 57, abilities 33 -> 29, items 3 -> 3. 23 of the 25 rows closed, no new row appeared, and the sizing above re-derived on the fresh artifact came out at 25 rows / 9,332 uses / 30.2% — still the largest single family. TWO REMAIN AND NEITHER WAS ATTRIBUTION: <code>disable</code> (914) parts on field 4 carrying the DISABLED MOVE, not a cause; <code>afteryou</code> (223) now parts further down the stream on a pre-existing defect this fix exposed. THREE MORE WRONG TAGS WERE FOUND WHILE READING AND ARE FIXED: every status-curing item and Healer wrote <code>[from]</code> <code>item:</code> / <code>[from]</code> <code>ability:</code> onto a <code>-curestatus</code> the authority reaches through a bare <code>cureStatus()</code> (<code>[msg]</code>). See	-status
		pXa

#	item	note
	docs/ENGINE.md 2026-08-13. ORIGINAL ROW: Grouping <code>data/all-mechanics-fire.json</code>'s 116 diverging rows by the SHAPE of what differs rather than by move name collapses them to a handful of families, and the largest by a distance is a <code>field 4</code> mismatch of one kind: the authority NAMES THE CAUSE of a line and we emit the bare line. `	
#235	FROZEN RELEASES AGE OUT OF THE LOADER, AND 168 OF 196 CAN NO LONGER BE OPENED — 2026-08-12. <code>tests/staged_status_counters.js</code> exited 1 with all 11 scenarios reading release <code>THREW</code>, which looked like an engine failure and was a baseline that had aged out. The requirements are LAYERED and each refusal names only the NEXT missing symbol: the original pin <code>6b5447db1738</code> predates <code>ROADMAP #222</code>'s split RNG streams (<i>"the stall counter would silently re-couple to the accuracy pin"</i>), and the first re-pin cleared that and immediately threw on <code>spreadL50</code> (<i>"every body would be built blank while this file believes it filled a spread"</i>). Both guards are CORRECT and neither may be worked around — a BEFORE arm with welded dice or blank spreads measures something other than what the file compares, and does it quietly. 30 of 196 snapshots export rngStreams, 28 export spreadL50, 28 export both, so 168 frozen engines verify, hold their bytes, and cannot be run — the same <code>DIGEST-SET-IS-NOT-A-LOADABLE-ENGINE</code> distinction <code>engine_release.js</code> learned three times (<code>mc_key</code>, <code>move-effects</code>, <code>pp.js</code>), arriving from the other direction: not a snapshot missing a file, but a loader that grew past the snapshot. Re-pinned to <code>6155acc0fb26</code> (2026-08-12T19:54), the OLDEST that satisfies both rather than the newest that loads — this file has twice silently re-pointed its BEFORE arm at a fresh cut and turned the comparison into a copy of the AFTER arm, and "newest that loads" is that failure on purpose. Result: 10 of 11 scenarios went from unmeasurable to release IDENTICAL / live IDENTICAL. The coverage lost is real and named: the new baseline postdates the slot binding, Fairy Aura, Raging Bull, Gravity and the OHKO branch, so anything this file would have caught in that window is gone. THE RESIDUE: <code>guts-is-not-halved-by-its-own-burn</code> throws on BOTH arms, so it is a fixture defect rather than a baseline one, and it was invisible while all eleven were throwing — handed out separately. FIXED 2026-08-12, CLOSED 2026-08-12: <code>rngStreams</code> and <code>spreadL50</code> are now in <code>game_differential.js</code>'s <code>need</code> list, so an aged-out release is refused AT LOAD with EVERY missing symbol named at once — <code>release 6b5447db1738</code> was frozen before <code>engine/medicham2-browser.js</code> exported: <code>natureL50</code>, <code>rngStreams</code>, <code>spreadL50</code> — instead of one symbol per full run. The deep checks stay: they document WHY each symbol matters, which a name in a list cannot, and they are the backstop if a caller ever bypasses the loader. AND THE TOOL FOR THIS ALREADY EXISTED: <code>node engine/engine_release.js compat <file></code> <code><symbols...></code> reports every release as PROVIDES / LACKS / PRUNED / UNLOADABLE, and the new refusal prints that exact command. I hand-rolled a scan script to rediscover what it already answers, which is the same shape as the mechanics runner existing and nothing gating on it. Its verdict confirms the count: 27 of 200 can serve this caller, 4 pruned, 168 predate an export, 1 broken — <code>5e8c391b3f04</code> is UNLOADABLE with Unexpected token <code>'</code>' , a release cut from a tree mid-edit	closed — measure
#236	A POISON-TYPE'S TOXIC CANNOT MISS AND OURS ROLLS FOR IT — 2026-08-12. Found while auditing what survives a switch, unrelated to switching, and deliberately NOT folded into that batch. <code>sim/battle-actions.ts</code> sets <code>accuracy true</code> outright for <code>move.id === 'toxic' && gen >= 8 && pokemon.hasType('Poison')</code> — it sits in the SAME condition as <code>move.alwaysHit</code>, above the <code>Accuracy</code> event, so nothing downstream can reintroduce a roll. We have no such branch, so a staged <code>Poison-type's Toxic</code> emitted `	-miss
		where the authority's carries <code>**1209 corpus</code> learn it, so the exemp **IT IS DETERMINISTIC, SUGGESTS**: this is no move that hits every t single staged fixture CANNOT SEE IT**: both accuracy never decides hid the OHKO branch (# belongs with the never arithmetic in <code>hitChance</code> as a special case insi Landed exactly there — branch (#230), because <code>(move.ohko)</code> and <code>OVERWR</code>I and our OHKO branch RE Derived, not typed: th of <code>BattleActions.prototype</code> printed before wiring FIRST PARSER OVER-MATC splitting the conditio
#238	KING'S SHIELD BLOCKED STATUS MOVES HERE AND MUST NOT — CLOSED 2026-08-13. <code>Battle#checkMoveBypassesProtect</code> (<code>sim/battle.ts:1300</code>) takes a FOURTH argument, <code>blockStatus = true</code>, and its first clause is <code>!(move.category !== 'Status')</code>	<code>blockStatus)</code> && <code>move.flags[it and **KING'S SHIELD</code> (<code>data/moves.ts:9933</code>; t unchanged and only set Toxic, Taunt, Will-O-W Shield and are refused Bunker. **THIS ENGINE before a byte changed, Aegislash: King's Shie

#	item	note	
#239	**A STATUS REFUSED BY AN IMMUNITY IS `	-immune	AND WE WROTE
#237	SOME MECHANICS ARE HARDCODED EXCEPTIONS IN THE SIMULATOR, NOT HANDLERS, AND EVERY DERIVATION BUILT TODAY IS BLIND TO THEM — WILL, 2026-08-13. He asked whether the ability that poisons Steel types is even in the regulation. It is: Corrosion, legal, carriers Salazzle and Glimmora, 36 corpus uses — and it has ZERO on* handlers . It is not an ability hook at all. It is a name written into the immunity check itself, <code>pokemon.js:1257</code> : <code>!(source?.hasAbility("corrosion")) && ["tox","psn"].includes(status.id))</code> . THIS IS A CLASS, NOT A CURIOSITY . Ten abilities are named directly in <code>dist/sim/*.js</code> rather than reached through a handler — sheerforce, klutz, prankster, multitype, megasol, levitate, guts, eeelevate, dancer, corrosion — and three carry NO handlers whatever. Levitate is the expensive one: 3,186 corpus uses and 0 handlers . Every derivation this project built on 2026-08-12 reads HANDLERS — the per-ability trigger derivation (<code>PRE.moveNeeds</code>), the preflight trigger clauses, the ACCMOD conformance sweep, the residual-order table. All of them are structurally blind to this shape , so the abilities arm cannot stage Levitate or Corrosion from a handler read, and a sweep that reports "no handler, nothing to check" is reporting its own blind spot as a clean bill. <code>data/tags.json</code> does carry rows for both, so <code>tag_dex</code> reached them by another route — but that is a second derivation happening to cover the gap, not coverage by design, and nothing checks that the two agree about which abilities exist. THE CONCRETE COST, MEASURED THE SAME NIGHT : a diverging game showed Toxic into a Steel Scizor — the authority emits <code>-immune</code> , we emit <code>-miss</code> , because our poison immunity check runs AFTER the accuracy roll instead of before it. Fixing that as an absolute immunity would have been wrong in the opposite direction, since a Salazzle Toxic into a Steel body LANDS. Will caught that before it shipped. The ordering fix and the Corrosion exemption went to the agent already in <code>hitChance</code> ; the general sweep — which mechanics in this format are hardcoded exceptions, and does anything verify our tags cover them — is NOT started	open — engine	
#240	SHOWDOWN RE-SORTS THE ACTION QUEUE ONLY WHEN THE NEXT ACTION IS A MOVE, AND WE RE-SORT ALWAYS — WILL, 2026-08-13. Found by him reading a diverging card: a turn where all four slots switch, Tyranitar arrives with Sand Stream, and the two engines order the remaining two switches oppositely. His hypothesis was that the sand turns on Excadrill's Sand Rush and makes it suddenly faster, and the mechanism is exactly right while the direction is inverted — it is OURS that applies it . <code>battle.js:2403</code> guards the re-sort: <code>`if (this.gen >= 8 && (this.queue.peek()?.choice === "move"</code>		<code>this.queue.peek()?.choice == this.queue.list)</code> if (a.pokemon after EVERY action, but action is a move — so speeds are never recom turn start, before the chainModify(2) if sandstorm SHOWDOWN'S OWN ORDER, before Excadrill's, wh start; we reverse that

#	item	note	
			the weather. Base spee Excadrill first either not the argument. **TH THE OPPOSITE DIRECTION scan to the shield's own actic other half — the authority do is the fact, and both sites are will keep contradicting each DO NOT SEE THE WEAT fires and Sand Rush DOES a opposite answers depending
#241	PART (3) ONLY: EIGHT CAUSES WHERE THE AUTHORITY ANNOUNCES A FAILURE AND THIS ENGINE SAYS NOTHING — THE ROW WAS SHRUNK TO ITS REMAINDER 2026-08-15 (MEASURE). THE OLD TITLE NAMED THE PART THAT IS FINISHED, which is precisely how a register comes to cost an agent: it read "A <i>GENERIC -fail NAMES THE MOVE'S USER AND WE NAME THE TARGET — WILL, 2026-08-13</i>", and the attribution case was discharged on 2026-08-15 with both streams matching byte for byte (the account is kept in full further down this row). THE COUNT IS AN UPPER BOUND AND SAYS SO RATHER THAN BEING QUOTED FLAT: the eight causes are read out of <code>data/game-differential.json</code>, generated 2026-08-13T07:36Z against release <code>0c5bb83c5744</code> — which predates #241(1)+(2), #255, #256, #259 and the attribution fix, five landings on the very emission paths this row is about. Nobody has re-run the top-tie-first differential since, so the honest statement is <i>at most eight, class confirmed live, exact count owed</i>. THE CLASS WAS CONFIRMED IN THE SOURCE, NOT IN THE STALE ARTIFACT: <code>medicham2-browser.js</code> carries a dated RETRACTION of the one <code>mvFail</code> that would have covered part of it — <i>**a staged Encore into a body that has never moved emits `</i>	-fail	p1a: Whimsicott on the a back the day it landed across the two arms, s than merely unnoticed. carrier (Whimsicott x3 WHOLE:** Third sightin clicks Thunder Wave in

#	item	note

#	item	note

#	item	note

#	item	note

#	item	note
#242	A DURATION-ONLY EFFECT EXPIRES FROM INSIDE THE RESIDUAL WALK AND OUR DERIVED ORDER TABLE CANNOT SEE IT — WILL, 2026-08-13. <i>"its the same problem of when tailwind officially ends and how that affects leftovers and residual damage"</i>, said of a card where Showdown runs Leftovers, Leftovers, the Infestation chip and THEN -sideend Tailwind, while we announce the Tailwind expiry FIRST. Same family as the Trick Room-end vs sandstorm-chip ordering he called earlier. CHASING IT FOUND THE HOLE IN THE INSTRUMENT, WHICH IS WORTH MORE THAN THE CARD. engine/residual_order.js derives the walk by enumerating effects that own an onResidual handler — 42 of them. Tailwind owns none. It is in the authority's walk purely because it has a <pre>duration : findSideEventHandlers(side, "onSideResidual", "duration") collects it, and then battle.js:340 fires the expiry at its SORTED POSITION — if (eventid === "Residual" && handler.end && handler.state?.duration) { handler.state.duration--; if (!handler.state.duration) { handler.end.call(...); continue; } } . So every duration-only effect in the format — Tailwind, Trick Room, the screens, Safeguard, Mist, Lucky Chant — expires from inside the residual and is INVISIBLE to the table we derive the order from. The 42-row table is not wrong; it is answering a narrower question than its name suggests, which is the Dex.forFormat -is-not-a-legality-filter mistake in a new place. TWO JOBS, AND THE SECOND IS THE REAL ONE: place the expiries correctly, and make residual_order.js enumerate by duration as well as by handler so its row count is the population rather than a subset. A THIRD DEFECT SITS ON THE SAME LINE: ours reads `</pre>	-sideend
		p2:

#	item	note
#243	THE LIVE FALLEN COUNT UPDATES AT TURN END, SO A LAST RESPECTS FIRED AFTER AN ALLY DIED THAT TURN IS ONE DEATH LIGHT — WILL, 2026-08-13. He asked directly: <i>"does last respects update mid turn if an ally dies first and then last respects hits"</i>. In the authority, YES, and the two fallen-count mechanics answer it OPPOSITELY — which is the part worth holding on to. Last Respects evaluates its <code>basePowerCallback</code> inside <code>getDamage</code> (<code>battle-actions.js:1375</code>), i.e. at the moment	CLOSED 2026-08-13 — <code>docs/ENGINE.md</code> . <code>fallenSettle(S)</code> derives both sides' counters at each of the authority's own moments: at <code>battleInit</code> after the roster is stamped, at the top of the action loop beside <code>flushAfterMoveSpends / receiverSweep</code> and ABOVE the <code>sideWiped</code> break, after the last action, and once more in the residual so the Supreme Overlord snapshot includes what the residual killed. One function, four call sites. Authority measured first at seed [1,2,3,4]: Houndstone's Last Respects after a Garchomp KOs the Milotic ally earlier in the same turn deals 69 against 36 with the ally alive — x1.917, BP 50 -> 100 —

#	item	note
	the move hits, and <code>side.totalFainted++</code> happens inside <code>faintMessages()</code> (<code>battle.js:2092</code>), which <code>runAction</code> calls after EVERY action. So an ally that dies earlier in the turn is already counted. <code>Supreme Overlord</code> reads the same field ONCE in <code>onStart</code> and freezes it in <code>effectState.fallen</code> , and never re-reads — so a body dying while Kingambit stands there does nothing for it, and only a Kingambit entering afterwards benefits. WE MODEL THE SPLIT CORRECTLY AND THE TIMING WRONG: <code>att._sf.fainted</code> is live and <code>fallenStuck</code> is the entry snapshot, exactly right, but <code>sFA.fainted = fallenCount(...)</code> runs once per turn in the end-of-turn block, whose own comment says so — <i>"a mid-turn kill is seen one action late — an approximation, stated"</i>. Declared rather than hidden, which is the right way to carry a gap, and it is still wrong: Last Respects is +50 base power per death, so being one death behind is 100 where the authority says 150, a third of the move off. INVISIBLE TO EVERY EXISTING TEST, because the census probe stages the death on a PREVIOUS turn (<i>"one ally really fainted first 1,101"</i>) and the whole-game differential pins the dice so the ordering that produces a same-turn kill is rare. The fix is to recompute where the authority does — after each action, not at turn end	while Iron Head from a Kingambit ALREADY OUT deals 70 against 70, x1.000. This engine read x1.000 on both and now reads x1.980 and x1.000. The split survived and is asserted: <code>ability boostsFromFallen</code> gained a probe that asserts Supreme Overlord did NOT become live, shown RED under a deliberate break (<code>_fallenStuck</code> swapped for <code>_sf.fainted</code>) because a probe that asserts a negative passes on the unfixed engine too. Census 561 -> 564 live / 0 missing. Whole-game differential BEFORE (<code>a2857a1bfd70</code>) and AFTER (<code>0c5bb83c5744</code>), releases verified to differ in exactly one file, same pool, 1219 games, both arms: 220/239 diverged in both, every class row identical to the instance — nothing moved, which is expected because Mode A's pins rarely produce a same-turn kill followed by a Last Respects, and is not a strength claim. The <code>Math.min(fainted,5)</code> at the read site was checked rather than assumed: the authority has NO cap on this move, ours first binds at six fallen allies, a bring-4 game tops out at three, so it is unreachable and is left in place with a comment naming it
#244	THE ROLLOUT SEED DROPS THE DEAD, SO EVERY PLAYOUT BELIEVES NOBODY HAS DIED — WILL, 2026-08-13, AND HIS INSTINCT REDIRECTED THIS ONE. I first reported that <code>board.js</code> prices Last Respects at a flat 50 because <code>movePower</code> 's stub has no <code>side.totalFainted</code> (measured: the callback returns <code>NaN</code>, the <code>bp > 0</code> guard rejects it, and it falls through to the printed 50). <i>**He pushed back — "wait rollout shouldnt be thinking like that? it should just be playing the games out right?" — and he was right.</i> <code>rollout_leaf.js</code> does not score a snapshot; it seeds MEDICHAM from the real board and PLAYS THE POSITION OUT, so <code>board.js</code> 's <code>movePower</code> is not in that path at all and my claim about rollouts was wrong. THE REAL DEFECT IS IN THE SEED AND IT IS WORSE. <code>buildSide</code> drops fainted bodies — <code>if (m && m.fainted) { stats.fainted++; continue; }</code> — correctly, because a corpse cannot act. <code>battleInit</code> then stamps <code>S.sFA.team = teamA.filter(Boolean)</code> , which is now SURVIVORS ONLY, and <code>fallenCount</code> is <code>sf.team.filter(x => x.fainted).length</code> → 0. So a rollout from a position where two of your team are already dead plays out with Last Respects at 50 instead of 150 and a Kingambit with no Supreme Overlord boost, and the deeper into a game the position is, the wronger it gets — precisely the phase both mechanics exist for. AND THE FIX FOLLOWS FROM WILL'S STATEMENT OF THE DESIGN, NOT FROM THE SYMPTOM. <i>** 2026-08-13: "miltanks rollout needs to just play the game out on medicham and have it match showdown perfectly thats the whole point. miltanks just chooses the actions."</i> That rules out the obvious patch. Threading a fallen COUNT through <code>battleInit</code> as an extra field would make this card right and would still leave the seed handing MEDICHAM a position that does not exist in the game — a side with four bodies where the real side has six, two of them dead. The seed must reconstruct the TRUE position, corpses included, and let the simulator derive everything it derives in a real battle. Anything the payout has to be TOLD is a second source for a fact the engine already owns, which is the rule this project breaks most expensively. A corollary worth stating while it is cheap: every other approximation on the seeding path is a defect by the same argument, and they should be swept for rather than waited for — the seed is the only place a correct simulator can still produce a wrong game. THIS IS THE PATH MILTANK ACTUALLY USES, which the <code>board.js</code> feature defect is not. <code>board.js</code> 's <code>movePower</code> fallthrough is real and separate and is the SMALLER half: it affects the feature vector MAG scores an action with, and it is a whole CLASS — 29 moves in this format carry a <code>basePowerCallback</code> , and the stub satisfies the weight and speed ones while silently returning the printed value for anything reading side or battle state. VERIFIED BY: <code>node tests/test-rollout-fallen.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	CLOSED 2026-08-13 (the seed) — SEARCH, R13; and 2026-08-13 (turn one) — ENGINE, #246. <code>rollout_leaf.buildSide</code> appends the roster's corpses, <code>battleInit</code> calls <code>fallenSettle</code> , and <code>tests/test-rollout-fallen.js</code> proves it dice-for-dice: Last Respects at 50/100/150/200 against a position holding 0/1/2/3 in the ground, with the 0 row as the control that says the fix does not invent deaths. REACH MEASURED, THEN EFFECT MEASURED: 8.75% of 183,840 decision points (<code>data/rollout-fallen-prevalence.json</code>), converting to an argmax flip on 6.1% of 131 paired decision points under common random numbers (#278, <code>data/argmax-paired.json</code>) — and three of that arm's eight sampled flips have Last Respects on the board. THE ROW'S OTHER HALF IS NOT CLOSED AND IS NOW #283, deliberately: <code>board.movePower</code> still builds no <code>side.totalFainted</code> , so the FEATURE vector prices the move at its floor. A row closed in part is a row whose leftovers have no home, which is the shape #269 left behind and #277 had to be opened for. The remainder is FIT-INVALIDATING and that was measured rather than assumed — <code>engine/feature_fixture.js</code> 's 58 per-feature column hashes MOVE when <code>movePower</code> 's callback path is disabled — so it waits for the refit, which is gated behind MEDICHAM by Will's own instruction
#245	THE GUARD BUILT FOR THIS EXACT BUG CANNOT SEE IT, BECAUSE IT WATCHES FOR AN ABSENT ROSTER AND THIS ONE IS PRE-FILTERED — 2026-08-13. <code>fallenCount</code> was written to fix an earlier version of #244 and its comment states the principle: <i>"the one case where the roster is absent is COUNTED rather than falling silently back to the arrays that caused this, because a quiet fallback here is indistinguishable from the bug."</i> Correct, and one shape too narrow. <code>MEDFAILS.fallenNoRoster</code> increments only when <code>sf.team</code> is missing or empty. In the rollout the roster is PRESENT and NON-EMPTY and has simply had the dead removed, so the counter stays at zero and <code>fallenCount</code> returns a confident 0. A capability counter that reports success while the capability is absent is this project's named failure mode arriving through the door its own guard was built to close. The fix is not another counter: the count must come from something that cannot be silently filtered — the seeded position knows how many have fallen and should SAY so, rather	CLOSED 2026-08-14 — SEARCH, R19. THE GUARD WAS IN THE WRONG FILE AND NO COUNTER WRITTEN INSIDE IT COULD EVER HAVE FIRED: <code>fallenCount(sf, act, bench)</code> sees three arrays, and nothing computable from them distinguishes <i>nobody died</i> from <i>somebody pruned the corpses before I was called</i>. So the check moved to the seam where both the seeded roster and the real position are in scope. <code>rollout_leaf.checkFallenSeeded</code> compares what the engine will actually COUNT — <code>S.sFA.fainted</code> , the field Last Respects and Supreme Overlord read — against <code>board.graveyard[side]</code> , a record <code>board.faint()</code> writes and that nothing on the seeding path filters. It is called at BOTH <code>battleInit</code> sites including the ranking path, it counts (<code>FALLEN_GUARD.checked</code> is the proof-of-firing, <code>mismatch</code> is the claim) and it SPEAKS once per process rather than once per rollout, which is how the last warning of this shape died. THE RED DEMONSTRATION IS A STANDING ASSERTION RATHER THAN A BREAK DONE ONCE BY HAND: <code>tests/test-rollout-fallen.js</code> rebuilds the pre-filtered roster and asserts, in the same block, that <code>MEDFAILS.fallenNoRoster</code> does NOT move while the new guard fires with both numbers named — and it was shown red twice besides, on the corpses

#	item	note	
	than have it re-derived from a list somebody else pruned. AND A NUMBER IN THAT SAME COMMENT IS STALE: it says Last Respects has 19,299 corpus uses; <code>data/tags.json</code> says 6,650 . One of the two is wrong and neither is derived at read time — the same prose-outliving-what-it-described shape as the fourteen handoffs, sitting inside the fix for a silent-fallback bug. VERIFIED BY: <code>node tests/test-rollout-fallen.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	being dropped (17 fail) and on the guard being disabled (8 fail). THE STALE NUMBER IN THE SAME COMMENT IS RE-DERIVED AND IS NEITHER OF THE TWO QUOTED: the comment says 19,299 Last Respects uses and this row said 6,650; <code>data/tags.json</code> says 6,970 today. It sits in <code>medicham2-browser.js</code> , which SEARCH may not edit, and is filed as #284	
#246	battleInit STAMPS fainted: 0 AND THE RECOUNT IS AT TURN END, SO THE FIRST TURN OF EVERY PLAYOUT IS STILL A TURN WHERE NOBODY HAS DIED — 2026-08-13. HANDED BACK BY SEARCH. #244 is closed on the SEED side: <code>rollout_leaf.buildSide</code> now appends the roster's corpses, <code>sfA.team</code> holds all six, and <code>fallenCount</code> returns the truth — proved by <code>tests/test-rollout-fallen.js</code> , which was shown RED first (12 failures, <code>fallenCount -> 0</code> at every N) and is green at N=1,2,3. It is green from turn TWO. <code>battleInit (medicham2-browser.js:11698)</code> writes the literal <code>sfA:{fainted:0,...}</code> and the only recount is <code>sfA.fainted=fallenCount(sfA,actA,benchA)</code> in the end-of-turn block, so at t=0 the count is 0 whatever the roster says. THAT IS THE DECISION-RELEVANT TURN: <code>rolloutAfterActions</code> forces the candidate click on turn 1, so a Last Respects being RANKED is priced at 50 while the position says 150. Measured on the fixture at N=3: 50 BP where the roster says 200. THE FIX IS ONE LINE — derive it where the literal is, fainted: fallenCount(...) after the roster is stamped, exactly as the turn-end block already does — and it is in ENGINE's file, which SEARCH may not edit while ENGINE holds it. It needs nothing further from SEARCH: the roster it would read is now correct. Same family as #243 (the count is one action late) and #245 (the guard cannot see either). VERIFIED BY: <code>node tests/test-rollout-fallen.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	CLOSED 2026-08-13 — <code>docs/ENGINE.md</code> , in the same pass as #243 because they are the same field in the same file and closing one alone leaves the count right at turn end and wrong at turn start. <code>battleInit</code> now calls <code>fallenSettle(5)</code> immediately after the two <code>team</code> writes. Measured on the probe before and after, with the knob varied FOUR ways because two arms cannot tell "the count is read" from "three corpses happen to be worth something": Last Respects on the FIRST action of a battle seeded with 0/1/2/3 already fallen dealt 51/51/51/51 and now deals 51/101/151/201 — 1x/2x/3x/4x, which can only be 50 + 50n read live. Identical results across a varied knob meant the knob was unwired, and it was: the literal <code>sfA:{fainted:0,...}</code> was the value a side has BEFORE it has a party, left standing as the value it has after one was handed in. #247 is NOT closed by this and was not touched: even a correct <code>sf.fainted</code> at t=0 is not what Supreme Overlord's entry snapshot wants	
#247	SUPREME OVERLORD'S ENTRY SNAPSHOT IS UNSEEDABLE, SO EVERY SEEDED BODY ENTERS THE PLAYOUT WITH _fallenStuck: 0 — 2026-08-13. Found sweeping the seeding path for #244. The ability reads a SNAPSHOT frozen when the body entered (<code>bringIn</code> stamps <code>nx._fallenStuck=sf.fainted</code>), which #243 records as the correct split. A body placed on the field by <code>battleInit</code> never went through <code>bringIn</code> , so it carries <code>buildMon's _fallenStuck: 0</code> for the whole playout — and #246 does not close this one, because even a correct <code>sf.fainted</code> at t=0 is not what the snapshot wants. THE FACT IS NOT ON THE BOARD AT ALL: <code>board.js</code> records <code>graveyard</code> (who is dead) and nothing about WHEN each body entered relative to those deaths, so this cannot be reconstructed by SEARCH from the position. It needs the board to remember the fallen count at each switch-in, which is upstream of both divisions. Small: one carrier in the regulation, Kingambit, 94 corpus uses of the ability against Last Respects' 6,650. VERIFIED BY: <code>node tests/test-rollout-seed.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	CLOSED 2026-08-13 — SEARCH. The row's own sentence was the fix: <i>"it needs the board to remember the fallen count at each switch-in"</i> . <code>board.switchIn</code> stamps <code>enteredWithFallen: graveyard[side].size</code> — the count AT THE MOMENT OF ENTRY, and <code>faint()</code> writes the graveyard before the replacement arrives, exactly as <code>faintMessages()</code> increments <code>side.totalFainted</code> before Showdown asks for one — and <code>rollout_leaf.seedHistory</code> copies it onto <code>_fallenStuck</code> . IT IS SEEDED IN buildSide AND DELIBERATELY NOT IN dmgMon: _fallenStuck multiplies base power inside <code>dmgRange</code> , so putting it in the shared builder would have moved fitted FEATURE values and silently invalidated <code>data/policy-weights.json</code> . Gate <code>tests/test-rollout-seed.js</code> , shown RED first; the observable is DAMAGE, inverted out of a control table over the snapshot that is asserted injective before it is used as a lookup, and the CONTROL is #243's split — a carrier already standing there when its allies die still reads 0, so a fix that read <code>sf.fainted</code> for every body would pass the first row and fail this one. PREVALENCE MEASURED AND IT IS THE SMALLEST OF THE FOUR BY TWO ORDERS OF MAGNITUDE: 116 of 190,378 decision points, 0.061% (<code>data/rollout-seed-prevalence.json</code> , 14,102 open-sheet games; only 201 decision points have the carrier active at all, because Kingambit is almost always brought with Defiant). Correct and very nearly inert, said plainly rather than dressed up	
#248	A BENCHED POKEMON ENTERS EVERY ROLLOUT AT FULL HP, WITH NO STATUS AND THE DATASET'S MOVES — 2026-08-13. Swept out of the seeding path beside #244 and it is the biggest of the four. <code>rollout_leaf.sideTeam</code> synthesises a benched body as <code>{species, hp: 1, status: '', boosts: {}}</code> because <code>board.bench()</code> returns SPECIES NAMES and the board keeps no state for a Pokemon that is not on the field — <code>switchIn</code> builds a brand-new object each time and the old one is discarded. So a Pokemon that pivoted out at 20% and burnt comes back to the playout clean and whole, on every sample, on both sides. AND IT DOES NOT CARRY ITS DECLARED MOVES: the bench synthesis passes <code>item</code> , <code>nature</code> and <code>pp</code> from the sheet and NOT <code>moves</code> , so <code>dmgMon</code> leaves <code>buildMon's</code> dataset-representative four in place — the identical sheet-reaches-one-consumer-and-not-the-next shape CLAUDE.md records three instances of on 2026-07-30. HP and status are public information in a real battle and the fix is a board-side memory, not a rollout heuristic; the moves half is one field in <code>sideTeam</code> . BOTH CHANGE WHAT MILTANK CLICKS, so each owes its own arm rather than a free ride	CLOSED 2026-08-13 — SEARCH, BOTH HALVES, AND IT IS THE LARGEST OF THE FOUR. MOVES: <code>sideTeam</code> passes the sheet's <code>moves</code> into the bench synthesis and <code>dmgMon</code> already preferred them — one field, and the CONTROL is that a benched species with no sheet still gets the dataset four, so a closed-sheet game is unchanged. STATE: <code>board.switchIn</code> records the OUTGOING body's hp and status into <code>lastSeen</code> before it overwrites the slot, <code>board.benchState()</code> reads it back, and <code>sideTeam</code> seeds from it; the board could not answer this before because <code>switchIn</code> builds a new object and the old one was discarded. BOOSTS DELIBERATELY STAY EMPTY — Showdown clears stat stages on switch-out, so carrying them would be a new bug wearing the shape of a fix, and the gate asserts it. PREVALENCE (data/rollout-seed-prevalence.json, 14,102 open-sheet b03 games / 190,378 decision points): a live benched body is hurt or statused at 17.053%. THE MOVESSET SHARE IS WITHHELD as of 2026-08-23 — see #402 — the generator resolved a species through <code>norm()</code> , which strips the hyphen <code>MC.mons</code> keys every forme with, so every forme was handed an EMPTY dataset moveset and scored as "the moves differ". Inflation only, magnitude unpublished; the hurt-or-statused figure is computed independently and stands. Two residues declared rather than glossed: Regenerator and Natural Cure heal SILENTLY on the way out, so the memory is a pre-heal HP and a pre-cure status (erring low on HP, which is the safe direction); and the COUNTERS hanging off a status — sleep turns and the toxic ramp, both of which medicham2 models and carries across a switch — are not seeded at all, which is #267	
#249	THE SEEDED PLAYOUT HAS NO HAZARDS, NO SCREENS AND NO GRAVITY, AND NOTHING SAYS SO — 2026-08-13. Third sweep finding. <code>rollout_leaf.applyField</code> translates exactly four things onto the seeded battle — weather, terrain, Tailwind and Trick Room — and <code>battleInit</code> starts <code>sfA.hz</code> absent and <code>sfA.sc</code> empty. <code>board.js</code> HAS the rest: <code>startSide</code> derives Stealth Rock, Spikes, Reflect, Light Screen, Aurora Veil and Safeguard from the setter move and its dex duration, and they are readable through <code>hasSide</code> . So every rollout switch-in walks onto a clean field, every screen the real	-sidestart	(<code>magnemite.js:647</code>) and <code>environment-and-playin</code> place: MAG's <code>deadSide</code> hazards on the wrong s **VERIFIED BY: <code>node tes</code> gate that decides it, <code>engine/register_reality.js</code> r

#	item	note
	board is under is deleted, and a search that already had to be taught not to spam Protect is being shown a cheaper switch than the game offers. Same shape as the terrain defect \$3.7 already closed — a field the leaf was GIVEN never reaching the engine — except here the leaf is not even given it. AND CHASING IT FOUND A DEFECT UNDERNEATH, WHICH IS WORTH MORE THAN THE SEEDING GAP: <code>board.noteMove</code> STARTS EVERY SIDE CONDITION ON THE MOVER'S OWN SIDE (<code>board.js:2675</code> , <code>if (move.sideCondition) board.startSide(side, ...)</code>) and four of them belong to the FOE. Derived from the format, not recalled: <code>stealthrock</code> , <code>spikes</code> , <code>stickyweb</code> and <code>toxicspikes</code> are all <code>target: foeSide</code> , while <code>reflect</code> and <code>tailwind</code> are <code>allySide</code> . So on every OFFLINE board — the fit's board — a hazard is recorded on the side that laid it. The LIVE path takes the side off `	
#250	EVERY SEEDED BODY CAN FAKE OUT, BECAUSE <code>_mvActs</code> IS o ON A BODY THAT HAS BEEN OUT FOR SIX TURNS — 2026-08-13. Fourth sweep finding, and the cheapest to state: <code>firstTurnOnlyRefused</code> gates on <code>mon._mvActs</code> , <code>buildMon</code> sets it to 0, and <code>dmgMon</code> carries HP, status, stages, item, ability, moves and PP across but nothing about how long the body has been standing there. <code>board.js</code> tracks the quantity already — <code>turnsActive</code> , incremented in <code>endTurn</code> , and its own comment says it exists so the fit can learn Fake Out without the move being named — and the seed does not pass it. Fake Out is 16,871 corpus uses. Also unseeded from the same hole: choice lock, Encore, Disable, Taunt, Substitute, Leech Seed, the perish count and Protect's consecutive-use counter on the FOE (<code>protectTurns</code> is passed for my side only, <code>buildSide(board, foe, dex, stats)</code> takes no fourth argument). VERIFIED BY: <code>node tests/test-rollout-seed.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	CLOSED 2026-08-13 — SEARCH, AND <code>turnsActive</code> WAS THE WRONG QUANTITY, WHICH THIS ROW PROPOSED. Measured wrong in BOTH directions before a line was written: <code>endTurn</code> counts every turn a body is on the field, so a LEAD is already at 1 on the turn it may still Fake Out, and a body that entered MID-TURN is at 1 on the very turn it came out to use the move — which is exactly when Fake Out is clicked. So the count is taken where a move happens: <code>board.noteMove</code> increments <code>moveActs</code> , ABOVE the <code>worked</code> gate exactly as the PP spend is and for the same reason — Showdown increments <code>activeMoveActions</code> at the top of <code>runMove</code> , above every BeforeMove refusal — and <code>rollout_leaf.seedHistory</code> copies it onto <code>_mvActs</code> . It needs no assumption about when an adapter calls <code>endTurn</code> . Gate <code>tests/test-rollout-seed.js</code> , RED first; the observable is the FLINCH read out of a played turn's trace, not the field, because reading <code>_mvActs</code> back would only prove the seed wrote what the seed wrote. PREVALENCE 17.655% of decision points had a first-turn-only move offerable in the seed that the game refuses (48,221 points with such a carrier active). ONE GAP DECLARED: a turn spent asleep, flinched or fully paralysed emits a cant line and no move line, so <code>noteMove</code> never fires and this undercounts — it errs by leaving Fake Out legal a turn longer for a body that has not moved once since it entered, which is the direction the seed already erred in. The rest of this row's list — choice lock, Encore, Disable, Taunt, Substitute, Leech Seed, the perish count and the FOE's Protect streak — is split out as #269, unfixed
#251	MAG BECOMES A FUTILITY GATE AND STOPS BEING A LEARNED SCORER — WILL, 2026-08-13. <i>"im thinking of cutting mag back in size to say, no fake out after turn 1, no poison on steel, no status moves into gholdengo, no prio moves when farig is on the board... just very simple rules and not the whole convoluted, protect and let my partner ko the threat."</i> THIS FIXES AN ORDERING FAULT AND IS NOT MERELY A SIMPLIFICATION. A learned MAG prunes what is bad ON AVERAGE and runs BEFORE DODUO, so it deletes precisely the actions that are bad alone and good in combination — Fake Out into a spread, Follow Me covering a setup turn, a weak click that baits a Protect — which is the entire class DODUO exists to find. The safe line is FUTILE (no other action rescues it) versus BAD (conditional, must survive to DODUO). THE GATE MUST BE DERIVED, NOT WRITTEN, AND ALL FOUR EXAMPLE RULES ARE WRONG AS TYPED: Fake Out is <code>activeMoveActions > 1</code> — first turn OUT, not turn 1 of the game, so a Fake Out user switching in on turn 6 may still use it; Corrosion (Salazzle, Glimmora) poisons Steel; Good as Gold is <code>breakable</code> and refuses only when <code>target !== source</code> ; Armor Tail is one of Farigiraf's THREE abilities and exempts Perish Song, Flower Shield and Rototiller. Category right every time, hand-written form wrong every time — the ban-list-of-four shape, and a prune list is worse than a ban list because it fails silently: a deleted option leaves no trace in any output. So the gate asks the ENGINE "does this action produce any effect on this board" rather than holding rules. OPEN SHEETS DO NOT REMOVE THE NEED: Fake Out's condition is battle STATE and appears on no sheet, and the sheet stops being true — six legal moves change an ability mid-battle plus 11 Mold Breaker carriers and a Mummy, and Knock Off has 4,078 corpus uses. It reads the LIVE BOARD, which on turn one is the same object and by turn four is not. BUILDABLE NOW — the only stage of the pipeline with no fitted parameters, so the quarantine does not block it. Needs a counter for what it pruned and a check that a pruned action really was a no-op: an over-pruning gate deletes a good play and nothing catches it, because both sides of every comparison share the blind spot	open — search
#252	THE FUTILITY GATE CARRIES A DECLARED PREDICTION, AND WILL BROKE THE SAFETY ARGUMENT THE SAME	deferred by decision — search; NOT A DEFECT (audited 2026-08-15: nothing here says a mechanic behaves wrongly — it says a SEARCH gate makes an unrecorded

#	item	note
	NIGHT — 2026-08-13. <i>"of course there is forbidden tech of clicking a prio move when farig is on the board because youre making a prediction of it switching out."</i> The rule in #251 was stated as "no action rescues it", and that quantified over MY PARTNER'S actions only. Quantified over the OPPONENT'S as well, most of the gate's membership stops being unconditional: a move follows the SLOT, so switching the slot rescues it. Priority into an Armor Tail Farigiraf is dead only while that Farigiraf is still there when the move resolves; the same holds for Toxic into a Steel and any status into Gholdengo. TWO HALVES, ONE ASSUMPTION-FREE: the UNCONDITIONAL half fails on the USER's own state (Fake Out out of position, a Choice lock, no PP) and asserts nothing; the OCCUPANT-DEPENDENT half asserts <i>"they will not switch"</i>. Three of the four original examples are in the second half. The gate may still cut them — it is usually right and the expected cost is small in a format where the read is rare — but it is a PREDICTION and must be recorded as one. DEFERRED BY WILL'S DECISION, NOT MISSED: <i>"id rather not engage with those type of scenarios at the moment but make a note of it for things to possibly adjust much much later into the project."</i> When taken up, the home is not the gate: an action that is dead under one opponent action and live under another is a COLUMN of the equilibrium matrix, not a row to delete before coordination	prediction. It was counted as an engine defect by the prose fallback, which matched the metaphor <i>"a Farigiraf is dead only while that Farigiraf is still there"</i>)
#253	THE WORD "ENDGAME" IS CLAIMED BY TWO MODELS IN OUR OWN LEDGER, AND THAT IS WHY THE ARCHITECTURE WAS HARD TO HOLD — WILL, 2026-08-13. <i>"wait didnt we have dusknoir as the endgame guy? its hard to keep track."</i> He is right and the confusion is ours, not his memory. <code>docs/MODELS.md</code>: 1451 scopes DUSK as the endgame TABLEBASE — exact, precomputed offline, shipped as a lookup because it cannot be solved under a turn timer. SLOWKING's Job: line in the same file reads <i>"the endgame — tell you the equilibrium-best move (and win %) on a live position"</i>, which is not distinguishable in wording. THE SPLIT, NOW WRITTEN DOWN: DUSK is small endgames, EXACT, offline then looked up; SLOWKING is any position, approximate (an equilibrium over ESTIMATED values), live per turn; HYPNO is about the OPPONENT rather than the position and is a deliberate deviation FROM SLOWKING. Only DUSK can say "guaranteed" — which means Will's own description of the late-game stage (<i>"which ones lead to a guaranteed win... hand these scenarios to miltank"</i>) is DUSK's job and was filed under HYPNO. Fixed in <code>docs/MODELS.md</code> and <code>docs/ALAKAZAM-v2-spec.md</code>. AND THE MATRIX IS KEPT EVEN IF GREEDY WINS: <i>"id still want slowing matrix even if greedy wins right? because then if we play opponents regularly we can adapt and become unexploitable."</i> Keeping it is right and is already ADR-003's position — VGC-Bench's policy beats a Worlds competitor and is ~100% exploitable, so winning on average and being safe are different axes. ONE CORRECTION TO THE REASONING: adapting to an opponent makes you MORE exploitable, not less. Nash never adapts and cannot be beaten in the long run; deviating to punish a habit opens a hole in you. What repeated play actually buys is the other direction — a FIXED policy degrades because they learn YOU, and greedy is a fixed mapping. The matrix is not the adaptation mechanism; it is the floor you measure your deviations against, and HYPNO is the deviation	open — direction
#254	EVERY HAZARD LANDS ON THE SIDE THAT LAID IT IN THE OFFLINE BOARD, AND THE LIVE PATH IS RIGHT — SO THE FIT AND THE GAME DISAGREE. FOUND BY SEARCH 2026-08-13 WHILE CHASING #249; SPLIT OUT AND VERIFIED INDEPENDENTLY. <code>engine/board.js</code> <code>noteMove</code> ends with <code>if (move.sideCondition) board.startSide(side, move.sideCondition, ...)</code> — <code>side</code> is the MOVER's, unconditionally. DERIVED FROM THE FORMAT, NOT RECALLED: of the 11 legal side-condition moves in this regulation, SEVEN are target: allySide and FOUR are target: foeSide — Stealth Rock, Spikes, Sticky Web, Toxic Spikes. The seven (Reflect, Light Screen, Aurora Veil, Safeguard, Tailwind, Quick Guard, Wide Guard) are correct BY ACCIDENT, because the mover's side is the right answer for them. The four hazards are recorded on the layer's own side, which is the side they can never be on. THIS IS NOT #249 AND MUST NOT BE CLOSED WITH IT. That row is the SEED carrying no hazards into a payout — a missing feature, and a rollout that ignores hazards is merely blind. This is a feature that is PRESENT AND INVERTED, and it sits in the board the FIT reads: <code>engine/board.js</code> is the offline board, the live path reads Showdown's `	-sidestart and gets it right. ** ENVIRONMENT MUST MATCH MAG fitted with the sh trained under broken m Rock hurts its own tea **SIZE, AND IT IS SMAL uses across the four (34, Spikes 15) against worse than a missing o can tell it is wrong. <code>move.target</code> and place t A hand-written hazard third time, and a haza the wrong side again. WRONG TOGETHER — WHICH 2026-08-13 before anyo <code>board.startSide(side, ...)</code> ; to set <code>deadSide</code> ("this READS the same way for take the mover's side, cancels — a hazard is that same wrong side. THAN THE ONE IT FIXES* both readers keep look fire for a hazard and believing it was not u IT IS ONE HELPER, NOT that answers "whose si <code>move.target</code> , called by condition lands on is decide it independentl expensively. <code>const foeSi</code>

#	item	note
		five other places in t match. **A PROBE MUST readable by p1 as "up when p1 tries to lay i passes on an engine th **CLOSED 2026-08-13 BY RADIUS BY SIX SITES.** the class, is exported noteMove , the two reac copies of the same mov engine/fit_policy.js:793 (T corpus_shift.js , feature_c three in board.js woul hazard hurts the team a different file name. THAN REPEATED: "credit None of the four hazar Tailwind 4, the hazard for them under every v alone; the alreadyUp s fact-one-function rule moves, where sideFor i FIX IS THE GUARD'S.** to this: run under bot columns moved**, becau clicks** and pre-set o the half the fix leave stealthrock / spikes up on flip is exercised in B b984c210828d -> d1be4f95d three by compiling a p the tree was never rev failed at HEAD and 11/ all six wrong candidat fails the ninth-site c live / 0 missing**, un exercises medicham2 , IS NO, MEASURED: THE F LIVE-PLAY DEFECT.** Th FP.decisionsFor under bo 1,731,851 candidate ve and 0 decisions, 0 gam not thin exposure alon Sticky Web 2, Spikes 1 ISOMORPHISM, construct mover's side and both is the new one with th derived read agrees. * come from noteMove - takes the side straigh
#255	GOOD AS GOLD REFUSES ALLY-AIMED STATUS MOVES IN THE AUTHORITY AND OURS ONLY REFUSES FOES — 2026-08-13, LEFT BY THE #241 PASS RATHER THAN HALF-FIXED. The handler's condition is <code>move.category === "Status" && target !== source</code> , so it refuses anything that is not the holder ITSELF — including a partner's Decorate, Coaching, Skill Swap or Heal Pulse, measured on the authority. Every branch here is <code>_isFoe</code> -gated, so a Gholdengo accepts support from its own side that Showdown refuses. THIS IS A REFUSAL DEFECT, NOT AN EMISSION ONE, which is exactly why it was correct to leave it: #241 parts (1) and (2) were a NARRATION pass, and folding a behaviour change into them would have made a three-game differential move unattributable. AND IT IS A REAL SET, NOT A CURIOSITY — Gholdengo is the single carrier and ally-aimed support into your own Gholdengo is ordinary doubles play. Fix it with its own before/after. CLOSED 2026-08-13, AND THE SWEEP DOUBLED THE SCOPE THE ROW STATED. Derived first: one carrier in the regulation (Gholdengo, filtered walk) and <code>refusesStatusMoves</code> has EXACTLY ONE member over the whole tag corpus (<code>abilities:goodasgold</code> , 3,043 uses), printed before anything was wired. The authority was measured one battle per route — a supporter aiming at its own partner — and **26 routes all print `	-immune p1b: Gholdengo

#	item	note
#256	FOUR MORE onTryHit REFUSAL SITES ANSWER WRONGLY, NAMED RATHER THAN SWEPT — 2026-08-13. Residue of #241, each written down instead of being folded into a pass that had already moved: Role Play and Spite reach <code>kind: 'pass'</code>, which carries no target to announce a refusal against, so the refusal has nowhere to go; Heal Pulse emits a second `	-fail <mover> on the ORDINA nothing refused it; **
#257	A --write ON A VERIFICATION RUN REPUBLISHED THE DAMAGE DIFFERENTIAL AT n=150 AND ORPHANED EVERY DOCUMENT CITING 6,000 — 2026-08-13. <code>tests/test-docs-current.js</code> went red on one new row: <code>docs/MEDICHAM-SPRINT-NOTES.md:5426 6000</code> not in <code>data/engine-diff.json</code>. The flagged line was weeks old and had never been wrong; an agent ran <code>test-engine-diff.js --n 150 --write</code> as a quick check and the artifact became <code>requested: 150, compared: 150</code>. The claim did not become false — its EVIDENCE was deleted. Fixed by re-running at the published size, never by editing the sentence down to 150, which would have quietly reduced a published result to whatever somebody last typed on a command line. THE GATE FIRED ON A LINE NOBODY TOUCHED AND THAT IS THE POINT OF IT: every other check here compares a document to itself, and this one compares a document to an ARTIFACT, so it notices the ground moving under prose that is sitting still. THE HABIT IS THE DEFECT. The whole-game differential already learned this and writes its debugging dump WITHOUT <code>--write</code> for exactly this reason; <code>test-engine-diff.js</code> has no such separation, so any <code>--n</code> plus <code>--write</code> republishes. Give it the same split — a verification run should be unable to overwrite a published measurement — and until it has one, a verification run passes <code>--out</code> or omits <code>--write</code>. CORRECTION 2026-08-13, MEASURED BY THE #255 PASS AND THE STATED MITIGATION DOES NOT WORK: <code>tests/test-engine-</code>	closed 2026-08-14 — <code>docs/MEASURE.md</code>

#	item	note
	diff.js HAS NO --write FLAG AT ALL. Line 728 writes data/engine-diff.json unconditionally on every run, and it takes no --out either (OUT_NAME is switched only by --plant). So "omit --write " is not a protection, it is a no-op — a --n 6000 verification run with no flags rewrote the artifact, and only reproduced the published 6000/6000/0 by luck of asking for the published size. The split this row asks for has to be BUILT; there is no argument that reaches it today. BUILT AND CLOSED 2026-08-14. The published figure was checked FIRST and is TRUE: data/engine-diff.json carries requested 6000, compared 6000, agreed 6000, disagreed 0 , generated 2026-08-14T06:36Z, so the documents citing 6,000 are backed by a real run and tests/test-docs-current.js is green on all 21 clauses — nothing had to be restated and nothing was withheld. The mechanism is engine/publish_guard.js, called by tests/test-engine-diff.js in place of the unconditional fs.writeFileSync . A publish below the published sample is REFUSED; the run still completes and its output goes to data/verification/<name>.n<sample>.json so a verification run loses nothing except the ability to republish, and process.exitCode is set to 3 so a refused run cannot read as a pass. --republish-smaller is the deliberate door and stamps sample_shrunk {from,to,at} INTO the artifact. A high-water mark per artifact lives in data/published-samples.json so the refusal survives a hand edit or a writer that never called the guard. The three conformance sections that used to re-read data/engine-diff.json BY NAME now amend the path publish() actually wrote — so a --plant run, which goes out of its way not to touch the artifact, no longer stamped three sections onto it anyway. tests/test-publish-guard.js (23 assertions) covers the guard, that the caller still uses it, and that every ratcheted artifact on disk is at or above its record; all three clauses were shown RED on a deliberate break (refusal disabled; artifact shrunk behind the guard; a bare write restored in the caller) before being trusted. DEMONSTRATED ON THE REAL DEFECT: tests/test-engine-diff.js --n 150 now exits 3, leaves the 6000-comparison artifact untouched, and writes data/verification/engine-diff.n150.json	
#258	SEVENTY-FIVE SILENT CATCH BLOCKS ABOVE THE FLOOR, 41 OF THEM MANUFACTURING A VALUE, ALL IN ENGINE'S AND SEARCH'S FILES — SHRUNK TO ITS REMAINDER 2026-08-15; MEASURE'S OWN FILES ARE CLEAR. Numbers re-measured on the day, not carried: run the gate for the per-file list, never this sentence. THE ORIGINAL ROW, KEPT WHOLE: SEVENTY-EIGHT NEW SILENT CATCH BLOCKS SINCE THE 2026-08-06 BASELINE, AND THE GATE HAS BEEN RED ALL WEEK WHILE EVERY REPORT CALLED IT PRE-EXISTING — 2026-08-13. tests/test-no-silent-failure.js exits 1: 744 catch blocks, 296 silent (40%), 220 baselined, 2 fixed, 78 NEW — and 101 of the silent ones MANUFACTURE a value, handing a made-up result downstream rather than merely skipping. That last class is this project's named failure mode in its purest form: <i>a capability was absent and everything reported success.</i> THIS ROW EXISTS BECAUSE "PRE-EXISTING" WAS DOING THE WORK THAT "KNOWN FAILURE" USED TO DO. Three separate agent reports named this gate red and moved on, each correctly observing they had not caused it; the sum of those correct observations is a check nobody acts on, which CLAUDE.md already says is not a check. THE INSTRUMENT WAS AUDITED FIRST AND IS SOUND. The baseline is keyed by a HASH OF THE CATCH BODY, not by line number, so it survives edits elsewhere and this is real debt rather than drift. Reading three flagged blocks found two false positives — engine/game_differential.js:2656 counts through STATE_FAILS.x = (STATE_FAILS.x **RE-MEASURED AND THE WHOLE REACHABLE MANUFACTURE SUBSET CLEARED — 2026-08-19 BY MEASURE. THE GATE IS STILL RED AND THE REMAINDER IS BLOCKED ON FILE OWNERSHIP, NOT ON JUDGEMENT.** Today's numbers, re-derived on the day as this row instructs and not carried: **827 catch blocks, 291 silent (35%), of which 102 MANUFACTURE a value; 201 baselined, 1 fixed, 91 NEW, of which 48 manufacture.** Both figures this row had been carrying (75 above the floor, 41 manufacturing) were stale in both directions. **ALL 24 OF THE NEW-MANUFACTURING BLOCKS THAT WERE REACHABLE THIS SESSION ARE FIXED**, taking NEW 91 -> 67 and NEW-manufacturing **48 -> 24**: engine/mod_audit.js x3 (three reads whose EMPTY answer read as "Champions changes nothing" inside the tool whose entire job is that question — the healer -at-30% shape), engine/champions_sim.js (a checkCanLearn exception became a LEGALITY CLAIM about this format; false stays as the safe direction and is now counted in an exported learnCounters), engine/million_targets.js x2, engine/click_counts.js , engine/scenario_catalogue.js , engine/mega_census.js x2, engine/fixture_legality.js (a legality scan that passed by walking LESS), engine/conformance.js , engine/diff_swarm.js x2,	o)+1 and :1783 sets i detector gained the (x

#	item	note
	engine/quarantine.js (a declared-divergence matcher that threw silently moved its cause into UNDECLARED and INFLATED the rate this file publishes – now recorded and printed beside the declared count), engine/gate_offfield_target.js , engine/merge_mega_into_engine.js , engine/rollout_switch_census.js , engine/rollout_switch_probe.js , engine/million_run.js x2, engine/leaf_engine_contrast.js x2, engine/seed_source_audit.js . Every repair keeps the conservative value and adds the reason – a counter, a named console.error, or both – because inventing a different answer is a second defect, not a fix. **THE 24 THAT REMAIN ARE NAMED AND EVERY ONE IS IN A FILE ANOTHER DIVISION HELD THIS SESSION**: engine/game_differential.js (6), engine/tag_dex.js (2), engine/medicham2-browser.js (1) and fifteen across tests/` (roster.js 5, probe_volatile_leaves, staged_board, test-farm-ram-guard, test-forme-assert, test-lownode, test-roadmap-register, test-switch-carry, test-unmodelled-clicks, test-web-quarantine-loaders, test-web-quarantine). NOT RE-BASELINED, unchanged. THIS IS NOT A FILED FAILURE: the gate is RED, it is red for a stated reason, the reason is that the remaining blocks belong to ENGINE, and the next owner of those files closes it	

#	item	note

#	item	note	
#259	ONE FROZEN BODY THAWS WITH AN ANNOUNCEMENT AND THE OTHER THAWS SILENTLY, ON THE SAME TURN — WILL, 2026-08-13, READING CARD 2. <i>"for some reason we announce toxapex thaw but not incin thaw."</i> Both p1 bodies are frozen on turn 1; on turn 2 the authority emits `	-curestatus	p1a: Toxapex

#	item	note
#260	"EVERY SECONDARY FIRES" SELF-CANCELS FOR ANY EFFECT WITH AN ESCAPE ROLL, SO THE BOTTOM ARM GUARANTEES FREEZE AND GUARANTEES THAW — WILL, 2026-08-13. <i>"bro guaranteed freeze in the lower bound is so broken"</i>, then, one message later, <i>"but i guess we get guaranteed thaw?"</i> — both right, and together they name a property of the instrument nobody had stated. <code>makeArm</code> pins <code>random(den)</code> to <code>o</code> in the bottom arm, and <code>chance = (num, den) => random(den) < num</code>, so every <code>randomChance</code> returns true. The freeze secondary <code>(randomChance(1,10))</code> fires AND the thaw check inside <code>frz.onBeforeMove (randomChance(1,4))</code> also fires, so a body is frozen and cures on its very next move without ever losing a turn. The bottom arm therefore never exercises frozen-turn behaviour at all, while appearing to guarantee it. THE EFFECT IS NOT UNIFORM AND THAT IS THE POINT: an effect with ONE roll compounds (paralysis is applied AND fully paralyses every turn; confusion is applied AND self-hits), an effect with an ESCAPE roll cancels (freeze). <i>"The worst corner"</i> is therefore the wrong mental model for this arm — it is <i>"every branch taken"</i>, which is the worst case only when there is one branch. NOT A DEFECT IN THE PIN, which is doing exactly what it says; it is a COVERAGE claim that needs narrowing, and the arm's own description (<i>"every secondary fires"</i>) is what misleads. AND A LATENT COLLISION FOUND WHILE CHECKING IT: <code>random(m)</code> returns the damage index when <code>m === DAMAGE_ROLL_SIDES</code> (16), so any <code>randomChance(x, 16)</code> would be read as a damage roll rather than a chance. Derived: no legal move in this regulation has a 1-in-16 secondary, so it is latent today and would surface silently the day one is added	open — measure
#261	THE GAME LETS A FROZEN BODY THAW ITSELF BY ATTACKING AND THIS ENGINE DOES NOT, BECAUSE <code>flags.defrost</code> HAS NO REPRESENTATION ANYWHERE HERE — WILL, 2026-08-13. <i>"we need to test that the moves that auto thaw you out (either by the user or getting hit by it) actually thaw."</i> He named both routes and the format has both while this engine has one. Derived, filtered to the regulation: <code>flags.defrost</code>	<code>rng()<0.25</code> , so a frozen 25% roll instead of th A COLLAPSED PAIR, WHIC data/tags.json tags **al about two of its own m target — they defrost both are Fire and frz.o

#	item	note
	— the USER thaws when it USES the move — has five members: Burn Up, Flare Blitz, Matcha Gotcha, Scald, Scorching Sands. <code>thawsTarget</code> — the TARGET thaws on being HIT — has three: Matcha Gotcha, Scald, Scorching Sands. They are different rules over overlapping sets. <code>grep -c defrost RETURNS 0 IN BOTH engine/medicham2-browser.js AND data/tags.json</code>. The authority's <code>frz.onBeforeMove</code> opens <code>if (move.flags["defrost"]) && !(move.id === "burnup" && !pokemon.hasType("Fire")) return;</code> — the frozen body simply moves. Ours has no such clause: the thaw branch reads only <code>`m.frzTurns>=3</code>	=== "Fire" && move.categor right thing happens fo the three non-Fire mem ones are covered by ty CLAIM ABOUT POKEMON RA (2026-08-13: *"no froz moves"* — correcting w not exist). It does ex A row heading that can game is the same hazar level up, and it cost ADDITION**: a <code>thawsUse</code> needs-Fire-type except to the moves that actu CONFOUND**: a frozen u for the target), a fro the tag), and a frozen (thaws by type alone) type route from the ta 570 live / 0 missing** carrying requiresUserTy the SAME reader spends callers — and <code>thawsTar</code> engine clause sits ABO return spends no turn i mechanics.js, two of the first; four green-then including the target r red on its own. Whole- 978 games: no count mo DISAPPEARS from the bo move: flareblitz where it corners
#262	A MIDDLE ARM WITH EVENT-ADDRESSED DICE, BECAUSE BOTH PINNED CORNERS PRODUCE GAMES NOBODY PLAYS — WILL, 2026-08-13. <i>"is it possible at all for us to run a middle bound? where things can miss, and secondary chances have a chance to proc? to get some more realistic scenarios to test... otherwise paralysis ends the mons usefulness... thats why games take years."</i> He is right about the mechanism and it is measurable: the bottom arm pins every <code>randomChance true</code> (see #260), so the full-paralysis roll fires EVERY turn and a paralysed body never moves again — which is a large part of why the median game hits the 12-turn cap when a real one ends at 7. WHY RANDOM DICE ARE NOT THE ANSWER, AND THIS IS THE WHOLE DESIGN PROBLEM: the arms are pinned for SYNCHRONISATION, not caution. Both engines must give the same answer to the same question or a divergence is a dice disagreement rather than a rule disagreement, and they make different numbers of draws in different orders, so a shared seeded stream desynchronises on turn one. A constant is the only mapping that survives that. THE WAY OUT IS TO ADDRESS THE DIE BY THE EVENT RATHER THAN BY POSITION IN A SEQUENCE: <code>hash(turn, attacker slot, move id, effect name) % 100 < chance</code>. Order-independent, so the engines agree without being in lockstep; some Rock Slides miss and some flinch; paralysis fires near 1-in-4 instead of every turn; games END. Across thousands of events the observed rates approach the printed ones, which also makes it the natural home for the rate work in #196/#28. THE COST, STATED: the two engines must NAME the event identically — same move id, same turn, same target — or the hashes differ and every roll disagrees. The normaliser already maps ids, so it is checkable, and it must be ASSERTED rather than assumed: a probe that plants a known event on both sides and compares the drawn value. AND IT INHERITS #260: an effect with an escape roll must not get "true" for both the apply and the escape, which a hash gives for free and a constant never can	ENGINE HALF LANDED 2026-08-13 — the engine names the event; the AUTHORITY half is still unwired and measures 0.0%. <code>medicham2</code> now carries a module-level event address written at four sites (top of the turn, top of the action, the announcement, the step driver) and <code>midEventDice / midEventLog</code> export it; <code>tests/test-middle-identity.js</code> DIFFS the two engines' address logs over 900 real games. Measured, <code>medicham2-matched</code>: acc 99.7%, dmg 98.8%, crit 98.5%, sec 91.0% — 98.3% pooled over 1,058 events, floored per category (pooling hid a break: deleting the per-target write left the pool at 91.2% over a floor of 90). Four deliberate breaks, three RED, and the fourth GREEN is itself the finding — the announcement-site write is redundant over 900 games and is now COUNTED rather than trusted. The row cannot close on ENGINE's side alone. <code>game_differential.js</code> builds its address inside the function it installs as <code>battle.prng.random</code>, and <code>Battle#random</code> is <code>this.prng.random(m,n)</code> — so <code>this</code> there is the PRNG and every authority address is <code>`<seed></code>

#	item	note
#263	THE STORE HAS NO SPREADS AT ALL, SO A REPLAY DIFFERENTIAL CANNOT ADJUDICATE DAMAGE OR SPEED — WILL, 2026-08-13, AND HE TALKED ME OUT OF LEADING WITH IT. I proposed running <code>engine/replay_differential.js</code> against real games as the realistic instrument. <i>"we dont know the sps on games that actually happened tho"</i>, then <i>"so the damage and speed would be off."</i> Measured on <code>data/games.bo3.json1</code> : 4,001 games, 47,856 sheet bodies, EVERY ONE <code>"evs": null</code> , ZERO with a real spread. An open team sheet shows species, ability, item, moves and nature — never the stat points. SPEED IS THE WORSE HALF AND THAT IS THE PART I MISSED: damage being off produces a wrong number, but speed being off produces a wrong TURN ORDER, which is categorical — every line after it diverges for a reason that is not a rule. <code>replay_differential.js</code> 's own header already makes this argument about reconstructing CHOICES (<i>"a reconstruction that guesses is worse than no test, because a guess that happens to be wrong looks exactly like an engine defect"</i>) and does not apply it to the spreads. THE DISTINCTION THAT MATTERS IS CONTROL, NOT REALISM: the synthetic differential's spreads are unknown AND IDENTICAL ON BOTH SIDES, so a divergence can only be a rule; a replay's spreads are unknown AND DIFFERENT from whatever the player actually brought, so a divergence may be either. Unknown-but-identical is fine; unknown-and-wrong is not. So the replay differential keeps a NARROWER job — sequences and categorical facts independent of exact numbers (did that ability fire, did that switch happen, was that move legal) — and cannot arbitrate a KO or a speed tie, which is most of what a game turns on. This is ROADMAP #68 restated with its consequence: it is NARROWED by the nature and can never be closed by it. Do not lead with replay; build #262 instead, which is synthetic and therefore controls its own spreads	open — measure
#264	A 100-ACCURACY MOVE CAN MISS, AND THE DIFFERENTIAL ASSERTS THAT IT CANNOT — WILL, 2026-08-13. <i>"i mean even if a move is 100 accuracy it could still miss due to evasion, bright powder, sand veil, etc."</i> He is right and the population is larger than the three he named. Derived and filtered to the regulation: 3 items (Bright Powder, Wide Lens, Zoom Lens), 6 abilities (Sand Veil, Snow Cloak, Compound Eyes, Hustle, Tangled Feet, Wonder Skin) and 3 moves (Double Team, Minimize, Sweet Scent) touch accuracy or evasion. The authority keeps <code>accuracy === true</code> (cannot miss) strictly apart from <code>accuracy === 100</code> (rolls, after boosts and modifiers): <code>hitStepAccuracy</code> runs the <code>Accuracy</code> event and the evasion stages ONLY when it is not <code>true</code> , so a printed-100 move into a +2 evasion body is a 60% move. WHAT IS CONFIRMED HERE: <code>hitChance</code> does the stage arithmetic, WIRE 129 fixed the bug where <code>accuracy</code> and <code>evasion</code> mapped to NULL so every Double Team silently vanished, <code>evasionIgnored</code> is counted, and the one place a 100 becomes never-miss is documented and CORRECT — rain Thunder, where Showdown itself writes <code>move.accuracy = true</code> and the artifact lossily records 100. WHAT IS NOT CONFIRMED, AND THAT IS THE ROW: that a printed-100 move into a boosted-evasion body actually misses HERE, at the right rate. The mechanism being present is not evidence it fires — this project's founding lesson — and no probe covers it. AND THE CLAIM IS LOAD-BEARING RATHER THAN COSMETIC: <code>game_differential.js</code> asserts <i>"a 100-accuracy move HITS"</i> as a PIN CLAIM on both corners. If evasion is live, that claim is false whenever a Bright Powder or a Sand Veil is on the field, and the arm would be asserting something the game does not do. PROBE SHAPE: a printed-100 move into a body at +0 evasion (never misses), the same into +2 (misses ~40% of the time), and the same into a Bright Powder holder — three arms, because two cannot separate the stage arithmetic from the item modifier CLOSED 2026-08-13 — AND HALF THE ROW WAS ALREADY TRUE, WHICH IS WHY MEASURING CAME FIRST. The OUTCOME was never wrong: <code>hitChance</code> has always returned the FINAL accuracy and the roll sites have always gated on that, so a printed-100 Ice Beam into a +2 body missed 38.3% here against the authority's 38.8%, and into a Bright Powder holder 9.7% against 10.3%. What was missing was the PROBE, and there are now two — a three-armed RATE probe (a pinned roll cannot tell 60 from 99, and the stage route and the item route are different code) plus an Aerial Ace control that must also be shown LANDING. THE REAL DEFECT WAS THE DRAW, wrong in both directions: the attack and stat-change sites gated on <code>acc < 100</code> , so a Coil or a Wide Lens (final 133 / 110) stopped drawing where the authority draws, and the two STATUS sites drew unconditionally including on the Infinity a Poison-type Toxic	closed 2026-08-13 — <code>docs/ENGINE.md</code>

#	item	note
	produces, where the authority draws nothing. <code>accMustRoll(acc)</code> is now one answer for all four sites and it is <code>isFinite(acc)</code>. THE AMENDMENT'S CHEAP PREDICATE WAS BUILT AND IS A MEASURED REGRESSION — middle-arm VOID 131/171 before, 133 with the predicate (worse than doing nothing, because it took draws away from the status sites), 100 with the authority's rule; usable games 40 → 71. And the speed premise does not survive measurement: the skip never avoided the pipeline (<code>hitChance</code> runs unconditionally and always did — the guard sat underneath it), and three interleaved rounds of 6,000 turns put the three variants inside each other's round-to-round spread. Census 565 → 567 live / 0 missing; four deliberate breaks shown RED. The corners cannot see this and that is now measured rather than suspected: two frozen releases differing in exactly one file, 200 games, top 53/171 and bottom 59/171 before and after. <code>game_differential.js</code> PIN_CLAIMS need a re-wording (they say "a 100-accuracy move HITS" and assert a FINAL accuracy of 100) — reported, not edited, since MEASURE owns that file.	
#265	FIXTURES WERE DECLARING SETS THE GAME WOULD REFUSE, AND NOTHING IN THE REPOSITORY ASKED — CLOSED 2026-08-13 BY MEASURE. CLAUDE.md's cardinal rule is that no Pokémon, item, ability or move outside the regulation may be NAMED, in "<i>every example, every illustration and every derived result</i>" — and it was being broken inside the tests, where it is worse than in prose: a fixture built on a set that cannot exist measures a game we do not play, and every number it produces inherits that. BOTH KNOWN INSTANCES WERE FOUND IN PASSING RATHER THAN BY A CHECK, which is the argument: <code>engine/feature_fixture.js</code> gave Venusaur a ROCKY HELMET (banned here since 2026-08-04) and it was spotted by an agent doing #254; seven illegal learnset pairs reached <code>data/fixture-learnset-baseline.json</code> because an agent sent to repair one faint happened to run the validator. THE VERDICT IS <code>TeamValidator#validateTeam</code> 's AND NOTHING RE-IMPLEMENTS IT. Will: "<i>i thought we were using showdowns team validator as the ultimate legality test.</i>" The first plan was species <code>isNonstandard</code>, then the ability against the species list, then the item, then <code>checkCanLearn</code> per move — four hand-written rules standing in for a table Showdown maintains, which is FACTS ARE GLOBAL broken inside the file written to enforce a rule. A piecemeal check only finds the rules somebody thought to write; the validator carries the 66-point budget, the 32-per-stat cap, the item clause and every ban the mod adds next. <code>engine/fixture_legality.js</code> therefore calls <code>champions_sim.checkLegal</code> — one shared validator instance, five proved-clean padding slots — and prints the authority's own sentences. THE POPULATION, DERIVED RATHER THAN LISTED: 334 js files, 627 set declarations in 26 files, 227 distinct sets, 55 rejected, 41 distinct verdicts. Four scanner rules each went in after it was caught being wrong on a real file: a literal counts as an entity only if it normalises to that entity's own id (<code>dex.species.get('p2')</code> answers Porygon2 and <code>p2</code> here is a SIDE); moves come only from ARRAY literals (<code>hit('vaporeon', 'icebeam')</code> clicks the move at that body, not from it — three false accusations); helpers are taken at TOP LEVEL only (<code>test-mechanics.js</code> redeclares <code>run / hit / at</code> in dozens of probe callbacks and a file-global name set paired an ability stamped on the ATTACKER with the DEFENDER); and a set is declared either as an object literal or by MUTATION, which is 64 real sets in <code>test-protocol-trace.js</code>. 507 construction sites carry no literal and are outside the population, correctly — a fixture that derives its bodies from the format cannot type a name that does not exist. THE GATE IS <code>tests/test-fixture-legality.js</code>, ratcheted on <code>data/fixture-legality-baseline.json</code>, keyed on the VALIDATOR'S OWN SENTENCE so the same illegal declaration moved to a new scenario is the same defect. Six clauses: a population floor (a scanner that finds nothing passes everything below it), no new verdict, the baseline may only SHRINK, every entry states a kind and a written reason, a stray literal that names nothing in the format (<code>'damp'ro</code> for Damp Rock at <code>test-protocol-trace.js:131</code> — the item resolves to nothing and the fixture silently runs without it), and a self-check that the ratchet still discriminates. SHOWN RED THREE WAYS: re-planting the real Rocky Helmet fails naming <code>engine/feature_fixture.js:87</code>; two invented baseline entries fire the stale-allowance clause; one with a bad kind fires the reason clause. SIX REPAIRS, ALL IN MEASURE'S OWN FIXTURE, COVERAGE MEASURED ACROSS EVERY ONE: Rocky Helmet → Miracle Seed (4 sites), Electric Seed → Mental Herb, Throat Spray → White Herb, Grimmsnarl's Thunder Wave → Taunt, and Hippowdon's WEATHER BALL, which no legal sand or Rock body in this regulation learns — the flip moved to a new board <code>sand-weather-ball</code> rather than displacing anything, after an in-place Heliolisk swap was tried and MEASURED to thin <code>allyHit</code> 4→3 and <code>abilityBlock</code> 3→2 while going silent nowhere. AND THE GUARD'S OWN IDENTITY CHECK WAS THE FINDING THAT OUTLIVES THE FIX: <code>verify()</code> decided "<i>is this the same fixture?</i>" from ROUND and the scenario LABELS, so editing a body inside a scenario that keeps its name reported every moved column as "<i>these features changed MEANING</i>" — falsely accusing <code>board.js</code> of a change it did not make, and collapsing	closed 2026-08-13 — docs/MEASURE.md

#	item	note	
	the refit/restamp distinction the whole file turns on. It now carries a BODY digest, backward-compatible with older stamps		
#266	THE MATCHER IS ARMED AS OF 2026-08-27 AND THE GATE IS RED AT 2 FAILED, DELIBERATELY: 21 ILLEGAL SETS IN SEVEN FILES ARE NOW VISIBLE — 15 NEW VERDICTS AND 15 NEW DECLARATIONS, NONE BASELINED, NONE REPAIRED, ALL OWED TO THE DIVISIONS THAT OWN THOSE FILES. It reads DEPTH and asks the format for every literal's role, so it catches the CLASS and not a fourth shape; 872 → 1308 declarations, 272 → 411 distinct sets, 0 strays. Previous headline, kept because a register that rewrites its own history is the thing this row is about: <i>"fifteen illegal fixture verdicts and fifteen declarations remain, and the real debt is not them — a positional-row shape this sweep cannot see hides 124 more sets of which 21 are illegal. Re-measured 2026-08-26 by MEASURE; the gate is green again after being red at five."</i> THE HEADLINE SAID FORTY-ONE FOR TWO DAYS AFTER IT STOPPED BEING TRUE, which is the thing this register keeps doing and the thing that costs an agent: <code>node tests/test-fixture-legality.js</code> exits 0 today over 348 files / 639 set declarations / 232 distinct sets, with every one of the 22 verdicts and 23 declarations on the baseline and no stale allowance. AND IT IS NOT AN ENGINE DEFECT AT ALL — it is fixture hygiene; it held the MEDICHAM gate shut only because the gate's prose fallback matched the words <i>"a row that does not exist"</i> two thousand characters into this row's own account of a fixture bug it had already repaired. THE ORIGINAL ROW, KEPT WHOLE: FORTY-ONE ILLEGAL FIXTURE DECLARATIONS ARE BASELINED AND OWED A REPAIR BATCH — OPENED 2026-08-13 BY #265. They are not MEASURE's files and each repair changes what its scenario measures, so folding forty into the pass that ADDS the check makes both unattributable — the same reasoning already recorded in <code>data/fixture-learnset-baseline.json</code>, of which this baseline is the superset. **BY FILE — verdicts touching each, which SUM ABOVE 41 because one sentence can be produced in several files (Snorlax	Swords Dance in eleven places across two): <code>tests/staged_board.js</code> 15, <code>tests/test-protocol-trace.js</code> 12** (several are mainline recollections the mod does not carry — <i>"Gengar can't have Levitate"</i>, Champions gives it Cursed Body; plus the 'dampro' typo), <code>tests/staged_status_counters.js</code> 7, <code>tests/test-perish-song.js</code> 3, <code>tests/test-volatile-duration.js</code> 3, <code>tests/test-game-diff.js</code> 2, <code>tests/test-switch-carry.js</code> 2, <code>tests/test-click-censoring.js</code> 2, and one each in <code>test-speed-tie.js</code>, <code>test-nature-differential.js</code>, <code>probe_volatile_leaves.js</code>. THE COST OF EACH IS THE SAME SHAPE: the body was chosen for a typing, a bulk, a speed or a pairing with another test, and a legal replacement moves the board rather than renaming it — <code>test-perish-song.js</code> and <code>test-volatile-duration.js</code> deliberately share bodies so a duration change cannot pass in one and fail in the other, and <code>test-speed-tie.js</code> is pinned to four 100-base-Speed bodies. NOTHING MAY BE RE-BASELINED INTO SILENCE: the gate fails on a baselined verdict that stops being produced, so a repair must remove its entry. PART ONE DONE 2026-08-14 (MEASURE): 41 verdicts → 32, 55 rejected sets → 45, the one stray literal → 0, gate ALL GREEN on the shrunk baseline with the population unchanged at 627/227. Ten came off, each named with its cost in the new <code>repaired</code> block of <code>data/fixture-legality-baseline.json</code>. The stray was the worst and no validator could ever have reported it: 'dampro' resolves to a row that does not exist whose <code>.name</code> is <code>''</code>, and an empty item is a LEGAL item, so the fixture built a Politoed HOLDING NOTHING and everything went green. Both red gates' fixture audits are green and <code>tests/test-perish-song.js</code> is green outright; the singer is AZUMARILL and the Taunt/Encore/Disable user is ALAKAZAM, both DERIVED (six legal bodies learn Perish Song+Protect, seven learn Taunt+Encore+Disable) and the cost is named: none of the seven can have Pressure, so the doubled-PP rate is no longer staged in that file. <code>tests/test-volatile-duration.js</code> still parts from Showdown on HP and that is NOT the repair — measured, not assumed: the pre-repair fixture was replayed and parts in all four scenarios by the same magnitude and direction. Its absolute verdict is WITHHELD because <code>engine/medicham2-browser.js</code> was being written at 02:26 during the runs; only the PAIRED claim is published. PART TWO, 2026-08-15 (MEASURE). 32 verdicts to 22; the REAL population was never 32 and is now 23. THE RULER WAS SHORT: <code>validateTeam</code> returns ONE complaint per Pokemon, so a Snorlax carrying four moves it cannot learn produced one sentence, and repairing the first made the second read as a NEW verdict. Every declared move now also goes through <code>checkCanLearn</code>, and the baseline ratchets those PAIRS beside the sentences — 34 before this batch, 23 after. THE CLASSES ARE THREE AND ONLY ONE IS EXPENSIVE: 0 illegal SPECIES, 33 re-aimable pairings, and 1 UNREACHABLE — Spore, which has ZERO legal carriers in this regulation, derived twice (whole-roster <code>checkCanLearn</code>, and an independent raw learnset walk through prevo and base-forme chains; both answer 0). <code>tests/staged_status_counters.js</code> stages BOTH sleep-counter scenarios on it, and the repair is not a rename: every clickable sleep move here is sub-100 accuracy (Hypnosis 60, Sleep Powder 75, Sing 55) and both scenarios run on <code>top-tie-first</code>, where every sub-100 move MISSES by construction. Milotic can legally learn Hypnosis, so the mechanic exists on the bottom arm and nowhere else — HELD, filed to the owner of the differential, and declared <code>unreachable: true</code> so it can never be read as an engine defect. REPAIRED: 10 verdicts / 11 declarations, all in <code>tests/test-protocol-trace.js</code> and <code>tests/test-click-censoring.js</code>, each onto a DERIVED legal carrier with its cost named — Rotom carries the Levitate this format never let Gengar have (its <code>abilities</code> is <code>{"0": "Cursed Body"}</code>), Reuniclus splits at Snorlax's exact 30 Speed, Mamoswine phazes, Meganium sets the terrain and throws the Hyper Beam, and Thief replaces Knock Off in the PART 5 acceptance case that <code>new Battle()</code> never validated. All 41 claimed protocol events still fire. A POPULATION HOLE, MEASURED AND CLOSED: a set written as a positional row <code>['species', [moves], ability, item]</code> was neither of the sweep's two shapes, and twelve such rows drive 200 games — five were illegal, including a Toxapex holding an item that does not exist in Gen 9. Repaired first, matcher added after: 627 to 639 declarations, verdict count unmoved. THE BASELINE COULD BE LAUNDERED AND NOW CANNOT: PRE-EXISTING is a claim about TIME and nothing checked it, so 'repair one' and 'excuse a new one' were the same edit. The 41 historical sentences are now a closed <code>origin</code> set; a PRE-EXISTING entry outside it fails by name, and an UNREACHABLE entity may never be called DELIBERATE. Shown RED on the full attempt — offender planted AND added to <code>verdicts</code> AND to <code>pairs</code> AND marked <code>unreachable</code>, exit 1. Still open: 22 verdicts / 23 declarations, all in files MEASURE does not own: <code>tests/staged_board.js</code> 16, <code>tests/staged_status_counters.js</code> 7, <code>tests/test-switch-carry.js</code> 2, <code>tests/test-click-censoring.js</code> 1 — the last being Farigiraf/Encore, where no legal body in this regulation learns Encore, Roar and Follow Me together and that fixture's one body plays all three roles. NO VERIFIED BY MARKER, AND THE REASON IS THE MARKER'S OWN LIMIT: <code>tests/test-fixture-legality.js</code> is a RATCHET with a baseline, so it is GREEN today with all 22 verdicts standing — its exit code says <i>no NEW illegal fixture, never the debt is gone</i>. Marking it would make <code>engine/register_reality.js</code> report this row STALE while 22 declarations sit in the baseline, which is the same false confidence one level up. A marker is only honest where the instrument's green means the row's claim is FALSE. PART THREE, 2026-08-26 (MEASURE). THE GATE WAS RED AT FIVE AND THREE OF THE FIVE WERE THE GATE. Full account: <code>docs/_reports/2026-08-26-fixture-legality.md</code>; CHANGELOG 5.151.0. Two were real illegal entities: Incineroar can't learn Knock Off. at <code>tests/test-protocol-trace.js</code>:437, introduced by <code>3050904d</code> the same night in a scenario whose own comment derives the two moves it is ABOUT and not the partner's set — repaired to Darkest Lariat, <code>canLearn</code> true, same job, and the slot's whole script is Protect; and Milotic can't learn Calm Mind. at <code>tests/probe_mental_herb_order.js</code>:105, which wrote <code>[IDLE === 'Calm</code>	RED — tests/test-fixt 27, deliberately and repo seven files, newly VISIBLE 15 declarations already on th

#	item	note
		Mind' ? 'Recover' : IDLE] against a constant IDLE = 'Calm Mind' — a branch that can never be taken, so nothing illegal was ever played, and the name was still typed next to a body that cannot carry it. Eight stale allowances removed, 22 → 15 verdicts and 23 → 15 pairs, all in <code>tests/staged_board.js</code> and all repaired by ENGINE in <code>24fe4c5c</code>; proved a repair rather than a scanner that stopped looking, because <code>git show 24fe4c5c^</code> declares all eight, HEAD declares none, and the file is still the sweep's largest contributor at 204 declarations. <code>origin</code> untouched at its closed 41. THE OTHER THREE FAILURES WERE THE INSTRUMENT ACCUSING ITS OWN FAIL MESSAGES. Helper detection judged a top-level <code>const</code> on <code>src.slice(at, at + 500)</code>, a window that runs off the END of the declaration; two files pair <code>const ok = (cond, label, extra) => {...}</code> with <code>const stage = rows => rows.map(r => ({ species: r[0], ..., moves: r[3] })),</code> so <code>ok</code> inherited <code>stage</code>'s evidence and every assertion in both files was scanned as a set declaration — which is how <code>'medicham='</code>, a fragment of a FAIL message, was read as the species <code>Medicham</code>. The window now stops at the next top-level declaration and can only SHRINK: 875 → 872 declarations, and the full set dump diff is exactly the three phantom rows and nothing else, strays <code>5 → 0</code>. AND THE ANSWER TO 'DOES THIS CHECK CATCH THE CLASS' IS NO, WITH A COUNT. Three shapes walk past it. (1) A POSITIONAL ROW WHOSE MOVES ARRAY IS NOT IN SLOT 2 — the same class matcher (C) was added for, and larger. <code>stage(rows)</code> in thirteen files (twelve under <code>tests/</code>, plus <code>engine/game-differential.js</code>) writes <code>['species', 'item', 'ability', [moves]]</code>, moves LAST; matcher (C) demands the array second, and the helper matcher files every string inside any <code>[...]</code> as a MOVE, so the species literal lands in the move list and the call is recorded as <i>'no species literal'</i>. A position-independent matcher finds 413 rows / 157 distinct sets, of which 124 are invisible to the gate today and 21 the validator REJECTS — 15 verdict sentences not on the baseline (Agility on Incineroar/Whimsicott/Milotic/Clefable, Tailwind on Incineroar/Garchomp, Knock Off on Garchomp, Thunder Wave on Garchomp, Explosion on Steelix/Reuniclus, Whirlwind on Snorlax, Swords Dance on Meowscarada, Body Press on Archaludon, Toxic on Clefable, Final Gambit on Basculegion), with 0 stray literals of its own. IT WAS MEASURED AND DELIBERATELY NOT ARMED: arming it reds the gate by fifteen verdicts in files this batch does not own, clause 7 rightly refuses to launder them as PRE-EXISTING, and the repairs are not renames — <code>probe_turn_order.js</code> stages Agility and Tailwind because the probe is about speed brackets and <code>probe_selfdestruct_winner.js</code> stages Explosion because the probe is about self-destruct, so a legal replacement means a different BODY and a different board. Same order matcher (C) followed: measure, repair, then arm. (2) A ROW WITH NO MOVES ARRAY AT ALL — <code>engine/validate_damage.js</code>'s golden master, <code>[att, ability, item, nature, stat, move, def, nature, {spread}, weather, defAb?, defItem?]</code>, two species and a move, all scalars. No matcher here can see it, including the generalised one, and it is the file where Choice Band, Choice Specs and an Amoonguss were found on 2026-08-25 by a human rather than by this gate and repaired in <code>5bb13e3b</code>. Audited by hand 2026-08-26: 36 rows, 0 problems — clean today, still unseen, still ungated. (3) RULE 1 IS LOOSER THAN ITS OWN SENTENCE: 'a literal is an entity only if it NAMES ITSELF' is implemented as <code>id === nrm(s)</code>, stripping punctuation from BOTH sides, so <code>'medicham='</code> names itself. NOT tightened — the proximate cause is fixed and a second narrowing heuristic on top is enumerating known-bad forms; named in the header instead. NO GAME NUMBER MOVED AND NONE COULD: no engine byte, no census, no differential, no release. OWED: the 21 repairs behind shape (1), arming that matcher, a gate over shape (2), and a re-run of <code>tests/test-protocol-trace.js</code>'s 200 games and <code>tests/probe_mental_herb_order.js</code> — this batch may not play a game, so zero movement there is a PREDICTION and not a measurement. PART FOUR, 2026-08-27 (MEASURE). THE MATCHER IS ARMED, IT CATCHES THE CLASS RATHER THAN A FOURTH SHAPE, AND THE GATE IS NOW RED AT 2 FAILED — 15 NEW VERDICTS AND 15 NEW DECLARATIONS, NONE BASELINED. Full account: <code>docs/_reports/2026-08-27-invisible-fixtures.md</code>. POSITION IS NO LONGER A RULE. The regex that demanded species-first / moves-array-second is gone; every array literal is split by DEPTH into own-level literals and nested-array literals, and every one is handed to <code>roleOf</code>, which asks the format. A group declares a set when exactly one own-level literal names a species AND the group carries a further component (item, ability, or a nested array). Both known orderings were proved from the live scan — <code>['garchomp', '', 'Rough Skin', ['Protect', 'Earthquake']]</code> and <code>['slowing', ['skillswap', ...], 'regenerator', 'leftovers']</code> — same code path, opposite orders, both attributed correctly. The component requirement is the measured line between a fixture row and a list of this project's own model names, not caution: dropping it matches 907 groups instead of 448 and accuses <code>engine/mew_farm.js</code>'s <code>BOOL_FLAGS</code> and <code>build/build_status.js</code>'s <code>inputs: ['magnemite']</code> of naming species that do not exist. MEASURED WHEN ARMED: the two readings of the nested array (every literal is a move, vs only literals that self-name a move) give IDENTICAL populations, verdicts and strays — the first is used so rule 2 has one implementation. Population 872 → 1308 declarations, 272 → 411 distinct sets, 15 → 40 rejected, 15 → 30 verdicts, strays 0 → 0. <code>origin</code> untouched at 41; no allowance added or removed. ALL 21 ARE REAL AND ALL 21 ARE PAIRING — no illegal species, no illegal item, no UNREACHABLE entity; the twelve accused moves carry between 4 and 102 legal carriers each. Verified against known-good cases before being believed: 371 of 411 sets ACCEPTED, all 16 accusations re-derived through <code>CS.canLearn</code> (16/16 false) beside 11 positive controls on the same bodies (11/11 true), <code>validatorThrew</code> 0, and 0 stray literals. ZERO REPAIRED, 21 FILED: every one is in a file MEASURE does not own and every repair is behavioural, which this batch may not verify — <code>probe_turn_order.js</code> (13) chose Agility in its own words <i>"priority 0, harmless to the board"</i> on bodies picked for their SPEED TIERS, so both a move swap and a body swap move the board; <code>probe_selfdestruct_winner.js</code> (2) is decided by who empties first; <code>test-effect-credit.js</code> (1) requires a body that moves no stat

#	item	note
		by itself; <code>test-encore-fail-silent.js</code> (1) phazes the format's only Suction Cups body; <code>test-resolution-order.js</code> (3) stages Toxic and Final Gambit AS the subject and Final Gambit has only FOUR carriers, none with Basculegion's 195 HP; and <code>engine/game_differential.js</code> (4 sites) is the instrument that produces the differential. ONE INSTRUMENT FAILURE, CAUGHT BEFORE THE REPORT: the first armed run was 3 FAILED, the third being clause 9 "<i>THE RATCHET NO LONGER DISCRIMINATES</i>" while it was naming all fifteen — its negative control was <code>findings[0]</code> , which the new verdicts displaced. A control that only holds while clause 2 is green is clause 2 wired to fire twice; it now picks a verdict the baseline allows, and says UNSTAGED rather than failing if the baseline empties. TWO FURTHER FINDINGS, FILED NOT GATED. (a) NINE isNonstandard: 'Past' SPECIES NAMED AS BARE LITERALS IN THREE FILES — <code>tests/bench-medicham.js:45</code> pins a roster containing Amoonguss, <code>tests/test-choice-lock.js:56</code> benches Rillaboom + Amoonguss, and <code>engine/playstyle.js</code>'s SUN/SAND/FAKEOUT role priors name Groudon, Gigalith, Rillaboom, Mienshao, Hitmontop, Purugly. All seven distinct species are <code>tier: 'Illegal'</code> and all seven are ABSENT from <code>MC.mons (mcKey</code> throws by name), so six live role-prior entries can never match a real game. Not armed, because a bare species list cannot be told from a codename list by shape. Whether the bench copes is NOT claimed — the one probe attempted returned null for every body including the five legal ones, so it measured nothing. (b) champions_sim.checkLegal CANNOT VALIDATE A SINGLE ONE OF THIS FORMAT'S 76 MEGA FORMES — measured: <code>legal 0</code>, <code>speciesClause 6</code>, <code>transformsInBattle 70</code> . The 70 are the authority being RIGHT (a mega is declared as base + stone, never as the forme); the 6 are an instrument defect, <code>fillerSets</code> skipping the subject by <code>id</code> and not by <code>baseSpecies</code> , so a Charizard-Mega-Y is padded with a Charizard. No fixture declares a mega today so no guard was written for a case that does not occur — but every <code>checkLegal</code> caller inherits it, and it is what blocks blind spot 2: the multi-species variant that would reach <code>engine/validate_damage.js</code>'s golden master adds exactly ONE extra verdict and that verdict is this artifact. NO GAME NUMBER MOVED AND NONE COULD — two files changed, neither in the 26 frozen <code>SOURCES</code> , nothing under <code>data/</code> written, no fixture edited. Cost: the gate is ~42s against ~1.5s, because the validator is asked about 411 sets instead of 272. OWED: the 21 repairs, each with its command in the report; <code>fillerSets</code> by <code>baseSpecies</code> ; a gate over shape 2; the nine bare literals; and the fact that this gate is not registered in <code>tests/run-all.js</code> , so its red does not propagate.
#267	A STATUS IS SEEDED AND THE COUNTER HANGING OFF IT IS NOT, SO SLEEP RESTARTS AND TOXIC RESETS — FOUND BY SEARCH 2026-08-13 WHILE CLOSING #248. The seed now carries <code>status</code> onto every body, active and benched, and carries NOTHING about how far through that status the body is. <code>buildMon</code> returns <code>slp: 0</code> and the toxic ramp starts at stage 0, so a body two turns into a one-or-two-turn sleep wakes late in every playout, and a <code>tox</code> at stage 5 chips at stage 1. THE ENGINE IS NOT THE PROBLEM AND ALREADY HAS THE HARDER HALF RIGHT: <code>medicham2's 2026-08-12</code> note walks the format's own condition table and records that <code>par/brn/psn/frz/slp</code> have no switch handler so their counters CARRY OVER, while <code>tox.onSwitchIn</code> sets <code>stage = 0</code> so the ramp RESTARTS — it models both. THE BOARD RECORDS ONLY THE STATUS NAME. Both worlds can supply the counter: the protocol's `	-status gives the turn it lan quantity is turn-turnIt second counter per con at scale and every one sets to zero. **CLOSED ONCE, on the board: <code>bx</code> — the same shape <code>pp i</code> belongs to the Pokemon body comes out — booke FIELD and carrying a <code>s</code> engine's own rule (<code>slp</code> that pivots out keeps <code>rollout_leaf.seedStatusClock</code> <code>par</code> and <code>brn</code> get nothi and <code>tox</code> is written evi its own <code>toxStageResetOn</code> living here. **NO CALL written by <code>endTurn al</code> and every fixture stam it. **Off by at most o AFTER it had already m it asleep marginally l / 192,912 decision poi some turns into <code>slp/to</code> CEILING, because the <code>s</code> **VERIFIED BY: <code>nodetes</code> gate that decides it, <code>engine/register_reality.js</code>' <code>ri</code>
#268	THE BOARD GIVES A PERMANENT HAZARD A DURATION AND COUNTS NO LAYERS, SO THE SEED CAN ONLY SEE IT BRIEFLY AND ONLY ONE DEEP — FOUND BY SEARCH 2026-08-13 WHILE CLOSING #249. Derived from the format, not recalled: Stealth Rock, Spikes, Sticky Web and Toxic Spikes carry NO <code>condition.duration</code> , because they last until they are removed. <code>board.startSide(side, cond, duration)</code> defaults an absent duration to <code>turn + 1</code> , so in the OFFLINE path — the fitter's board — a hazard is up for exactly one turn; the live path passes <code>fieldDuration</code>'s default of 5 and is wrong for longer. Measured on a fixture: laid <code>true</code>, one <code>endTurn()</code> later <code>false</code> , while the real rocks are still there. SO deadSide GOES BACK TO ZERO AND THE MODEL RE-LAYS STEALTH ROCK, which is the same symptom #254 measured for the write-only variant and the reason that row insists the write and the reads move together. Layers are the second half: <code>sideConditions</code> is a Map of expiry, not a count, so a seeded Spikes or Toxic Spikes is always ONE layer and the difference between one and two Toxic Spikes is poisoned against badly poisoned. THIS IS NOT SEARCH'S: fixing it moves <code>deadSide</code> and <code>setupTurns</code> , which are fitted FEATURES, so it invalidates <code>data/policy-weights.json</code> and belongs with the refit. <code>tests/test-rollout-seed.js</code> §6 prints the fixture's before/after on every run so the day it closes is visible.	closed 2026-08-14 — SEARCH; gate <code>tests/test-seed-clock.js</code>; artifact <code>data/rollout-clock-prevalence.json</code>

#	item	note
	CLOSED 2026-08-14 BY SEARCH — AND THE "FTT-INVALIDATING" CLAIM IS CORRECTED, MEASURED RATHER THAN ARGUED. It is fit-AFFECTING in kind and immaterial in size: over the fitter's own replay of 800 human games / 139,340 candidates scored, <code>deadSide</code> fires 927 times with the fix and 925 without it — a difference of TWO candidates — and no other feature moves at all (the whole <code>feature_coverage</code> table and <code>feature_fixture</code>'s 58 per-feature column hashes are byte-identical). The reason is the format rather than the code: 60 hazard clicks across 14,288 open-sheet games, and not one hazard is laid in the first 120 games of the fit corpus. <code>setupTurns</code> does not move at all, exactly as #254's own note predicted — no hazard carries a <code>condition.duration</code>, so <code>dur()</code> is o whatever <code>alreadyUp</code> says. THE FIX IS ONE FUNCTION AND IT ASKS THE FORMAT, NOT THE CALLER: <code>startSide</code> now decides permanence itself — the same shape <code>sideFor</code> uses one screen down, because whose side a condition lands on and how long it lasts are both FACTS — from a set derived in <code>derived()</code> (a move declaring no <code>condition.duration</code>) with the <code>hazard</code> tag as a dex-free fallback, so the live adapter's guessed <code>fieldDuration</code> of 5 is overruled without <code>magnemite.js</code> being touched. The FIRST version read only <code>derived()</code> and was MEASURED WRONG on a board that had never scored a feature — four Spikes read one layer deep, the defect wearing the shape of the fix — which is why the tag path exists and why <code>sideCounters</code> counts every fallback. LAYERS ARE THE SECOND HALF AND ARE COUNTED NOW: the Map's value is <code>{until, layers}</code>, clamped at the tag's own <code>maxLayers</code>, and <code>board.sideLayers</code> is the one reader; <code>applySideState</code>'s <code>Math.min(1, ...)</code> — a constant 1 — is gone, so three Spikes take 1/4 of a switch-in rather than 1/8 (asserted behaviourally: −17 HP at one layer, −35 at three) and Toxic Spikes can reach the second layer that makes a body BADLY poisoned. Prevalence: a hazard is up at 0.271% of decision points, 0.08% of them older than five turns, 0.015% more than one layer deep — real, correct, and very nearly inert, which is stated rather than dressed up. VERIFIED BY: <code>node tests/test-seed-clock.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	
#269	EVERY DURABLE VOLATILE IS STILL UNSEEDED, AND THE LIVE BOARD ALREADY HAS SOME OF THEM — SPLIT OUT OF #250 BY SEARCH 2026-08-13. The seed carries HP, status, stages, item, ability, moves, PP, the Protect streak (mine), the move-action count and the entry snapshot. It carries NONE of: choice lock (<code>_lock / _lockT</code>), Encore (<code>_encoreMove</code>), Disable, Taunt, a multi-turn lock (<code>_mtLock</code>), a charge turn (<code>_charging</code>), Substitute (<code>_sub</code>), Leech Seed (<code>_seededBy</code>), a partial trap (<code>_trap</code>), Recharge, the perish count, or Gigaton Hammer's lockout. ** magnemite.js ALREADY WRITES <code>board.startVolatile / endVolatile</code> FROM `	-start
		/

#	item	note	
#270	THE SEEDED FIELD HAS NO CLOCK, SO A WEATHER WITH ONE TURN LEFT LASTS THE WHOLE PLAYOUT — FOUND BY SEARCH 2026-08-13 WHILE SWEEPING THE SEED. <code>applyField</code> sets <code>S.field.weather</code> and <code>S.field.terrain</code> and never <code>weatherT / terrainT</code>, and the engine's tick is <code>if (field.weatherT > 0 && --field.weatherT <= 0)</code> — so ZERO MEANS NEVER EXPIRES. A sun the real board has two turns of runs for sixty in every rollout, mispricing every Fire move, every Solar Beam, every weather-scaled ability and every Aurora Veil for the whole playout. THE SAME FILE ALREADY DOES IT CORRECTLY ONE FUNCTION AWAY: <code>applyMegaWeather</code> sets <code>weatherT</code> from <code>MEDI.weatherTurns(w, item, TAGS)</code>, which reads Heat/Damp/Icy/Smooth Rock off the body — so the seed's two weather paths disagree with each other, which is the shape that produced the mega-weather defect in the first place. THE TWO HALVES ARE NOT EQUALLY BLOCKED: the terrain clock IS on the board today (<code>pseudoWeather</code> holds an expiry, and <code>board.fieldLeft</code> reads it back), so that half is three lines; the WEATHER clock is not (<code>setWeather</code> takes no duration and records none), so the board must first learn it — from `	-weather	<code>plus the dex duration</code> <code>weatherTurns already cc</code> <code>#250 BATCH**:</code> it is on half of it inside a fo result unattributable. BIGGEST OF THE FOUR BY decision points, with every one of them for 192,912 decision point scans.) **THE BOARD RE DURATION**, and that i <code>MEDI.weatherTurns(w, item</code> <code>already calls</code>, and a s the FACTS-ARE-GLOBAL b answers in the first p with no caller change because the rock is th beside <code>applySideState</code>, l callers build the fiel otherwise grow the sam leaves. **AN EXPIRED W since zero is the engi over is the bug spelle representable and simp **NOT FIT-INVALIDATING meant, so <code>deadWeather</code> identical — that the b is a real and separate here. Two declared lim live board through -we unknown and the clock weather early rather t in the artifact is a C the tag artifact and r BY: <code>node tests/test-seed-cl</code> decides it, checked ag than by anyone rememberin
#271	A KNOCKED-OFF ITEM IS STILL ON THE BOARD'S BODY, SO EVERY DAMAGE FEATURE AND EVERY ROLLOUT KEEPS APPLYING AN ITEM THAT IS GONE — FOUND BY SEARCH 2026-08-13, AND IT IS THE CLAUDE.md LESSON BROKEN IN THE PLACE THAT LESSON WAS WRITTEN ABOUT. <code>board.switchIn</code> copies the item off the SHEET onto the slot object; <code>board.noteItem</code> — the one thing `	-item	/

#	item	note	
#272	FEINT AND PHANTOM FORCE WENT THROUGH THE SHIELD AND LEFT IT STANDING, SO THE PARTNER'S CLICK WAS STILL REFUSED — CLOSED 2026-08-14. The differential's shape regrouping named <code>-activate <slot> feint</code> at 15 games and <code>-activate <slot> phantomforce [broken]</code> at 7, the largest concentrated cause after the Tailwind ordering, and it was not a missing line. <code>hitStepBreakProtect</code> is step 5 of <code>moveSteps</code> (<code>sim/battle-actions.ts:755</code>) and this engine had no step 5 at all; both moves carry <code>ignoresProtect</code>, so the MOVE landed and the shield stood behind it. Staged on the authority first: Incineroar Protects, Feint, Garchomp's Dragon Claw at the same body — Showdown lands the partner's click and ends the turn with <code>p2a volatiles: []</code>, <code>medicham2</code> refused it with `	-activate	p2a
#273	<code>tests/probe_red_demo.js</code> IS RED WITH 31 STALE REVERSALS, AND A STALE REVERSAL IS A DEMONSTRATION THAT HAS NOT RUN — FILED 2026-08-14 BY ENGINE WHILE LANDING #272. The file's whole job is the standing gate <i>no check is committed until it has been shown failing on a known-bad input</i>: for each wire it patches the engine (or strips the tag) and asserts the probe flips. 31 of its rows now report THE REVERSAL NO LONGER MATCHES THE ENGINE — a later wire rewrote the lines the patch anchors on, so the demonstration silently stopped running, which is this project's signature failure wearing the uniform of the check built to prevent it. Two of them are a different shape (<code>THE STRIP NO LONGER APPLIES</code> — <code>icescales</code> no longer carries <code>damageReduce</code>, <code>airlock</code> no longer carries <code>weatherSuppression</code>, so the ENTITY or the TAG left <code>data/tags.json</code> and nobody noticed). MEASURED, NOT ASSUMED, THAT #272 DID NOT CAUSE IT: #272 adds one line inside <code>_STEPS</code>, which is a region several reversals anchor near, so the count was taken twice — with that line and with it removed — and read 31 both times. Not fixed in the landing session on purpose: re-aiming 31 anchors is a separate job and folding it in would make neither result attributable. The fix is not "re-aim the anchors" either — an anchor that is a literal source string will go stale again on the next wire,	CLOSED 2026-08-26 BY ENGINE — SAME DEFECT AS #449, FIXED IN THE SAME PASS, AND THE STRUCTURAL CAUSE THIS ROW NAMED ON 2026-08-25 IS THE HALF THAT WAS ACTUALLY MISSING. This row said it: <code>"tests/probe_red_demo.js declares no ABRA-EXIT line, so classifyExit cannot tell its refusal from its red."</code> It does now. <code>node tests/probe_red_demo.js</code> reads 200 demonstrations: o HOLLOW, o COULD NOT BE APPLIED, exit 0, <code>ABRA-EXIT 0 VERDICT-GREEN</code>; <code>data/register-reality.json</code> moves this row from <code>green:false / VERDICT-RED / CONFIRMED</code> to <code>green:true / VERDICT-GREEN</code>, and <code>openDefectClause</code> drops from 2 red rows (#218, #273) to 1 (#218). The predicted residue in this row's own account — <i>an anchor that is a literal source string will go stale again on the next wire</i> — is still true and is now COUNTED rather than mistaken for a red: the run prints <code>N COULD NOT BE APPLIED</code> on every pass, green included. Account and the per-row diagnosis are on #449.	closed 2026-08-26 — instru <code>tests/probe_red_demo.j</code> VERIFIED BY: <code>node test:</code>

#	item	note
	which is what the 31 are. It wants a reversal expressed against something that moves with the code. CLOSED 2026-08-15 (MEASURE), ON THE INSTRUMENT'S OWN EXIT CODE RATHER THAN ON A READING OF IT. <code>node tests/probe_red_demo.js</code> exits o with 200 demonstrations, o failed, and greps to o rows saying THE REVERSAL NO LONGER MATCHES THE ENGINE and o saying THE STRIP NO LONGER APPLIES — the 31 are gone, and the file is larger than it was when they were filed. The two strip rows are the interesting half: they were re-aimed at reachable carriers, and the class was given an ORACLE rather than a patch — <code>skipUnreachableAbility</code> asks the regulation whether any legal species carries the ability BEFORE stripping it, so the next ability #175 drops arrives as a named <code>N/A</code> instead of as a red nobody can act on. Two rows read <code>N/A today</code> (<code>aurabreak</code> , <code>transistor</code> ; neither has a legal carrier), which is the oracle answering, not a demonstration skipped. THE RESIDUE IS DECLARED AND IT IS NOT A DEFECT, SO IT MAY NOT HOLD A GATE. The sentence above — <i>an anchor that is a literal source string will go stale again on the next wire</i> — is still true of this file: the anchors were RE-AIMED, and <code>probe_red_demo.js</code> names four re-anchoring in its own comments (lines 496, 1311, 2926, 3240). That is a design improvement to the demonstration harness, nothing in the game is wrong because of it, and no instrument measures it; if it is wanted it needs its own row rather than a closed row's footnote. VERIFIED BY: <code>node tests/probe_red_demo.js</code> — a closed row is checked as hard as an open one, and this instrument carries no baseline, so its green means the 31 are gone rather than forgiven. REOPENED 2026-08-23 BY MEASURE — THE CLOSE NO LONGER HOLDS, AND THIS ROW'S OWN LAST PARAGRAPH PREDICTED IT. <code>data/register-reality.json</code> (generated 2026-08-23T06:13:06Z) ran this row's own VERIFIED BY command and read exit 1 after 55-3 s, classified VERDICT-RED , so the row files as PREMATURE CLOSE — the only one in the register. The 2026-08-15 close rested on <code>200 demonstrations, 0 failed</code> ; that is no longer what the instrument says. NO MECHANISM IS CLAIMED AND THE FAILING ROWS WERE NOT READ — the pass that reopened this was forbidden to run anything that plays a board. The leading candidate is this row's own residue paragraph, that an anchor which is a literal source string goes stale on the next wire, and eight days of wires have landed since; but a candidate is not a measurement, and it could equally be a refusal spelled as exit 1, which the exit-code classifier states in its own header it cannot tell apart from a red. WHOEVER PICKS THIS UP READS THE FAILING ROWS FIRST AND SAYS WHICH IT IS. RUN 2026-08-24 BY MEASURE AND THE FAILING ROWS WERE READ AS THIS ROW ASKED — IT IS TWO DEFECTS, NOT ONE. <code>node tests/probe_red_demo.js</code> on release <code>3929459bb195</code> , engine quiet and <code>engine/medicham2-browser.js</code> byte-unchanged across the run: 200 demonstrations, 14 failed — 12 STALE REVERSALS and 2 GENUINE FAILURES. THE 12 ARE THE INSTRUMENT DEFECT THIS ROW OWNS, and they are this row's own predicted residue arriving on schedule — an anchor that is a literal source string goes stale on the next wire: WIRE 118 Tailwind inside the same turn, WIRE 123 the slower entry weather setter, WIRE 129 the attack-site roll knowing who it is aimed at, WIRE 130 sound/Infiltrator through the doll, WIRE 130 a second Substitute costing nothing, ROADMAP #31 the turn re-sorting around the mega's Speed, ROADMAP #81 a blocked High Jump Kick paying its crash, WIRE 2 a failed Protect resetting the counter, WIRE 7 Protean's STAB, WIRE 9 a MISS naming its targets, WIRE 12 the Life Orb toll refused by a miss, and ROADMAP #259 a frozen body clicking a pivot move. ONE OF THE TWELVE IS A DIFFERENT SHAPE AND IS WORSE: WIRE 9's patch APPLIED and produced an engine that cannot play a turn (<code>_mvMissed is not defined</code>), so that reversal is not merely un-aimed — the code around its anchor has moved out from under it. THE 2 GENUINE FAILURES ARE NOT INSTRUMENT DEFECTS AND ARE NAMED HERE RATHER THAN ABSORBED. Both read <code>shipped-arm=false (must be true)</code> , so the SHIPPED engine fails the assertion, and both reverted arms read false too, so the knob is not being watched either: (1) <i>a spread move takes x0.75 rounded half up on 4096ths, not a truncation</i>; (2) <i>Life Orb is chainModify([5324,4096])</i>, not <i>Math.floor(d 1.3)*</i>. Both are WIRE 4 and both are in the DAMAGE pipeline. NO MECHANISM IS CLAIMED: whether the engine regressed or those two fixtures went stale in a way that reports FAIL rather than STALE was not diagnosed here. Life Orb was one of four ENGINE landings on 2026-08-23, which is a lead and not a diagnosis. AND THE VERDICT THAT REOPENED THIS ROW WAS NOT EVIDENCE. <code>data/register-reality.json</code> , generated 2026-08-23T11:48:06Z, records this instrument at exit 1 after 4,980 ms; a complete run takes ~25 s and prints 200 demonstrations, and the child timeout is ten minutes — so that entry is an instrument that STOPPED EARLY, classified VERDICT-RED because this file declares no <code>ABRA-EXIT</code> line for its exit codes. That is precisely the refusal-spelled-as-exit-1 this row's own last paragraph said it could not rule out. The reopen was right; the thing quoted as its evidence was not. INSTRUMENT OWED: nothing for the 12 — this file is its own instrument and it names them by row — but the two WIRE 4 damage failures want an ENGINE row of their own, because an instrument row is the wrong home for a live	

#	item	note
	damage defect. RE-SCOPED 2026-08-24 — THE TWO WIRE 4 DAMAGE FAILURES WERE THE HARNESS AND ARE WITHDRAWN AS ENGINE DEFECTS; THIS ROW'S REMAINING SCOPE IS SMALLER AND ONE MEMBER IS A NEW SHAPE. Re-derived 2026-08-23 by ENGINE: all four typed constants (Flamethrower 64, Heat Wave 51, Close Combat 80, Close Combat + Life Orb 104) are exactly the PRE-#304 span draw min + floor(0.5*(max-min+1)) over the rows' own bands; #304 made <code>rng5 = () => 0.5</code> name index 7 instead, moving the values to 64/52/79/103. The engine matches the AUTHORITY at all sixteen indices on the demonstrations' own bodies, and <code>tests/test-damage-roll-support.js</code> is green on the same subjects. Both rows now sweep the full band with a printed control that must not move, so they cannot go stale on a die convention again. NEITHER IS #339 (the spread DRAIN's per-target heal, untouched) NOR #338 (the Life Orb TOLL). The old <code>rng5</code> also answered accuracy, crit and secondaries, so 90-accuracy Heat Wave MISSED at indices 0 and 1 and CRIT at 15 — two thirds of its band was unreadable and the row could not have been re-aimed as it stood. THE COUNT IN THE BRIEF WAS ALREADY STALE: HEAD read 15 failures, not 14. After the edit: 200 demonstrations, 13 failed — 12 stale reversals plus ONE HOLLOW ROW, and the hollow one is a different shape filed on its own row (WIRE 7 the Sitrus is eaten between the two attackers, <code>reverted-arm=true</code>). Account: <code>docs/_reports/2026-08-23-spread-rounding-life-orb.md</code> READ 2026-08-25 BY MEASURE UNDER A NO-PLAY CLAUSE — THE INSTRUMENT WAS NOT RUN, AND THE REGISTER ENTRY HOLDING THE GATE SHUT IS STILL THE WRONG ONE. No verdict for this row was taken; <code>node tests/probe_red_demo.js</code> drives the engine through turns and was out of scope. What was established WITHOUT running it: a stale reversal is repaired only by editing <code>tests/probe_red_demo.js</code>, since <code>revertedEngine</code> fails on <code>!src.includes(find)</code> where the <code>find</code> strings live in that file and <code>src</code> is <code>engine/medicham2-browser.js</code>. That file has not been touched since commit 2784ffab, 2026-08-23 22:44 — BEFORE the 2026-08-24 reading of 200 demonstrations / 13 failed / 12 stale — while <code>engine/medicham2-browser.js</code> has been committed six times since (65782219, 382e9989, 65a9c5c4, 3ab94955, 665d497d, da34ee4a). Engine movement can only ADD staleness to a literal-source-string anchor. So the 12 cannot have self-repaired and the row is live; the exact count is OWED and is not asserted here. AND data/register-reality.json STILL CARRIES THE BAD ENTRY. Generated 2026-08-23T11:48:06Z, it records this instrument at <code>exit 1</code> after 4,980 ms against a run that takes ~25 s and prints 200 demonstrations — an instrument that STOPPED EARLY, published as <code>VERDICT-RED</code>. As of 2026-08-25 that is still the entry <code>openDefectClause</code> reads, so this row is holding the MEDICHAM gate shut on a REFUSAL SPELLED AS A RED while being independently red for a different, hand-measured reason. THE ENTRY WAS NOT HAND-EDITED: it is a generated artifact, a hand-flip of <code>green</code> there is the erase-the-evidence failure <code>register_reality.js</code>'s own header warns about, and correcting it honestly requires the probe to run. THE STRUCTURAL CAUSE IS NAMED: <code>tests/probe_red_demo.js</code> declares no <code>ABRA-EXIT</code> line, so <code>classifyExit</code> cannot tell its refusal from its red — <code>engine/gate_fail_and_silent.js</code> solved exactly this for #241 with <code>declareExit</code>, and the same treatment here would stop a crashed run being published as a defect. OWED: <code>node tests/probe_red_demo.js</code> (engine quiet, <code>engine/medicham2-browser.js</code> asserted byte-unchanged across the run), then <code>node engine/register-reality.js</code>.	
#274	<code>engine/selftest.js</code> 's CLEAN-DATA CHECK IS RED ON NINE FILES AND AT LEAST ONE IS A FALSE POSITIVE — FILED 2026-08-14 BY ENGINE WHILE LANDING #272. <i>"every raw reader of the ladder store declares why"</i> fails naming <code>engine/click_counts.js</code>, <code>engine/medicham2-browser.js</code>, <code>engine/mega_census.js</code>, <code>engine/mega_sets_from_sheets.js</code>, <code>engine/replay_differential.js</code>, <code>engine/rollout_switch_census.js</code>, <code>engine/sheet_usage.js</code>, <code>tests/test-parse.js</code>, <code>tests/test-side-guard-chooser.js</code> — nine files with neither a clean filter nor a <code>RAW-STORE-OK</code> declaration. The simulator's entry is a COMMENT, not a read: <code>medicham2-browser.js:3151</code> is WIRE 149's prose recording a corpus scan (<i>"Scanned data/games.ladder.json1, 51,445 games..."</i>) and the file performs no store read at all, so the check is matching a filename inside a sentence. That does not clear the other eight, and it does mean the count cannot be read as "nine undeclared readers". Predates #272 and is untouched by it — the matched line is outside that change and the diff introduces no ladder reference. Routed to OPS/MEASURE rather than fixed here: the readers span four divisions and the checker's own matching rule is the first thing that needs deciding	open — routed
#275	THE CALLER'S FIELD TYPES TAILWIND AS FOUR AND TRICK ROOM AS FIVE, WHATEVER IS LEFT ON THEM —	open - the measurement is owed. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: landed 2026-08-14 — SEARCH; UNMEASURED, owes its

#	item	note
	FILED 2026-08-14 BY SEARCH WHILE CLOSING #270. <code>miltank.js:836</code> and <code>:1032</code> build <code>twA</code>: <code>this.board.hasSide(side,'tailwind') ? 4 : 0</code> and <code>tr: this.board.hasField('trickroom') ? 5 : 0</code>, and the four R-gates plus <code>rollout_switch_probe.js</code> and <code>leaf_position_contrast.js</code> each build the same object with the same two literals — seven copies of one fact, and all seven say a Tailwind with one turn left has four. It is #270 exactly, in a different file: the position is running a clock and the seed hands over a constant. THE REMAINDER IS ALREADY ON THE BOARD AND ALREADY READ: <code>board.sideLeft(side,'tailwind')</code> and <code>board.fieldLeft('trickroom')</code> return it, and <code>rollout_leaf.applySideState</code> has used <code>sideLeft</code> for the screens since #249. DELIBERATELY NOT FOLDED INTO THE #267-#270 BATCH: it changes the seeded speed order of a live search in seven callers at once, so it owes its own arm rather than a free ride, and landing it inside a four-row batch is what makes the batch's own result unattributable. The right shape is the one #270 used — one <code>applyFieldClock</code> -style read at the leaf, so no caller grows the same two lines an eighth time. LANDED 2026-08-14 BY SEARCH AND IT IS UNMEASURED — IT STILL OWES ITS ARM. <code>rollout_leaf.applySpeedClocks</code>, called from <code>applyFieldClock</code>, reads <code>board.sideLeft</code> and <code>board.fieldLeft</code> and OVERWRITES the caller's two numbers, so all of them are fixed without being edited — eight sites in seven files, counted rather than quoted (<code>miltank.js</code> ×2, <code>rollout_r1/r2/r3.js</code>, <code>rollout_switch_probe.js</code>, <code>leaf_position_contrast.js</code>, <code>argmax_paired.js</code>; this row said seven and <code>argmax_paired.js</code> had grown the eighth since); <code>miltank.js</code>'s two copies are deleted rather than corrected, and the other six are now inert. NEITHER KEY IS SPELLED AT THE LEAF: <code>board.speedSideKeys()</code> answers from <code>derived()</code>'s <code>speedSide</code> (the side conditions whose dex condition declares an <code>onModifySpe</code> handler) and falls back to the <code>doublesSideSpeed</code> tag when no feature has been scored yet, counted at both tiers; <code>board.roomFieldKey()</code> is the declared irreducible <code>GAME_RULES</code> already held. An INFINITE remainder — what <code>sideLeft</code> returns for a permanent condition since #268 — is refused and counted, because the engine's tick reads a positive counter as running and zero as off, so Infinity would be this row's bug with the sign flipped. Gate: 10 assertions in <code>tests/test-seed-clock.js</code>, shown RED first (7 failed, including the CONTROL), asserted from BOTH SEATS, with the caller deliberately passing the old constants in every arm so a green row can only mean the board overruled it. NO HEAD-TO-HEAD WAS RUN AND NONE COULD BE: writing agents were live in the tree and every downstream figure is quarantined behind MEDICHAM, so this changes the seeded speed order of a live search and NOTHING here says it chooses better	own arm; gate tests/test-seed-clock.js . STATUS WORD CLARIFIED 2026-09-05 BY MEASURE: <code>landed</code> describes the CODE and a human reads it as a closure; the same cell says the change is UNMEASURED and owes its own arm, so the row is open for that arm. No claim altered, nothing re-run
#276	THE BOARD'S WEATHER NEVER EXPIRES FOR THE FEATURES EITHER, SO <code>deadWeather</code> AND EVERY WEATHER-SCALED FEATURE BELIEVE A FIVE-TURN SUN IS UP ON TURN FORTY — FILED 2026-08-14 BY SEARCH WHILE CLOSING #270. <code>board.setWeather</code> records the word and, since #270, WHEN it was set and what the setter held — but <code>board.weather</code> itself is still a bare string that only ever changes when a new weather arrives. #270 fixed the SEED, which reads the clock and deletes an expired weather before the payout; it deliberately did not expire the board's own field, because that moves fitted feature values and therefore invalidates <code>data/policy-weights.json</code> . Sized from the store on the same scan as #270: a weather is up at 32.2% of decision points with a mean of 2.95 turns left, so the population of points where the board is wrong about the weather is large. The fix is <code>board.weather</code> becoming an accessor over <code>weatherSince</code> + <code>MEDI.weatherTurns</code> — the same shape #271's item accessor used, and for the same reason — and it belongs with the refit. INSTRUMENT OWED: a gate that sets a weather on the board, advances past <code>MEDI.weatherTurns</code>, and FAILS while <code>board.weather</code> still names it and <code>deadWeather</code> still reads false. <code>tests/test-seed-clock.js</code> is GREEN and decides #270 — the SEED clock — not the board own field, so a VERIFIED BY pointed at it would make a live defect read CONFIRMED. CLOSED 2026-08-17 BY SEARCH, IN ONE BATCH WITH #283, AND WILL'S RULING WAS READ CORRECTLY RATHER THAN CONSERVATIVELY: he forbade RUNNING the refit early, not fixing the features it will be run against — data/policy-weights.json is already quarantined and already owes one, so moving a fitted value adds to an existing debt rather than creating a new one. <code>board.weather</code> is an accessor over <code>weatherLeft()</code>, which is <code>MEDI.weatherTurns(word, rock, TAGS)</code> - <code>weatherAge()</code> — the ONE implementation, and <code>rollout_leaf.applyFieldClock</code> calls it now instead of keeping its own copy of the same subtraction. THE WORD IS STILL RECORDED (weatherWord()) AND THAT IS LOAD-BEARING, NOT BOOKKEEPING: the accessor answers the FEATURE's question and would otherwise have made the seed's own expiry branch — and <code>fieldClockCounters.weatherExpired</code> with it — silently unreachable, a dead guard wearing the shape of a fixed one. <code>engine/miltank.js</code> and <code>tests/test-seed-clock.js</code> both read	-weather line), so a naive ver: nothing ever reaches i refuses an active weat derived rather than ch up', so re-laying it s idempotence gets wrong IS MEASURED AND NAMED* **27 of 58 features an 51,399 candidate vecto is the row's own class stab / eff2 / effHalf , accuracy , chargeTurn , abilities), abilityBlock beneath them. **AND TH MEASURED, NOT ASSUMED* THEN setWeather , so ev hashes are byte-identi refit is owed' would b feature_shift.js runs an z Two declared limits, b

#	item	note
	the word now. RE-DECLARING A WEATHER THAT IS ALREADY UP DOES NOT RESTART IT, AND WITHOUT THAT CLAUSE THE WHOLE ROW IS A NO-OP : both worlds announce one weather twice (the click through <code>noteMove</code> , then the store's <code>w</code> event or the live `	
#277	THE REMAINDER OF #269 — SIX VOLATILES THE SEED STILL DROPS, EACH WITH A NAMED REASON — FILED 2026-08-14 BY SEARCH SO THAT CLOSING #269 IN PART LOSES NOTHING. #269 closed the three the engine's own <code>durationVolatiles()</code> table holds (Taunt, Encore, Disable). These six are not in it and each is blocked differently, which is why they are one row and not six: <code>substitute</code> — the consumer is <code>_sub</code> , the substitute's REMAINING hp, and the wire never states it, so any number would be invented; <code>leechseed</code> — <code>_seededBy</code> is a reference to the BODY that drains, not a count; <code>healblock</code> — the consumer is <code>_healBlock</code> and <code>_vol.healblock</code> is read by nothing (<code>medicham2-browser.js:10160</code>), so this one is a pure vocabulary translation and is the cheapest of the six; <code>perishsong</code> — <code>_perish</code> , which the engine CLEARS on switch-out (<code>perishClearedOnSwitch</code>), so seeding it means putting that rule in a second place; <code>throatchop</code> — <code>_noSound</code> ; and the CHOICE LOCK (<code>_lock / _lockT</code>), which is not on the board at all. AND THE FOE'S PROTECT STREAK IS STILL IN THE WRONG PLACE, unchanged from #269 as filed: <code>buildSide(board, foe, dex, stats)</code> takes no <code>protectTurns</code> and <code>mag_bot.js</code> 's tracker is gated <code>if (mine)</code> , so the opponent's consecutive Protects are never counted. It should not be caller state — <code>board.noteMove</code> already reads <code>move.stallingMove</code> , so a counter there serves both sides and both worlds. <code>rollout_leaf.unseededVolatiles()</code> prints the first five with their reasons on every gate run, so this row's contents are readable from the code rather than only from here. WORKED 2026-08-14 BY SEARCH: THREE OF THE SIX ARE CARRIED, THREE ARE REFUSED BY NAME, AND THE PROTECT STREAK IS CLOSED. <i>Carried</i>: <code>healblock -> _healBlock</code> and <code>throatchop -> _noSound</code> , as <code>VOL_ENGINE_FIELD</code> — a declared protocol-to-engine join beside <code>VOL_MOVE_FIELD</code> , written ABOVE the <code>_vol</code> table so neither leaves a consumer-less duplicate, and the gate checks BOTH halves (the key is the format's own condition NAME, which is the word the wire carries, and the field is one the engine still reads). THE CHOICE LOCK WAS NOT MISSING, IT WAS PRIVATE: the expression sat inline in <code>board.candidates()</code> and nowhere else, so the FIT knew a locked body had one legal move and the PLAYOUT did not — it is now <code>board.choiceLockOn(mon, dex)</code> , called by both, and the seed writes <code>_lock</code> with the engine's own <code>_lockT</code>: Infinity discriminator. <i>Refused, each by name</i>: <code>substitute</code> (<code>_sub</code> is the doll's remaining hp and the wire never states it), <code>leechseed</code> (<code>_seededBy</code> is a body reference, and the board cannot hold it either — Leech Seed declares no <code>condition.duration</code> , so the live adapter's table falls back to 3 and the board forgets a seed the real game keeps until the body leaves), <code>perishsong</code> (the two counters are OFF BY ONE, measured: the wire's <code>perish3</code> is stored as three board turns while the engine holds <code>_perish = 4</code> at that instant and kills at the end of the turn it reaches 0, so the board reads ZERO on the very turn the body dies — seeding it would end the count early or drop the lethal turn, and the repair is in the live adapter). THE PROTECT STREAK IS NOW <code>board.noteMove</code> 's, counted off the dex's own <code>stallingMove</code> flag plus the engine's <code>stallCounterFeeds</code> tag, reset by any other move, cleared for free by a switch because <code>switchIn</code> builds a new body — so the FOE's consecutive shields are counted for the first time, in the fit's board as well as the live one, and <code>mag_bot.js</code> 's species-keyed map survives only as a counted fallback. THE FIT IS UNMOVED AND IT WAS CHECKED, NOT ARGUED: <code>engine/feature_fixture.js</code> 's 58 per-feature column hashes are byte-identical against a deliberate break restoring the old inline lock and removing the streak write, and the gate proves the shared lock answers exactly <code>isChoice</code> over all 148 legal items, which is the whole input to <code>candidates()</code> . Gate: 40 assertions in <code>tests/test-seed-clock.js</code> with two behavioural arms (a silenced body REFUSES its sound click; a locked body's other move never appears on the menu), shown RED on three separate deliberate breaks	worked 2026-08-14 — SEARCH; 3 carried, 3 refused by name, streak closed; gate tests/test-seed-clock.js
#278	THE PAIRED ARGMAX RUN — REACH BECOMES EFFECT, AND IT IS 47.3%. RUN AND CLOSED 2026-08-14 BY SEARCH. R13 through R16 each published a PREVALENCE and each said, correctly, that it is a CEILING ON REACH and not an effect — a body priced holding an item it no longer holds may still be the same argmax. This is the run that converts one into the other. Six arms, each a COMMIT, compiled with <code>Module._compile</code> into <code>require.cache</code> (#254's technique, because three other agents were live in the tree), so the delta between consecutive arms is ONE landing and attribution is a subtraction rather than an assertion; the simulator, the dex, the tags, the priors, the engine data and the weights all come from one frozen release and are byte-identical across every arm. Over 131 paired decision points under common random numbers keyed on the CANDIDATE'S IDENTITY rather than its index, the whole of the seed work moves MILTANK's argmax on 62 of them —	closed 2026-08-14 — SEARCH; generator engine/argmax_paired.js; artifacts data/argmax-paired.json + data/argmax-paired-n12.json + data/argmax-paired-n100.json

#	item	note
	47.3% — and moves MAG’s argmax on ZERO. Per landing: #244 6.1%, #254 0.0%, #247/#248/#249/#250 46.6%, #271 0.0%, #267/#268/#269/#270 9.9%. THE PAIRED NULL IS ZERO AND THIS RUN DEMONSTRATES IT THREE TIMES — the identical-dice control (0/131), and two arm pairs whose change provably cannot reach these positions (#254, #271). THE UNPAIRED FIGURE IS NOT THE COMPARATOR: the same arm against itself with only the dice changed disagrees on 62.6%, which is R3’s finding from another direction and is why the run is paired at all. THE ONE THING CRN CANNOT SETTLE IS SWEEP RATHER THAN ARGUED: common random numbers pair the SEED, not the TRAJECTORY, so some flips are a rerouted die — the rate falls 53.4% → 47.3% → 41.2% across n=12 → 40 → 100, so an 8.3x budget costs 12 points and the residual is real and is not most of it. WHETHER THE NEW CHOICE IS BETTER IS UNMEASURED, AND THE NUMBER SAYS SO RATHER THAN THE PROSE: <code>head</code>’s pick is worth +9.70 points over <code>pre</code>’s by <code>head</code>’s own leaf, and the identical statistic on the UNPAIRED control — where neither pick is better by construction — is +7.96. That gap is argmax selection bias, it shrinks with budget the way selection bias does, and it carries no direction. A FLIP IS NOT A WIN; the thing that answers “better” is an SPRT and this is not one. #271 AND #269 CONTRIBUTE EXACTLY ZERO AND THAT IS THE HARNESS, NOT THE FIX: the decision points come from <code>joint_rows.js</code>’s replay, which calls neither <code>board.noteItem</code> (whose one caller is the live protocol reader) nor <code>board.startVolatile</code> — so every rate here is a FLOOR on the live board, and closing that is PRIORITIES 13e and is MEASURE’s	
#279	THREE MOVES ALWAYS CRIT IN THIS FORMAT AND WE ROLL A DIE FOR THEM, AND ON AN ORDINARY MOVE THE AUTHORITY ROLLS WHERE WE DO NOT — 2026-08-14. Found by the middle arm before the middle arm works, which is the argument for building it. Its address dump on a voided game: the authority draws <code>`dmg</code>	<code>flowertrick</code> and NO crit; we draw <code>dmg</code> <code>flowertrick</code> AND crit

#	item	note
#280	NO MILTANK MEASUREMENT IN THIS PROJECT'S HISTORY HAS EVER RUN ON MORE THAN ONE CORE, AND THE RUNNER HAD NO IDEA HOW BIG ITS OWN WORKERS ARE — FOUND AND FIXED 2026-08-14 WHILE RUNNING #57's R4. Three compounding costs, each a tool that already existed and was not reached for. (1) --conc IS NOT PARALLELISM. <code>mew.js:967</code> is <code>Promise.all</code> over async workers in ONE process — no <code>worker_threads</code>, no <code>child_process</code>. The simulator is synchronous and CPU-bound, so in-process concurrency overlaps nothing and only multiplies GC pressure. <code>mew_farm.js</code>'s header has said "CONC MUST BE 1" since 2026-07-25 with the measured table (8 procs × conc 4 = 11 games/s; 8 × conc 1 = 38), and R4's arms were launched at <code>--conc 6</code> in a single process anyway. Measured: probe at conc 2 = 34.25 s/game, real run at conc 6 = 34.7 s/game — identical, and that evidence was in hand before the run started. One core of sixteen, 13% load, 8h28m for 880 games. (2) MILTANK WAS NOT FARMABLE AT ALL. <code>mew_farm</code>'s unknown-flag guard correctly REFUSED <code>--miltank2</code> rather than dropping it (the 2026-07-31 <code>--opponent-model</code> lesson working exactly as designed), and nobody had ever added the miltank family to the lists — so every MILTANK number ever published came from a single process. Added <code>miltank</code>, <code>miltank2</code>, <code>miltank-no-defer</code>, <code>no-raw</code>, <code>closed-sheets</code>, <code>explain</code> to <code>BOOL_FLAGS</code> and <code>miltank-n</code>, <code>miltank-foe</code>, <code>miltank-preview-n</code>, <code>mega</code> to BOTH the forwarding list and <code>KNOWN_VALUE</code> — the asymmetry is a trap and is now commented: <code>KNOWN_VALUE</code> only silences the error, so a value flag added there alone reproduces the exact bug the guard exists to stop. (3) <code>sprrt.js --watch</code> EXISTS AND WAS NOT USED. Run 1 decided at 625 games of 2,326 (73.1% wasted); run 2 decided at 84 of 880 (90.5% wasted). Combined, run 2's 8.5 hours needed about five minutes. THE RAM GUARD IS THE PART THAT IS A CHECK RATHER THAN A LESSON. <code>mew_farm</code>'s default is core-based (0.75 × cpus, capped 16) and picks 12 on this 13.3 GB machine; a MILTANK worker measured 663–867 MB fresh and 1,948 MB after an 8h28m run, so 12 workers is ~10 GB before growth. The machine swaps, and a swapping desktop does not look slow, it looks FROZEN — which is the force-quit Will reported twice, misdiagnosed both times as something outside this repo (orphan processes, then vmmem at 3.8 GB; vmmem measured 0.12 GB the second time and was not the cause). New <code>engine/ram_cap.js</code> — its own module because <code>mew_farm</code> spawns at require time, so a test importing the cap from there would launch the farm as a side effect. <code>tests/test-farm-ram-guard.js</code> pins the regression, is auto-discovered by <code>run-all.js</code>, and was shown RED on a deliberate break (<code>granted 12</code>, exit 1) before being trusted. It capped its first real run 4 → 2 unprompted and printed <i>"This run is NARROWER than the one you asked for"</i>. VERIFIED BY: <code>node tests/test-farm-ram-guard.js</code> — the register row and the gate that decides it, checked against each other by <code>engine/register_reality.js</code> rather than by anyone remembering.	closed 2026-08-14 — <code>engine/ram_cap.js</code> + <code>engine/mew_farm.js</code> + <code>tests/test-farm-ram-guard.js</code>
#281	R4 RE-RUN, THREE ARMS — MILTANK BEATS MAG, AND MAG'S OWN SWITCH FEATURE COSTS IT ~15 POINTS. RUN 2026-08-14, QUARANTINED ON ARRIVAL. #57's re-run of the gate <code>mag_bot.js:786</code> records as never having happened. THE THREE ARMS EXIST BECAUSE ONE RUN WOULD HAVE CONFOUNDED TWO CHANGES: <code>mag_bot.js:752</code> forces <code>`switching: SWITCHING</code>	ROLLOUT, so live MIL head-to-head differs i reachable. <code>mew.js</code> doe: only), so the two are must be set deliberate 1):** MAG with switchi CI [60.3, 69.6]**, SPR magnemite.js:748 says tl policy beats not switc estimates the cost at a live defect in the s search alone (880 game **94.3% of 265 decisiv 30 decisive pairs. **T forcing switching ON t confound, it CRIPPLED the honest margin. **R

#	item	note
		off vs MILTANK switchi 200 games the raw spli the direction run 1 pr FINDING**: 34.2% in ru the board the POLICY d otherwise does, and no score". **AFFORDABILIT 9,655 logged decisions decision** (max 24,299 80-200 s against a 7-m AGREE ON EVERY COUNT** QUARANTINED ON ARF STAMP, NOT A CAPTION are provisional; the director Stamped against the release
#282	THE LIVE ADAPTER READS A VOLATILE'S DURATION OFF MOVES THIS REGULATION DOES NOT CONTAIN — FILED 2026-08-14 BY SEARCH WHILE CLOSING #277's TRANSLATION. engine/magnemite.js:130 builds its duration table with for (const m of dex.moves.all()) and skips only !m.exists — no isNonstandard filter, which is the walk CLAUDE.md names by name ("Dex.forFormat <i>IS NOT A LEGALITY FILTER</i>"). Measured on the format rather than argued: the move that shares the heal-block volatile's name is isNonstandard: 'Past' and carries condition.duration: 5 , while the ONE legal carrier in this regulation is Psychic Noise, whose blocksHealing tag says 2 — the same 2 Showdown's own durationCallback returns for it. So a live board records a heal block as five turns where the game says two, and since #277 that number is now carried into every payout. The table is keyed BOTH by volatileStatus and by the move's NAME, so the illegal row wins on the name key; the sound lock is unaffected only by luck (its legal carrier is the name). Not fixed here for two reasons: magnemite.js is the live protocol adapter and is not SEARCH's, and the repair is not simply adding the filter — with the illegal row gone the table has NO entry for that volatile and the fallback is a bare 3, so the duration has to come from the tag the engine already reads. The seed carries whatever the board holds and says so	CLOSED 2026-08-14 — SEARCH, R19. Artifact data/seed-source-audit.json (engine/seed_source_audit.js),gate tests/test-seed-clock.js \$7, shown RED on TWO deliberate breaks. THE FILTER ALONE WOULD HAVE TRADED A WRONG 5 FOR A WRONG 3, AND THAT WAS DEMONSTRATED RATHER THAN AVOIDED: the naive fix — legality filter added, nothing else changed — fails three of the new arms with Heal Block -> 3 . A DURATION IS A PROPERTY OF THE CONDITION, NOT OF THE MOVE, and Showdown's own registry answers for both, so the walk now covers the format's 500 legal moves (of 954 that exist), asks each which volatiles it can start — volatileStatus , secondary , secondaries , self ; Psychic Noise declares none of the first kind and reaches it through a 100% SECONDARY — and takes the number by running the authority's own durationCallback with that legal move as the effect. That is the same function the server runs, so the 2 is DERIVED and self-correcting rather than transcribed. THE WHOLE MEASURED EFFECT IS ONE KEY: 84 keys to 56, exactly one corrected value (healblock 5 to 2) and 34 keys that had no legal carrier at all and can never be looked up; taunt, encore, disable and throatchop are byte-identical and are asserted as controls. 18 durations are computed by the authority, 7 refused (the trapping family wants this.random) and use the declared value with the refusal COUNTED, and o are ambiguous — measured, because a -start line does not name the move that caused it. The behavioural arm asserts the corrected number reaches the seeded body's _healBlock , not just the table NOTATION REPAIRED 2026-09-04 BY MEASURE — pipe characters inside inline code were cutting this status cell, so the register read the row OPEN against its own closure. Only the notation changed; no claim was altered or removed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: cheap check from committed artifacts only — data/seed-source-audit.json exists and carries the volatileDurationTable and basePowerCallbackClass keys the closure names, and engine/seed_source_audit.js and tests/test-seed-clock.js are on disk; nothing was re-run here.
#283	board.movePower BUILDS NO side.totalFainted , SO THE FEATURE VECTOR PRICES A FALLEN-COUNT MOVE AT ITS FLOOR — THE REMAINDER OF #244, OPENED 2026-08-14 BY SEARCH SO THAT IT HAS A HOME. #244 is closed on the SEED and this is not the seed: movePower calls a move's basePowerCallback against a hand-built stub whose side is { sideConditions: {} } , so Last Respects' 50 + 50 * pokemon.side.totalFainted returns NaN, the bp > 0 guard rejects it and the printed 50 is used — in every damage FEATURE MAG ranks an action with, at every depth of the game. IT IS A CLASS AND WAS COUNTED AS ONE: data/seed-source-audit.json records 29 legal moves carrying a basePowerCallback , of which 6 read state the stub does not build — Last Respects (totalFainted), Assurance (hurtThisTurn), Beat Up (allies), Payback (queue / newlySwitched), Spit Up (stockpile) and Water Shuriken (battle). A fix aimed at one member leaves the other five. IT IS FIT-INVALIDATING AND THAT IS MEASURED RATHER THAN ASSERTED: engine/feature_fixture.js 's 58 per-feature column hashes MOVE when movePower 's callback path is disabled, so the instrument is live and a change here reaches data/policy-weights.json . Not taken: Will's ruling is that the refit waits for MEDICHAM, and it belongs in the same batch as #276. INSTRUMENT OWED: a gate that FAILS while board.movePower returns the printed floor for the 6 basePowerCallback moves data/seed-source-audit.json enumerates as reading state the stub does not build. tests/test-rollout-fallen.js is GREEN and decides #244/#245/#246 — the SEED roster — not the feature path; engine/seed_source_audit.js counts the class and always exits o. CLOSED 2026-08-17 BY SEARCH, IN THAT BATCH — AND THE FIRST THING IT FOUND IS THAT THE CLASS OF SIX WAS WRONG IN BOTH DIRECTIONS. engine/seed_source_audit.js derived that six by SUBSTRING-MATCHING each callback's source against STUB_HAS_NO , a hand-typed set of fifteen field names — the ban-list-of-four shape CLAUDE.md warns about. It caught Water Shuriken on the battle inside hasAbility("battlebond") (the callback answers 15 perfectly well), and it MISSED Triple Axel (899 uses), Rage Fist (585) and Avalanche (29), none of whose fields are in that set. Asked of the callbacks instead — a member is one that THROWS or returns a non-number against the stub, which is the defect itself rather than a proxy for it — the real class is seven, carrying 9,163 corpus uses.	closed 2026-08-17 — SEARCH; gate tests/test-board-clock-power.js; artifact data/feature-shift.json

#	item	note
	WHAT IS SUPPLIED: <code>side.totalFainted</code> from <code>board.graveyard[side].size</code> (Last Respects, 7,306 uses, 50 -> 50/100/150/200); <code>side.pokemon</code> from the party so the FORMAT's own <code>onModifyMove</code> builds <code>move.allies</code> (Beat Up, 337 uses, which used to throw and fall back to its printed o — so <code>damaging</code> was false and the move was invisible to the ranking, the Rock Slide bug a fourth time); and <code>move.hit</code> = 1 (Triple Axel, whose ANSWER does not move — 20 either way — but whose printed value stops being a silent fallback). The ally filter is not re-implemented: <code>onModifyMove</code> is run on an <code>Object.create</code> copy because <code>move.allies.shift()</code> mutates, and the blast radius was measured rather than assumed — of the 29 legal callback moves exactly one also carries an <code>onModifyMove</code>. WHAT IS REFUSED, BY NAME AND DERIVED RATHER THAN LISTED (<code>board.unmodelledBasePower()</code>), the shape <code>rollout_leaf.unseededVolatiles()</code> uses); Rage Fist (<code>timesAttacked</code>) and Avalanche (<code>attackedBy</code>) need a per-body hit ledger the board does not keep, and building one would need a write in the live adapter AND in the offline replay or the fit and the player diverge; Payback (<code>queue.willMove</code>, 7 uses) is asking the TURN's resolution order, which is the search's question; Assurance COMPUTES its printed 60 and that is correct where it is asked, because a decision point is the top of a turn and Showdown clears <code>hurtThisTurn</code> at <code>nextTurn</code>; Spit Up returns <code>false</code> — the move fails — which is right with no Stockpile up, and the board records no layer count. THE FIT-INVALIDATION IS MEASURED (<code>data/feature-shift.json</code>): 12 of 58 features and 6 of 18 joint features move, on 0.687% of 51,399 candidate vectors over 300 fit games, 28.33% of games — <code>bp</code>, <code>isStatus</code>, <code>dmgFrac</code>, <code>koTarget</code>, <code>killIsRoll</code>, <code>koFirst</code>, <code>killsThreat</code>, <code>eff2</code>, <code>effHalf</code>, <code>tgtBulk</code>, <code>tgtMayProtect</code>, <code>defMismatch</code>, and six joint columns beneath them. R19'S MEASUREMENT IS CORRECTED HERE RATHER THAN LEFT: it said the 58 fixture hashes MOVE for this row, and they do not — R19 disabled the WHOLE callback path, which reaches Low Kick and Gyro Ball, and no fixture board carries Last Respects, Beat Up or Triple Axel, so this change is byte-identical on the frozen fixture. The invalidation is real and lives in the corpus, not in the fixture. VERIFIED BY: <code>node tests/test-board-clock-power.js</code> — shown RED on two deliberate breaks (the pre-#283 stub side; <code>onModifyMove</code> never run), with Beat Up's <code>isStatus</code> 1 -> 0 and the Last Respects vector's <code>dmgFrac</code> 0.429 -> 1.000 as the BEHAVIOURAL arms	
#284	A CAPABILITY COUNTER'S OWN COMMENT CARRIES A CORPUS NUMBER THAT IS 2.8x THE DERIVED ONE — RESIDUE OF #245, FILED 2026-08-14 BY SEARCH. <code>medicham2-browser.js</code>'s WIRE 125 note beside <code>fallenCount</code> says Last Respects has 19,299 corpus uses; #245's own row then quoted 6,650 from <code>data/tags.json</code>; the artifact says 6,970 today. Three numbers, one fact, none of them derived at read time — the prose-outliving-what-it-described shape this repository has recorded more times than any other, sitting inside the fix for a silent-fallback bug. Small, and worth doing because of WHERE it is: that sentence is the JUSTIFICATION for a guard, so a reader weighing whether the guard is worth keeping is weighing it against a number wrong by a factor of three. The derivation is one line — <code>data/tags.json</code>'s <code>moves.lastrespects.uses</code> — and the right repair is to stop transcribing it rather than to retype it. SEARCH may not edit that file	open — ENGINE, documentation defect
#285	THE DOCS-CURRENCY CENSUS WAS EXEMPTING FIGURES ON THE STRENGTH OF ITS OWN COMPLAINT ABOUT THEM, AND --update WAS THE #258 LAUNDERING BUTTON A SECOND TIME — FILED AND FIXED 2026-08-15 BY MEASURE. Three defects in one gate, found while chasing <code>docs/ABRA-whitepaper.md</code>: 12 untraceable figures, was 11 — a red SEARCH correctly refused to file as known. (1) SELF-REFERENCE. <code>engine/docs_scan.js#allArtifactNumbers</code> unions every <code>data/*.json</code>, and <code>data/docs-currency-baseline.json</code> IS one: it records the census's own findings as strings (<code>"docs/ABRA-whitepaper.md</code>	0.073
		<code>data/slowking-eval.json"</code>), figure down as an OFFE building it worse — re from 12 to 5 in ONE ru just called them untra citation_mismatches sinc excludes it: a gate ma ARRIVES THROUGH THE RE data/open-work.json is € so ANY figure quoted i quoted the offending i copied the row into th EVIDENCE FOR the figur regression still under source' must never bec the sweep for other pr — this file. **IT WAS closing it takes docs/ restores docs/ROLE-FAI that document from the zero), and puts the wh Those seven are a DISC row about a ratchet wi row happening. **The r --update DID BOTH JOBS did before #258: ok(ad
#286	<code>weatherSetupHelpsPartner</code> GUARDS ITSELF ON A FIELD NOTHING EVER WRITES, SO ITS "ONLY IF THE WEATHER IS NOT ALREADY UP" CLAUSE HAS NEVER BOUND — FILED	)!==" before setting once in the live tree* the repository is the

#	item 2026-08-17 BY SEARCH WHILE CLOSING #276. engine/board.js jointFeaturesFor reads norm(A.__weather **FIXED AND THE GATE IS GREEN – 2026-08-19 BY MEASURE, WITH THE SECOND DEFECT FIXED TOO AND A NEW ARM BUILT BECAUSE THE OLD ONE COULD NOT SEE IT.** engine/board.js now has stampSky(cands,board) – the assignment the field never had – and BOTH candidate producers call it: board.candidates (the fitter, the fixtures, the rollouts) and magnemite_candsFor, which builds its menu from the live REQUEST and does not route through the first. Stamping only one would have been the fitting/playing mismatch this repository names by its casualties, and the live half is PROVEN rather than assumed: a 4-game self-play run reports skyStamped=6779, jointSkyUnstamped=0, skyStampNotAnArray=0 . The value is board.weather , the expiry-aware accessor #276 built, never __weatherWord . **THE SECOND DEFECT THIS ROW RECORDED IS ALSO FIXED**: the loop reads setter.__weather instead of always A.__weather , so it asks about the half of the pair whose click is being judged. **AND THE STAGED ARM STRUCTURALLY CANNOT SEE THAT ONE** – both candidates of a real pair come off the same board and carry the same stamp, so the wrong-half read agrees by accident; reverting it left gate_weather_guard.js at exit 0. A fourth pass was added: with only the SETTER's field populated, the feature must read the same in either argument position. That is exact rather than sampled, so it cannot false-positive, and it is red on that revert. **BOTH HALVES SHOWN RED ON A DELIBERATE BREAK FIRST.** Gate exit 1 -> 0; selftest 9 cases, 0 failing. **REACH, MEASURED WEIGHT-FREE OVER THE FIT'S OWN PAIR ROWS** (joint_rows.build with a ZERO ranker and K above the longest candidate list, so no pair is dropped by a quarantined number; both arms from one process, so ENGINE editing the simulator under the run cannot separate them): over **2,000 corpus games, 21,757 joint decision points and 1,211,091 pair evaluations**, the feature fired **344 times before and 219 after – 125 suppressed, 36.3% of every firing of this column was a REDUNDANT weather setup**, on 64 of 21,757 decision points (0.29%). At 300 games the same figure reads 49.2% of 59 fires, so treat the small-sample version as noise around the large one. **THE ARGMAX-FLIP RATE IS WITHHELD, NOT UNMEASURED**: each flipped pair's score moves by exactly the fitted weight for this column, and that vector is quarantined – publishing a rank change would be quoting it. **A SEARCH-OWNED FILE (engine/board.js , and engine/magnemite.js) WAS EDITED UNDER A MEASURE BRIEF**, on the router's instruction, and that is on the record here rather than implied. tests/test-wiring.js caught the first attempt outright – __dropDeadVolatiles returns {cands,user,doubles}`, not an array – which is exactly what that file exists for; board/feature/wiring tests all green after	note	assigns it, on a candi !==w , which is true w and the pair scores "m when the sun is alread one line above says th the weather is not alr nothing"*), which is w carries **+1.311**, th board.js 's own note, so answer already – board is the expiry-aware on MOVES A FITTED JOINT C standing instruction i INSTRUMENT OWED: an ar weather setter beside ALREADY UP and FAILS w engine/feature_shift.js is BY MEASUREMENT 2026-08 THE CHEAP GATE FOR IT occurs **exactly once engine/board.js:483 – wit line copied into a fro anywhere**. So the gua empty w , and it has r same three lines and i B],[B,A]] and reads A. field would ask about BUILT, MEASURED AND RE prefixed fields read a engine/ ** and only ONE __seed , __setDB , __u __ABRA_BOARD_DATA ; see. Seven false posit ignore, which is the f casualties, so it is N with the alternative r specifies – stand a we type with that weather reads 1. A static scan IS BUILT – 2026-08-18 engine/gate_weather_guard engine/feature_fixture.js 's candidate pair the shi weatherSetupHelpsPartner = the function itself ca type and no weather an handed. THE ARM: the s Board's own door, boar candidates and vectors REPAIR CONTROL: the sa which must drop to 0 – proof this gate is not decoration. It goes th itself, because jointFe candidate objects are calling it green is th shape. **MEASURED: 2 c 2 staged, 2 STILL FIRI when the field is popu fit-invalidating and t node engine/gate_weather_
#287	data/seed-source-audit.json NAMES THE basePowerCallback CLASS BY SUBSTRING-MATCHING A HAND-TYPED LIST OF FIFTEEN FIELD NAMES, AND IT IS WRONG IN BOTH DIRECTIONS – FILED 2026-08-17 BY SEARCH WHILE CLOSING #283. engine/seed_source_audit.js STUB_HAS_NO is a literal array of fifteen strings tested with src.includes(k) against each callback's source text. Measured: it reports Water Shuriken as reading battle , which is the substring inside hasAbility("battlebond") – that callback answers 15 correctly and has never fallen back – and it MISSES Triple Axel (move.hit , 899 corpus uses), Rage Fist (pokemon.timesAttacked , 585) and Avalanche (pokemon.attackedBy , 29), because none of those field names is in the list. So the artifact's readUnmodelledState: 6 is neither an upper nor a lower bound on anything; the derived answer is seven callbacks that cannot produce a number, carrying 9,163 uses. This is the ban-list-of-four shape CLAUDE.md opens with, sitting inside the artifact that DEFINED a register row's scope. AND THE ARTIFACT NOW CARRIES A FALSE CLAIM: its openAndNotFixed block states that #244's remainder is not fixed because it owes a refit, which stopped being true when #283 closed. The repair is one line – call board.unmodelledBasePower(dex, board) , which asks the callbacks instead of their spelling and is self-correcting – and it is FILED RATHER THAN DONE because regenerating that artifact today would bake ENGINE's in-flight data/tags.json into it, and a measurement is a photograph. Until it is re-run, the file's	closed 2026-08-19 — MEASURE; gate engine/gate_seed_source_audit.js exit o (was 1); artifact data/seed-source-audit.json regenerated	

#	item	note
	basePowerCallbackClass and openAndNotFixed blocks are WITHHELD, not annotated CONFIRMED BY MEASUREMENT 2026-08-18 BY MEASURE, IN BOTH DIRECTIONS AND IN THE FALSE CLAIM. Read back off data/seed-source-audit.json (generated 2026-08-15T03:51Z, unchanged since): basePowerCallbackClass.readUnmodelledState is 6, and its readUnmodelledStatelist still carries watershuriken -> [battle] — the substring inside hasAbility(battlebond), a callback that answers correctly and has never fallen back — while ragefist sits in rows with unmodelledReads: []. Both directions of the row's claim reproduce exactly. AND THE FALSE CLAIM IS STILL IN THE FILE: openAndNotFixed asserts that #244's remainder is unfixed pending a refit, and #244 has read CLOSED to the register's own closed-detector since #283 landed — so the artifact asserts a register row is open that the register says is closed. board.unmodelledBasePower EXISTS (engine/board.js:3239) and is exported (:4438), so the one-line repair the row describes is available and unblocked in the code; what blocks it is that regenerating the artifact today bakes ENGINE's in-flight data/tags.json into it. basePowerCallbackClass and openAndNotFixed stay WITHHELD. INSTRUMENT OWED: a check comparing the artifact's readUnmodelledStatelist against board.unmodelledBasePower(dex, board) — asking the callbacks instead of their spelling, which is self-correcting where a fifteen-string list is the ban-list-of-four shape THE OWED INSTRUMENT IS BUILT — 2026-08-18 BY MEASURE, ON BOTH OF THE ROW'S CLAIMS. engine/gate_seed_source_audit.js compares the artifact's readUnmodelledStatelist against board.unmodelledBasePower(dex, board) — asking the callbacks instead of their spelling — and separately checks every row the artifact's openAndNotFixed block asserts is open against the register's closed-detector, which is IMPORTED from quarantine.js and never copied. The derived side is a UNION over an empty board plus every canonical fixture board, stated as a decision: unmodelledBasePower takes the caller's board, so a callback can answer on one position and fall through on another (beatup reads allies), and naming one board would make the verdict a property of that board. MEASURED on the artifact as generated 2026-08-15T03:51Z: claimed {assurance, beatup, lastrespects, payback, spitup, watershuriken}, derived {avalanche, beatup, payback, ragefist} — 4 claimed and not derived, 2 derived and not claimed, plus #244 asserted open while the register reads it CLOSED. Exit 1, all three ways. The row's earlier figure of seven derived callbacks is NOT reproduced here and is not repeated: it came from a different derivation, and unmodelledBasePower reports only the callbacks that cannot produce a number at all. It regenerates nothing, for the reason this row already gives. VERIFIED BY: node engine/gate_seed_source_audit.js FIXED AND THE GATE IS GREEN ON BOTH CLAIMS — 2026-08-19 BY MEASURE, AND THE ROW'S OWN PREMISE FOR NOT DOING IT WAS MEASURED AND FOUND FALSE. STUB_HAS_NO IS GONE. engine/seed_source_audit.js now derives the class by calling board.unmodelledBasePower(dex, board) — asking each callback to produce a number and recording the ones that cannot — over a UNION of an empty board, its own staged board and every canonical fixture board, with the board count recorded in the artifact and a fixture that will not build REPORTED rather than swallowed. Nothing in the file reads a callback's spelling any more, so it is self-correcting. The derived class is 4: avalanche, beatup, payback, ragefist — and the row's earlier count of seven including Triple Axel is exactly the kind of figure a derivation fixes without an edit, because #283's mv.hit = 1 made Triple Axel answerable after that list was typed. THE FALSE CLAIM IS GONE TOO, AND ITS STATE IS NOW DERIVED: openAndNotFixed keeps its AUTHORED caveat text and reads OPEN/CLOSED from docs/ROADMAP.md through quarantine.roadmapRowIsClosed, imported and never copied; #244 reads CLOSED, so the caveat is DROPPED with its reason recorded rather than annotated, and an unreadable register WITHHOLDS every caveat instead of publishing on an unchecked premise. THE ROW SAID REGENERATING WOULD BAKE ENGINE'S IN-FLIGHT data/tags.json INTO THE ARTIFACT. MEASURED: IT DOES NOT. With engine/tags.js and data/tags.json both made unreadable by a preload, the regenerated artifact is BYTE-IDENTICAL apart from its timestamp — the file's own header already declared it does not read tags, and that declaration is now checked rather than trusted. THE GATE HAD ITS OWN LESSON IN THE SAME PASS: arm 2 read .row off a single object, and when the repair turned openAndNotFixed into {checkedAgainst, rows, dropped} it found ZERO assertions and went GREEN — a check that stops looking and calls the silence a pass. Found by breaking the artifact deliberately and noticing the arm did not shout. It now accepts three shapes and REFUSES anything else with exit 2. Gate exit 1 -> 0 (4 claimed-not-derived, 2 derived-not-claimed and 1 false claim all to zero); shown red on two separate deliberate breaks, one per arm	

#	item	note	
#67	ILLUSION IS NOT MODELLED, AND TWO MECHANICS FILED UNDER OTHER NAMES ARE ACTUALLY THIS — ROW ADDED 2026-08-15, CITED SINCE BEFORE IT EXISTED. <code>docs/ENGINE.md:201</code> has been declaring a gap "with the reason 'Illusion is not modelled (ROADMAP #67)', which is still true" against a register row that did not exist — <code>tests/test-roadmap-register.js</code> caught it as <i>"1 item(s) a ledger schedules and the register does not name — these exist only in a file nobody opens"</i>. The citation was right and the row was missing, which is the inverse of the failure the register-reality work spent the night on: there, rows outlived their defects; here, a defect outlived its row. IT IS LIVE, AND THE EVIDENCE IS FRESH. ENGINE's 2026-08-15 pass over the 69 diverging mechanics found <code>bittermalice</code> and <code>nightdaze</code> diverge because of Illusion, not because of the moves they are filed under — the staged body is displaying as something it is not, so the comparison parts on the name before it ever reaches the move. Both remain in the 53. WHAT IT WOULD TAKE: the display name and the true species must be separable in the stream, which touches every line that names a body, so this is not a tag wire — it is a change to how the engine identifies a Pokemon to the protocol. NOT SCOPED HERE deliberately: sizing it belongs with whoever takes it, and a row that guesses its own cost is how #266 came to say 41 against a true 32	CLOSETED 2026-08-18 — see ROADMAP #160, and the filing above was wrong. The EVIDENCE stands and is more precise than the row states: <code>bittermalice</code> (<code>Zoroark-Hisui</code>) and <code>nightdaze</code> (<code>Zoroark</code>) are the ONLY two rows in <code>engine/all_mechanics_fire.js</code> 's whole 964-row population whose carrier holds Illusion, and both diverge with <code>switch: a different body</code> — the ability announcing the body under another name. What was wrong is the STATUS: Illusion is not open work, it is SHELVED, and <code>engine/game_differential.js</code> has said so in its own words since 2026-08-13 (<i>"a body pretending to be another body makes every divergence it causes unreadable as a rule defect"</i>), dropping every team carrying a legal carrier from the sample. <code>engine/all_mechanics_fire.js</code> did not read that shelf, which is the whole of ROADMAP #291. It does now — <code>GD.CLOSET_SPECIES</code> is imported, never re-derived — so the two rows are staged, played and printed with the owner's reason and stop counting: the mechanics clause reads 49, not 51. DO NOT MODEL ILLUSION.	
#289	THIS ENGINE COULD NOT EMIT A ZERO-MAGNITUDE STAT LINE AT ALL, AND THE MISSING LINE WAS THE SMALL HALF — FIXED 2026-08-18 BY ENGINE. <code>medicham2-browser.js</code> 's trace emitter opened <code>bst(m,eng,d,from){ if(!d) return; }</code>, and <code>d</code> is the delta that actually LANDED — so a body already at a cap produced no protocol line. Showdown emits one: <code>sim/battle.ts:2073-2078</code>, the branch taken when <code>boostBy</code> is falsy, with <code>msg</code> decided at :2040-2044 (<code>-unboost</code> when <code>`boost[boostName] < 0</code>		<pre>target.boosts[boostName] == does not override boost source alone**: three Swords-Dances to +6, I</pre>

#	item	note
#290	THE ordering FAMILY IS A REAL TURN-ORDER DEFECT, NOT THE RETIRED SPEED-TIE ARTEFACT — MEASURED 2026-08-18 BY ENGINE. ordering is the shape "same event, different slot", and its biggest member is two `	move lines naming the same like the pinned speed documents that history CANNOT BE ONE**: CHANG ROOT — the selection s tie is resolved by the constant pinned die — speed_tie: "to the EARLIER argued: order_probe in pair with both bodies' moves' priority read o games: 16 tied, 9 NOT 20, 30, 32, 42, 56, 64 under Trick Room with first, medicham2 the . whose priorities match ordering — 220 games, t announcement families. QUEUE-BUILD TIME**, an reading is weak eviden OWED**: a gate that FA order_probe carries sp conjunction is the def reads zero the day the RUN — 2026-08-18 BY ME KILLED.** node engine/q this row asked for: it speed_tied:false AND san games, and it has thre absent* can never read probe array, or a prob defect is present; exi to register_reality.js del artifacts before being DIFFERENT priority pas conjunction turns two LAST ANSWERABLE ARTIFA A/middle): 26 move-vs-CONJUNCTION across 11 42, 56, 64, 76, 76, 14 every die pinned on bo three are Protect. **A THAT THEY ARE NOT THE COINCIDENCE WAS.** The count are two DIFFEREN is **228 games**, and divergence_shape.shapeOff(**229**. Cross-tabbed: inside the class event ordering that shape as the two SETS differ by them as one number was cause. **WHAT IS ESTAB move-vs-move pairs onl never a count of it; e extrapolation wearing here. **THE EVIDENCE I THAN QUOTING IT**: dat 04:09Z and release 48i was cut at 05:05Z, so WITHHELD rather than a same command decides t order-probe **THE ARTIF 2026-08-18 BY MEASURE. release (978ca8fe72c9 , longer exits 2 on a stale artif: the conjunction — not sp 5 distinct games, at speed are Tailwind or Protect. This and the two do not subtract: runs. The probe covers move ordering class and never a cc
#291	THE MECHANICS CLAUSE DID NOT HONOUR THE CLOSET AND THE ROSTER CLAUSES DID, SO TWO INSTRUMENTS DISAGREED ABOUT A DECISION THE OWNER HAD ALREADY MADE — FIXED 2026-08-18 BY ENGINE. tests/roster.js declares DEFERRED , seven entities Will shelved BY NAME with his own quote and date, and its header states the contract: "A row in here is still staged, still played against the authority, and still printed on every run with its reason and its date. The only thing it stops doing is holding the MEDICHAM gate shut." engine/quarantine.js prints that shelf and its three deliberate-roster clauses respect it. engine/all_mechanics_fire.js honoured neither that shelf nor the differential's. Measured before a line changed: abilities:forewarn and items:metronome sat inside the failing count of 53 — and metronome	closed 2026-08-18 — ENGINE; gate tests/test-closet-scope.js , shown RED on the pre-fix artifact (6 checks failing) with a CONTROL asserting the shelf is 1.04% of rows and not a blanket

#	item	note	
	was the ONLY item in it, so the item clause read 1 where the honest answer is 0 — and moves:bittermalice / moves:nightdaze sat in the moves count on ILLUSION carries the differential drops whole teams for (ROADMAP #160, #67). IMPORTED, NEVER RE-DECLARED: roster.js now exports DEFERRED and game_differential.js exports CLOSET_SPECIES (derived from the ABILITY, so a carrier added next regulation is covered), and the mechanics runner reads both. A copy of seven names would agree the day it was written and disagree the first time either moved — the ban-list-of-four shape. THE SUBTRACTION IS PUBLISHED, NOT APPLIED QUIETLY: summary[kind] carries diverged, diverged_including_shelved, shelved_by_owner and shelved_by_owner_diverging, and the gate's own tail prints how many rows the shelf removed. MECHANICS CLAUSE 53 -> 49 (moves 37 -> 35, abilities 15 -> 14, items 1 -> 0)		
#292	THE WHOLE-GAME GATE PRINTED A COMPOSITION FROM ONE RUN BESIDE A HEADLINE FROM ANOTHER, AND A RATE FROM ONE PIN BESIDE A BASELINE FROM ANOTHER — FIXED 2026-08-18 BY ENGINE. Two instances of one shape. (1) wholeGameClause took its headline from data/game-differential.json and its shape counts from data/divergence-report.json, a separate file written by a separate run, with nothing comparing them. Measured 2026-08-17: the headline read 287 of 1539 off release 0c5bb83c5744 while the composition beside it (emission 132, rule 91, ordering 39, unparsed 34, field 5) came from a report whose own run block records 983 games, 303 diverged, release a81663f17c0c . Shaping the current artifact gives EMISSION 116 / RULE 83 / ORDERING 52 / UNPARSED 35 / FIELD 1 — field alone was wrong by 5x. The fix is STRUCTURAL rather than a freshness check: the composition is computed from the artifact the headline came from, through engine/divergence_shape.js, so there is no other run it could describe. (2) The same clause compared the run's rate against data/whole-game-baseline.json without reading the mode the baseline records. Measured: baseline A/top-tie-first/pins:ef342837b791 at 30.8%, run A/middle/pins:1fd77b835ee2 at 61.2%, and the clause printed AND IT GOT WORSE about a 30-point rise that is entirely the instrument changing corner — the same run's own top-tie-first arm read 14.7%. The artifact's own pins.why says "ONE PIN IS ONE CORNER." The trend is now WITHHELD , not annotated, when the modes differ; the VERDICT does not move, because correctness decides it and zero is zero under any pin. AND unparsed IS NAMED FOR WHAT IT IS — not a fifth kind of disagreement but the comparator admitting it has no grammar for the pair; the dominant class is printed beside the count (12 of 12 are drag: a different body, a single `	drag	` line rather than a pair)
#293	A REFUSED PRIORITY MOVE NAMED A NEIGHBOUR'S ABILITY THAT DOES NOT REFUSE PRIORITY, AND AN ALLY-AIMED HELPING HAND NAMED ITS OWN USER — FIXED 2026-08-18 BY ENGINE. Two dropped facts, both narration, both derived. (1) priorityBlockAbilities() reads the blocksMove tag and keeps only members whose param says what === 'priority'; the line that found the HOLDER for the `	cant	announcement asked T/ filter. Good as Gold c it refuses STATUS move standing beside the re implementations of one the one deciding the N Scizor clicking Feint

#	item	note	
#294	THE ACCURACY ROLL WAS ONE PER MOVE WHERE THE GAME ROLLS ONE PER TARGET, SO ROCK SLIDE COULD NOT HIT ONE AND MISS THE OTHER — FIXED 2026-08-18 BY ENGINE. Will: <i>"rock slide and heat wave can hit one and miss the other."</i> <code>engine/medicham2-browser.js</code> declared the divergence at its own roll site — <i>"ONE ROLL PER MOVE, NOT PER TARGET... a SPREAD move rolls once, against no defender, and gets the attacker's modifiers only"</i> — and the authority is <code>hitStepAccuracy(targets: Pokemon[], pokemon, move)</code> (<code>sim/battle-actions.ts:690-692</code>), which takes an <code>ARRAY</code>, sets <code>this.battle.activeTarget = target</code> per iteration and ends each one in <code>randomChance(accuracy, 100)</code> at :737. THE DISTRIBUTION WAS WRONG, NOT ONLY THE VARIANCE: a 90-accuracy spread move into two live bodies is $81 / 18 / 1$ in the authority and was $90 / \mathbf{0} / 10$ here, so the eighteen-percent middle case did not exist in this engine at all. Reach: Rock Slide 18,122 clicks, Heat Wave 11,121, plus Dazzling Gleam, Hyper Voice, Matcha Gotcha, Eruption — every sub-100-accuracy spread move in the format. THE STATED OBJECTION WAS OBSOLETE AND THAT IS WHY IT LANDED NOW: <i>"it would change how much rng every existing seeded run consumes"</i> was true of a <code>SEQUENCE</code>; the pinned corners feed a <code>CONSTANT (scalar = () => spec.corner</code>, so extra draws change nothing — measured, both corner arms bit-identical at 59/260 and 75/260 before and after) and the middle arm feeds <code>EVENT-ADDRESSED</code> dice keyed on <code>`hash(seed\</code>	turn\	category\
#295	THE REACH SHELF MIXES TWO UNITS AND SO IS TWO THRESHOLDS WEARING ONE NUMBER — FOUND 2026-08-18. <code>engine/quarantine.js</code> 's <code>REACH_SHELF</code> is the integer 25, and it is compared against <code>TEAMS</code> for an ability (denominator 26,232 open-sheet teams) and against <code>CLICKS</code> for a move (denominator 1,259,717). So the same literal means 0.095% of teams or 0.0020% of clicks — a move clears the bar at roughly 48x lower relative usage than an ability. That is not a judgement anyone made; it is an artefact of two usage artifacts having different denominators, and it is invisible because both print as a bare integer beside a name. The shelf is NOT wrong today and this is not urgent: Will set it at 25 deliberately (2026-08-18, <i>"leave it at 25"</i>) and the five rows holding the mechanics clause shut — <code>cursedbody</code> 2,177 teams, <code>toxicdebris</code> 1,840, <code>disable</code> 1,799 clicks, <code>regenerator</code> 1,596, <code>poltergeist</code> 1,383 — clear any threshold in the plausible range, so no verdict turns on it. It becomes load-bearing the moment anyone tunes the number, which is exactly when nobody will notice the units. THE FIX IS A RATE, NOT A BIGGER INTEGER: express reach as a share of its own denominator and shelve below one rate, so the two populations are asked the same question. Print the denominator beside every row so the unit is never implicit again. Whoever does it must re-derive the shelved/counted split under the new rule and say which rows <code>MOVED</code>, because a re-unitised threshold is a new threshold and the 34/15 split published in <code>CHANGELOG 5.36.0</code> does not carry across it FIXED 2026-08-18 BY MEASURE, AND IT MOVED NOTHING, WHICH IS THE TEST IT HAD TO PASS. The anchor is unchanged and is now named in its own unit: <code>REACH_SHELF_CLICKS = 25</code>, which is where the number has always come from (<code>tests/roster.js:1517</code> <code>USAGE_SHELF_BELOW = 25</code>, applied at :1522 as <code>clicks >= USAGE_SHELF_BELOW</code> — a clicks threshold, ruled on as a clicks threshold; Will, 2026-08-18: <i>"leave it at 25"</i>). The common unit is OCCURRENCES PER STORED GAME, which both instruments already express and neither has to be reinterpreted to reach: a move's rate is <code>clicks / click-counts.store_games</code> (64,846), an ability's is <code>teams / sheet-usage.sheet_games</code> (13,116). The teams threshold is <code>DERIVED</code> at the anchor's rate — $25 \times 13,116 / 64,846 = \mathbf{5.06\ teams}$, so 6 teams is the smallest counting observation — and the comparison is integer cross-multiplication (<code>n * 64846 < 25 * denom</code>), so no float boundary can move a row and the move shelf is preserved bit-for-bit at <code>n < 25</code>. THE SPLIT REPRODUCES EXACTLY: 34 counted, 15 shelved, before and after, with no row moving in either direction. It reproduces because no diverging ability sits between 6 and 24 teams — the lowest counted one is <code>angerpoint</code> at 25 — and every shelved row is a move decided by the	CLOSED 2026-08-18 — MEASURE	

#	item	note
	unchanged anchor. A RE-UNITISED THRESHOLD IS STILL A NEW THRESHOLD and the equality is a measured property of today's population rather than of the change: an ability diverging at 6-24 teams used to be shelved and now COUNTS, which is the direction the defect demanded. ANCHORING THE OTHER WAY DOES NOT REPRODUCE, WHICH IS WHY THE ANCHOR IS NAMED RATHER THAN ASSUMED: taking the ability figure as the anchor (25 teams = 1.906 per thousand games) puts the move shelf at 124 clicks and shelves twelve currently-counted moves, from <code>move:ficklebeam</code> (112) down to <code>move:attract</code> (30). Same fix, opposite result. THE DENOMINATOR NOW TRAVELS WITH EVERY COUNT — <code>reachOf</code> carries <code>denom / denomLabel</code> , the clause's own summary prints <code>2177 teams/13,116 open-sheet games</code> , and <code>--reach</code> prints the population and the applicable shelf on every row — because the actual defect was that a bare integer beside a name cannot carry its unit and one header line is not a fix. Two more literals were untangled in the same pass: <code>coverageClause</code> reads the CLICKS anchor because its loop walks <code>click-counts.json.moves</code> and nothing else, and the decision-impact sample floor became <code>DECISION_POINTS_FLOOR</code> — the same 25 in a THIRD unit (paired decision points), which is exactly how one name came to mean three things. Shown RED on a deliberate break first: reverting <code>below()</code> to the bare integer fails <i>"an ability at 6 teams (above the derived shelf) COUNTS where the bare integer shelved it"</i> (quarantine selftest 77/1)	
#296	THE GATE READ THREE KEY NAMES THE VERDICT ARTIFACT HAS NEVER WRITTEN, SO ZERO ROWS HAVE EVER CROSSED THAT WIRE — FOUND AND FIXED 2026-08-18 BY MEASURE. <code>engine/quarantine.js</code> 's open-defect clause read <code>`rr.rows</code>	<code>rr.verdicts</code> , <code>v.command</code> <code>cmd</code> and <code>green</code> . **Thre every run the clause h bucket, withRed was s open row names an inst a failing measurement* loudest. #258 has carr instrument exits 1 tod FAILURE TO THE LETTER* comparing the two file a capability was absen crashed, nothing was s true. **THE FIX IS A D tolerant reader (<code>rr.res</code>
#297	THE CLOSED-DETECTOR LET PROSE OUTRANK AN <code>open</code> STATUS CELL, HIDING THREE LIVE DEFECT ROWS INCLUDING THE LARGEST FAMILY IN THE WHOLE-GAME DIFFERENTIAL — FOUND AND FIXED 2026-08-18 BY MEASURE. <code>roadmapRowIsClosed</code> cashed in #148's prescription in ONE direction only: a status cell saying <code>closed</code> outranked the prose scan, and a status cell saying <code>open</code> did not. So a row whose narrative contains <code>CLOSED 2026-08-11</code> about a part that IS closed read as a closed ROW. MEASURED BEFORE CHANGING IT, over all 217 register rows: EXACTLY FIVE verdicts move — #20, #177, #218, #220, #224 — every one from closed to open, and every one has a cell beginning with the literal word <code>open</code> . THREE OF THE FIVE ASSERT BREAKAGE: #218, #220, #224. So the MEDICHAM gate's open-defect clause has been blind to three live engine-defect rows, and <code>engine/open_work.js</code> , which shares the detector, has been omitting them from the work list this project prints instead of typing. #220 IS THE EXPENSIVE ONE. Its <code>-fail</code> vs <code>-singleturn ... protect</code> family reads 238 games of the 695 in <code>data/game-differential.json</code> — the largest single family in the artifact, 34% of the divergence — in a row the gate could not see. Its head says <code>CLOSED 2026-08-11</code> about the WEATHER RESIDUAL half; it was reopened the next day and its cell has said <code>open</code> — <code>engine</code> DEFECT ever since. THE FIX IS THE SAME RULE AS NOT A DEFECT : an explicit ruling in the cell outranks a guess over prose, and the direction is the safe one — it can only ever make the gate MORE shut. <code>open</code> must START the cell, so a closed row mentioning an open question is unaffected. Shown RED on a deliberate break first: removing the guard fails <i>a cell that says OPEN outranks a CLOSED 2026-08-11 inside the row narrative.</i> CONSEQUENCE: the open-defect population goes 5 -> 8 — #218, #220 and #224 join it, all as debt, none with an instrument. That is three defects the register always	CLOSED 2026-08-18 — MEASURE

#	item	note
	knew about and the gate did not. VERIFIED BY: <code>node engine/quarantine.js --selftest</code>	
#298	THE WHOLE-GAME CLAUSE PUBLISHES A RATE WITH NO RELEASE CHECK, SO TODAY'S HEADLINE 690 of 1230 DESCRIBES BYTES THE TREE NO LONGER HAS — FOUND 2026-08-18 BY MEASURE, NOT FIXED. <code>mechanicsClause</code> refuses an artifact whose <code>release</code> differs from <code>data/engine-release.json</code> (<i>that is not a weaker answer, it is an answer about other bytes</i>), <code>decisionImpact</code> refuses one, and the new <code>orderProbeClause</code> refuses one. <code>wholeGameClause</code> does not compare releases at all. Measured: <code>data/game-differential.json</code> is stamped release <code>6875c8ace00e</code>, generated 2026-08-18T04:09:52Z; <code>data/engine-release.json</code> reads <code>488fd1bf3f7c</code>, cut 2026-08-18T05:05:47Z for #294 — the per-target accuracy roll, which is the largest single behaviour change of the day and lands directly on spread moves. So the printed 56.1%, its class composition, the 228-game <code>ordering</code> class and the 238-game Protect family are all measured against a build that no longer exists, and the clause says nothing about it. A CAPTION IS NOT A QUARANTINE and this does not even have a caption. NOT FIXED IN THIS PASS DELIBERATELY: making the headline clause refuse would withhold the project's most-quoted engine figure while ENGINE is cutting releases hourly, which is a judgement with an owner rather than a repair. INSTRUMENT OWED: the same release comparison the other three clauses already make, plus a decision about whether a stale rate is WITHHELD or reported as <code>PRE-RELEASE FIXED 2026-08-18 BY MEASURE, AND THE HEADLINE WAS WITHHELD BEFORE IT WAS RE-MEASURED</code>. <code>wholeGameClause</code> now makes the same release comparison its three siblings make, and on a mismatch NOTHING MEASURED COMES BACK: no rate, no <code>diverged</code>, no <code>games</code>, no class composition, and none of those numbers in the verdict string either. What comes back is which release it wanted and which it got, plus the command that repairs it — the only fact the artifact still supports. That is CLAUDE.md's ruling applied rather than re-argued: <i>the figure must be WITHHELD, not annotated; printing it with a caveat is the bug</i>. It refuses a MISMATCH and not an ABSENCE, exactly as <code>orderProbeClause</code> does: an unstamped artifact, or a tree with no <code>engine-release.json</code>, still answers. Four selftest cases drive the shipping function on injected artifacts and assert the RED case twice — that the clause fails, AND that the figures are absent from what it returns — because a withheld number still in the string is the same bug with a different word in front of it. The exit mapping is <code>clauseExit</code>, one implementation shared with <code>--order-probe</code>, and a withheld verdict maps to 2 and never to 0. THE FIGURE WAS THEN RE-MEASURED RATHER THAN LEFT WITHHELD: <code>engine/game_differential.js --games 1230 --write</code> on the tree's own release, 995 games delivered (the swarm's pool, not the request), giving 696 of 995 = 69.9% stamped to <code>978ca8fe72c9</code>. It is NOT comparable with the 690/1230 this row was filed about — and the paired arm proves the difference is not the engine: replaying release <code>6875c8ace00e</code> under today's steering gives 700 of 995, the same class composition to the game. VERIFIED BY: <code>node engine/quarantine.js --selftest</code>	closed 2026-08-18 — the clause withholds every figure on a release mismatch, names both ids, and the differential was re-run so a live number exists: 696 of 995 = 69.9% on <code>978ca8fe72c9</code> . Was: open — measure DEFECT, engine/quarantine.js
#299	THE WHOLE-GAME HEADLINE IS COMPUTED OVER A DENOMINATOR THAT INCLUDES 350 VOID GAMES, AND THE ARTIFACT DOES NOT CARRY THE NUMBER THAT WOULD FIX IT — FOUND 2026-08-18 BY MEASURE WHILE RE-RUNNING THE DIFFERENTIAL. <code>engine/game_differential.js</code> declares its own rule in the middle arm's description: <i>a game whose draw counts diverge is VOID, not a divergence</i>. Its stdout reports all three figures — <code>DIVERGED</code> (primary arm middle): 700 of 995, <code>VOID</code> (instrument desync): 350 of 995, <code>DIVERGED</code> among the 645 usable games: 628 (97.4%) — and the ARTIFACT persists only <code>games: 995</code> and <code>diverged: 700</code>. So <code>wholeGameClause</code> publishes 700 (less declared) over 995, and by the run's own arithmetic 72 of those 700 diverged games are themselves void: 700 — 628 = 72, and 995 — 350 = 645. THE HONEST FIGURE IS WORSE, NOT BETTER. On the games where the instrument is valid the engines disagree about 628 of 645 = 97.4%, where the clause's own ratio is 69.9%. Neither number can be published by the clause today, because the void count exists only in a console line nobody parses. This is not a request to change the bar: the clause is green only at zero either way. It is that the most-quoted number in this project is a ratio of two quantities that do not describe the same population, and a reader cannot tell from the artifact. MEASURED on release <code>978ca8fe72c9</code>, 995 games, arm A/middle, census <code>2e3953f1f882</code>, pool <code>631d4ea60a80</code>; the identical split appears on release <code>6875c8ace00e</code> under the same steering. INSTRUMENT OWED: the artifact must persist <code>void</code> and <code>usable</code> beside <code>games</code> and <code>diverged</code>, and then <code>wholeGameClause</code> must publish the usable rate — a clause cannot derive a denominator that was never written down. LANDED 2026-08-18 BY ENGINE, AND THE POPULATION WAS NOT WHAT THE VOID RULE SAID IT WAS. <code>mid_void</code> is now in the artifact — <code>void_games</code>, <code>usable_games</code>, <code>diverged_among_usable</code>, <code>diverged_rate_over_usable</code>, and every verdict the check reached	CLOSED 2026-08-18 — engine EMPHASIS MARKERS REMOVED FROM THE FRONT OF THIS CELL 2026-09-04 BY MEASURE — the leading asterisks defeated the register's cell clause, so the row read OPEN against its own CLOSED verdict. Only the notation changed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: <code>tests/test-middle-draw-scope.js</code> is on disk and carries its <code>--mid-carry-nth</code> arm at seven sites; it was not run here.

#	item	note
	BY CAUSE. Printing the causes killed the headline: 340 of 350 voided games had drawn no comparable address at all, 275 of them because NEITHER engine rolled a die in the whole game (median 12 turns, the cap). Under the count-based check an empty side meant the streams had parted; under shared-address identity there is nothing to disagree about, and the rule already said so two clauses down for the OUTCOME log while contradicting itself for the full one. A dice-free game is the most trustworthy evidence in the run, not the least. AND THE REAL DESYNC WAS #302, ONE FILE OVER. Void on the same 797-game sample: 350 (old rule) -> 11 (rule corrected) -> 8 (#302 fixed). VERIFIED BY tests/test-middle-draw-scope.js, whose \$6 re-runs itself with <code>--mid-carry-nth</code> and fails if the broken arm passes	
#300	THE CATCH BLOCK THAT REPORTS A FAILED SHOWDOWN WRAP CANNOT RUN, SO A BAD SHOWDOWN_PATH KILLS THE DIFFERENTIAL WITH A TEMPORAL-DEAD-ZONE ERROR NAMING NEITHER CAUSE NOR CURE — FOUND 2026-08-18 BY MEASURE, BY WALKING INTO IT. <code>engine/game_differential.js:244</code> assigns <code>MID_WRAP_ERROR = e.message</code> inside the catch that guards the middle-arm wrapper; <code>MID_WRAP_ERROR</code> is declared <code>let</code> at :743, five hundred lines below. The assignment is therefore in the variable's temporal dead zone and THROWS. Reproduced: a <code>SHOWDOWN_PATH</code> carrying one trailing space (<code>set VAR=x && ...</code> in cmd keeps it) makes the <code>require</code> fail, and the process dies with <code>ReferenceError: Cannot access 'MID_WRAP_ERROR' before initialization</code> at line 244 — which names the reporting variable and says nothing about the path, the <code>require</code>, or the wrapper. THE ERROR PATH IS THE PART THAT DOES NOT WORK, which is this repository's founding shape pointed the other way: not a capability absent and everything reporting success, but a capability whose FAILURE REPORT is the thing that is broken. It is latent while the wrap succeeds, so no run that worked has ever touched it, and <code>MID_WRAP_ERROR === null</code> is asserted by a pin claim that can never see this branch. Cost measured at one wasted run and about ten minutes of looking for an engine bug that was a shell quoting mistake. The repair is to hoist the declaration above the try. INSTRUMENT OWED: an arm that loads the driver with a deliberately wrong <code>SHOWDOWN_PATH</code> and asserts the process reports the <code>require</code> failure rather than a <code>ReferenceError</code>	open — engine DEFECT, engine/game_differential.js (ENGINE; filed by MEASURE, not fixed)
#301	switch lookups that MISSED: medicham 22, showdown 0 ON A LINE THE RUN ITSELF MARKS MUST READ 0 , AND NOTHING GATES IT — FOUND 2026-08-18 BY MEASURE. The whole-game differential prints this counter with its own bar attached and its own consequence spelled out beside it: <i>a miss means that side PASSED while the other switched</i>. So on those lookups the two engines are not playing the same game at all — one body left the field and the other did not — and every line after it is compared across two different positions. MEASURED: 22 misses on the medicham side and 0 on the authority's, on release 978ca8fe72c9 over 995 games, and the identical 22/o on release 6875c8ace00e under the same steering, so it is not a regression from either of today's landings. It is reported here rather than repaired because <code>engine/game_differential.js</code> and the simulator are ENGINE's and were live while this ran. The counter is not persisted in the artifact either, so nothing downstream can read it. INSTRUMENT OWED: the miss count published in the artifact and a clause that fails while it is non-zero, since a run marked MUST READ 0 that reads 22 is currently caught only by a human reading stdout	open — engine DEFECT, engine/game_differential.js (ENGINE; filed by MEASURE, not fixed)
#302	THE AUTHORITY'S REPEAT INDEX SURVIVED THE GAME BOUNDARY, SO THE TWO ENGINES DREW DIFFERENT VALUES FOR THE SAME EVENT IN EVERY MEASURED GAME — FOUND AND FIXED 2026-08-18 BY ENGINE. The middle arm's die is <code>`FNV1a(seed</code>	turn category

#	item	note
#303	THE LARGEST CLASS IN THE WHOLE-GAME DIFFERENTIAL WAS A SHARED DIE READ BACKWARDS ON ONE SIDE — FOUND AND FIXED 2026-08-18 BY ENGINE. <code>-damage field 3</code> was 226 of 491 diverging games while <code>tests/test-engine-diff.js</code> reads 0 of 6000 at each corner and <code>game_differential.js</code> 's own header says <i>"the engines agree about damage"</i>. Both were true: the corner is exactly where the defect cannot show. Showdown's <code>random(16)</code> is an INDEX and <code>randomizer</code> is <code>tr(tr(d*(100-i))/100)</code>, so i=0 is the MAXIMUM; <code>medicham2</code> draws a POSITION in its span, <code>d.min + floor(u*(d.max-d.min+1))</code>, increasing in u. The PINNED arms already encode the correspondence — <code>CORNER_TOP</code> sits beside <code>damageIndex: 0</code>, <code>CORNER_BOTTOM</code> beside <code>damageIndex: 15</code> — and the middle arm handed Showdown <code>floor(u*16)</code>, the inverse. A shared die read backwards on one side is ANTI-CORRELATED, which is worse than two independent dice. Arm split, one 969-game run, complete populations: 226 in <code>middle</code>, 2 of 155 in top-tie-first, 0 of 183 in bottom-tie-first; sampled HP deltas both signs to +/-18, median 5 — the span, not a rounding step. Fix narrowed to <code>MID_CAT === 'dmg' && m === DAMAGE_ROLL_SIDES</code> because <code>random(16)</code> elsewhere is a 1-in-16 chance (#260). Paired on 797 games, the mapping the only thing that moved: <code>-damage field 3</code> 204 -> 137, diverged 452 -> 409, void 8 -> 4. TWO THIRDS OF THE FAMILY SURVIVES AND IS NOT EXPLAINED — the standing candidate is a REAL <code>medicham2</code> defect, <code>damage_interior</code> : 20 producible HP values against the authority's 14 on one staged hit, 6 of them the authority can never emit. VERIFIED BY <code>tests/test-middle-damage-roll.js</code>, which derives the mapping from the pinned arms' own declarations and re-runs itself with <code>--mid-damage-uninverted</code>	CLOSED 2026-08-18 — engine; the 137-game residual is #304 EMPHASIS MARKERS REMOVED FROM THE FRONT OF THIS CELL 2026-09-04 BY MEASURE — the leading asterisks defeated the register's cell clause, so the row read OPEN against its own CLOSED verdict. Only the notation changed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: <code>tests/test-middle-damage-roll.js</code> is on disk and carries its <code>--mid-damage-uninverted</code> arm at five sites; it was not run here.
#304	medicham2's DAMAGE ROLL IS A UNIFORM DRAW OVER THE INTEGER SPAN; SHOWDOWN'S IS ONE OF SIXTEEN FLOORED INDICES — FOUND 2026-08-18 BY ENGINE, NOT FIXED. Showdown computes <code>tr(tr(d*(100-i))/100)</code> for <code>i</code> in 0..15, so its support has holes and repeats; <code>medicham2</code> computes <code>d.min + floor(u*(d.max-d.min+1))</code> (<code>medicham2-browser.js</code>:18336), which is every integer in the span. Measured by the driver's own <code>damage_interior</code> block and published in <code>data/game-differential.json</code> : on one staged hit showdown 14 distinct values, medicham 20, six of which — 110, 113, 116, 119, 122, 125 — the authority can never emit, worst per-value probability gap 7.5 points. EVERY CORNER INSTRUMENT IS BLIND TO IT BY CONSTRUCTION (the endpoints agree), which is why <code>test-engine-diff</code> reads 0/6000. <code>dmgRangeOneHit</code> ALREADY takes a <code>rolls</code> OUT parameter <i>"in Showdown's own random(16) index order"</i> with exactly one caller, so the fix is to select <code>rolls[index]</code> instead of interpolating. It moves EVERY damage number in every rollout and wants its own pass and its own probe. FIXED 2026-08-18 BY ENGINE. It is the SUPPORT that is wrong, not a rounding stage: 16 indices is a hard ceiling, the randomizer's own floor creates REPEATS below it (<code>baseDamage/100</code> per step) and every stage after it — STAB :262, the type chart :270-284, burn :285, <code>ModifyDamage</code> :293 — scales the survivors apart into HOLES. The loop now selects <code>rolls[damageRollIndex(u)]</code>. The 14/20 shape holds across a widened fixture of nine staged hits (both categories, a resist, a 4x, an item after the die, two spreads, BP 40-120): Ice Shard reads 5 authority values against 25, Ice Beam 9 against 33 — and the two rows whose SUPPORTS matched still failed the PAIRED clause, so a support count alone understates it. PAIRED, 797 games, <code>MEDI_DAMAGE_SPAN_DRAW=1</code> the only difference: <code>-damage field 3</code> 137 -> 8, diverged 409 -> 330, 51.1% -> 41.1%, and BOTH CORNER ARMS UNMOVED at 163/191 — which is the prediction, since <code>rolls[0]</code> IS <code>d.max</code>. The PRE arm reproduces #303's published POST exactly. Standing artifact republished: 969 games, 344 diverged, 2 VOID, 35.4%. Landing it exposed TWO census	CLOSED 2026-08-18 — engine; 8 games survive, candidate is the declared per-hit multi-hit randomizer EMPHASIS MARKERS REMOVED FROM THE FRONT OF THIS CELL 2026-09-04 BY MEASURE — the leading asterisks defeated the register's cell clause, so the row read OPEN against its own CLOSED verdict. Only the notation changed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: <code>tests/test-damage-roll-support.js</code> is on disk; closure not re-verified 2026-09-04.

#	item	note	
	probes and one gate clause that were PINNING the defect — <code>spreadFoes</code> demanded 51, a value the authority cannot emit at any roll; PART 3b of <code>tests/test-game-differential.js</code> FAILED IF THE INTERIOR WAS IDENTICAL. VERIFIED BY <code>tests/test-damage-roll-support.js</code>		
#308	TEN MECHANICS PARTED THE BOARD AND FOUR MORE WROTE A LEAF NOTHING COMPARED — FOUND BY <code>engine/all_mechanics_fire.js</code> 's board arm, FIXED 2026-08-19 BY ENGINE . Ten rows came back STATE: spite removed no PP at all, <code>syrupbomb</code> applied no Speed drop, <code>corrosivegas</code> / <code>stuffcheeks</code> / <code>teatime</code> all left a berry the authority had taken, <code>berserk</code> paid +3 off a three-hit move, <code>sleepstalk</code> and <code>snore</code> were refused by the sleep gate in the one state they exist for, <code>focuspunch</code> wrote a `	move	line the authority ne the target 8 HP down. WRONG AND THE CORRECTI IDENTICAL on both engi
#309	THE WHOLE-GAME END-STATE RE-RUN IS OWED AFTER THE TRACE FIX, AND THE PUBLISHED FIGURE IS PRE-FIX. The board-state run (2026-08-20) measured 83 of 797 games ending on a different board , of which the "narrate identically, end differently" population was 9 — and Trace copying a different ability was 9 of the 15 in the pre-correction count . The Trace selection defect was then fixed (CHANGELOG 5.53.0) and no corpus measurement has been taken since . So the standing different-board figure describes an engine that no longer exists, and the population most likely to have moved is the one Trace lived in. RE-RUN <code>engine/game_differential.js</code> --end-state against a cut release with the census and team pool pinned , and publish the new split by class. Do NOT quote 83 / 9 / 51.0% as a current result. The run reads the team pool LIVE and OPS appends hourly, so pin all three or the sample moves underneath it (CLAUDE.md, "A RELEASE FREEZES CODE")	CLOSED 2026-08-21. Re-run on release <code>9b216aeaa84</code> , census pinned <code>38da8c3e9005</code> , team store <code>data/team-pool-frozen</code> , --baseline cleared: DIFFERENT-END-STATE 83 -> 72 on the SAME 797 games (swarm, coverage, <code>speed_ties</code> and <code>end_reasons</code> blocks byte-identical to the baseline). The population this row named — protocol never parted, "narrate identically, end differently" — went 9 -> 0. Mechanism corroborated three ways: end-state families <code>active[].ability</code> 44->32 and <code>party.ability</code> 16->4 (-24 ability leaves, and essentially nothing else moved), <code>traceChoiceDie</code> =215 / <code>traceChoiceNoDie</code> =0 (the die reached the run with zero silent slot-0 fallbacks), turn-1 board identity <code>778/797 -> 783/797</code> . The only SOURCES file differing from the baseline release is <code>medicham2-browser.js</code> , and every hunk in it is the #310 Trace work, so the -11 is the Trace fix and nothing else. THE LEVEL IS A LOWER BOUND AND SO WAS THE PUBLISHED 83 — see #314: the state comparator fails its own planted proof in BOTH artifacts. The DELTA survives (same ruler, blind in identical places, paired deterministic replay, and the families that moved are ones the comparator demonstrably reads: a different species on the field 8/8, an ABILITY on a BENCHED body 5/5). Artifact <code>data/verification/gd-endstate-309.json</code>	closed — MEASURE
#310	EVERY ROLLOUT STILL TAKES A FIXED TRACE INDEX; ONLY THE DIFFERENTIAL GOT A STREAM. <code>battleInit</code> now accepts <code>opts.rng</code> and <code>game_differential.js</code> passes the arm's stream, so Trace samples uniformly among eligible foes there. Every other caller passes nothing, keeps the old fixed eligible[0], and increments <code>MEDSEEN.traceChoiceNoDie</code> — loud rather than silent, by design. That counter is non-zero on every self-play and rollout run in the repo. Closing it means handing those callers a stream, which MOVES EVERY SEEDED RUN — the same class of debt as the per-target accuracy roll (#294, #57). Not a bug to fix in passing: it needs its own pass, and everything downstream of it owes a re-run	open — engine DEFECT, deferred by cost	
#311	THE SURVIVING TURN-ORDER DIVERGENCES ALL NAME A MEGA FORME — A LEAD, NOT A FINDING. After the Speed-chain and Gale Wings fixes (#290, CHANGELOG 5.49.0) the four remaining --order-probe pairs share a signature the Talonflame family did not: every one involves a mega forme , with speed gaps of 500 and 382 — showdown <code>Excadrill @572</code> vs <code>medicham Sableye-Mega @72</code> , showdown <code>Aerodactyl-Mega @422</code> vs <code>medicham Sableye-Mega @40</code> , <code>Banette-Mega @105</code> vs <code>Swampert-Mega @224</code> , <code>Raichu-Mega-Y @200</code> vs <code>Meowstic-M-Mega @170</code> . That points at mega-evolution timing and the action queue re-sorting around a forme change , not at the Speed arithmetic already corrected. IT HAS NOT BEEN STAGED. It is one draw of a pool that grew by roughly a thousand games. Stage it — does the authority re-sort the queue when a body mega-evolves mid-turn, and when does the new forme's Speed take effect — and if the lead dies, close this row and say so ; it has not earned a fix	CONFIRMED 2026-08-21 BY ENGINE, AND THE CAUSE IS NOT THE MEGA — THE PRIORITY BRACKET IS FROZEN AT THE TOP OF THE TURN. Showdown re-sorts at the tail of EVERY <code>runAction</code> for <code>gen>=8</code> (<code>sim/battle.ts:2916-2923</code>) and the re-sort re-runs <code>getActionSpeed</code> , which recomputes <code>action.priority</code> through <code>ModifyPriority (:2639-2646)</code> . <code>medicham2</code> freezes the bracket once per turn — <code>engine/medicham2-browser.js:13928</code> for(<code>const it of acts</code>) <code>it._pri=actionPriority(it,field)</code> ; — and <code>turnOrderKey (:9126-9132)</code> re-reads only <code>spe</code> . The comment above that line asserts Showdown "resolves an action's priority when it is queued and never again", and that claim about the authority IS the defect. <code>_resortTail (:14206)</code> already declared the residue, as does #240's closing note. Mega was the visible case, not the mechanism: walking <code>D.abilities.all()</code> filtered to the regulation for <code>onModifyPriority / onFractionalPriority</code> , exactly 3 of the 76 legal base->mega pairs change a priority-touching ability (Banettite GAINS Prankster; Sablenite and Meowsticite LOSE it) — and all four surviving --order-probe pairs name one of those three, each with the sign a frozen bracket predicts. Probe <code>tests/probe_mega_priority.js</code> exits 1: ARM 1 (Banette+Banettite) showdown moves it first, <code>medicham</code> last; ARM 2 (Sableye+Sablenite) the reverse; turn 2 agrees in both, isolating the cause to mid-turn re-derivation; ARM 3 (Aerodactylite, no priority ability) agrees throughout as the repair control, and both no-mega controls agree. SURFACE IS WIDER THAN MEGA and is named rather than claimed: the same line makes the authority re-derive brackets after every move, so Grassy Glide (terrain-dependent priority, legal here), Skill Swap, Simple Beam, Entrainment and	CLOSED 2026-08-21 — F <code>e3313a0</code> landed the bracke row was held open for are do pool unchanged from the pre 149 -> 147 diverged of th now PASSES — real disagr Census regenerated and the j unchanged. THE HONES THAN DRESSED UP: the probe population it targeted <i>"move-vs-move pairs only; i remaining 29 have a diffi one being widened.</i> Gale' regain.js

#	item	note	
		Worry Seed can all move a bracket for the remaining queue — un-staged. FIXED 2026-08-21 in e3313a0 — _resortTail re-derives _pri over the remaining queue through actionPriority, the same one function the turn-top sort calls. (That commit touched no register row and no ledger; the account went to docs/MEDICHAM-SPRINT-NOTES.md, which is why this cell said CONFIRMED for a defect that was already repaired.) THE GAIN DIRECTION IS NOW STAGED TOO, and it had never been observed: the counter had seen talonflame 1->0 and banette-mega 0->1 over 200 self-play games and no 0->1 on a Gale Wings body, so the REGAIN rested on the source looking stateless. Will, 2026-08-21: "can Talonflame REGAIN Gale Wings mid-turn if it heals back to full HP" — yes, on the authority AND here. A Roserade @ Choice Scarf (213) Life Dewes a Talonflame (168) from 138/153 to exactly 153/153 as action 0, and both engines then move it AHEAD of a Jolteon at 182. Both controls — heal removed, ability removed — put it back behind. THE ROW STAYS OPEN ON THE RE-RUNS IT DECLARES OWED, NOT ON THE MECHANIC: every seeded artifact is stamped pre-fix and the census has not been regenerated since	
#312	THREE DAMAGE ROWS ROOT-CAUSED AND UNFIXED, AND TWO OF THEM WANT THE SAME CONSUMER. All three survived #304 and are byte-identical to the pre-#304 artifact, so they are not roll-index residue. Shell Side Arm (data/moves.ts:16223): onModifyMove recomputes physical against special from RAW stats and flips move.category, also setting flags.contact — the authority emits [anim] Shell Side Arm Physical and deals 99 where we deal 88 off the declared "Special", and no tag exists for this shape. Sand Force (data/abilities.ts:3950): the handler is a disjunction (Rock or Ground or Steel) and engine/tag_dex.js's onType regex takes the first alternative only , so the artifact says "Rock" and a Bulldoze fixture never matches — while engine/fixture_preflight.js derives all three off the same handler, two producers of one fact disagreeing , the artifact_audit shape in a new place. Metronome (item) (data/items.ts:4023): the tag damageMultOnRepeat is complete and correct and grep finds NO CONSUMER in medicham2-browser.js. Hustle is diagnosed and deliberately unfixed because it needs the SAME consumer Sand Force does — base-power stage plus type plus weather, which no branch of dmgRange serves today. Land that consumer once and measure both; landing them separately makes a bad result unattributable	closed 2026-09-04 — ALL FOUR NAMED BULLETS ARE FALSE AT HEAD. Shell Side Arm is register row #518, CLOSED, and the engine carries categoryPicked with the knob MEDI_NO_CATEGORY_PICK restoring the unchosen behaviour. Sand Force: data/tags.json abilities.sandforce.params.damageBoost.onType is ["Rock", "Ground", "Steel"], all three alternatives, folded into the base-power chain. The Metronome item has TWO live census rows on tag item/damageMultOnRepeat — "the Metronome item makes damage CLIMB over consecutive uses of one move" and "the Metronome counter RESETS when the holder changes move". Hustle is live in engine/medicham2-browser.js as {side:"att",mult:0.8,when:"physical"}. Closed by MEASURE 2026-09-04 as register hygiene against HEAD 73cca9f8; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account docs/_reports/2026-09-04-register-closures.md. docs/MEASURE.md THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT	
#313	slushrush LOOKS LIKE A GENUINE ENGINE GAP AND IS THE ONLY ONE OF THE TWENTY UNEXPLAINED ABILITIES THAT DOES. Beartic at 50 x 2 = 100 clears Feraligatr's 78, so the turn order genuinely should change once snow is up, and the harness cannot currently show it. Twenty abilities still cannot say why they did not fire (down from 98 across both legs); the largest coherent group is ally mechanics — friendguard, telepathy, hospitality, curiousmedicine — which need a rung where the PARTNER is a live participant rather than another explanation, not a new predicate. Do NOT re-derive ally-only by the isAlly route: it matched 26 abilities and MOST MEAN THE OPPOSITE (Competitive's `if (!source		target.isAlly(source)) return; wrong explanation is worse t
#314	STATE_PLANTS AIMS AT SLOTS WITHOUT CHECKING ANYBODY IS STANDING THERE, AND applied MEANS THE CALLBACK RETURNED TRUTHY. Two separate holes in the board comparator's own proof, filed by MEASURE 2026-08-20 and not fixed. 22 instances across 18 plants land on a corpse at a late boundary and now prove nothing, because the post-faint group is held on both sides after the corpse-asyymetry fix. And applied records only that the plant callback returned truthy, not that the board moved — the bench-HP plant takes t[t.length-1], which is a fainted body, and Math.max(0, 0 - 1) changes nothing. The driver's own comments already document that exact hazard for the ACTIVE-HP plant. A plant that cannot move the board is a test that cannot fail , and this one gates the number the whole MEDICHAM finish line now rests on	open — instrument DEFECT. MEASURED 2026-08-21, so this row is no longer DEBT: tests/probe_bench_plants.js over 8 pairs reports 19 plants APPLIED AND NOT CAUGHT, o never applied. The six BENCH plants this row was filed about are now PROVEN 5/5 (item, status, toxic stage, typing, stat stage, ability) — that half is fixed. docs/ENGINE.md still says " 41 of 42 are caught, the one hole is the benched-HP plant "; that sentence is CONTRADICTED by its own instrument and must not be read as current. The 19 are INDETERMINATE rather than confirmed blind, because this row's own second hole — applied means the callback returned truthy, not that the board moved — makes the label unreliable. Consequence: planted_state_proof_ok is FALSE in every --state / --end-state artifact, so DIFFERENT-END-STATE is a LOWER BOUND wherever it is quoted, including the 83 published in CHANGELOG 5.54.0, which did not disclose it. Artifact data/verification/bench-plants-309.json	open — instrument DEFECT tests/probe_bench_pla plant(s) applied and n was pointed at it and immed reality.json — a live def engine/register_real; withdrawn the same pass. TI while any plant is applied-an name. Until then this row is
#315	A FAINTED MEGA'S FORME REGRESSES IN SHOWDOWN AND NOT HERE. sim/battle.ts:2554-2557 and :2568-2571 return a fainted mega to its base forme; medicham2-browser.js leaves it evolved. Measured in the 797-game end-state run: 3 species, 2 maxhp and 2 types leaves differ on bodies both engines call dead. These were deliberately NOT held by the post-faint skip (CHANGELOG 5.54.0) precisely because they are the one thing about a corpse a later board can still be wrong about — a re-entry, a party-wide effect or a count over living formes can read them. Small, real, and cheap	open — engine DEFECT. DUPLICATE POINTER 2026-09-04 by MEASURE: the TYPES half of this row is register row #503, which is CLOSED and owns it. What survives here and is still OPEN is the species and maxhp half, which nothing at HEAD restores on the faint path and no census row covers.	
#316	THREE QUARANTINE CLAUSES READ PASS WHILE THEIR INSTRUMENT REFUSES TO RUN, SO THEY ARE MEASURING 2026-08-11 ARTIFACTS — MEASURED 2026-08-21. tests/roster.js exits 1 on ALL THREE stages (items, abilities, moves) at the FIXTURE AUDIT, before a single game is played: the CONTROL CLICK IS NOT INERT. The single failing line, identical on all three stages, is "the inert click focusenergy moves NO board leaf in either engine over 3 turns" — and the very next line reads "ok — and the two engines agree on all 288 leaves while doing it". THIS IS NOT AN ENGINE DEFECT AT ALL. The roster stages an entity beside a control click that is supposed to move nothing,	closed 2026-08-21 — instrument DEFECT, both halves landed. VERIFIED BY: node tests/roster.js --selftest — REPAIRED 2026-09-04 BY MEASURE. The old marker carried a SHOWDOWN_PATH=... prefix and a literal --release <id>, so nothing ever ran it; both were decoration, since tests/roster.js:129 already requires engine/showdown_path.js (which resolves the sibling checkout and SETS the env var) and tests/roster.js:145 already does ER.open(ARG('--release')) or null, where null is the newest release. --selftest is the command that decides THIS row rather than the entities: the single line that was red — "the inert click focusenergy moves NO board leaf in either engine over 3 turns" — is selftest arm 1, and main() returns its failure count, so the exit code is red exactly when the instrument is untrustworthy and green when it is not. A stage run was NOT chosen deliberately: --	

#	item	note
	so any board difference can be attributed to the entity. Focus Energy applies <code>vol.focusenergy</code> and spends <code>pp[].focusenergy</code> — both are leaves the board comparator reads, so the control moves the board by construction. The likely trigger is that the comparator got BETTER: once it began reading volatiles and PP, a click that had always set a volatile stopped qualifying as inert. The control did not change; the ruler did. (CORRECTED 2026-08-21: <i>this row first said "88 of the first ~106 read can't learn Focus Energy", a learnset story reported by a subagent and published without being checked. Measured directly, the string can't learn appears ZERO times in any stage's output. The wrong diagnosis pointed at a shape rule deriving illegal clicks; the real one is a control-inertness definition.</i>) Meanwhile <code>engine/quarantine.js</code> reports deliberate <code>roster / items , / abilities</code> and <code>/ moves</code> as PASS, because it reads the stored artifacts and those are dated 2026-08-11. A GREEN CLAUSE RESTING ON A STALE ARTIFACT IS WORSE THAN A RED ONE, BECAUSE NOTHING ABOUT IT LOOKS WRONG — it is the same shape as the fourteen stale handoffs and the ban list of four, arriving through a gate instead of through prose. Three of the five clauses currently holding the MEDICHAM finish line OPEN are in this state, so "5 of 8 pass" overstates what is actually verified. Two things are owed and they are separate: (1) restore a usable control — either choose a click that moves no leaf the comparator reads, or exclude the control's OWN volatile and PP leaves from the inertness assertion while keeping every other leaf in it (the second is narrower and does not weaken the attribution, since the control's own volatile is never what an item or ability is being credited for); (2) make <code>quarantine.js</code> REFUSE a roster clause whose artifact is older than the engine it claims to describe, exactly as the whole-game and mechanics clauses already do ("MEASURED AGAINST A DIFFERENT ENGINE... <i>that is not a weaker answer, it is an answer about other bytes</i>"). Until (2) exists this can silently recur — CLOSED 2026-08-21. BOTH HALVES LANDED, AND THE DIAGNOSIS ABOVE IS HALF RIGHT AND HALF AN ARTEFACT OF THE COMMAND IN THIS ROW'S OWN VERIFIED BY CELL. <code>tests/roster.js</code> reads <code>--stage</code> , not <code>--kind</code> ; <code>--kind items</code> is silently ignored and the run falls back to <code>--stage spine</code> , so the three 'stages' measured above were the same stage three times and the retraction of the learnset diagnosis was itself wrong. Asked with <code>--stage</code> , there were TWO independent defects, both of them the RULER moving rather than the control: (1) THE INERTNESS ASSERTION. PP became a compared leaf 2026-08-11 (<code>c42b066</code> lifted the roster's own hold) and <code>vol.focusenergy</code> on 2026-08-12 (<code>13e94cb</code>), so a control that had not changed began moving two leaves the comparator reads. (a) is not available and that is a measured fact, not a preference: <code>pass</code> is refused for a healthy active body, so a control clicks a move and a move spends PP. Took (b), DERIVED — <code>INERT_SELF</code> reads the control move's OWN <code>volatileStatus</code> and its own PP row off the format and exempts those two anchored leaf paths and nothing else; a control declaring anything more (a boost, a status, a heal, a non-self target) FAILS a new selftest clause by name, so a swap cannot silently widen it. Shown red both ways before being trusted: <code>--selftest --inert substitute</code> exempts exactly <code>vol.substitute / pp.substitute</code> and still CATCHES the quarter of the user's HP Substitute takes inside a handler, on 48 hp leaves in both engines; <code>--selftest --inert swordsdance</code> is refused by the cap with <code>'declares boosts {"atk":2}'</code>. (2) THE LEARNSET CLAUSE, WHICH NOTHING HAD EVER SEEN FIRE HERE. <code>tests/staged_board.js</code> gained a <code>checkCanLearn</code> clause on 2026-08-12 (<code>b61230a</code>) and it shut the roster down completely — items 239 refusals, abilities 440, moves 1,290 — unnoticed for ten days because the roster is not in <code>tests/run-all.js</code> . Derived: 55 of this format's 347 legal species can learn Focus Energy and 292 cannot, and the roster hands it to every body it stages. <code>fixtureAudit</code> now takes a single exempt move id with a REQUIRED written reason, prints every exempted pair, and exempts the control click and nothing else. ALL THREE STAGES NOW RUN AND REPORT ENTITIES — items 3 DIFFER / o DID-NOT-FIRE / 136 MATCH, abilities 10 / 1 / 117, moves 157 / 0 / 306, on release <code>b240433ae8af</code> . (3) AND THE CLAUSE NOW REFUSES A STALE ARTIFACT, in the whole-game clause's own words: the three roster clauses were reading <code>PASS</code> off files generated 2026-08-11 on release <code>96361d523e20</code> while the instrument could not start. <code>rosterStage()</code> withholds on a release mismatch — counts absent from the object AND from the sentence — with six selftest probes driving the SHIPPING function on injected artifacts. Live: the gate went from 3 of 8 failing to 6 of 8, the three new reds being the three withholds MARKER NOT MACHINE-RUNNABLE — NOTED 2026-08-23 BY MEASURE, ROW OTHERWISE UNTOUCHED. This row's <code>VERIFIED BY</code> carries an environment prefix and a literal placeholder, so <code>engine/register_reality.js</code> refuses it rather than shell-executing it, and the row files as INSTRUMENT UNRUNNABLE — which is not agreement and is not evidence either. The command is a correct instruction to a human; it is not an instrument. See #381. SUPERSEDED 2026-09-04 BY MEASURE — the marker question is settled in this row's STATUS CELL;	<code>stage moves</code> is red today on #319's five entities, which would file this closed row as PREMATURE CLOSE on somebody else's defect. Stated limit, since one exit code cannot carry three claims — the stale-artifact withhold half is decided by <code>node engine/quarantine.js --selftest</code> and the control-shape cap by <code>node tests/roster.js --selftest --inert substitute</code> , both named in the account above and neither carried by this marker.

#	item	note
	the paragraph above is dated evidence and is left standing rather than edited out.	
#318	THE MOVE STAGE HANDS EVERY MOVE IN THE FORMAT TO ONE CARRIER THAT CANNOT LEARN 331 OF THEM — MEASURED 2026-08-21, NOT FIXED. With ROADMAP #316's control-click exemption applied, 632 learnset refusals remain across the three roster stages and 87% sit on FOUR fixed cast bodies: Goodra-Hisui 349, Dragapult 106, Weavile 77, Torterra 26. It is not 421 accidents, it is two design decisions seen many times — <code>move/*</code> rules stage the move under test on ONE bulky QUIET body so the comparison is about the move rather than the body, and DELIVERY picks one boring vehicle per type without asking who is throwing it. The clause is REPORT-ONLY for the roster as of #316, declared with its reason and its concentration printed on every run: the roster does not build teams, it builds boards — <code>scaffold()</code> multiplies HP by 4, 6 or 8 and <code>buildPair()</code> constructs bodies directly in both engines, so the rig is already a position no validator would pass. THE COST IS STATED: a shape rule that derives a carrier which cannot legally hold the entity is no longer refused. The repair is to derive the move stage's carrier from the move's OWN learnset and the delivery vehicle from the THROWER's — which changes the carrier's types, bulk and speed and therefore changes what every move scenario measures, so it is its own batch, in the same words <code>data/fixture-learnset-baseline.json</code>'s header uses about the eight pairs that preceded it. This count must reach zero and the clause must then go back to FAIL MARKER NOT MACHINE-RUNNABLE — NOTED 2026-08-23 BY MEASURE, ROW OTHERWISE UNTOUCHED. This row's VERIFIED BY carries an environment prefix and a literal placeholder, so <code>engine/register_reality.js</code> refuses it rather than shell-executing it, and the row files as INSTRUMENT UNRUNNABLE — which is not agreement and is not evidence either. The command is a correct instruction to a human; it is not an instrument. See #381. SUPERSEDED 2026-09-04 BY MEASURE — the marker question is settled in this row's STATUS CELL; the paragraph above is dated evidence and is left standing rather than edited out. AND IT CANNOT BE DECIDED FROM DISK EITHER — NOTED 2026-08-23 BY MEASURE. The learnset-refusal count this row is scored on is printed to stdout and written to no artifact: <code>data/roster.moves.json</code> carries <code>counts</code>, <code>scope</code>, <code>reds</code>, <code>mirror</code> and <code>results</code>, and no refusal figure at all, so the 632 can be re-read only by a human watching a run go past. INSTRUMENT OWED: the LEARNSET CLAUSE IS REPORT-ONLY count published into the roster artifact, so this row can be decided by a file rather than by a person.	open — instrument DEFECT, filed by ENGINE while repairing #316. NO MARKER, AND THAT IS THE MEASURED ANSWER — 2026-09-04 BY MEASURE. The old marker was <code>SHOWDOWN_PATH=...</code> <code>node tests/roster.js --stage moves</code>, which <code>engine/register_reality.js</code> refused to read, so this row has been counted as live instrument coverage while nothing ever ran. Dropping the prefix does NOT repair it, because the exit code cannot carry this row's claim: <code>tests/roster.js</code> passes <code>learnsetMode: 'report'</code> to <code>fixtureAudit</code>, so the refusals land in a printed bucket and never in <code>bad</code>, and <code>bad</code> is the whole of <code>process.exit(bad ? 1 : 0)</code>. A marker here would report GREEN on 632 refusals. The debt is already declared in the account above and is now this row's only coverage — see the INSTRUMENT OWED declaration there.
#319	157 OF 500 MOVES PART FROM THE AUTHORITY BY 1–4 HP, AND NOTHING HAS SEEN IT SINCE 2026-08-11. The roster's move stage, runnable again after ROADMAP #316, reports 157 FIRED-AND-BOARDS-DIFFER / o DID-NOT-FIRE / 3o6 FIRED-AND-BOARDS-MATCH on release <code>b240433ae8af</code>, against the last artifact anybody could read (2026-08-11, release <code>96361d523e20</code>) which said o DIFFER. It is almost certainly ONE defect family, not 157: 217 of the 240 parted leaves are <code>hp</code> or <code>party.hp</code> (Torterra 1334 vs 1333, Goodra-Hisui 1215 vs 1218, Sceptile 79 vs 75), and 97 of the 157 entities are staged by the single rule <code>move/plain-attack</code> — a plain hit into the punching bag. The tail is 21 <code>status / present-in-one-engine-only</code> and 11 <code>boosts.* off-by-one</code>. DO NOT READ THIS AS 157 BROKEN MOVES, and do not compare it to the 2026-08-11 figure as a regression: those are different bytes, which is exactly what #316's withhold now enforces. It needs the damage arithmetic diffed on ONE staged hit against the authority before anything is attributed MARKER NOT MACHINE-RUNNABLE — NOTED 2026-08-23 BY MEASURE, ROW OTHERWISE UNTOUCHED. This row's VERIFIED BY carries an environment prefix and a literal placeholder, so <code>engine/register_reality.js</code> refuses it rather than shell-executing it, and the row files as INSTRUMENT UNRUNNABLE — which is not agreement and is not evidence either. The command is a correct instruction to a human; it is not an instrument. See #381. SUPERSEDED 2026-09-04 BY MEASURE — the marker question is settled in this row's STATUS CELL; the paragraph above is dated evidence and is left standing rather than edited out. RE-SCOPED 2026-08-23 BY MEASURE — IT IS 5, NOT 157, AND THE FIVE ARE NAMED. <code>data/roster.moves.json</code> on release <code>c36782953dee</code> (generated 2026-08-23To6:06:12Z) reads 5 FIRED-AND-BOARDS-DIFFER / o DID-NOT-FIRE / 47o FIRED-AND-BOARDS-MATCH / 22 COULD-NOT-STAGE / 3 DEFERRED-BY-OWNER out of 500 in scope. THIS IS NOT A BEFORE/AFTER WITH THE 157 AND MAY NOT BE QUOTED AS ONE — different release, and #316's withhold rule exists precisely so that a roster count is never compared across bytes. What it does establish is that this row's SCOPE is now five named entities, each with its own home: <code>dragoncheer</code>, <code>fakeout</code>, <code>psychup</code> and <code>transform</code> are one fact about the <code>vol.focusenergy</code> leaf (#384), and <code>matchagotcha</code> is a drain heal one HP short (#339). The items stage carries the other two — <code>bigroot</code> (#385) and <code>greninjite</code> (#356).	open — engine DEFECT, filed by ENGINE while repairing #316. VERIFIED BY: <code>node tests/roster.js --stage moves — FIRED-AND-BOARDS-DIFFER must be o. REPAIRED 2026-09-04 BY MEASURE: only the <code>SHOWDOWN_PATH=...</code> prefix was removed, and it was decoration — <code>tests/roster.js:129</code> requires <code>engine/showdown_path.js</code>, which resolves the sibling checkout and sets the variable itself. No <code>--release</code> is passed ON PURPOSE: <code>ER.open(ARG('--release'))</code> or <code>null</code> takes the newest release, and a pinned id ages out and strands the marker (LESSONS §12). Stated honestly, the oracle is WIDER than the claim — <code>bad</code> is FIRED-AND-BOARDS-DIFFER + DID-NOT-FIRE + failed red rows, so green means all three are zero and red does not by itself say which. It writes no artifact without <code>--write</code>, so a register pass cannot republish <code>data/roster.moves.json</code>; it does play the whole move stage and is multi-minute.</code>

#	item	note	
	When those four rows close this one has nothing left to describe and closes with them.		
#320	THE ENTRY TRANSFORM COPIED A BODY AND NEVER ANNOUNCED IT, SO IMPOSTER WAS THE LARGEST FAMILY OF THE LARGEST DIVERGENCE CLASS — CLOSED 2026-08-21. event missing from medicham2 is 55 of the 147 diverging games at release b240433ae8af ; rolled up by engine/divergence_report.js and grouped by the SHOWDOWN-side event, its joint-largest family is 10 games, 10 causes, all of them -transform ... [from] ability: Imposter.medicham2 emitted nothing. THE ENGINE HAD DECLARED IT — the transformsOnEntry header read "NO `	-transform	LINE IS EMITTED"*, tri one. ROADMAP #210 then site**, so the shared precisely so the two r announcing it and the emits inside the primi callers get it free; i an effect at data/abilit the move passes none). CONSTRUCTION** – trans zero throughout, becau absent; the new MEDS! is asserted for EQUALI **MARKER NOT MACHINE-R OTHERWISE UNTOUCHED.** prefix and a literal p rather than shell-exec UNRUNNABLE – which is command is a correct i See #381. **SUPERSEDED ACCURATE WHEN IT WAS N standing rather than e wording superseded is, environment prefix and refuses it rather than INSTRUMENT UNRUNNABLE either. The command is instrument."* **BOTH H APART, AND NEITHER MOV this cell read SHOWE so the sentence was co 05:32, ROADMAP #521) r imposter-transform-line.js , placeholder. The -r pi tests/_live_release.js redi store, and without it time. **(2) THE RUNNER SAFE its -r clause. ' lifted from 429e8d84 tonight's rewrite – re classifyMarker() also ret tests/test-imposter-transfori filed as INSTRUMENT UN seven days stale, and classifyMarker()' changed n row in the pass that wrote th exit code is green.
#321	KINGAMBIT'S fallenundefined IS THE AUTHORITY'S OWN TYPO AND THIS ENGINE REFUSES IT ON PURPOSE — CLOSED 2026-08-27 BY ENGINE. NOTHING WAS EMITTED AND NOTHING SHOULD BE. Filed 2026-08-21 asking for the line to be mirrored; three things landed after it and settled it the other way. (1) THE MECHANISM, DERIVED not recalled. data/abilities.ts:4724 guards onStart on pokemon.side.totalFainted and :4728 is the ONLY assignment to effectState.fallen , INSIDE that guard; :4732 's onEnd has NO guard and fires unconditionally from sim/battle-actions.ts:103 (switch-out, before the replacement s	switch	) and sim/battle.ts:2553 property that was neve from initState (sin and clearVolatile delete actions.ts:117 , AFTER the captured line carries in the official simula a Defiant control on t and the exit line is i and the exit lines are counter reading, and t THE ROW'S OWN VERIFIE engine/quarantine.js decl – one reader subtracts clauses, so the five g 5 declared). Emitting tl it, and it would put l o), which is the cens ARE LIVE AND PROBED** counts 1/2/3, on switc general question this 22): board-material no reproducing the author node engine/status.js – th DECLARED / THE AUTHOI and node tests/test-mecha live at 754/754.
#322	THE PRE-TURN CHARGE ANNOUNCEMENT WAS AT ORDER o AND THE AUTHORITY PUTS IT AT 107, WHICH IS BELOW THE SWITCHES AND BELOW THE MEGAS — CLOSED 2026-08-21. ordering is 29 of the 133 diverging games at release 6a05dd9ad60d and does NOT collapse: 28 distinct causes. Grouped by SHAPE(node engine/divergence_report.js --class ordering --all), its LARGEST single family is one line in the wrong place — **5 games `switch <> -singleturn	...	focuspunch plus 2 game:

#	item	note
#323	CLASS A OF <code>data/mutation-coverage.json</code> SAYS "NOTHING IN THE SIMULATOR IMPLEMENTS THIS FACT" AND IT IS WRONG ON AT LEAST ONE ROW I CHECKED BY HAND — MEASURED 2026-08-22 ON RELEASE <code>6fb9ebd3b704</code> . The A/B/C/D classifier decides A from exactly two things: no <code>TAGS.param/has/withTag/reactorsTo</code> call names the tag, and no NAME branch names the carrier. It does not look at the OTHER TAGS ON THE SAME CARRIER, and a sibling tag is a third route it cannot see. <code>move:leechseed / immunityGate</code> is graded A with the string	open — measurement DEFECT, <code>tests/mutation_harness.js</code> (MEASURE)

#	item "Nothing in the simulator implements this fact", and the Grass immunity is implemented — <code>medicham2-browser.js:18616</code> reads <code>perTurnHP.immuneType</code> and refuses the seed on a Grass body. The same shape covers most of the class: <code>move:lowkick / weightBased</code> sits beside a read <code>variablePower {kind:'targetWeightKg'}</code> , <code>move:acrobatics / readsOwnItem</code> beside a read <code>variablePower {kind:'userNoItem'}</code> , <code>move:heatwave / inflictsBurn</code> beside a read <code>statusInflict</code> , <code>move:closecombat / lowersUser</code> beside a read <code>statChange</code> . 31 of the 36 class-A carrier x tag rows carry an ARMED census probe that PROVES the mechanic works , which is the artifact telling you the same thing from the other side. The file's own <code>cannotSee</code> block says a third route exists; the PER-ROW <code>defectWhy</code> contradicts it, and the per-row string is the one that gets read into a report. That is the caption-is-not-a-quarantine failure at row scale. The fix is to the STRING and to the ranking, not to the rule: A means "this tag is a second, unread copy of the fact", never "this mechanic is missing", and a sibling-tag column belongs in <code>defectEvidence</code> beside <code>nameBranchAt</code>	note	
#324	119 VALUES IN data/tags.json ARE READ BY NOTHING, AND THAT — NOT CLASS A — IS WHAT THE MUTATION ARTIFACT HAS BEEN HOLDING. MEASURED 2026-08-22, RELEASE 6fb9ebd3b704 , 198,840 SHEET ENTRIES. Classes B and C of the sweep name a param the artifact carries and the simulator dereferences NOWHERE: it consumes the tag's membership and substitutes its own number. 130 carrier x tag x param rows, of which 11 are provenance metadata (see #325) and 119 are facts . Ranked by share of corpus sheet entries: <code>move:protect / shieldsUser</code> , <code>{volatile,duration,bypassable}</code> and <code>stallCounterChecks</code> , <code>{counter,duration,deletesOnFailure}</code> and <code>targetClass</code> , <code>{target,mustPressure}</code> at 67.75%; <code>move:fakeout / firstTurnOnly.{menuDisable,whileActiveMoveActions}</code> and <code>targetClass.*</code> at 12.44%; <code>item:lifeorb / damageMultAll.costsPerAttack</code> at 10.69% — the row rediscovered the expensive way on 2026-08-21 after sitting here for sixteen days; <code>move:suckerpunch / needsTargetToAttack.needs</code> 7.11%; <code>`move:flareblitz</code>	<code>wavecrash / recoil.readFrom</code> 6.42% and 5.74%; <code>move:matchagotcha / drain</code> . <code>{num,den}</code> 4.36% and <code>move:gigadrain / drain.{num,den}</code> 1.22%; <code>move:knockoff / readsTargetItem.muilt</code> = 1.5 2.65%; <code>move:lightscreen</code>	<code>reflect / halvesDamage.muilt</code> engine uses 2732/4096)
#325	THE MUTATION BATTERY PERTURBS PROVENANCE FIELDS AND SCORES THE RESULT AS READ-AND-IGNORED , WHICH IS PURE NOISE IN THE DEFECT COUNT — MEASURED 2026-08-22. 11 of the 130 shadowed-param rows mutate a field that is not a fact about the game at all: <code>move:toxic / neverMissesFromUserType.from</code> = <code>"DERIVED:BattleActions.prototype.hitStepAccuracy"</code> , <code>`move:feint</code>	<code>phantomforce / breaksProtect.from</code> , <code>move:solarbeam</code>	<code>knockoff / conditionalPower</code> tag", <code>ability:levitate / type</code> <code>ability:earlybird / nameImpl</code> is the tag_dex CITATIO nothing should ever re no information and inf the same class of erro denominator has rows i names in the provenanc <code>nullParamsSkipped`</code> are alre
#326	THE MUTATION BATTERY'S HAND-DECIDED CONSTANTS WERE PINNED TO ENGINE TEXT THAT MOVED, AND BOTH REFUSED TO RUN — FOUND AND REPAIRED 2026-08-22. Two independent staleness failures stopped the first re-run of <code>tests/mutation_harness.js</code> in sixteen days. (1) The triage calibration hand-decided <code>taunt / forbidsStatusMoves</code> as class A against release <code>032b4a2979dd</code> , when nothing named the tag; ENGINE has since WIRED it (<code>forbidByVolatile()</code> calls <code>TAGS.withTag('move','forbidsStatusMoves')</code> and dereferences <code>.forbids</code>), so the rule returned D and the sweep refused. The rule was right and the CASE was stale — a distinction the refusal message could not draw, because a calibration case recorded no build. (2) The planted <code>speedMult</code> stub anchored on the literal text <code>s*+_sm.muilt</code> , which became <code>_mods.push(+_sm.muilt)</code> when the speed modifiers became an accumulated list; <code>plant()</code> THREW rather than planting nothing, which is the behaviour that makes this cheap instead of catastrophic. Repaired: every calibration case now carries <code>decidedAgainst</code> , the refusal tells the reader to check it before concluding the rule is wrong, and the A branch is calibrated on a synthetic tag and carrier that exist nowhere — no lookup and no name branch is the DEFINITION of A, so that case cannot be closed by a fix. There is no genuine class-A exemplar left in this engine to pin, which is itself the news	CLOSED 2026-08-22 — MEASURE	
#327	THREE MECHANICS ARE GENUINELY ABSENT FROM medicham2 AND ALL THREE ARE WORTH LESS THAN 0.3% OF THE CORPUS — MEASURED 2026-08-22, FILED SO THEY ARE NOT RE-DISCOVERED, NOT SO THEY ARE FIXED. After hand-reading every class-A row of <code>data/mutation-coverage.json</code> that the census cannot prove (5 of 36), exactly three survive as real gaps and one — <code>move:leechseed / immunityGate</code> — is a false positive (see #323). What is real: (a) <code>immunityGate</code> on <code>move:trick</code> — the simulator contains no lookup for the tag and the string <code>stickyhold</code> appears NOWHERE in it, so Trick succeeds against Sticky Hold; Sticky Hold has 0 uses in 198,840 corpus sheet entries , and <code>refusesItemLoss</code> , the other tag that would carry it, is also unread. (b) <code>punishesMinimize</code> on <code>move:heavyslam</code> (467 uses) and <code>move:bodyslam</code> (215) — neither the tag nor the string <code>minimize</code> appears in the simulator, so the never-miss and the x2 into a Minimized	closed 2026-09-04 — FALSE AT HEAD, on all three "genuinely absent" mechanics. Sticky Hold: three live census rows including "Sticky Hold keeps the item through Knock Off and Trick, and lets a Sticky Barb go", and <code>stickyhold</code> DOES appear in the simulator where the row said it appears nowhere. Minimize: live census row "Body Slam cannot miss a minimized body and hits it for double", and <code>minimize</code> appears 20 times. Metronome item: the two <code>item/damageMultOnRepeat</code> census rows named in #312. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT, engine/medicham2-browser.js (ENGINE; filed by MEASURE, not fixed, LOW)	

#	item	note	
	body are both absent; Minimize is 32 sheet entries, 0.016% , and the mechanic only fires against one. (c) <code>damageMultOnRepeat</code> on <code>item:metronome</code> — the escalating <code>[4096,4915,5734,6553,7372,8192]</code> table is unread; 27 sheet entries . The carrier's own usage is the wrong ruler here and it is the trap this row exists to name: Heavy Slam is clicked 467 times, but the mechanic's reach is Minimize's 32, not Heavy Slam's 467. Do not schedule these against #324, which is worth two orders of magnitude more		
#328	THE MORPEKO SWITCH DIVERGENCE IS NOT A FORME-DATA GAP AND NOT AN ORDERING ONE — REFUTED WITH A CONTROL, AND THE REAL CAUSE IS THREE DEFECTS ON ONE SHAPE. 2026-08-22, ENGINE. #204 was routed on the premise that ten <code>event missing from medicham2</code> games name Morpeko and that the missing forme rows caused them. Three of the 40 cards in data/divergence-turns.json mention Morpeko and one is the switch case , and it is not a switch ORDER divergence at all: <code>medicham2</code> emits no line for that slot, which is why the class is <code>event missing from medicham2</code> . <code>Speed</code> , <code>comparePriority</code> , <code>switchin_order.js</code> and <code>data/switchin-order.json</code> are all irrelevant — those govern the order entry ABILITIES resolve in, not which `	switch	lines are emitted. ** releases identical but output. **C1, ENGINE** when the body leaves t hangry against the auth isPermanent flag, Hung back. **C2, INSTRUMENT buildPair (:2213) *prec BODY CAN NEVER BE SWIT every PLAYED game is b ["morpeko","dragonite"],"esp [null,null,null,null] . The r pass . One fact, two co **SUFFICIENCY PROVED W onBoundary one boundar A DIFFERENT DEFECT AND BOTH engines hold abor abomasnow ; the Showdo misses too, and the ga does NOT clear it. Ove {sw:'abomasnowmega'} co HERE ON PURPOSE** – C1 die agent that day) an in SOURCES and is th moves a file under a m o8-22-formes.md`
#288	TESTING READS A CORPUS THAT MOVES UNDER IT, AND AN ARTIFACT RECORDS WHICH BYTES OF CODE MADE IT BUT NOT WHICH GAMES — WILL, 2026-08-17: "lets keep scraping games but for our testing purposes i just want a frozen corpus so it doesnt move" . OPS appends continuously and should keep doing so — <code>data/games.bo3.jsonl</code> and <code>data/games.ladder.jsonl</code> both moved at 21:01 on the day this was filed. What must stop moving is what TESTING reads . Two channels leak today. (1) THE DIFFERENTIAL'S TEAM POOL , already filed as #81 WIRE 5 and stated in <code>engine/game_differential.js:4037</code> in its own words: <i>"buildSwarm reads data/games.bo3.jsonl and data/games.ots.jsonl LIVE, dedupes to distinct teams and picks by a STRIDE over the matching set — so ONE appended game shifts the stride and changes which teams get played"</i> , with the note that both files moved during that wire's own test runs. The mitigation there asserts WHICH TEAM KEYS were picked, so a moved pool is visible afterwards; it does not stop the pool moving. <code>data/team-pool-frozen</code> already exists and is half the mechanism. (2) EVERY uses COUNT , which lives in <code>data/tags.json</code> and is regenerated from the live store — two divisions reported it independently on 2026-08-15 as <i>"~3,271 moved lines are uses counts following the store"</i> . THE COST IS MEASURED, NOT HYPOTHETICAL AND NOT LARGE : the Metronome closet entry states 19 uses and the artifact reads 25, with nobody having edited either — the corpus grew underneath a typed number, which is this repository's signature failure in miniature. Every impact ranking built on <code>uses</code> — the divergence work order, the bring guide, the mechanics priority — is therefore against a moving target at the margin. THE SECOND HALF IS THE PROVENANCE GAP AND IT IS THE REASON NOBODY NOTICED : an artifact carries an <code>engine_release</code> id saying which bytes of CODE produced it and NOTHING saying which GAMES, so two runs can differ entirely in corpus and look identical in provenance. A corpus snapshot id belongs beside the release id in every artifact that reads a game file. NOT A FREEZE OF THE STORE . Ingest keeps running; testing reads a stamped snapshot. SEQUENCING, DELIBERATE : this lands BEFORE the MAG refit and AFTER the MEDICHAM gate closes. The refit is the run that most needs a stable corpus and is the one place where fitting against a moving target would be invisible; doing it before the gate closes would change the corpus definition under an in-flight measurement, which is the failure this row exists to prevent, committed while building the fix	open — MEASURE; sequenced after the MEDICHAM gate and before the refit	
#317	FUR COAT CARRIES NO DEFENCE MULTIPLIER, SO medicham2 DEALS DOUBLE PHYSICAL DAMAGE THROUGH IT — FOUND 2026-08-21 BY ENGINE, BY ACCIDENT, NOT FIXED . Furfrou was the first draft of <code>tests/test-bracket-regain.js</code> 's second foe and the board parted on the very first physical hit: <code>p2.party.furfrou.hp medicham 90 showdown 120</code> on a Talonflame Facade, medicham dealing exactly double . <code>data/abra-tags.js</code> gives <code>furcoat</code> a single tag, <code>breakable</code> , with no multiplier param at all, so the halving never reaches <code>dmgRange</code> — this is a	closed 2026-09-04 — FALSE AT HEAD. <code>data/tags.json</code> abilities.furcoat.params.condStatMult is <code>{stat:"def",mult:2,when:"always"}</code> , the engine consumer names this row in its own comment (2 occurrences of the row number in <code>engine/medicham2-browser.js</code>), and the live census row reads "Fur Coat halves physical damage with no condition at all, and Mold Breaker takes it back". Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE	

#	item	note	
	engine/tag_dex.js DERIVATION gap and not a missing branch in the simulator, the same shape as #312's Sand Force, where the derivation read one alternative of a disjunction and the artifact then looked complete. It appeared in ALL THREE arms of that file including both controls, so it could not have flipped that verdict, and the cast was changed to a body with an inert ability so <code>stateDiv === null</code> could be ASSERTED rather than excused. 8 corpus sheets, so it is small — and small is exactly the size at which a defect survives, since nothing in the ranked queues will ever surface it. The derivation must be shown its own membership before it is wired: <code>breakable</code> is carried by a large family and the multiplier is not, so a predicate that over-matches here would silently halve damage through abilities that do no such thing	CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT, filed by ENGINE while answering an unrelated question	
#350	THE OPEN-DEFECT CLAUSE CALLS A ROW "NO INSTRUMENT DECIDES IT" WHEN WHAT IT MEANS IS "MY VERDICT ARTIFACT IS OLDER THAN THAT ROW" — MEASURED 2026-08-22 ON #318 AND #319. <code>engine/where.js</code> <code>--gates</code> reads <code>docs/ROADMAP.md</code> live and reports that both rows NAME an instrument. <code>openDefectClause</code> in <code>engine/quarantine.js</code> reads <code>data/register-reality.json</code> and reports both under <code>debt</code> , whose line reads <i>"open row(s) assert breakage with NO instrument that decides them"</i>. Both tools are computing the right answer to their own question and one of the two sentences is false. <code>data/register-reality.json</code> was generated 2026-08-22T01:55:12Z; commit <code>76e4a67</code> put the #318 row into the register at 02:33Z, thirty-eight minutes later, so the clause has no verdict for a row that has carried a <code>VERIFIED BY</code> marker since it was written. A SECOND, INDEPENDENT CAUSE SITS UNDER IT: <code>register_reality.js</code> 's <code>SAFE</code> guard accepts only a plain <code>node <repo script>.js [--flags]</code> , and the marker on both rows begins <code>SHOWDOWN_PATH=... node tests/roster.js --stage moves</code> , so a fresh run would classify them <code>INSTRUMENT UNRUNNABLE</code> rather than giving them a verdict — moving them out of <code>debt</code> into the clause's third bucket, never into <code>withRed</code> . NEITHER READING CHANGES THE GATE: <code>ok</code> is <code>withRed.length === 0</code> , currently 3 rows (#218, #241, #258) out of 16 open-and-asserting-breakage, and <code>debt</code> has never held the clause shut. THE DEFECT IS THE BOOKKEEPING, AND <code>where.js</code> ALREADY DOES ITS HALF: since commit <code>cac12fa</code> it prints <i>"NAMING A GATE IS NOT HAVING A VERDICT"</i> , names the artifact's age, and lists the rows with no verdict. The quarantine side says nothing. WHAT WOULD DECIDE IT: <code>openDefectClause</code> reads the row it is already parsing for a <code>VERIFIED BY</code> marker and splits <code>debt</code> into "names nothing" and "names an instrument, no verdict recorded", and compares <code>register-reality.json</code> 's <code>generated</code> against the mtime of <code>docs/ROADMAP.md</code> , printing STALE when the artifact is older. RELATED AND NOT THE SAME ROW: #108 is <code>status.js</code> against <code>provenance.js</code> over artifact safety; this is the open-defect clause against <code>where.js</code> over the register's own coverage. Same shape, second pair — which is itself the argument for fixing the shape rather than the instance. Account: <code>docs/_reports/2026-08-22-register-reconciliation.md</code>	open — instrument DEFECT (MEASURE; measured 2026-08-22, NOT fixed in this pass because ENGINE was live in <code>tests/roster.js</code> and <code>quarantine.js</code> runs the roster stages)	
#349	THE DIVERGENCE ROLLUP'S RANKING CAN NAME THE LINE WE EMITTED RATHER THAN THE ONE THAT DIFFERS, AND A HUMAN READING 40 CARDS FOUND ~20 ROOT CAUSES WHERE THE ROLLUP OVER THE SAME RUN FOUND NONE — MEASURED 2026-08-22. The class name describes the COMPARATOR, not the defect. <code>event missing</code> from <code>medicham2</code> means "the authority emitted a line and we did not", which is a property of the DIFF and is the same sentence whether the cause is Encore's failure condition, Imposter's transform announcement or a Life Orb recoil. Two defects with nothing in common land in one bucket and one defect lands in several depending on which line differed first, so the rollup can only ever report how the comparator classified things. Over the same 961-game run its buckets were <code>"-fail 7 games"</code> and <code>"ordering 29 games"</code>; Will grouped by MECHANISM instead and recognised Encore, Roost, Toxic Debris and Matcha Gotcha on sight. TWO CONCRETE RANKING DEFECTS. (1) <code>max_uses</code> is <code>Math.max</code> over the entities named by both lines of a pair, so the head of a usage-ordered list can name the line WE emitted: the report's own head read <i>"126,170 clicks, Protect"</i> when the differing line was <code>`start</code>	typechange	[from] <code>protean</code> . **That h <code>entityStanding</code> collects over all of them. (2) <code>uses</code> : null printed as 0, BOTTOM of a usage-order half is NOT confirmed block reading <code>data/met</code> the observation may pr Recorded as observed-a ACTION WILL ASKED FOR CARDS. ** <code>--dump-games</code> costs a few hundred by Raise it to the low hu WHAT WOULD DECIDE from which side's line — so a ours
#347	THE ROSTER CREDITS SPIKY SHIELD ON A SCENARIO THAT CANNOT FAIL — OBSERVED 2026-08-22, INSTRUMENT WEAKNESS, MECHANIC BELIEVED COVERED ELSEWHERE. <code>all_mechanics_fire</code> reports <code>spikyshield</code> as RESOLVED / NO-DIVERGENCE with <code>setup: []</code> — nothing attacks into it — and marks both <code>volatile:spikyshield</code> and <code>volatile:stall</code> uncomparable. So <code>punishesContact</code> (<code>onContact: true</code>, <code>fraction: 8</code> , correctly derived) never fires in that row, and a green there is a green about nothing. THE MECHANIC ITSELF IS WELL COVERED AND THIS ROW IS NOT A CLAIM THAT IT IS BROKEN: 40 census rows touch <code>protect/shield/stall</code>, including <i>"Spiky Shield hurts the attacker it blocked"</i> with a <code>Protect</code> control that must toll o while <code>Spiky Shield</code> tolls 22, <i>"a Spiky Shield answers before the type chart"</i>, and <i>"Spiky Shield and Baneful Bunker refuse Encore exactly as Protect does"</i> for the status half (#238). The	closed 2026-09-04 — FALSE AT HEAD. <code>data/roster.moves.json</code> at HEAD stages <code>spikyshield</code> against a scenario that CAN fail — "Avalugg-Hisui shields a lethal Flash Cannon; Salazle beside it carries the same move, does NOT click it, and must die" — and reports FIRED-AND-BOARDS-MATCH with a knob-cleared control and a recorded state delta. The stage carries o FIRED-AND-BOARDS-DIFFER and o DID-NOT-FIRE on release <code>8ad06030e129</code> . Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — instrument DEFECT (ENGINE owns <code>tests/roster.js</code> and <code>engine/all_mechanics_fire.js</code> ; observed by Will 2026-08-22, filed by MEASURE)	

#	item	note
	census is the strong instrument; the roster row is the weak one. Note also that the authority itself announces Spiky Shield as <code>move: Protect</code> on both <code>-singleturn</code> and <code>-activate</code>, so our naming there is correct and the renderer is faithful — that is NOT a defect and is recorded here so it is not filed as one later. WHAT WOULD DECIDE IT: the roster's scenario builder must refuse to credit a mechanic whose <code>setup</code> is empty, or must stage a contact attacker into the shield; a row that cannot fail is a test that cannot fail, which is #314's lesson applied to a different instrument. Source <code>docs/CARD-REVIEW-2026-08-22.md</code> §F2	
#348	<code>tools/lownode.cmd</code> IS SOUND AND ITS ARGUMENT QUOTING IS A LIVE HAZARD THAT HAS ALREADY TRUNCATED A RELEASE REASON — OBSERVED 2026-08-22. The wrapper itself passes: <code>tests/test-lownode.js</code> is 4/4 including exit-code propagation, and measured directly it returned 7 for <code>process.exit(7)</code> and 1 for a module-not-found with stdout and stderr intact. The hazard is the arguments. An absolute Windows path splits on its drive colon across the Bash-to-<code>cmd</code> boundary, and <code>start</code> mis-parses a quoted argument containing spaces — <code>--seed "a vs b"</code> launches <code>cmd</code> rather than <code>node</code>. It has already cost something: a quoted <code>--why</code> string was truncated to a fragment on an engine-release cut, so a release carries a reason that is not the reason anybody typed. THE WORKAROUND IS REPO-RELATIVE PATHS AND NO SPACES IN A QUOTED ARGUMENT, and that is a rule somebody has to remember, which is what this register exists to replace. WHAT WOULD DECIDE IT: an arm in <code>tests/test-lownode.js</code> that passes an argument containing a space and an argument containing a drive-letter path and asserts the child received them byte-identically — shown RED against the current wrapper before any change. Related and NOT the same: #300, the differential's unreachable catch block on a bad <code>SHOWDOWN_PATH</code>. Source <code>docs/CARD-REVIEW-2026-08-22.md</code> §F3	open — instrument DEFECT (OPS/tooling; observed 2026-08-22, filed by MEASURE, no fix attempted)
#346	THE WHOLE-GAME HARNESS MANUFACTURED DIVERGENCES ON A DOUBLE KO, AND CARD 25 WAS ENTIRELY THAT — FOUND AND FIXED 2026-08-22, WITH NO REGISTER ROW UNTIL NOW. <code>engine/game-differential.js</code> mirrors medicham2's actives into Showdown's forced-switch replacement choice and the fallback had no memory of what the other slot just took, so on a double KO on a side down to its last usable body it sent <code>"switch 4, switch 4"</code>. Showdown refused it — <i>"The Pokemon in slot 4 can only switch in once"</i> — eight times, and the return value of <code>battle.choose</code> was DISCARDED on that path, so every refusal was swallowed; <code>requestState</code> stayed <code>switch</code> and the next loop reported <i>"showdown stopped asking for a move"</i>. <code>chooseAction</code> already carried a <code>claimed</code> set; this path did not. With the choice repaired the two engines agree on ALL 136 LINES of that game and both end it with the same winner. There was never an engine defect in it. THE claimed SET ALONE WAS NECESSARY AND NOT SUFFICIENT, AND THAT IS THE FINDING: the moment the refusal stopped being swallowed, a 43-game batch threw on a different shape — a body FAINTED in Showdown and at 139 HP here, so the boards had already parted and NO answer reproduces medicham2's placement. Throwing there would have moved a REAL engine divergence into the instrument's error column, so the mirror has a third answer, <code>cannot</code>, and the game stops with a named end reason AFTER the turn's bookkeeping runs — load-bearing, because the comparator call four statements later is what records the earlier, real divergence. Counted apart as <code>forced_switch_unmirrorable</code>, expected non-zero, DELIBERATELY NOT GATED. <code>choices_refused</code> counts one <code>battle.choose()</code> the authority returned false for: 0 across 129 pinned games, 0 in a 12-game full-driver run, exactly 1 under deliberate sabotage, asserted. Landed in commit <code>ff7a2bc</code> with <code>tests/test-forced-switch-mirror.js</code>. CONSEQUENCE FOR EVERY EARLIER FIGURE: the repaired game runs 11 turns instead of 10 and <code>chooseAction</code> ranks on run-level accumulators, so from baseline pair 12 onward every later game may play differently — class counts either side of this are comparable only as distributions. Filed CLOSED so it is not re-discovered	closed 2026-08-22 — ENGINE/instrument, commit <code>ff7a2bc</code>; filed retrospectively by MEASURE 2026-08-22 because the landing carried no register row
#345	THE AUTHORITY ANNOUNCES A VOLATILE COUNTER'S EXPIRY AND WE ARE SILENT, AND NOTHING PROVES THE TWO ENGINES END ON THE SAME TURN — OBSERVED 2026-08-22 BY WILL ACROSS THREE CARDS. Three cards, three counters, one question: do they tick and expire on the same turn? Showdown emits <code>-end ... Infestation [partiallytrapped]</code> [<code>silent</code>] twice and <code>-end ... Throat Chop</code> once, and we emit nothing. The state behind them IS compared — <code>board_state.js:661</code> reads <code>trapped_by_move</code>, and the file notes the victim's half is <i>"the half that decides whether a switch is legal"</i> — but the comparator's own proof is not clean: <code>probe_bench_plants.js</code> lists a MOVE TRAP counter off by one among its 19 applied-and-not-caught plants (7 of 8 pairs catch it), which is #314's second hole. So the field is read, a lie about it is mostly detected, and the instrument cannot say so with authority. AN UNANNOUNCED EXPIRY AT THE RIGHT MOMENT IS COSMETIC; ONE A TURN LATE IS A BODY THAT CANNOT	closed 2026-09-04 — FALSE AT HEAD on all three counters, and the same-turn question the row asked is answered by the census rather than argued. Throat Chop: "the silence lasts the turn it lands and ONE more, and its end is a line". Infestation: "the trap chip names the MOVE that bound the body, not the condition". Perish: "the perish counter reads 3 / 2 / 1 at the ends of turns 1, 2 and 3". Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code>. <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, expiry turn unprobed)

#	item	note
	SWITCH WHEN IT SHOULD. Infestation blocks switching, and Perish Song's last tick KILLS — <code>duration: 4</code>, the cast-time <code>perish3</code> is <code>[silent]</code>, and the residual prints whatever the counter now reads, one step per end of turn. NOT COVERED BY #242, which is CLOSED and settled the residual ORDER and which of 90 walk participants announce; and not covered by #234, CLOSED, which fixed the <code>[from]</code> attribution naming the condition where Showdown names the move. Neither asserts that a counter expires on the same TURN. WHAT WOULD DECIDE IT: stage Infestation, Throat Chop and Perish Song separately, run each to expiry on both engines, and assert both end on the same turn and emit the same <code>-end</code> line — with Perish Song's arm asserting the body dies on the same turn, since that is the one where a tick off by one is a game	
#343	A FAILED ROOST LEAVES THE USER STILL FLYING ON THE AUTHORITY AND GROUNDED HERE, AND IT IS BOARD-MATERIAL — STAGED AND DELIBERATELY NOT FIXED 2026-08-22. Found while landing #342's <code>null</code> -against- <code>false</code> classification and stated in that commit's own account, with no register row until now. We emit the <code>-fail ... heal</code> line correctly and THEN ALSO the <code>-singleturn</code>; the authority emits NEITHER, because <code>runMoveEffects</code> marks the target <code>false</code> and <code>selfDrops</code> skips false targets. The consequence is not the two lines: it is that the Roost user's Flying type is suppressed here on a turn the authority leaves it Flying, so a Ground move that should be refused connects, and every damage and speed read for that turn is taken against the wrong type. Left undone on purpose — the rule in this repository is one family per pass, and the Roost family wants its own hand list rather than a free ride inside the Encore/Yawn batch. WHAT WOULD DECIDE IT: stage a Roost that fails (a user already at full HP) on both engines and assert (a) neither line is emitted and (b) a Ground move into that user is refused on both. Source <code>docs/CARD-REVIEW-2026-08-22.md</code> §E3 and commit <code>11bab14</code>. Adjacent: #147 wired Roost's grounding in the first place; #242 records that Roost is residual order 25, announces nothing, and gives its type back TWENTY orders above the Grassy Terrain heal	closed 2026-09-04 — FALSE AT HEAD. Live census row "a Roost that heals NOTHING grounds nothing — the self-rider is skipped when the primary failed", with counters in <code>engine/medicham2-browser.js</code> filed under this row number (6 occurrences). Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code>. <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; staged 2026-08-22, filed by MEASURE, fix deliberately deferred)
#344	A FAINTED BODY STAYS IN ONE OF OUR ACTIVE SLOTS WHERE SHOWDOWN CLEARS IT INSIDE <code>faintMessages</code> — OBSERVED 2026-08-22, CONFIRMED AS REAL STATE, CONSEQUENCE UNMEASURED. A Dragonite read <code>fainted=true</code> together with <code>where=active</code>. Showdown sets <code>pokemon.isActive = false</code> inside <code>faintMessages</code> (<code>sim/battle.ts:2563</code>); we leave the body in the active array until something replaces it. Confirmed to be real state rather than a labelling artifact — <code>rosterSnapshot</code> reads membership of the active array itself, not a flag. IT IS NOT THE CAUSE OF CARD 25 (see #346, which was entirely the harness) and in the repaired game it produced zero divergent lines, so it is recorded as an observation and NOT asserted as a defect. It is nevertheless the same <code>faintMessages</code> region as #331 (the batching) and #315 (a fainted mega's forme), and <code>isActive = false</code> at <code>:2563</code> is what makes the authority's <code>toString()</code> print a bare side id — so if #331 is fixed this may fall out with it. WHAT WOULD DECIDE IT: a boundary comparison asserting no body with <code>fainted=true</code> is a member of either active array at the moment the authority has already cleared it, plus a search for a read path (<code>bringIn</code>, targeting, spread-move target selection) that would behave differently with a corpse in the slot. Until one of those produces a consequence this row makes no claim about a mechanic. Source <code>docs/CARD-REVIEW-2026-08-22.md</code> §E4 REFUTED AND CLOSED 2026-08-27 BY ENGINE. THE AUTHORITY DOES NOT CLEAR THE SLOT — IT CLEARS A FLAG, AND THE DUMP WAS COMPARING OUR MEMBERSHIP AGAINST THEIR FLAG. <code>faintMessages</code> sets <code>pokemon.isActive = false</code> (<code>sim/battle.ts:2563</code>) and does NOT touch <code>side.active</code>: the only writers of that slot in the whole simulator are <code>switchIn</code> (<code>sim/battle-actions.ts:136</code>), <code>swapPosition</code> (<code>:1597-8</code>) and the request mirror at <code>:2690</code>. So the corpse sits in the authority's slot exactly as it sits in ours. MEASURED, NOT ARGUED — <code>tests/probe_corpse_in_slot.js</code> wraps <code>Battle#faintMessages</code> and reads the authority's own object the instant it returns, over the pinned pool (129 pairs, release <code>f6a3b35ed665</code>): 1,202 fainted bodies were STILL MEMBERS of <code>side.active</code>, o of them read <code>isActive === true</code>, and the control is clean — o of 69,626 off-field bodies were members, so membership does distinguish. THE PARITY IS EXACT: over 1,594 turn boundaries, 59 of them holding a corpse (174 corpse-slots), the two engines' active arrays matched on every slot, o mismatches, compared by <code>rosterKey</code> through the one door with o fallbacks; the planted-occupancy control fired 227 of 227. THE ROW'S EVIDENCE WAS AN INSTRUMENT DEFECT AND IT IS FIXED. <code>rosterSnapshot</code> answered <code>where</code> with <code>acts.indexOf(m)>=0</code> on the medicham half and <code>p.isActive</code> on the showdown half, and <code>engine/replay_one.js:387</code> printed <<< A FAINTED BODY IN AN ACTIVE SLOT beside the MEDICHAM rows only — an alarm on one engine and a clean row on the other for behaviour they agree on. Shown RED first: 41 of 41 fainted bodies read <code>medicham where=active / showdown where=bench</code>; both	closed 2026-08-27 — REFUTED. Not a state divergence: the two engines hold the corpse in the same slot, measured 1,202 times with a control that can fail. The observation was <code>rosterSnapshot</code> comparing membership against a flag, and both that field and <code>replay_one.js</code>'s one-sided alarm are fixed. The one consequence found beyond today's <code>ModifySpe</code> work is #495

#	item	note
	halves now read MEMBERSHIP (the flag is kept beside it, named), the annotation is symmetric, and the same probe reads o of 41. THE CONSEQUENCE IS MEASURED, WHICH IS WHAT THIS ROW ASKED FOR. <code>isActive</code> has exactly one read a corpse can reach that an <code>hp / fainted</code> gate does not already dominate — <code>findEventHandlers</code> (<code>sim/battle.ts:1053-1067</code>), which for a corpse target collects NOTHING: not its own handlers, not any <code>onAlly / onAny / onFoe</code> on the field, and not the <code>instanceof Side</code> block below it. That block was recomputed on every call whose target was a corpse. Over the same pool, 13 event names reached a corpse and exactly TWO suppress anything: <code>ModifySpe</code> (211 calls, 39 handlers — choicescarf 24, tailwind 8, sandrush 3, par 2, swiftswim 1, chlorophyll 1), which is the channel landed today by the replacement batch, and <code>ModifyPriority</code> (4 calls, 2 handlers, both galewings), filed as #495. Every other row is <code>o</code>. The control is the SAME recompute run against LIVE bodies — <code>267</code> of <code>6,493</code> sampled reads non-zero over <code>17</code> event names — so a zero above is a fact about the corpse and not about the recompute. <code>--verify-inert</code> replays every game with both wrappers disarmed and the verdicts are identical, so the instrument did not move what it measured. NOTHING IN THE SIMULATOR CHANGED AND NOTHING WAS EXPECTED TO: the release re-cut to the SAME id <code>f6a3b35ed665</code> over an identical tree, and the pinned re-run (<code>--games 1200, arm middle, cap 12, data/team-pool-frozen, census pin 9446a684709d</code>) reads 961 games, 15 raw diverged, board-material 9 of 961, whole-game 10 of 961, census 765 live / 765 probed / o missing — every figure identical to the baseline, as predicted before the run. Account: <code>docs/_reports/2026-08-27-fainted-active-slot.md</code>. VERIFIED BY: <code>node -r ./tests/_live_release.js tests/probe_corpse_in_slot.js --games 1200 --verify-inert</code>	
#341	DRAGON TAIL AND CIRCLE THROW IGNORE SUCTION CUPS — CONFIRMED, PROBED RED 2026-08-22. The phaze branch reads <code>refusesForcedSwitch</code>; the DAMAGING branch does not, so a damaging phaze move drags a body the authority refuses to move. Staged: Tyrannitar's Dragon Tail into Malamar — the authority emits <code>-activate p2a: Malamar / ability: Suction Cups</code> and nobody moves; we emit a <code>drag</code> of Snorlax. Malamar is the only legal carrier of Suction Cups in Reg M-B and is brought 1,340 times across the two human stores. The membership is derived and small, which is why it can be closed cleanly: exactly two abilities and one move carry <code>onDragOut</code> in the whole authority — Suction Cups (legal, one carrier), Guard Dog (zero legal carriers) and Ingrain (a legal move that zero legal species can learn here). Champions overrides none of the three. That is why this is the only member of the family that can fire, and it is why #175's rule holds: do not add a Guard Dog row by hand — <code>tests/test-tag-params-derived.js</code> proves the predicate already covers it, its absence IS the legality filter, and a regulation that brings a carrier would pick it up with no edit. WHAT WOULD DECIDE IT: the staged Tyrannitar-into-Malamar pair above, asserting NO body moves and the <code>-activate</code> line is present, with a non-carrier control that must still be dragged	closed 2026-09-04 — FALSE AT HEAD. The DAMAGING branch now reads the tag — <code>TAGS.param("ability", suppressedAbility(m, tg), "refusesForcedSwitch")</code> — with the counter <code>forcedSwitchRefusedDamaging</code> and a <code>DRA_G_ABILITY_FIRST</code> control, and the live census row is "Suction Cups cannot be dragged out by Roar OR by Dragon Tail, and Mold Breaker ignores it". Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code>. <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; probed red 2026-08-22 by Will, filed by MEASURE)
#342	WE EMITTED AN EXTRA -fail ON THE PHAZE REFUSAL BECAUSE WE HAD NO MODEL OF null AGAINST false — FOUND AND FIXED 2026-08-22, AND THE FIX HAD NO REGISTER ROW UNTIL NOW. Showdown's move pipeline carries a FOUR-VALUED result (<code>battle-actions.ts:1156-1170</code>) and three independent sites branch on it: <code>null</code> means handled, stay silent; false means failed, announce it. We modelled neither, which is why we were silent where we should speak (Encore, Yawn) and speaking where we should be silent (the phaze refusal, which printed <code>-activate ... suctioncups PLUSa -fail</code>, where <code>onDragOut</code> returns <code>null</code> and the authority writes <code>-fail</code> only on <code>hitResult === false</code>). One root, opposite symptoms. <code>applyMoveVolatile</code> now classifies every refusal as FAIL or SILENT and a caller that cannot classify one announces NOTHING. RED FIRST ON 5 OF 10 ARMS — Encore into a body that has not moved; a second Encore onto the volatile it just made; Encore into a body whose last move was Encore, which was a STATE bug (we actually applied it); a second Yawn; and the phaze case running the other way. All 10 green, five counters at exact equality. THE OVER-FIRE PROOF IS THE PART THAT MATTERS, because over-firing is what got the previous attempt retracted: 355 games, pool pinned identically, arms differing only in <code>medicham2-browser.js</code>, ATTRIBUTED BY TRACE DELTA rather than by tally — of the 44 games whose first delta is our failed-move line, ZERO agreed-then-parted. Yawn's <code>onTryHit</code> half is DELIBERATELY LEFT UNDONE: our <code>canTakeStatus</code> is wider than <code>runStatusImmunity</code> and includes Safeguard, where the authority is silent, and announcing there is the exact over-fire that cost the last attempt. Ours also printed the raw id <code>suctioncups</code> where the authority prints <code>Suction Cups</code>. Landed in commit <code>11bab14</code>; the account is <code>docs/CARD-REVIEW-2026-08-22.md §E3</code> and <code>docs/MEDICHAM-SPRINT-NOTES.md</code>. This row is filed CLOSED so the fix is not re-discovered, not so it is re-done	closed 2026-08-22 — ENGINE, commit <code>11bab14</code>; filed retrospectively by MEASURE 2026-08-22 because the landing carried no register row
#340	THE BENCH IS IN A DIFFERENT ORDER FROM THE AUTHORITY'S, AND BOTH OF OUR INSTRUMENTS ARE	closed 2026-09-04 — FALSE AT HEAD, and the blindness the row describes is gone with it. The live census row is the authority rule verbatim: "a drag indexes into the bench

#	item	note
	BLIND TO IT BY CONSTRUCTION — DIAGNOSED 2026-08-22, DIE HYPOTHESIS REFUTED, LANDED AS A DIAGNOSIS WITH NO ROW UNTIL NOW. The drag divergence was hypothesised to be the die and it is not: both engines roll a uniform die, consume exactly one draw, and under pinned dice draw the same value at the same address. What differs is the ORDER of the list the die indexes into. <code>sim/battle-actions.ts:125-132</code> swaps the outgoing body into the arriving body's party slot; we remove-and-append. Measured: 34.8% of side-boundaries hold the same bench in a different order, and 0 of 3,118 index positions map to the same body — zero fixed points, so a uniform index disagrees with probability 1. INVISIBLE TO BOTH INSTRUMENTS, AND THAT IS THE IMPORTANT PART: the protocol comparison sees no line, and <code>board_state.js</code>'s <code>partyMap</code> keys the party BY SPECIES — correctly, because index-keying manufactured a divergence in 123 of 179 games — and the same correct choice makes it blind to WHERE a body is standing. BLAST RADIUS IS LARGER THAN THE DRAG: <code>bringIn</code>'s default replacement is <code>_live(bench)[0]</code>, so a reordered bench moves every faint replacement and every pivot with no explicit target, in rollouts and in <code>board.js</code> features. Measure it; do not assume it neutral. WHAT WOULD DECIDE IT: a boundary-by-boundary comparison of the two party arrays by INDEX (not by species) on a pinned run, asserting the same body stands at the same index, with the 3,118 positions as the denominator. Source <code>docs/CARD-REVIEW-2026-08-22.md §E1</code>; the diagnosis landed in commit <code>11bab14</code> and carried no register row	in the AUTHORITY's order — a body that switches out takes the incoming body's party slot". Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code>. <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; diagnosed 2026-08-22, filed by MEASURE, fix not attempted)
#338	LIFE ORB RECOIL DID NOT FIRE FOR A WHOLE TURN IN A REAL GAME, WHICH IS THE BEHAVIOURAL HALF #324 EXPLICITLY EXCLUDES — OBSERVED 2026-08-22 BY WILL READING CARD 24 (index 37); NOT PROBED SINCE. <code>data/tags.json</code> gives <code>lifeorb</code> a <code>damageMultAll</code> of <code>mult 1.3</code> plus <code>costsPerAttack: "1/10 max HP"</code> — the multiplier is machine-readable and the cost is an English string, so nothing can derive the recoil from the tag and whatever applies it is hardcoded elsewhere, which is why it fires in two cards and not in a third. On this card Gholdengo landed a resisted Shadow Ball and never paid its 16 HP; the stream runs to the <code> upkeep</code> marker with no Life Orb line at all, so it is absent for the turn, not merely late. Life Orb appears in 17,168 of 19,401 bo3 games and the roster credits it <code>FIRED / diverged false</code>, so two live instruments both call it clean. NOT COVERED BY #324, AND THE DISTINCTION IS THE POINT: <code>#324</code> registers <code>item:lifeorb / damageMultAll.costsPerAttack</code> as one of 119 tag values read by nothing and says in its own words that <i>"none of these is a behavioural defect on its own — the mutation moved nothing"</i>. This row is the counterexample to that sentence for one member of the 119. WHAT WOULD DECIDE IT: replay card 24's index 37 on both engines and assert the recoil line is present, then stage a resisted hit, a super-effective hit and a hit that would leave the holder at 1 HP — a recoil applied conditionally is what the two-of-three card pattern suggests	closed 2026-09-04 — FALSE AT HEAD. Live census row "the Life Orb toll is refused by a move that MISSED", the knob <code>MEDI_ORB_STALE_RANGE</code> restores the stale-range behaviour, and <code>data/roster.items.json</code> at HEAD reports <code>lifeorb FIRED-AND-BOARDS-MATCH</code> on release <code>8ad06030e129</code>. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code>. <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, unprobed)
#339	A SPREAD DRAIN HEALS ONCE OVER THE SUMMED DAMAGE WHERE THE AUTHORITY HEALS AND ROUNDS PER TARGET, SO WE HEAL 1 HP LESS ON A QUARTER OF THEM — OBSERVED 2026-08-22 BY WILL READING CARD 1 (index 0). Showdown heals per target, interleaved, with an <code>[of]</code> attribution on each; we heal once at the end and drop the attribution. The arithmetic is merged too, and that half is not narration: <code>dealT</code> accumulates across targets and the drain heals half of the accumulated total once, where the authority rounds per target (<code>battle.ts:2168</code>). The two disagree exactly when both hits deal odd damage — 25.0% of two-target spread drains — and we heal 1 HP less. Legal spread-drain moves in this regulation: Matcha Gotcha (8,182 uses) and Parabolic Charge (151); Parabolic Charge is <code>allAdjacent</code>, so up to three drains and three roundings. PARTIALLY ADJACENT TO #324, which lists <code>move:matchagotcha / drain.{num,den}</code> at 4.36% of corpus sheet entries as a param the simulator dereferences nowhere — that is the derivation half; this is the arithmetic half and <code>#324</code> asserts nothing about it. WHAT WOULD DECIDE IT: a staged two-target Matcha Gotcha whose per-target damage is odd on both bodies, asserting the healer's HP matches the authority to the point, with an even-damage control that must agree both before and after the fix CORROBORATED BY THE ROSTER 2026-08-23 ON RELEASE <code>c36782953dee</code>: <code>data/roster.moves.json</code> (generated 2026-08-23T06:06:12Z) reads <code>matchagotcha FIRED-AND-BOARDS-DIFFER</code>, and every parted leaf is the HEALER short by one — <code>goodrahisui party.hp showdown 583 ours 582 at turn 3, 592 / 591 at turn 4</code> — which is the direction and the magnitude this row predicts. IT IS CORROBORATION AND NOT A SECOND MEASUREMENT OF THE 25.0%: the roster stages one scenario, so it says the arithmetic parts and says nothing about how often.	closed 2026-09-04 — FALSE AT HEAD. Live census row "a SPREAD drain rounds once per body, not once over the summed damage", the knob <code>MEDI_DRAIN_LUMP_ROUND</code> restores the lumped rounding, and <code>data/roster.moves.json</code> at HEAD reports <code>matchagotcha FIRED-AND-BOARDS-MATCH</code> where this row recorded it FIRED-AND-BOARDS-DIFFER. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code>. <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE)
#337	onTryImmunity IS NOT DERIVED, SO SIX LEGAL MOVES CARRY NO IMMUNITY GATE AT ALL — OBSERVED 2026-08-22 BY WILL READING CARD 35. Endeavor's gate is <code>onTryImmunity(target, pokemon) { return pokemon.hp < target.hp; }</code>, and returning false produces a type-immunity-style	closed 2026-09-04 — FALSE AT HEAD. <code>data/tags.json</code> <code>moves.endeavor.params.immunityGate.hook</code> is <code>onTryImmunity</code>, the engine reads <code>immunityGate</code> at 40 sites, and the live census row is the row's own equal-HP case: "Endeavor at equal HP is IMMUNE — no damage AND no damage LINE". Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code>; this edit

#	item	note
	block (<code>-immune</code>), not <code>-fail</code> . On card 35 both bodies were at 135/135 — Whimsicott and Raichu share max HP at L50 and 0 EV — so <code>135 < 135</code> is false and the authority answered <code>-immune</code> ; we ran the <code>damageCallback</code> (<code>target.hp - pokemon.hp = 0</code>) and announced a zero-damage line reading "drops to 100%". Our tags for endeavor carry no immunity condition at all. Legal moves on this hook: Attract, Endeavor, Leech Seed, Switcheroo, Trick, Worry Seed. Trick and Switcheroo matter most — a missing immunity gate means an item swap that should be blocked goes through, which is board-material. Recurring shape: a fact that lives in a HANDLER rather than a declarative field is not picked up by <code>tag_dex</code> , and the roster then credits the entity clean because nothing staged the condition — joins #317 (Fur Coat) and #312 (Sand Force). ADJACENT AND NOT THE SAME ROWS: #327 covers <code>immunityGate</code> on <code>move:trick</code> against Sticky Hold specifically (0 corpus uses), and #323 records <code>move:leechseed / immunityGate</code> as a classifier FALSE POSITIVE because the Grass immunity is implemented through a sibling tag. Neither covers the hook. WILL'S REQUIREMENT, AND IT IS THE DISCIPLINE THAT GOT THE LAST -fail ATTEMPT RETRACTED: STAGE THE POSITIVE ARM TOO. Three cases, all three asserted: user HP greater than target HP gives <code>-immune</code> with no damage line; user HP EQUAL to target HP gives <code>-immune</code> as well, because the gate is a strict less-than (the 135/135 case, easy to miss); user HP less than target HP works, the target drops to exactly the user's HP with a real <code>-damage</code> line. A test that only proves the immunity is a test that ships an Endeavor which never works	moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE)
#336	SLEEP: THE DISTRIBUTION IS MODELLED CORRECTLY AND WHETHER THE DRAW IS ADDRESSED IDENTICALLY HAS NEVER BEEN CHECKED — RAISED 2026-08-22, VERIFICATION OWED, NO CLAIM OF BREAKAGE MADE. Champions overrides <code>slp</code> and uses <code>this.sample([2, 3, 3])</code> ("<i>1/3 chance for a Pokemon to wake up on turn 2</i>") where mainline uses <code>this.random(2, 5)</code> giving 2, 3 or 4 — a different distribution AND a different draw shape, since <code>sample</code> calls <code>random(3)</code> . Our <code>SLEEP_TURNS_LOST = 1 + 2/3</code> is exactly the expected value of <code>sample([2,3,3])</code> , one third times one plus two thirds times two, so the distribution half is right. What is unverified is the ADDRESS: whether both engines consume the same number of draws at the same point, which is what decides every later die in a pinned game. Reading mainline here is the documented way to be wrong. WHAT WOULD DECIDE IT: a pinned staged sleep on both engines printing the draw count and the address consumed, against the Champions override read from the mod and cited by line. THIS ROW ASSERTS NOTHING ABOUT A MECHANIC BEING WRONG and deliberately carries no defect token in its cell: a row that overstates its scope costs a whole agent, which is more than the defect it names	open — verification owed, MEASURE. NOT A DEFECT: raised by Will 2026-08-22 as a question, not as an observed divergence
#334	THE CONFUSION SELF-HIT DRAWS A DIFFERENT INDEX FROM THE AUTHORITY, AND THE FORMULA IS RULED OUT AS THE CAUSE — OBSERVED 2026-08-22 BY WILL READING CARD 36; ROOT CAUSE UNPROBED. Our formula is correct and documented: no STAB, no type chart, no crit, no screen, no burn, sixteen sides at <code>85+r</code> , matching <code>getConfusionDamage</code> (<code>battle-actions.ts:1850</code>), with ONE gap declared rather than hidden — an <code>onModifyAtk</code> ability, so a burned Guts body hits itself harder. The move is typed with the wildcard category, so no item or type boost can reach it; that hypothesis is ruled out. Authority <code>108 -> 75</code> against our <code>108 -> 78</code> is 33 against 30, and <code>30/33 = 0.909</code> sits inside the 85-100% band, so it is a different index, not a different multiplier. Note the authority takes TWO draws here — <code>randomChance(33,100)</code> for whether to self-hit, then <code>randomizer(damage)</code> — so a different order or category shifts every later die address too, and this may be an ADDRESSING defect rather than a damage one. WHAT WOULD DECIDE IT: pin the dice and print the index each engine selected for the same staged self-hit, plus the draw COUNT consumed at that address. Do not fold this into #333 until the index is read: one is a scheme and the other is an address	open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, root cause unprobed)
#335	A PER-TARGET SECONDARY LANDS ON THE WRONG BODY — #294 FIXED THE ACCURACY ROLL AND NOT THE SECONDARY ROLL. OBSERVED 2026-08-22 BY WILL READING CARD 40; NOT PROBED SINCE. Muddy Water is a spread move with a 30% accuracy-drop secondary that the authority rolls per target. Showdown dropped Spiritomb's accuracy; we dropped Snorlax's. Same shape as #294 — "<i>the accuracy roll was one per move where the game rolls one per target</i>" — one hook over, and #294 is CLOSED, so its row must not be read as covering this. The sleep difference in the same card is probably DOWNSTREAM of this: once a secondary fires on a different body every later die address shifts, so it must not be treated as an independent defect until this is fixed. WHAT WOULD DECIDE IT: a scripted per-target secondary stream on a spread move with a sub-100% secondary, asserting all four outcomes (neither, first only, second only, both) are reachable — the exact discriminator #294 used for the accuracy roll, shown RED on the pre-fix tree there, and never run for secondaries	closed 2026-09-04 — FALSE AT HEAD. <code>engine/medicham2-browser.js</code> carries <code>secAddrFromLastTarget</code> and <code>secAddrMovedByFire</code> with the knob <code>MEDI_SEC_ADDR_PER_TARGET</code> restoring the shared address, which is this repo's proof that the per-target secondary address is wired and can be shown to move. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, unprobed)
#333	THE MULTI-HIT DAMAGE PATH WAS NEVER CONVERTED TO THE SIXTEEN-INDEX ROLL, SO #304'S FIX COVERS SINGLE HITS ONLY — PROVED BY ARITHMETIC 2026-08-22	closed 2026-09-04 — FALSE AT HEAD. The multi-hit path carries <code>perHitDamageLoop</code> and draws <code>damageRollIndex</code> per packet (<code>_pkArr.push(_pks[i].band[damageRollIndex(_au)])</code>), and the live census

#	item	note
	FROM CARDS 38 AND 39. <code>Battle#randomizer</code> is <code>tr(tr(base * (100 - random(16))) / 100)</code> : sixteen discrete values between 85% and 100% of the base. Twin Beam is the clean case because both hits are 40 BP, so the base is identical for both. Authority 185 -> 130 -> 75 , hits of 55 and 55 — one base, the same index twice. <code>medicham2</code> 185 -> 125 -> 75 , hits of 60 and 50. No single base can produce both under that scheme: 60 requires <code>B >= 60</code> and 50 requires <code>trunc(B x 0.85) <= 50</code> , i.e. <code>B < 60</code> . Contradiction. So either our base changes between hits of a constant-BP move or our draw is not the sixteen-index one. This is the unconverted residue of #304, whose own status cell already names it — <i>"8 games survive, candidate is the declared per-hit multi-hit randomizer"</i> — and a uniform draw WOULD produce 60 and 50 from one base because it can hit any integer in the interval. It also explains Will's observation that multi-hit damage "evens out": a uniform draw is unbiased about the same mean, so totals converge while individual hits scatter. WHAT WOULD DECIDE IT: a staged constant-BP two-hit move on both engines under pinned dice, asserting the two hits are equal whenever the authority's two are equal, plus a per-hit support count in the style of <code>#304's damage_interior</code> block	row is "each hit of a multi-hit move arrives as its own damage packet, as Showdown emits it". Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; the multi-hit half of #304, which is CLOSED and must not be read as covering it)
#329	<code>onDamagingHit</code> IS WRONG TWICE ON ONE HOOK — IT RESOLVES BEFORE THE MOVE'S OWN SECONDARIES WHERE THE AUTHORITY RESOLVES IT AFTER, AND IT FIRES ONCE PER MOVE WHERE THE AUTHORITY FIRES ONCE PER HIT. OBSERVED 2026-08-22 BY WILL READING DIVERGENCE CARDS 29 AND 37; NOT PROBED SINCE. Two defects, filed as one row because they are one derivation. ORDER: <code>sim/battle-actions.ts:1099</code> runs <code>secondaries()</code> and <code>:1121</code> runs <code>runEvent('DamagingHit')</code> , so secondaries resolve FIRST; card 29 is Flare Blitz's burn against a Stamina Defense boost in the opposite order here. FREQUENCY: card 37 has the authority laying one Toxic Debris layer per hit of Dual Wingbeat where we lay one after the move completes. Twenty legal abilities ride this hook — Aftermath, Cursed Body, Cute Charm, Effect Spore, Electromorphosis, Flame Body, GooeY, Illusion, Innards Out, Justified, Mummy, Poison Point, Rough Skin, Sand Spit, Spicy Spray, Stamina, Static, Toxic Debris, Wandering Spirit, Weak Armor — and several are board-material rather than cosmetic: Aftermath and Innards Out deal damage, so the attacker's HP when the secondary lands differs; Weak Armor changes Speed and therefore turn order; Cursed Body disables a move; and Mummy overwrites the attacker's ability, so running it before a secondary the NEW ability would have modified resolves that secondary under the wrong ability entirely. WHAT WOULD DECIDE IT: two staged pairs run on both engines — (a) a contact Flare Blitz into a Stamina carrier, asserting the same relative order of the burn and the boost, and (b) a two-hit contact move into a Toxic Debris carrier, asserting TWO layers rather than one. Source <code>docs/CARD-REVIEW-2026-08-22.md §A1</code>, release <code>6a05dd9ad60d</code> , census pin <code>2cab3179f5fc</code> , 961 games / 133 diverged	closed 2026-09-04 — FALSE AT HEAD. Live census row "a two-hit volley raises DamagingHit between its arrivals, not once below both" — both halves of the row in one assertion — and the two behaviours are knobbed apart in <code>engine/medicham2-browser.js</code> : <code>MEDI_REACT_BATCHED</code> (where each firing lands) and <code>MEDI_VOLLEY_REACT_DRAWN</code> (how many times it fires), so the wiring is fallible rather than assumed. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, unprobed)
#330	<code>onSwitchInPriority</code> IS NOT MODELLED AT ALL — WE SORT SWITCH-IN EFFECTS BY SPEED AND THE AUTHORITY SORTS BY A DECLARED PER-ABILITY PRIORITY FIRST. OBSERVED 2026-08-22 BY WILL READING CARDS 26 AND 27; NOT PROBED SINCE. <code>Battle#comparePriority</code> is order, then priority, then speed — speed is only the third tiebreak — and this engine sorts entry effects on speed alone. Sixteen abilities declare the field in mainline, Champions overrides none, and five have legal carriers in Reg M-B: Unnerve +1 (6 carriers incl. Tyranitar), Klutz +1 (3), Mimicry -1 (1, Stunfisk-Galar), Forecast -2 (4, Castform + formes), Hospitality -2 (2, Sinistcha and Sinistcha-Masterpiece). Card 26 is the proof and it runs the wrong way round from speed: Torkoal is base-20 Speed and Sinistcha base-70, yet Drought fired FIRST on the authority because Hospitality carries -2 . We put the faster body first and got the reverse. Same family as the Intimidate-vs-Drizzle card. WHAT WOULD DECIDE IT: a staged entry pair on both engines with a Hospitality carrier faster than a weather setter, asserting the weather setter resolves first; plus the derived membership printed before wiring, because a name-matched implementation of five carriers is the shape this repository bans. Source <code>docs/CARD-REVIEW-2026-08-22.md §A2</code> CLOSED 2026-08-24 BY MEASURE — THIS FINDING WAS ALREADY STALE WHEN IT WAS FILED, AND IT IS RECORDED SO NOBODY RE-OPENS IT. The priority key landed on 2026-08-22 and is in the COMMITTED bytes: <code>engine/switchin_order.js</code> derives every ability declaring <code>onSwitchInPriority</code> from <code>Dex.forFormat</code> into <code>data/switchin-order.json</code> (generated 2026-08-22T04:48Z) — 16 declare it, 5 have a legal carrier in this regulation: Klutz +1 (3 carriers), Unnerve +1 (6), Mimicry -1 (1), Forecast -2 (4), Hospitality -2 (2); Intimidate (19 carriers) and Drizzle (2) declare none and sort on speed alone, and no legal ITEM declares one. <code>engine/medicham2-browser.js:5534</code> reads that table with <code>MEDFAILS.switchInPriorityTableMissing</code> guarding an absent one, and BOTH entry comparators apply it (<code>:5553</code> and <code>:5738</code>). The list was RE-DERIVED from the format rather than trusted and matched the artifact exactly; the earlier claim that 7 abilities declare it had	closed 2026-08-24 (MEASURE) — the mechanic was already implemented on 2026-08-22, so this row was two days stale on the day it was filed and was holding the open-defect clause shut for nothing. VERIFIED BY: <code>node tests/test-resolution-order.js --only a2-red</code> — REPAIRED 2026-09-04 BY MEASURE. The old marker was <code>data/switchin-order.json</code> , a DATA FILE and not a command, so <code>engine/register_reality.js</code> had nothing to run and this row's coverage was a citation of an artifact. Neither of the two things the old cell named is an instrument: <code>engine/switchin_order.js</code> prints the derivation and falls off the end at exit o whatever it finds, and a <code>MEDSEEN</code> counter is a number in a process, not an exit code. Arm <code>a2-red</code> IS the row: card 5 rebuilt, Torkoal base Speed 20 against Sinistcha base Speed 70, and it fails unless the streams agree clean, PART under the surgical revert <code>entry-sort-speed-only</code> , and the <code>switchInPrioritySeparated</code> delta is exactly 1 — a knob that moves, in the arm's own accounting. Its sibling <code>a2-control-equal-priority</code> (Vanilluxe in Sinistcha's place, both arrivals priority o) is the control and is not carried by this exit code. UNMEASURED AND SAID SO: this file declares <code>ABRA-HEAP: 6144</code> for its FULL 26-arm run because each arm opens a frozen release into one process, and <code>--only</code> runs one, so the default heap is expected to hold — if it does not the verdict is EXIT CODE UNDECLARED (134), which is loud rather than false. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, unprobed)

#	item	note	
	counted Zero to Hero and Imposter, which declare nothing here and hang off a different hook. Proved live with a knob that moves — the FASTEST body on the board resolves LAST carrying Hospitality (-2) and FIRST carrying Drizzle (o), both engines agreeing on both arms — with MEDSEEN.switchInPrioritySeparated = 60 over 120 real games, so the layer is not decoration. Account: docs/_reports/2026-08-24-entry-order.md §4		
#331	WE ANNOUNCE A FAINT AT THE MOMENT OF LETHAL DAMAGE AND THE AUTHORITY BATCHES IT UNTIL THE ACTION COMPLETES. OBSERVED 2026-08-22 BY WILL READING CARDS 28 AND 34; NOT PROBED SINCE. this.add('faint', pokemon) lives inside faintMessages() (sim/battle.ts:2549), which runAction calls at :2832 after the action finishes, so Showdown completes the move's own consequences — the recharge flag, Stone Axe's hazard, Fling's item — and announces deaths afterwards. Two consequences beyond the ordering itself. (1) faintMessages(lastFirst = false, ...) : the lastFirst argument is why Final Gambit faints the USER before the target on the authority and the reverse here. (2) It also explains a wording difference for free — Pokemon#toString() returns p2a: Name while active and bare p2: Name once not, and faintMessages sets isActive = false at :2563, so the authority's -hitcount names an already-dead body (p2: Azumarill) where ours names a live one (p2a: Azumarill). One root, two visible symptoms. LIKELY WORTH MORE THAN ITS CARD COUNT: a faint announced early shifts every line after it, so games currently filed under other classes may be downstream of this one. Imposter (#320) predicted ~10 and delivered 13 for exactly that reason. WHAT WOULD DECIDE IT: stage a lethal Stone Axe and a lethal Final Gambit on both engines and assert the `	faint	line's position relat: faint. Source docs/CAF the same row: #315, a other half of the same
#332	A BERRY IS EATEN BEFORE the upkeep MARKER AND BEFORE FAINT PROCESSING HERE, AND AFTER BOTH IN THE AUTHORITY — THIS ONE CHANGES WHO SURVIVES. OBSERVED 2026-08-22 BY WILL READING CARD 30; NOT PROBED SINCE. The authority's residual case runs fieldEvent('Residual'), then add('upkeep'), then faintMessages(), and only then eachEvent('Update') — and Sitrus hooks onUpdate, so it eats after the turn-end marker AND after deaths are processed. We eat during the residual sequence. Two ways that moves the board rather than the narration. (a) A body that residual damage would kill is dead before the berry can be eaten on the authority; eating mid-sequence heals it above zero and it lives. (b) With several residuals in sequence, eating mid-sequence gives a larger HP pool to every residual that follows — body at 20, poison 15 then sandstorm 7: authority 20 -> 5 -> 0 DEAD, ours 20 -> 5 -> berry 36 -> 29 ALIVE. THE RESIDUAL ORDER ITSELF IS CHECKED AND CORRECT AND IS NOT PART OF THIS ROW: data/residual-order.json carries sandstorm at order 1 / onFieldResidual and Leftovers at order 5 / subOrder 4, derived from the authority, and the simulator reads the numbers from that file rather than hardcoding them (#221, #242). Will raised this as the KO-flipping hazard; the residual half is sound and the BERRY half is where it bites. WHAT WOULD DECIDE IT: stage a Sitrus holder at an HP the residual chain exactly kills, on both engines, and assert the body is dead on both. Source docs/CARD-REVIEW-2026-08-22.md §A4	closed 2026-09-04 — FALSE AT HEAD, including the KO-flipping half. The row's own staged assertion is a live census row verbatim — "a body the residual chain kills is DEAD before its Sitrus is reachable" — beside "a Sitrus eats in the group that dropped it — the order-8 seed, not next turn" and "the Sitrus is eaten BETWEEN the two attackers, so the second hit is survived". Closed by MEASURE 2026-09-04 as register hygiene against HEAD 73cca9f8 ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account docs/_reports/2026-09-04-register-closures.md . docs/MEASURE.md THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; observed by Will 2026-08-22, filed by MEASURE, unprobed)	
#351	MOODY RAISES AND DROPS DIFFERENT STATS FROM THE AUTHORITY IN 8 OF THE 121 WHOLE-GAME DIVERGENCES, AND THE INSTRUMENT CANNOT SAY WHETHER THE TWO ENGINES EVEN SHARE THE DIE — FILED 2026-08-22 BY MEASURE WITH NO VERDICT. Observed on data/game-differential.json, release 13ba05093aa3, 961 games, generated 2026-08-22T19:15Z: six causes in -boost field 3 and two in -unboost field 3, e.g. `	-boost\	p2a\

#	item	note

#	item	note

#	item	note
#354	WE FLINCH WHERE THE AUTHORITY MOVES — 2 OF THE 121, AND THIS ONE IS INSIDE THE ARM'S ADDRESSED SET. FILED 2026-08-22 BY MEASURE WITH NO VERDICT. Two causes (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z): ``\`	-immune\ p1b <> \
#355	TWO TAILWINDS EXPIRING ON THE SAME TURN COME OUT IN THE OPPOSITE ORDER, AND #242 SETTLED WHERE IN THE WALK RATHER THAN WHICH SIDE — 2 OF THE 121, FILED 2026-08-22 BY MEASURE. One cause with n=2 (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z): `ordering :: \`	-sideend\ p2:\`

#	item	note

#	item	note	
#357	RAGE FIST IS PERMANENTLY 50 BASE POWER HERE BECAUSE THE TIMES-HIT COUNTER DOES NOT EXIST, AND THE TAG SAYS SO ITSELF — FILED 2026-08-22 BY MEASURE. One of the 121 (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z), and it is a KO: `~damage field 3 :: \	-damage\	p2a\

#	item	note	
#358	A SUBSTITUTE BREAKS ON THE AUTHORITY AND SURVIVES HERE, AND IT WAS TRIAGED AS AN ANNOUNCEMENT SHAPE — RECLASSIFIED AND FILED 2026-08-22 BY MEASURE. Cause (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z): `extra event emitted by medicham2 :: \	-end\	p2a\
#359	POLTERGEIST NAMES THE ITEM IT IS ABOUT TO THROW AND WE SAY NOTHING — FILED 2026-08-22 BY MEASURE. Cause (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z): `event missing from medicham2 :: \	-activate\	p2b\

#	item	note	
#360	TELEPATHY ANNOUNCES AN ACTIVATION AND WE ANNOUNCE AN IMMUNITY — THE RIGHT BOARD, THE WRONG LINE TYPE. FILED 2026-08-22 BY MEASURE. Cause (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z): `unrelated event mismatch :: \	-activate\	p2a\
#361	THE MULTI-HIT COUNT DIVERGENCE REPRODUCES, AND THIS ROW WAS FILED ON A MOVE THAT CANNOT CARRY IT — HEADLINE CORRECTED IN PLACE 2026-09-04 BY MEASURE. It read <i>"THE MULTI-HIT COUNT DIVERGENCE DOES NOT REPRODUCE, AND THE ROW EXISTS SO IT IS NOT RE- DISCOVERED AS NEWS — FILED 2026-08-22 BY MEASURE WITH NO VERDICT"</i> until then, and that sentence was doing the damage: a reader triaging this register skipped the mechanism on a null result taken against a fixture that is immune to it. THE 2026-08-22 INVESTIGATION IS DATED EVIDENCE AND IS KEPT VERBATIM BELOW, NOT REWRITTEN. The wire-queue triage read one game on <code>data/game-differential.json</code> release 603d9a69d5a3 (generated 2026-08-22T06:47Z) where Showdown's Scale Shot hit 4 times and ours hit 5. On the current artifact (release 13ba05093aa3 , 961 games, generated 2026-08-22T19:15Z) scaleshot occurs twice and BOTH are coverage lists — it is not in the cause table at all. Three explanations and nothing separates them: it was fixed between the two releases; the game that carried it now stops at an earlier divergence (the differential records only the FIRST divergence per game); or the pool moved. #103 IS NOT THIS DEFECT — that one answered the MEAN 3.1 to every question and is DONE, and a constant 3.1 cannot produce a 5 — and #333 is multi-hit DAMAGE, whose fix does not touch the count. THE COUNT'S DRAW CARRIES #351'S CAVEAT: <code>sim/battle-actions.ts</code> draws a non-2-5 count as <code>random(h[0], h[1]+1)</code> , which executes in none of the three methods <code>game_differential.js</code> wraps to derive a die address, so whether the two engines share that	open — engine DEFECT, 6 board-material games of 961 on release 8ad06030e129 , cause unprobed. The cell read <code>open — OBSERVED ONCE</code> on release 603d9a69d5a3 and NOT REPRODUCED on 13ba05093aa3; unprobed, no verdict filed (MEASURE 2026-08-22) until MEASURE corrected it 2026-09-04	

#	item	note
	draw is not established by this run either. WHAT WOULD DECIDE IT: stage Scale Shot under a pinned die and assert the same number of <code>-damage</code> lines on both engines. At 1 game it does not earn an evening; the row is here so the next reader knows it was looked at. CORRECTED 2026-09-04 BY MEASURE — THE FIXTURE WAS IMMUNE, SO THE NULL RESULT WAS TRUE AND IRRELEVANT. Scale Shot is not a <code>multiaccuracy</code> move. DERIVED on <code>gen9championsvgc2026regmb</code> under the regulation filter <code>x.exists && !x.isNonstandard && x.tier !== 'Illegal'</code>, not recalled: <code>scaleshot</code> is legal, <code>multihit</code> [2,5], <code>accuracy</code> 90, <code>multiaccuracy</code> <code>false</code> — its accuracy is checked ONCE, at <code>hitStepAccuracy</code>, so it cannot show the per-arrival divergence this row was opened on, and staging it would have proved nothing whichever way it fell. EXACTLY TWO MOVES IN THIS REGULATION CARRY THE FLAG, and the row named neither: <code>populationbomb</code> (hits 10, accuracy 90) and <code>tripleaxel</code> (hits 3, accuracy 90). <code>multiaccuracy</code> repeats the accuracy check per arrival after the first, at a call site inside the hit loop rather than at <code>hitStepAccuracy</code>, so at 90 accuracy the two engines part about 10% of the time per extra arrival — one extra check for Triple Axel, nine for Population Bomb. ON THE CURRENT PINNED ARTIFACT IT REPRODUCES: release <code>8ad06030e129</code>, arm <code>middle</code>, <code>--end-state</code>, census <code>9446a684709d</code>, pool <code>0d103fb9fa87</code> — 6 board-material games of 961, games 7 / 19 / 21 / 31 / 57 / 77, <code>tripleaxel</code> <code>x3</code> and <code>populationbomb</code> <code>x3</code>. In every one Showdown's stream continues with further <code>-damage</code> lines while ours emits <code>-hitcount <slot> 1</code>, and in three of them a body the authority has killed is still standing for us: <code>p1.party.blastoise</code> 97 alive against <code>o fnt</code>, <code>p2.party.maushold</code> 27 against <code>o fnt</code>, <code>p2.party.raichu</code> 84 against <code>o fnt</code>. The same pair dominates that run's unshared-address table (<code>9 crit populationbomb [sd only]</code>, <code>9 dmg populationbomb [sd only]</code>, <code>8 acc partingshot [sd only]</code>), which is the dice desynchronisation behind most of the 8 VOID games — so this mechanism pays twice, in the board count and in the void count. THE SCALE SHOT HALF IS NOT RETRACTED: <code>scaleshot</code> still does not appear in the current cause table, and that remains a true null about Scale Shot's own 2-5 count draw and #351's caveat. What is retracted is the headline's generalisation from it to the mechanism as a class. Account: <code>docs/_reports/2026-09-04-the-77-by-mechanism.md</code> section M1. INSTRUMENT OWED: a probe that stages Triple Axel and Population Bomb under pinned dice and asserts both engines take the SAME NUMBER OF <code>acc</code> DRAWS AT THE SAME ADDRESSES — the arrival count is the symptom and the address is the claim, and a probe that asserts only the count can go green on a coincidence. Nothing in the repository stages it.	
#362	A SIMULTANEOUS DOUBLE WIPE IS A WIN ON THE AUTHORITY AND A DRAW HERE — WINNER-DECIDING, AND INVISIBLE TO EVERY PROTOCOL COMPARISON WE RUN. FILED 2026-08-23 BY MEASURE. THE AUTHORITY, READ IN THE CHECKOUT TONIGHT: <code>sim/battle.ts:2605</code> — <code>this.win(faintData && this.gen > 4 ? faintData.target.side : null)</code> inside <code>checkWin</code>'s <code>if (this.sides.every(side => !side.pokemonLeft))</code>, and <code>faintData</code> is the LAST entry <code>faintMessages()</code> shifted off the queue at <code>:2546</code>. So in Gen 5+ both sides emptying is not a draw: the side whose body fainted LAST wins — the cartridge rule that the self-destructing player loses, written as a queue position. <i>(The citation carried in was <code>:2603</code>; that is where the <code>checkWin</code> signature sits in this checkout and the <code>win()</code> call is two lines below. Stated so the next reader does not re-derive it.)</i> OURS HAS NO SUCH RULE: <code>battleResult(5)</code> in <code>engine/medicham2-browser.js</code> (<code>:24478</code> when read at 2026-08-23T01:40Z; that file is under active ENGINE edit and the number had already moved to <code>:24529</code> by 02:00 — grep the SYMBOL, never this line) compares live-body counts, falls through to total HP fraction, and returns 0.5 for o-against-o four lines below — its own header calls that <i>"dead-even HP tie at the 20-turn horizon"</i>, which a mutual wipe is not. MEASURED, NOT ARGUED: <code>tests/probe_selfdestruct_winner.js</code> board <code>w3-simultaneous</code> — one boom on turn two takes all four remaining bodies — reads showdown <code>winner="B"</code> with <code>pokemonLeft</code> <code>p1=0</code> <code>p2=0</code> against medicham2's draw. It is declared KNOWN-OPEN in that file and reported rather than swept in, and the disagreement is IDENTICAL clean and under that file's surgical revert, so it is not the self-KO faint position (#331's region) and not the arm. WHY NO INSTRUMENT WE OWN CAN SEE IT: every whole-game comparison here compares PROTOCOL LINES or BOARDS. Two engines can emit identical lines to the character and still disagree about who won, so <code>engine/game_differential.js</code> has no class for this and never will until a winner is compared. THE BLAST RADIUS IS THE LEAF, NOT THE NARRATION — <code>battleResult</code> is what the sealed rollout and the Tower end screen read, so every mutual wipe in every rollout is scored half a win, including the ones the authority scores as a loss. WILL'S RELATED CASE IS CARRIED AS HIS HYPOTHESIS AND NOT AS A FACT: perish counters resolve in SPEED order, so the slowest body faints last and its side would win. #115 landed the speed-ordered residual walk and its own status cell already notes <code>battle.ts:2604</code> awards a double KO to the last faint —	open — engine DEFECT (ENGINE; WINNER-deciding, staged and measured by <code>tests/probe_selfdestruct_winner.js</code> <code>w3-simultaneous</code>, filed by MEASURE 2026-08-23). The perish-order case is a HYPOTHESIS UNDER DERIVATION and is not a claim of this row

#	item	note
	ENGINE is deriving the perish half now, and nothing in this row asserts it. WHAT WOULD DECIDE THE FIX: a faint-ORDER field written at every faint site in the engine (battleResult cannot acquire the rule without one), then w3-simultaneous promoted from reporting to ASSERTING equality against battle.winner , with the perish-counter board as the second scenario once ENGINE's derivation lands. INSTRUMENT OWED: a faint-ORDER field written at every faint site in the engine, and w3-simultaneous promoted from reporting a KNOWN-OPEN verdict to ASSERTING that our result equals the authority's battle.winner .	
#363	THE -boost field 3 CLASS IS 9 OF THE CURRENT 96 WHOLE-GAME DIVERGENCES AND THE ARTIFACT CAN NAME A MECHANISM FOR ONLY 2 OF THEM — FILED 2026-08-23 BY MEASURE WITH NO VERDICT. Read, not re-run, off data/game-differential.json : release 59bb68aa89a9 , generated 2026-08-22T22:50:24Z, 777 games / 96 diverged / o threw, census pin 80e648f34d56 , team pool b2b61ec40281 . Nine causes, each n=1 , and every one is a magnitude-2 raise on the same slot with a different STAT — ` \	-boost\ p2a\

#	item	note
#364	THE 22 DAMAGING self MOVES FAIL THROUGH A DOOR THAT HAS NEVER BEEN MEASURED, AND THE DECLARATION LIVED ONLY IN A CHANGELOG — FILED 2026-08-23 BY MEASURE, NO VERDICT. #343 staged the ONE heal-primary Status member (a failed Roost, where we emit the <code>-fail</code> and then also the <code>-singleturn</code> and ground a body the authority leaves Flying) and that pass DECLARED the rest open rather than sweeping them in — correct, the scope of a fix is the scope that was probed. But the declaration existed only in CHANGELOG 5.74.0 and in no register row, which is the same failure as a figure that lives only in prose: nothing could print it, so nothing could act on it. MEMBERSHIP DERIVED FROM THE FORMAT TONIGHT, FILTERED TO THE REGULATION, NOT TYPED: 25 legal moves carry a self effect; 3 are Status — Baton Pass, Roost, Shed Tail — and exactly one of those is heal-primary (Roost, self.heal true); 22 are damaging. Ten carry <code>self.boosts</code> (Close Combat, Superpower, Hammer Arm, Ice Hammer, Headlong Rush, Draco Meteor, Leaf Storm, Overheat, Make It Rain, Armor Cannon), eleven <code>self.volatileStatus</code> (Hyper Beam, Giga Impact, Outrage, Thrash, Petal Dance, Raging Fury, Uproar, Blast Burn, Hydro Cannon, Frenzy Plant, Rock Wrecker), one <code>self.onHit</code> (Burn Up). THIS IS NOT A TAIL: 32,135 clicks of 424,933 in the clean corpus — 7.6% — and Close Combat alone is 18,600, with Draco Meteor 3,291, Make It Rain 2,811, Overheat 1,879 and Hyper Beam 1,708. THE DOOR IS THE ONE #343 FOUND: <code>runMoveEffects</code> writes <code>damage[i] = false</code> on a refused hit, <code>spreadMoveHit</code> sets <code>targets[i] = false</code>, and step 4 <code>selfDrops</code> skips a false target — so for each of the 22 the open question is whether our self-rider fires on a hit the AUTHORITY refused (a Protect, a type immunity, a miss, a Substitute). For the ten boost members that is a stat stage that does or does not exist; for the eleven volatile members it is whether the user is locked or recharging, which decides the whole of the following turn. NOTHING HERE CLAIMS THEY ARE WRONG. The failure path has not been measured on any of the 22, on either engine, and no <code>DEFECT</code> token is filed — an unprobed member is not a defect, and calling it one would corrupt the open-defect clause exactly as much as missing a real one. WHAT WOULD DECIDE IT: one staged pair per <code>self SHAPE</code> rather than per move — Close Combat into a Protect asserting the Defence drop on both engines or on neither; Hyper Beam into a Ghost asserting <code>mustrecharge</code> on both or on neither, which is the board #161 records WIRE 43 getting backwards once already; Burn Up into a target that refuses it for the <code>onHit</code> shape. Three probes cover 22 members, and the shapes are derived, not listed. INSTRUMENT OWED: one staged pair per <code>self SHAPE</code> — boosts, volatileStatus, onHit — asserting the self-rider fires on both engines or on neither when the authority refused the hit.	open — UNMEASURED, 22 members / 32,135 clicks (7.6% of clean clicks); declared open by #343's pass and unprobed since. No verdict filed and no DEFECT token (MEASURE 2026-08-23)
#365	THE DIFFERENTIAL MIRRORS A FORCED SWITCH BY SPECIES NAME, SO IT CANNOT EXPRESS A RENAMED BODY OR ONE ALREADY ON THE FIELD — ONE INSTRUMENT DEFECT WEARING THREE FACES. FILED 2026-08-23 BY MEASURE. <code>mirrorForcedSwitch</code> in <code>engine/game_differential.js</code> (:3743 when read at 2026-08-23T01:40Z; that file is under active ENGINE edit and had moved to :3754 by 02:00 — grep the symbol, and it is EXPORTED) turns medicham2's replacement into a Showdown choice with <code>roster.findIndex((q, n) => !claimed.has(n) && !q.isActive && !q.fainted && id(q.species.id) === id(body.name))</code>. A miss writes <code>cannot</code>, which becomes <code>mirrorImpossible</code> and then, at the call site, <code>endReason = "the boards parted — medicham2's placement cannot be expressed to showdown"</code> at :3533. The message already distinguishes the two ways to miss: "does not have under that name" — the body was RENAMED (transform, forme change) — and <i>"has but cannot switch in (fainted/active)"</i> — the body is ALREADY ON THE FIELD. Neither is expressible by a name-keyed lookup, and both are the instrument, not the engine. MEASURED ON THE CURRENT ARTIFACT (<code>data/game-differential.json</code>, release 59bb68aa89a9, 777 games, 2026-08-22T22:50:24Z):	open — INSTRUMENT DEFECT in <code>engine/game_differential.js</code>, measured (<code>forced_switch_unmirrorable</code> 11, 2 of 777 games on the primary arm, release 59bb68aa89a9); ENGINE owns the file, MEASURE owns every figure that rides it (filed 2026-08-23)

#	item	note
	declared_gaps.forced_switch_unmirrorable = 11, against 959 slots mirrored and 7 legitimately passed, first instance "p1: slot 1 holds froslass, which showdown has but cannot switch in (fainted/active)". The middle arm ends 2 of 777 games on it, top-tie-first 2 and bottom-tie-first 5 — and one of those end reasons names froslassmega, the renamed face sitting in the published record. Every game that ends this way is a game whose remaining turns were never compared. THREE STAGED SCENARIOS ARE THIS ONE MECHANISM AND WERE PROVED TO BE, NOT ASSUMED: imposter-copies-the-body-opposite, hungerswitch-flips-every-turn and roar-drag-whoever-is-standing-there in tests/staged_board.js return the identical SHORT verdict at the identical turn with the identical endReason under all four arms, with err=null, stateDiv=null and the engines agreeing on every board, byte-identical on releases 18b227eee69f and 39631097fcc7. Not the arm, not the dice, not the engine. AND NOTHING GATES THEM: tests/staged_board.js is not test-*.js so tests/run-all.js does not discover it, while seven test-*.js files require it as a library — test-assert-mode, test-forme-assert, test-middle-stall-address, test-perish-song, test-roster-arm-pin, test-switch-back-renamed, test-volatile-duration — and tests/test-forced-switch-mirror.js drives the exported helper directly. One fix, nine instruments. #346 IS CLOSED AND IS NOT THIS ROW: it fixed the fallback's lack of memory of what the other slot had just taken on a double KO. This is the KEY itself, and #346 must not be read as covering it. WHAT WOULD DECIDE IT: mirror on the roster INDEX medicham2 chose rather than on a name, then stage a transform and a forme change through a forced switch and assert the game does not stop. Sources docs/_reports/2026-08-22-wrong-arm-callers.md and docs/_reports/2026-08-22-drag-and-fail.md. INSTRUMENT OWED: a clause that fails while declared_gaps.forced_switch_unmirrorable is non-zero, plus a staged transform and a staged forme change driven through a forced switch.	
#366	tests/test-precharge-order.js PUBLISHED "83 CHECKS" MEASURED ON AN ARM IT NEVER NAMED, AND 21 OTHER CALLERS ARE IN THE SAME STATE WITH 7 ALREADY CLEARED — FILED 2026-08-23 BY MEASURE. PRIMARY_ARM on the live driver is middle (read off it, not assumed), so a caller that omits arm silently gets live dice. test-precharge-order.js was screened under three arms and it MOVES: under bottom-tie-first four of its arms produce MORE protocol — 11→12, 14→16, 12→13, 10→11 lines — because secondaries fire and crits land that the middle arm did not draw. ITS VERDICT DOES NOT FLIP TODAY, because both engines share the arm and agree either way; what is weaker is the CLAIM. "The two protocol streams do not part" means less on a board where fewer things happened, and docs/ENGINE.md:1596 and :1650 publish "five arms, 83 checks" measured exactly there. THE POPULATION IS 22, NOT THE 27 FIRST REPORTED, derived by parsing each playGame call site's balanced argument list instead of grepping for the string arm: : three files pass the arm as an ES6 SHORTHAND ({ script, arm } , no colon) and were accused wrongly, and two never call the driver's playGame at all — replayGame contains it as a substring. AND IT IS BIGGER IN THE OTHER DIRECTION: 4 more reach it indirectly through tests/staged_board.js 's exported runOne(sc, patchedSrc), which takes no arm parameter at all, and 7 more are the driver's OWN self-proofs — planted_divergence_proof, directed and knock_off_roadmap_80 are published blocks of data/game-differential.json riding them, and a plant that has to beat live dice is a weaker red than one on a corner. 14 of the 22 publish a figure; 8 were screened and 7 are byte-identical across middle, top-tie-first and bottom-tie-first — INERT WITH EVIDENCE, which is the noise floor the one positive rests on. WILL'S RULE GOVERNS THE REPAIR — fix one at a time, re-measure, withdraw only what MOVES. Seven figures have already been CLEARED rather than withdrawn on suspicion, which is the point of screening before retracting. A CORRECTION IS OWED TO docs/ENGINE.md, which publishes 27 in two places; ENGINE owns that ledger. THIS ROW DOES NOT WITHDRAW THE 83. It has not been re-measured on a named arm, so what is established is that the instrument is arm-dependent — not what the number becomes. Source docs/_reports/2026-08-22-wrong-arm-callers.md. INSTRUMENT OWED: a check that every playGame call site names an arm, so an unpinned caller fails BY NAME instead of being audited by hand once a quarter.	open — MEASUREMENT PROVENANCE (MEASURE); 1 of 22 measured arm-dependent, 7 screened INERT, 14 unscreened, plus 4 indirect through SB.runOne and 7 inside the driver. The published "83 checks" is a WITHDRAWAL CANDIDATE and is NOT withdrawn (filed 2026-08-23)
#367	THE 5.74.0 RETRACTION'S MECHANISM IS NOT SUPPORTED BY EITHER ARTIFACT: THE TEAM-STORE FIELD HAS EXISTED SINCE 2026-08-07 AND BOTH RUNS DECLARE THE SAME FROZEN POOL. FILED 2026-08-23 BY MEASURE, WITH NO VERDICT ON THE REMAINDER. CHANGELOG 5.74.0 revised its divergence-rate-over-usable from 9.7% to 11.69% and gave two reasons: that --team-store data\team-pool-frozen "did not take", and that " data/game-	\ null in engine/game_diff the file is under acti unconditionally in the LOUD EITHER WAY ... an u whole problem"*, and ; 16:34** — fifteen days **AND THE FIRST IS CON rather than re-run: th

#	item	note
	differential.json RECORDS NO TEAM-STORE FIELD AT ALL". THE SECOND IS FALSE. `STEER_STAMP.team_store_pinned_to = TEAM_STORE \	22T21:17:11Z, **961 ga steering.team_store_pinned steering.team_pool_teams = **777 / 96**) carries TEAM_STORE is the sc data/team-pool-frozen/ is **109,006,606 bytes ag pool cannot yield the frozen pool. What actu against **1,597**) and b2b61ec40281), and bui i.e. **the two runs RE REQUESTED COUNT IS REC ARTIFACT STORES IT.** block in either file – But <i>buildSwarm</i> comput both runs carry **9 co **seven of the nine si 213 for HEAD . Invert the earlier run and ** runs asked for materia roughly 960**, and 120 intervals. coverage_sto field is *"null on a f and neither was --until- CONFIG RATHER THAN TO matching.length && picked.l protect at **99/99** an Identical starvation i same pool. So neither requested against 961 (80.9%). **The shortfa pool, present in both live store.** **WHAT R integer was passed, an Nothing in the record to find out. **NEITHER BE ENTERED IN A RETRAC on 777 games and 9.7% samples, not a pinned is the CAUSE that was REAL DEFECT IS THAT NO team_store_pinned_to any engine/replay_one.js:240 , one; no gate, no prove published claim's asse or its team_pool_digest receipt** – which is t and it is why a wrong GENERAL row (a corpus artifact that reads a case where the id alre IT**: a provenance cla pool the artifact does gap at a single reques clause that fails when own steering.team_store_p
#368	tests/test-state-differential.js WAS RED AND THEN GREEN ON THE SAME TREE IN ONE EVENING — OBSERVED 2026-08-22, CAUSE UNKNOWN, NO MECHANISM CLAIMED. Two independent passes, hours apart, same evening. The first (docs/_reports/2026-08-22-wrong-arm-callers.md) recorded exit 1 on all three arms — injected 86, 3 / 3 failures, exit 1/1/1 , arm-independent but for a pool-cache line — and printed the three clauses: <i>"the proof ran 1 plants and 42 are declared — CANNOT PLANT — the clean game's very first board already differs, so there is no agreeing board to plant into"; "no switch-in ever reordered Showdown's side.pokemon in this sample, so PART 3 tested nothing"; "no game in this sample held weather across two boundaries, so PART 5 tested nothing"</i>. The second (docs/_reports/2026-08-22-runner-blind-spot.md) recorded it GREEN, exit 0, in those words: <i>"Same tree, same evening, different answer."</i> NO MECHANISM IS CLAIMED AND NONE IS INVENTED HERE. One candidate is on the record and is explicitly NOT this row's verdict: the first report observed of a neighbouring gate that its message <i>"varies between runs because the team pool is drawn live from the store"</i>, which OPS appends to — and two of the three clauses above are fixture-REACHABILITY complaints that a moved pool could plausibly flip. The third is not a fixture complaint at all (<i>the clean game's very first board already differs</i>), so a single explanation is not even available. Untested either way. WHY IT IS FILED RATHER THAN FIXED TONIGHT: an ENGINE agent was live in engine/medicham2-browser.js across both passes and tests/run-all.js is being edited now, so any verdict taken now describes a moving tree. Per CLAUDE.md a red gate is fixed in the session that saw it or waived by Will by name — which makes <i>"which of the two answers is the gate's"</i> the thing that has to be settled first, and it	open — OBSERVED, red → green on one tree in one evening; cause UNKNOWN, no mechanism claimed, no verdict filed (MEASURE 2026-08-23)

#	item	note
	is not a status. A GATE THAT ANSWERS TWICE IS WORSE THAN ONE THAT ANSWERS RED, because both answers are quotable and neither can be checked. WHAT WOULD DECIDE IT: three runs on a settled tree at one pinned release, <code>--team-store data/team-pool-frozen</code> and a pinned census, with all three verdicts recorded. If they still differ, the subject is the gate's own fixture search and not the engine. INSTRUMENT OWED: three runs of this gate on a settled tree at one pinned release, one pinned census and <code>--team-store data/team-pool-frozen</code>, with all three verdicts recorded.	
#369	<code>engine/register_reality.js --list</code> SAYS IT RUNS NOTHING AND IT STILL OVERWRITES THE VERDICT ARTIFACT, DESTROYING EVERY VERDICT IN IT — TRIPPED AND SELF-REPORTED 2026-08-23 BY MEASURE. The file's own header advertises <code>node engine/register_reality.js --list # coverage only; runs nothing</code>, and that is TRUE of the INSTRUMENTS — every row printed <code>--list: not run</code> and no gate executed. It is not true of the ARTIFACT. The run ends <code>wrote data/register-reality.json</code>, and it replaced the settled 2026-08-22T01:55:12.569Z artifact — marked: 31 rows over <code>distinct_commands_run: 22</code>, 0 stale rows, 2 PREMATURE CLOSE (<code>tests/probe_red_demo.js</code>, <code>engine/gate_offfield_target.js</code>) and 1 INSTRUMENT UNRUNNABLE — with one carrying no verdicts at all: 271 insertions and 142 deletions against HEAD. RECOVERED ONLY BECAUSE THE FILE IS TRACKED. <code>git checkout -- data/register-reality.json</code> restored it byte for byte and the working tree is clean of it; nothing was lost. Had it been untracked it would have been unrecoverable, which is this repository's standing rule about deletion arriving through a door labelled read-only. THE BLAST RADIUS IS A GATE, NOT A FILE. <code>openDefectClause</code> in <code>engine/quarantine.js</code> reads this artifact to decide which open register rows are backed by a RED instrument; every row it cannot find a verdict for falls into <code>debt</code> and can no longer hold the gate shut. An artifact wiped to all- NOT RUN therefore makes the clause print <i>"no open row names an instrument that is RED"</i> for exactly the reason that should have made it loudest — which is the same shape as the three-key mismatch (<code>rows / verdicts</code> against <code>results</code>, <code>command</code> against <code>cmd</code>, <code>exit</code> against <code>green</code>) that this very clause records having survived unnoticed until 2026-08-18. A wipe is quieter than that was, because the key names still match. FILED BY THE AGENT THAT CAUSED IT, on a read of the header that was accurate and incomplete. WHAT WOULD DECIDE THE FIX: <code>--list</code> and <code>--json</code> write nothing, or write to a separate path, and the write site is guarded rather than the mode documented. INSTRUMENT OWED: a self-test asserting that a coverage-only invocation leaves <code>data/register-reality.json</code> byte-identical, in the style of the <code>--selftest</code> already in that file	open — INSTRUMENT DEFECT in <code>engine/register_reality.js</code> (MEASURE owns the file); tripped, self-reported and the artifact restored from HEAD the same minute — no verdicts lost (filed 2026-08-23)
#370	THE DERIVED RETRACTION REGISTRY MATCHES ON A ROUNDED VALUE, SO ONE RETRACTED FIGURE ACCUSED 52 DOCUMENT LINES THAT SAY A DIFFERENT NUMBER — MEASURED 2026-08-23 BY MEASURE, AND IT IS LIVE AT BASELINE TODAY. <code>retractionViolations</code> in <code>engine/docs_scan.js</code> accepts a hit when <code>Number(e.value.toFixed(f.dp)) === f.value</code>. That tolerance was built for a real case — a document writing 63% for a retracted 63.2% IS restating it — and it has no bound, so <code>(9.7).toFixed(0)</code> is "10" and a retracted 9.7% matches every bare 10% in the repository. MEASURED: when one register row caused 9.7% to enter the registry, that single entry produced 56 violations, of which 52 were a bare 10% — a 93% false-positive rate on one entry, across <code>docs/ENGINE.md</code>, <code>docs/GLOSSARY.md</code>, <code>docs/LITERATURE-v2.md</code>, <code>docs/MILTANK.md</code>, <code>docs/ABRA-whitepaper.md</code> and <code>docs/CARD-REVIEW-2026-08-22.md</code>, hitting <i>"Flare Blitz's 10% burn"</i>, <i>"top 10% in singles"</i>, <i>"the 0-10% or 90-100% bin"</i> and <i>"~10% of its useful size"</i>. THIS IS NOT ABOUT THAT ONE FIGURE AND IT IS NOT NEW — IT IS FIRING AT BASELINE RIGHT NOW. With that entry removed the clause sits exactly at its baseline of 10 keys, and 12 of the 17 surviving raw violations are rounding collisions rather than exact matches: <code>docs/BACKLOG.md:215</code> <i>"we produce the single most common set 48% of the time"</i> and <code>docs/archive/SESSION-2026-08-02.md:113</code> <i>"Sucker Punch's 48% failure prices the missing opponent model"</i> are both accused of restating the retracted mirror figure 47.5%, and neither is about it. A SECOND, INDEPENDENT HALF: isDistinctive ADMITS ANY PERCENT. It is <code>`f => f.pct \</code>	<code>f.value >= 1000</code>, so a <code>tw</code> <code>docs/ENGINE.md</code> <code>carrie</code>

#	item	note	
#371	SHOWDOWN REFUSES A MOVE AND WE EXECUTE IT — 8 OF THE 47 NON- event-missing DIVERGENCES, EVERY MEMBER BOARD-MATERIAL, THREE SUB-CAUSES ON ONE SIGNATURE. FILED 2026-08-23 BY MEASURE. Read, never re-run, off git show HEAD:data/game-differential.json (commit 192d917): release c66976713feb , showdown 20ad99ffc9a5 , census pin 80e648f34d56 , team store pinned to data/team-pool-frozen (8,778 teams, 1,968 picked), 961 games / 95 diverged / o threw , generated 2026-08-23To2:40:37Z. Triage: docs/_reports/2026-08-23-remaining-56.md (ea26208). THE SIGNATURE: the authority writes `	-fail\	SLOT and we write the authority's \

#	item	note	
#372	THE TARGET EATS ITS BERRY BEFORE THE ATTACKER'S OWN LIFE ORB OR RECOIL CHIP ON THE AUTHORITY AND AFTER IT HERE — 2 OF THE 47, AND THE HIGHEST REAL CORPUS USAGE OF ANY GROUP IN SCOPE. FILED 2026-08-23 BY MEASURE WITH NO VERDICT. Read off <code>git show HEAD:data/game-differential.json</code> (commit 192d917), release <code>c66976713feb</code> , 961 games / 95 diverged / 0 threw , generated 2026-08-23T02:40:37Z; triage docs/_reports/2026-08-23-remaining-56.md (<code>ea26208</code>). Two causes: <code>`ordering::\`</code>	-enditem\	p2a\
#373	THE TWO SIDES' detailschange LINES COME OUT IN THE OPPOSITE ORDER WHILE THE MEGA COUNTS AGREE EXACTLY — 2 OF THE 47. FILED 2026-08-23 BY MEASURE WITH NO VERDICT. Release <code>c66976713feb</code> , 961 games, generated 2026-08-23T02:40:37Z (triage <code>ea26208</code>). Two causes: <code>`ordering::\`</code>	detailschange\	p1a\
#374	WE EMIT A SEPARATE -activate WHERE THE AUTHORITY ATTRIBUTES THE SOURCE WITH [from] ON THE EFFECT LINE — THE TWO MEMBERS OF THE TRIAGE'S GROUP K THAT #360 DOES NOT COVER. FILED 2026-08-23 BY MEASURE WITH NO VERDICT. Release <code>c66976713feb</code> , 961 games, generated 2026-08-23T02:40:37Z (triage <code>ea26208</code>). Two causes: <code>`unrelated event mismatch::\`</code>	-start\	p2a\

#	item	note	
#375	first divergences SAMPLES 60 OF 95 GAMES AND CARRIES 22 OF THE 47 CAUSES, SO A QUARTER OF THE DIVERGENCE TABLE IS UNATTRIBUTABLE BY CONSTRUCTION — MEASURED 2026-08-23 BY MEASURE, AND IT IS WILL'S OWN CARD-REVIEW ACTION ITEM. Will: <i>"the cost of a card is a few hundred bytes; the cost of a missed root cause is an evening."</i> On release c66976713feb (961 games / 95 diverged / o threw, generated 2026-08-23T02:40:37Z, read from git show HEAD:data/game-differential.json at 192d917), 25 of the 47 non- event-missing causes carry nothing but a normalised cause string — mentions: [], max_uses: undefined , no species, no preceding lines, no sdAfter / meAfter . Two families die on it outright: the four unattributed refusals of #371(c) and five of the six -boost rows of #363, with several ordering rows behind them. THIS IS NOT #349. That row is the RANKING naming the line we emitted rather than the one that differs; this is the SAMPLE CAP — a second door onto the same wall, fixed in a different place. THREE ASKS, EVERY ONE ALREADY IMPLIED BY AN OPEN ROW: (1) record showdown_before , sdAfter and meAfter for EVERY divergence, or at minimum one entry per distinct CAUSE; (2) publish the stripped [from] / [of] attribution and the body's SPECIES beside each cause, so the normaliser can stay lossy for the comparison and lossless for triage (#363's INSTRUMENT OWED, arriving here through a second door); (3) publish MEDFAILS.entryOrderTie and MEDFAILS.replaceOrderTie in the artifact so a declared switch-order tie can be told from an ordering defect (#353's ask, filed 2026-08-22 and still unmet). AN AGENT WAS PART-WAY THROUGH RAISING THIS AND WAS STOPPED, so whoever takes it should look for partial work in the tree before starting. WHAT WOULD DECIDE IT: re-run the differential on the same three pins with the fuller dump and count how many of the 47 acquire a mechanism — that number is the measurement, and until it exists no count of unattributed divergences in this repo means anything. INSTRUMENT OWED: per-cause context in data/game-differential.json rather than a 60-game sample.	open — instrument DEFECT (MEASURE owns it; MEASURED on release c66976713feb — 60 of 95 games sampled, 22 of the 47 causes carried, 25 carrying nothing but a normalised string, 2026-08-23)	
#376	THREE PROTECT/DETECT ORDERINGS ARE EXACT SPEED TIES, AND A TIE HAS NO CORRECT ANSWER — THIS NEEDS A SCORING DECISION FROM WILL RATHER THAN A FIX. FILED 2026-08-23 BY MEASURE. Release c66976713feb , 961 games, generated 2026-08-23T02:40:37Z (triage ea26208). Three causes: `ordering :: \	move\	p1b\

#	item	note
#377	AN EMPTY BENCH WAS CONSULTED BELOW THE ABILITY INSTEAD OF ABOVE IT, SO A PHAZE INTO NO BENCH ANNOUNCED THE WRONG THING AND LEFT THE WRONG MOVE RESULT BEHIND — LANDED 2026-08-23 BY ENGINE, REGISTERED 2026-08-23 BY MEASURE. The authority asks <code>this.battle.canSwitch(target.side)</code> as a CONJUNCT above <code>runEvent('DragOut') (sim/battle-actions.ts:1353)</code>, so an empty back skips the whole body: no DragOut event, so an <code>onDragOut</code> ability is never consulted and no <code>-activate</code> is written. This engine asked the ability first, in BOTH doors, ~5,900 lines apart in one file. Three symptoms, one ordering: an empty-bench Roar wrote no <code>-fail</code> and no <code>[still]</code> where the authority writes both; Suction Cups announced itself at an empty bench on both doors where the authority is silent; and after a LEGAL Suction Cups refusal the mover's own result	open — MEASUREMENT OWED (MEASURE); the engine change landed 2026-08-23 by ENGINE and ENGINE proposes the closing text

#	item	note
	was false here and true on the authority. THE THIRD IS STATE, NOT NARRATION, AND IT IS WHY THIS COULD NOT BE CALLED COSMETIC: on that arm the board agreed and the protocol was byte-identical, and moveThisTurnResult still disagreed in the OPPOSITE direction from the other arm, which decides Stomping Tantrum's base power the following turn. One reader, canDragIn(bench) , quoted at sim/battle.ts:1563 / :1571 , now answers both branches. Shown RED on demand under TWO knobs (MEDI_DRAG_ABILITY_FIRST=1 restores the ordering, MEDI_DRAG_REFUSAL_FAILS=1 restores the state write), and the bench knob reads 2 / 1 / 0 in the authority's own possibleSwitches across the arms so an unwired fixture would have been visible. WHAT IS OWED IS THE MEASUREMENT, NOT THE FIX. No census regeneration, no whole-game differential and no roster run happened in the landing pass, so the mechanics count did not move and no movement may be claimed. Expectation stated before any run, per the pinned-pool rule: this is a LAB mechanic (an empty bench needs three of a side's four bodies dead with two standing, and Malamar is the format's only legal Suction Cups carrier), so the lab should move and the pinned pool is expected to sit still. Whirlwind and Circle Throw were not staged — same tag, same branches, an argument rather than a measurement. Accounts: docs/_reports/2026-08-23-phaze-empty-bench.md , docs/_reports/2026-08-23-announce-failure-class.md . INSTRUMENT OWED: tests/probe_announce_failure.js needs SHOWDOWN_PATH and -r ./tests/_live_release.js , so no runnable VERIFIED BY can be written for it until #381 is answered; until then the census and the whole-game differential are what decide this row	
#378	A TARGET THE AUTHORITY HAD ALREADY REMOVED FROM ITS ARRAY WAS STILL DRAGGED BY THE DAMAGING PHAZE, SO A DIFFERENT BODY STOOD IN THE SLOT FOR THE REST OF THE GAME — LANDED 2026-08-23 BY ENGINE, REGISTERED 2026-08-23 BY MEASURE. BOARD-MATERIAL. The authority zeroes its targets array all the way down (sim/battle-actions.ts:1063 a Substitute-eaten hit becomes null , :1080 a damage[i] === false becomes false , :1353 the forceSwitch walk skips both); this engine's damaging forcesSwitch loop walked targets with only the Protect filter applied. Measured on the staged board BEFORE a byte moved: Dragon Tail into a pure-Fairy Clefable — both engines write -immune , and this one then wrote `\\	drag\
		p2a: Weavile . From that different HP, with dif refused body had lost happened** (clefable.bo authority). **THE FIX IS SAID RATHER THAN GL step driver itself wal SIX sites — type immun absorbing ability, the seventh refusal added stages the type-immuni MEDSEEN.forcedSwitchTar, and a guard nothing re GREEN with the control target. **WHAT IS OWED damaging phaze is unco is that the lab moves is asserted. Account:
#379	A GATE THAT COULD NOT ANSWER WAS PUBLISHED AS A GATE THAT FOUND A DEFECT — FOUND AND FIXED 2026-08-23 BY MEASURE. engine/register_reality.js 's runUncached() read ANY non-zero exit as green: false , and in this file green: false is a MEASURED RED: verdict() turns it into CONFIRMED — "the row and the world agree" — on an open row, and into PREMATURE CLOSE, an accusation against whoever closed it, on a closed one; openDefectClause in engine/quarantine.js turns it into withRed , which is what holds the MEDICHAM gate clause shut. MEASURED ON THE PRE-FIX BYTES, engine/gate_offfield_target.js (#224) AND engine/gate_fail_and_silent.js (#241) BOTH EXIT 2, both printing CANNOT ANSWER — #224 because every artifact it reads was measured against other bytes, #241 because its artifact ran on release c66976713feb against a tree of 33d871e6db92 and its sample had moved as well. Neither gate had a finding. Both were being published as red evidence of a live engine defect, which is a gate saying something untrue in the direction that gets gates ignored (#148), arriving through the RULER rather than through the engine. THE FIX IS A CLASS, NOT status === 2 . Exit 0 is GREEN and exit 1 is RED — the universal conventions — and any other non-zero code is not a verdict unless the instrument declares it on the way out in one line-anchored machine-readable line (ABRA-EXIT 2 CANNOT-ANSWER , ABRA-EXIT 3 VERDICT-RED), uppercase and hyphenated for the same reason the register marker token itself is: gate_offfield_target.js prints the legend [0 clean, 1 the placeholder is back, 2 cannot answer] on EVERY run including its green ones, so a tolerant search for those words would have called every clean run a refusal. CLASS-SAFETY IS DEMONSTRATED RATHER THAN CLAIMED: gate_offfield_target.js WAS NOT EDITED and stops being published as red anyway, through the undeclared-code rule. What it cannot catch is a refusal spelled as exit 1 with no declaration — irreducible, stated in the classifier's own header, and counted as exit_codes_undeclared so the coverage is driven down rather than assumed zero (it reads 0 on the 2026-08-23 run). verdict() 's null case now splits three ways (INSTRUMENT UNRUNNABLE / INSTRUMENT CANNOT ANSWER / EXIT CODE UNDECLARED), all three in BAD so the tool still exits 1. SHOWN RED FIRST: RR_CANNOT_ANSWER_AS_RED=1 restores the pre-fix behaviour exactly	closed 2026-08-23 — MEASURE

#	item	note
	and fails 11 assertions by name; clean it is 51/51. CONSEQUENCE STATED PLAINLY: #241 leaves withRed , so a refusal no longer holds the gate shut — the clause did not open, because #258 is OPEN and its instrument measured exit 1 the same day. Account: docs/_reports/2026-08-23-cannot-answer-vs-red.md . VERIFIED BY: node engine/register_reality.js --selftest	
#380	"WHAT DOES THIS EXIT CODE MEAN" IS DECIDED IN TWO FILES, AND THE OPEN-DEFECT CLAUSE NOW SAYS SOMETHING FALSE ABOUT THE ROWS IT BUCKETS — MEASURED 2026-08-23 BY MEASURE, NOT FIXED. tests/run-all.js has always treated exit 2 as <i>"I COULD NOT RUN, NOT I FAILED"</i> for every script it runs, generically and in those words; engine/register_reality.js treated the same code as a red verdict until 2026-08-23 (#379). Two files deciding one fact, disagreeing, both continuing to work — and they still each own a copy. THREE THINGS ARE OWED AND NONE WAS DONE, EACH FOR A STATED REASON. (1) openDefectClause prints <i>"open row(s) name an instrument that WOULD NOT RUN"</i> over a bucket that now also holds instruments which RAN and declared CANNOT-ANSWER: the bucket is right and the sentence is false. (2) An OPEN row whose instrument REFUSED probably ought to make the clause ok: false, cannot_answer: true , exactly as orderProbeClause already does in the same file on the same reasoning — a clause that cannot be computed FAILS — so the gate stays shut for the honest reason rather than on a fictitious red; as it stands such a row moves to unrunnable and stops holding the clause at all. (3) The classifier should be ONE implementation both callers use. WHY NOT NOW: (1) and (2) are edits to engine/quarantine.js , which the pass that found this was forbidden to RUN, and an unverified edit to the file that computes the MEDICHAM gate is worse than the wording; (3) needs either an edit to tests/run-all.js (ENGINE was live in tests/) or making engine/register_reality.js requirable, which today would WRITE its artifact on require. Note for whoever takes (3): run-all.js FAILs on exit 3 and gate_fail_and_silent.js uses exit 3 for REGRESSION — that stays red only because the gate now declares ABRA-EXIT 3 VERDICT-RED . Also unreached and left alone: register_reality.js 's if (row.closed && !green) is a real !green collapse, currently shadowed by the green === null branch above it. Account: docs/_reports/2026-08-23-cannot-answer-vs-red.md	open — instrument DEFECT (MEASURE)
#381	THE REGISTER'S EVIDENCE BASE, MEASURED: FIVE ROWS CARRY A VERIFIED BY NO RUNNER CAN LAUNCH, AND THIRTY-FIVE OPEN ROWS ASSERT BREAKAGE WITH NOTHING AT ALL DECIDING THEM — MEASURED 2026-08-23 BY MEASURE. Derived over docs/ROADMAP.md with quarantine.roadmapRowIsClosed and roadmapRowSaysBroken , the one closed-detector, on the register as it stood after this pass's own edits. (1) FIVE MARKERS ARE INSTRUCTIONS TO A HUMAN WEARING AN INSTRUMENT'S CLOTHES. engine/register_reality.js 's SAFE pattern accepts only `node <engine	tests build>/<file>.js [--flags] , <executed. #316, #318, SHOWDOWN_PATH=... no a literal <id> ; every neither agreement nor BREAKAGE** (#318, #319 process has checked si a patch: either the ru the environment it alr in the chain) and to e rows carry INSTRUMEN as a verdict. **The fi relaxing SAFE into a exists to prevent. **(THE OPEN-DEFECT CLAUSE INSTRUMENT OWED .** marked and 8 declare a project. That is not a statement that nothing in total out of 295 re been checked against t OWED: a runner that ca SHOWDOWN_PATH` ,and breakage rows — printed on than rediscovered
#382	engine/move_result_state.js COMPARES THE ONE PIECE OF STATE THAT DECIDED TWO SILENT DEFECTS, AND IT IS READ BY ONE PROBE AND BY NOTHING AT SCALE — BUILT 2026-08-23 BY ENGINE, REGISTERED BY MEASURE. It reads moveThisTurnResult / moveLastTurnResult off the authority and _mvRes / _mvResLast off medicham2 at the same turn boundary engine/board_state.js already uses, keeps the four values apart (true / false / null / undefined) rather than collapsing them — three of the field's four consumers test === false but data/abilities.ts:5176 tests !== undefined , so a null/undefined collapse would silence exactly that one — and encodes anything outside the four as UNEXPECTED(...) . --selftest is 18 clauses driven through the shipping functions. ON ITS FIRST RUN IT CAUGHT TWO STATE DEFECTS NOTHING IN THIS REPOSITORY COULD SEE, on arms whose boards were identical and one of whose protocol was byte-identical, both deciding Stomping Tantrum's base power the following turn (#377). IT IS NOT WIRED INTO engine/board_state.js , deliberately: promoting it to a compared board field changes what every whole-game run counts, and the pass that built it could not size that. THE COST OF LEAVING IT IS THE ABSENCE IT WAS BUILT TO END, ONE LEVEL UP —	open — INSTRUMENT OWED (MEASURE decides the promotion; ENGINE built the comparator)

#	item	note
	every whole-game divergence triaged as narration-only today is triaged without this field being compared, and the two defects above are the receipt that a byte-identical protocol proves nothing about it. INSTRUMENT OWED: either the field promoted into <code>engine/board_state.js</code> , or a declared <code>NOT_COMPARED</code> entry there naming this row as its reason, plus a re-baselined whole-game count measured under the promotion rather than compared across it. Account: <code>docs/_reports/2026-08-23-announce-failure-class.md §3</code>	
#383	ABOUT SEVENTY OF THIS ENGINE'S ~84 mvFail SITES WRITE A BARE -fail WHERE THE AUTHORITY NEVER WRITES ONE, AND THE WHOLE-GAME DIFFERENTIAL CANNOT COUNT IT BY CONSTRUCTION — MEASURED 2026-08-23 BY ENGINE, NO VERDICT, FILED BY MEASURE SO IT IS NOT RE-DISCOVERED. Derived rather than noticed: <code>grep "'-fail'" sim/battle-actions.ts</code> returns exactly twelve sites and every one pairs the -fail with an attrLastMove — [still] in ten, <code>[notarget]</code> in two — so there is no site anywhere in the authority that writes a bare <code>-fail</code> with the <code>`\`</code>	move\ line left intact. Here with no <code>attrStill</code> , and a (11 do). **IT IS INVIS RIGHT**: <code>engine/game_d(f[i] === 'move' ? f.slice(0, 4</code> it, including <code>[still]. 1</code> (Showdown names one no user) and it is not a that this asymmetry ha **IT IS NOT CLAIMED HA reason it is written d from outside and an ev tests/probe_fail_and_silent its field-repeat famil comparison that keeps MEASURE decision about change. Account: <code>docs/</code>
#384	THE vol.focusenergy LEAF PARTS FROM THE AUTHORITY ON FOUR MOVES AT ONCE AND IN BOTH DIRECTIONS, AND IT IS ONE FACT ABOUT THE VOLATILE RATHER THAN FOUR MOVE BUGS — MEASURED 2026-08-23 BY MEASURE FROM THE ROSTER, FILED FOR ENGINE. <code>data/roster.moves.json</code> , release <code>c36782953dee</code> , generated 2026-08-23T06:06:12Z, four of the stage's five FIRED-AND-BOARDS-DIFFER rows and the ONLY leaf that parts in any of them: <code>dragoncheer</code> (p2b torterra, showdown 0 ours 1, turns 1/2/3), <code>psychup</code> (pia goodrahsui, showdown 0 ours 1, turn 2), <code>fakeout</code> (p2a sableye, showdown 1 ours 0, turn 2), <code>transform</code> (pia goodrahsui, showdown 1 ours 0, turn 2). BOTH DIRECTIONS APPEAR, WHICH IS WHY THIS IS NOT "WE FORGOT TO SET IT". READ, from the running checkout rather than recalled: <code>data/moves.ts:5984</code> — Focus Energy's <code>onStart</code> opens <code>if (target.volatiles['dragoncheer']) return false;</code> — and <code>data/moves.ts:4069</code> — Dragon Cheer's opens <code>if (target.volatiles['focusenergy']) return false;</code> The two volatiles are MUTUALLY EXCLUSIVE on the authority. Both conditions then carry an identical silent arm for exactly four effect ids — <code>['costar', 'imposter', 'psychup', 'transform'] (:5987 and :4070)</code> — and two of the four roster rows are <code>psychup</code> and <code>transform</code> , i.e. the copy path, and a third is the exclusion partner. CHAMPIONS OVERRIDES ONE OF THEM AND ONLY COSMETICALLY: <code>data/mods/champions/moves.ts:241</code> is <code>dragoncheer: { inherit: true, flags: {..., sound: 1} }</code> — the condition is inherited — and there is no <code>focusenergy</code> key in the mod at all. THE FOURTH ROW, fakeout , IS NOT EXPLAINED BY THAT AND IS NOT ATTRIBUTED HERE: the roster's control click IS Focus Energy (#316), so a Fake Out arm losing the volatile on turn 2 may be the control, not the move, and saying which needs a staged pair. WHAT WOULD DECIDE IT: the exclusion staged both ways round (Focus Energy into a Dragon Cheer body and the reverse, each asserting the second application is refused on both engines), plus a Psych Up and a Transform onto a body holding each volatile, with the <code>[silent]</code> arm asserted as a line and the volatile asserted as a leaf — those are two different claims and the roster only reads the second	closed 2026-09-04 — FALSE AT HEAD. Live census row "Focus Energy and Dragon Cheer REFUSE each other - the second one to arrive does not land", and membership is derived off <code>exclusiveWith</code> rather than a hand-listed family, with <code>MEDI_CRIT_VOLATILE_BLIND</code> restoring the unwired state. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; measured and filed by MEASURE 2026-08-23, ENGINE owns the fix and the closing text)
#385	BIG ROOT'S DRAIN MULTIPLIER ROUNDS DIFFERENTLY FROM THE AUTHORITY AND THE ERROR COMPOUNDS ACROSS TURNS — MEASURED 2026-08-23 BY MEASURE FROM THE ROSTER, FILED FOR ENGINE. <code>data/roster.items.json</code> , release <code>c36782953dee</code> , generated 2026-08-23T06:05:47Z: <code>bigroot</code> is one of the item stage's two FIRED-AND-BOARDS-DIFFER rows, and every parted leaf is the healer's HP — <code>kangaskhan party.hp</code> showdown 693 ours 692 at turn 2, then 714 against 712 at turn 3. THE GAP GROWS, WHICH IS THE INTERESTING HALF: one HP, then two, so it is a per-tick rounding rule rather than a one-off. READ: <code>data/items.ts:488-494</code> — the item is <code>onTryHealPriority: 1</code> with <code>onTryHeal(damage, target, source, effect)</code> returning <code>this.chainModify([5324, 4096])</code> , i.e. the authority applies 5324/4096 through its CHAINED-MODIFIER rounding rather than a plain multiply-and-floor. <code>data/mods/champions/items.ts</code> carries no <code>bigroot</code> key, so Champions does not override it. AND THE EFFECT LIST IS FIVE, NOT ONE — <code>['drain', 'leechseed', 'ingrain', 'aquaring', 'strengthsap']</code> at <code>:490</code> — so a fix aimed at drain alone closes one of five doors. ADJACENT BUT NOT THE SAME AS #339, which is the SPREAD drain healing once over a summed total where the authority rounds per target; this is the MULTIPLIER's own rounding and the two can be wrong independently.	closed 2026-09-04 — FALSE AT HEAD. <code>data/tags.json</code> <code>items.bigroot.params.healMultBySource.mult</code> is <code>1.2998046875</code> , which is 5324/4096 in Showdown fixed point over all five drain sources; the live census row is "Big Root multiplies the ALREADY-ROUNDED drain, in Showdown fixed point, not the raw fraction"; and <code>data/roster.items.json</code> reports <code>bigroot</code> FIRED-AND-BOARDS-MATCH. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (ENGINE; measured and filed by MEASURE 2026-08-23, ENGINE owns the fix and the closing text)

#	item	note
	WHAT WOULD DECIDE IT: one staged Big Root drain whose pre-multiplier heal is chosen so that 5324/4096 and 1.3 round apart, asserted to the point against the authority, with a no-Big-Root control that must agree before and after; and the same body run across the other four heal sources so the class is not closed on one of them	
#386	THE ONE TOOL BUILT TO PRINT WHAT THE REGISTER CANNOT SEE ABOUT ITSELF IS FED BY A SINGLE ARTIFACT THAT IS 11.4 DAYS OLD, SO IT PRINTS UNREGISTERED: 0 AND THAT ZERO CARRIES NO INFORMATION — MEASURED 2026-08-23 BY MEASURE. engine/open_work.js's whole justification is that a register cannot audit itself, and its UNREGISTERED half is the stronger of its two signals because it is measured rather than filed. Run 2026-08-23: measured_disagreements: 0, unregistered: 0. The reason is in its own output — the only instrument it reads is data/interaction-matrix.json, generated 2026-08-11T22:00:06Z, and it prints the warning itself: "11.4 days old. Anything read out of it describes the engine of that day, not this one." IT IS DOING EXACTLY WHAT IT WAS BUILT TO DO AND THE ANSWER IS STILL EMPTY , which is worse than a red: a zero from a blind instrument reads identically to a zero from a clear one. Measured against it the same day, the reports in docs/_reports/ filed nine defects with no register row on 2026-08-22 and at least four more on 2026-08-23 (#377, #378, #382, #383 among them), every one of which UNREGISTERED reported as zero. THE FIX IS NOT TO REGENERATE THE MATRIX — that makes one number current and leaves the tool reading one instrument. It is that UNREGISTERED must read the artifacts that actually measure this engine today: data/game-differential.json's causes, the three data/roster/*.json stages, and data/all-mechanics-fire.json, each matched against the register by subject and each stamped with its own age. INSTRUMENT OWED: those four sources wired into open_work.js's measured half, and a refusal — not a zero — when every source it reads is older than the engine	open — instrument coverage, NOT A DEFECT: the tool is correct and its input set is one artifact wide, so its zero may not be read as "nothing is unregistered" (MEASURE 2026-08-23)
#387	THE WHOLE-GAME HEADLINE SILENTLY CHANGED QUANTITY IN AUGUST AND NOTHING LABELS WHICH ONE IT IS, SO THE REGISTER'S OWN TREND LINE COMPARES TWO DIFFERENT MEASUREMENTS — MEASURED 2026-08-23 BY MEASURE. wholeGameClause published a RAW rate (diverged / games) until the declared-divergence exemption landed 2026-08-18, and has published an UNDECLARED rate ((diverged - declared - decision_cleared) / games) since. Both are correct and neither is named in the sentence it prints: the string is <n> of <games> = <pct>% DIVERGE, with the raw figure appearing only inside a parenthetical and only when something was subtracted. #218 therefore carries 39.6% (raw, 2026-08-12), 69.9% (undeclared, 2026-08-18) and 8.0% (undeclared, 2026-08-23) in one cell as if they were one series, and the raw-to-raw pair — 39.6% against 8.5% — has to be reconstructed by hand from a field the sentence does not print. AND THE ARTIFACT OFFERS THREE MORE NUMBERS THE HEADLINE IS NOT. The same run carries arms[].diverged at middle 82, top-tie-first 68 and bottom-tie-first 77 of 961; the headline reads 77 because 82 minus 5 declared is 77. A COORDINATOR READ THAT COLLISION AS THE GATE QUOTING A CORNER BY OMISSION AND ORDERED A CORRECTION AND A DEFECT ROW ON THE STRENGTH OF IT. Refuted by driving the shipping function on perturbed copies of the artifact: the headline is invariant to both arm figures and moves only with the top-level diverged and with the declared set (#218). NO ARM-SELECTION DEFECT EXISTS AND THIS ROW IS NOT ONE — it is registered because the near-miss cost a correction cycle, and because the same collision will recur every time raw minus declared lands on one of the other two arms. WHAT WOULD FIX IT: the clause names its quantity in the sentence (undeclared), prints the RAW rate beside it unconditionally rather than only when something was subtracted, and prints the arm it describes by id; and the artifact carries the published headline as its own field so a reader never has to reconstruct which of five numbers the gate meant. INSTRUMENT OWED: an assertion that the published headline string names its quantity and its arm, in the style of the composition-vs-headline check #292 already added to this clause	open — reporting clarity, NOT A DEFECT: every number the clause prints is correctly computed; what is missing is the label that says which question it answers (MEASURE 2026-08-23)
#388	A SUBSTITUTE REFUSES AN ON-ENTRY STAT DROP, AND EVERY GAME THIS ENGINE HAS EVER PLAYED DROPPED A STAT THROUGH A DOLL — LANDED 2026-08-23 BY ENGINE, REGISTERED AND CHECKED AGAINST THE ARTIFACT 2026-08-23 BY MEASURE. The authority gates the drop on the target's substitute and answers a bare -immune — data/abilities.ts:2191 (Intimidate) and :4710 (Supersweet Syrup), and the Champions mod overrides thirteen abilities of which NEITHER is one, so mainline IS the Champions handler here. applyEntryDrops walked the foes with only a `!f\`	\f.fainted guard. **FIXED the single consumer of intimidate (18,772 uses inherits it with no ed engine's subBlocks hel and would have let an **MEASURED , NOT ASSERT 08-23T09:02:29Z: **643 the probe ability/onSwit intimidate[true] subbed :

#	item	note
#389	A RATCHET THAT REPUBLISHES ITS OWN ARTIFACT ON A RED RUN LAUNDERS THE REGRESSION IT JUST CAUGHT — <code>tests/test-unmodelled-clicks.js</code> , FOUND 2026-08-23 BY MEASURE WHILE SWEEPING THE CLASS BEHIND #390. At HEAD the file's <code>fs.writeFileSync(OUT, ...)</code> sits outside every conditional and carries <code>generated: new Date().toISOString()</code> , so <code>data/unmodelled-clicks.json</code> is rewritten on every invocation including a failing one. The demonstration, recorded in the fixing agent's own header: remove <code>healbell</code> from the baseline so the live set appears to have grown — run 1 FAILS the set did not GROW ... NEW: <code>healbell</code> , exit 1, and writes the grown set; run 2, the run a person naturally does after seeing red, reads <code>3 -> 3</code> , exit 0, and <code>healbell</code> is baseline forever. Nothing else changed between the two runs. This is CLAUDE.md's founding failure inside the instrument rather than the engine — a capability absent and everything reporting success — and it is the same shape as the torn read that produced <code>344 attributed games</code> where the settled artifact had 450. It is the ONLY sibling the class sweep found: of the six <code>tests/test-*.js</code> that write into <code>data/</code> with no flag, four are generators and <code>test-json-nan-guard</code> writes only when its count FALLS. NOT CLOSED HERE, DELIBERATELY. A green-only write is in the working tree, written by another agent tonight, uncommitted, and I did not run it — the file is on this brief's do-not-touch list. INSTRUMENT OWED: the two-run demonstration above, re-run against the fix, with the artifact hash shown identical across the red run THE CLASS IS WIDER THAN THIS FILE, AND A GATE FOR IT WAS BUILT THE SAME NIGHT. <code>tests/test-red-run-writes.js</code> (another agent, 2026-08-23) asserts the PROPERTY structurally rather than enumerating known-bad files — three bugs in this repository have come from a ratchet whose detector was a list — and its own header reports five more instances found by reading, one of which is <code>tests/test-game-diff.js</code> publishing its artifact after its comparator had FAILED ITS OWN PLANTED-DIVERGENCE PROOF: an instrument declaring itself trustworthy immediately after failing the test of its trustworthiness. It also records that four instruments already carried an ad-hoc write-detecting regex (<code>test-rollout-gates</code> , <code>test-site-data-fresh</code> , <code>test-publish-guard</code> , <code>test-register-reality-readonly</code>) — the machinery was present four times and enforced nowhere. NO VERIFIED BY MARKER IS PLACED YET: that file is untracked at filing time and MEASURE has not run it; the marker belongs on this row the moment it is committed and has been run once. THE GATE LANDED AND IS COMMITTED; THE ROW STAYS OPEN BECAUSE MEASURE HAS NOT VERIFIED IT AND THE FLOOR STILL NAMES A LAUNDERER — 2026-08-24. <code>tests/test-unmodelled-clicks.js</code> now carries a green-only write (<code>write_policy: "Written ONLY by a run that PASSED (or by an explicit --accept)"</code>) and is committed. Four more instances were confirmed one at a time and fixed — <code>test-tag-consumed.js</code> (laundering demonstrated on the verbatim ratchet logic: <code>REGRESSED (was LIVE) → STILL DEAD</code> on the second run), <code>test-game-diff.js</code> (now exits at the planted-divergence proof, BEFORE a game is played; its header had claimed this for months and it was false), <code>test-forme-assert.js</code> and <code>test-switch-back-renamed.js</code> . <code>tests/test-red-run-writes.js</code> asserts the PROPERTY — a column-o <code>data/</code> write with a later failure signal — over 387 files rather than enumerating known-bad ones, proves its own detector against 2 plants and 3 controls every run, and was shown RED on a planted new offender. THE ENUMERATION HANDED OVER WAS ITSELF WRONG IN THE DIRECTION THAT MATTERS: it called <code>tests/test-mechanics.js</code> the milder class and it is the WORST instance in the tree — a full self-baselining ratchet on <code>data/mechanics-census.json</code> , the artifact that steers <code>all_mechanics_fire.js</code> , that <code>quarantine.js</code> reads for a MEDICHAM gate clause, and that the whole-game differential pins as <code>steering.input_digest</code> . It was fixed separately (the census floor	open — instrument DEFECT. The gate is built, committed and auto-registered; MEASURE has NOT run it, and <code>tests/test-mechanics.js</code> still sits in its floor tagged LAUNDERS. Closes on the floor shrinking plus one verified run (MEASURE 2026-08-24)

#	item	note
	may FALL and may never RISE, demonstrated two-run before and after) but <code>tests/test-mechanics.js</code> is still inside <code>ACCEPTED in tests/test-red-run-writes.js</code> with tier <code>LAUNDERS</code>, and the floor may only shrink by somebody removing it. THAT REMOVAL, PLUS ONE MEASURE RUN OF THE GATE, IS WHAT CLOSES THIS ROW. Not closed on another agent's report. Accounts: <code>docs/_reports/2026-08-23-red-run-writes.md</code>, <code>docs/_reports/2026-08-23-red-run-writes-class.md</code>, <code>docs/_reports/2026-08-23-census-launders.md</code>. VERIFIED BY: <code>node tests/test-red-run-writes.js</code>	
#390	THE LIVING-DOCS GATE REWROTE ITS OWN BASELINE ON EXACTLY THE CLASS OF COMMIT IT FIRES ON, FOR AS LONG AS IT HAS EXISTED — FOUND AND FIXED 2026-08-23 BY MEASURE. <code>tests/test-docs-current.js</code> compared its baseline as JSON text with one key removed by <code>regex (generated)</code>, so any key added later became a write trigger. The key that was added later is <code>changelog_top_at_baseline</code>, which tracks the CHANGELOG top version — and the living-docs rule REQUIRES a CHANGELOG bump on precisely the commits that trip this hook. Not a coincidence three times, a loop. Fraction of the observed rewrites carrying real information: 0 of 3. <code>086dd25</code>, <code>4383241</code> and <code>483f529</code> are each <code>2 insertions(+)</code>, <code>2 deletions(-)</code>, each a timestamp and a version, each a solo commit; no list, count or figure set moved in any of them. Causal link proved WITHOUT touching <code>CHANGELOG.md</code> (held by another agent): setting the baseline's <code>changelog_top_at_baseline</code> to <code>5.90.0</code> — the state a bump produces — made the gate write, byte-for-byte the shape of all three commits, and the file was restored afterwards. FIX: keys are now classified PROVENANCE (generated , changelog_top_at_baseline) versus RATCHET (by , rule , version_pins , unversioned_exempt , archive_grandfathered , known); the comparison runs over PARSED content with the provenance keys dropped, so re-indentation and key reordering are no longer triggers either; and a new clause fails BY NAME on any key in neither set, which is the part that stops the recurrence — <code>changelog_top_at_baseline</code> became a trigger because nobody had to say what kind of key it was. The write stays green-only and monotone, so the ratchet can still tighten itself (the 2026-08-22 audit's <code>66 → 65</code> retired citation would still land). A BLANKET "NEVER WRITE DURING A CHECK" WAS REJECTED ON MEASUREMENT, not taste: it would have stopped the ratchet tightening except when somebody remembered a flag. SHOWN RED FOUR WAYS — (A) tonight's exact trigger now writes nothing, hash identical; (B) a planted stale <code>unversioned_exempt</code> entry still writes, <code>ratchet tightened: 1 entry retired</code>; (C) an unclassified key exits 1 by name and the baseline is NOT written on the red run; (D) with the hook's new staging block deleted, the observed defect reproduces on demand in a throwaway repo and the tree is left dirty. <code>tests/test-docs-current.js</code> reads <code>23 passed / 0 failed</code> (was 22). THE HOOK HALF IS PROVEN ONLY IN A SCRATCH REPO — <code>.githubhooks/pre-commit</code> now <code>git add</code> s the gate's artifact when a GREEN run moved it, by NAME and never by sweeping the tree (several agents run against this repository at once) — and has not yet run in anger here; the first real commit that trips it is its true test. VERIFIED BY: <code>node tests/test-docs-current.js</code>	closed 2026-08-23 — the write trigger is fixed and demonstrated red four ways; the hook's staging half is unproven in this tree, and both files are uncommitted at filing time
#391	DECISION NEEDED FROM WILL — tests/test-effective-identity IS RED, THE RED IS THE RULER RATHER THAN THE ENGINE, AND MEASURE DELIBERATELY DID NOT MOVE THE REFERENCE POINT. MEASURED 2026-08-23 BY MEASURE. 18 of the 19 assertions pass, including all three behavioural pins that carry the file's real claim (<code>effective()</code> resolves all five fields together; the derivation reproduces all 62 hand-written <code>MEGA_ABIL</code> entries; <code>.ability</code> is the base forme's on all 62 buildable stone-holders before the choice and the mega's on all 62 after a real turn). Only the raw-count ratchet fails. What the counter counts, split by a lexer: repo-wide 45.2% of the 1,597 "raw reads" are not reads (189 prose + 533 writes). Of the 130 new matches across 30 files: 44 prose, 12 writes, 122 .baseStats reads that are structurally impossible defects (a live body carries <code>st</code>, not <code>baseStats</code>; <code>buildMon('garchomp', {})</code> has no <code>baseStats</code>; the one dangerous shape, <code>baseSpecies.baseStats</code>, occurs 0 times repo-wide), and of the remaining <code>.ability</code> reads ~27 touch a live body and every one deliberately wants the effective value and gets it. ZERO STALE-IDENTITY READS IN THE 130. AND THE HEADLINE ACTIVELY RECOMMENDS THE WRONG MOVE: the gate has been reported red by name seven times by seven agents (869, 1048, 1471, 1561, 1596, 1597) and two of those readings were BELOW the baseline of the day, because the ratchet is per-file and the headline is a total — an agent reading <code>1597</code> vs <code>1198</code> naturally concludes "restamp". The 2026-08-11 wholesale restamp (<code>234 → 1,198</code>, adopting 964 reads unreviewed on "<i>restamp i guess</i>") did not hold a week. The counter has made people edit comments to satisfy it twice on the record. 62% of the red is the calendar: 24 of the 30 files did not exist when the baseline was taken. THE THREE OPTIONS, PICK ONE. (A) declare the SIX files that are exact twins of existing DECLARED entries — <code>engine/board_state.js</code>, <code>tests/probe_bench_leaves.js</code>,	closed 2026-08-23 — MEASURE. The decision was overtaken: the counter was retired rather than restamped or waived, on Will's instruction, and none of options A, B or C was adopted

#	item	note
	engine/all_mechanics_fire.js , engine/million_run.js , engine/rollout_seed_prevalence.js , tests/test-immunity-gate.js — each duplicating a written reason, 48 of the 130, then re-measure (MEASURE's recommendation: it adopts nothing and leaves the other 82 visible). (B) accept the printed baseline, 1603 across 107 files (--propose prints it and writes nothing) — a REVIEWED re-baseline rather than 2026-08-11's unreviewed one, but still 130 matches adopted. (C) waive it by name: <i>"the raw-count ratchet is red on 130 matches that MEASURE walked on 2026-08-23 and found to contain zero stale-identity reads; it stays red until #176 is taken."</i> NOTHING WAS RESTAMPED AND data/effective-identity-baseline.json IS UNTOUCHED (still generated: 2026-08-11 , count: 1198 , 80 files) — narrowing the counter against a baseline recorded under the old counter is a restamp wearing a lexer, and #183 is what that costs. What DID change: the failure headline now leads with the FILE count and says the total is not what fails; a PROSE/WRITE/READ classifier prints beside every growing file; --split and --propose exist; destructured reads are counted and are 0 today. AND #183'S REAL DEBT IS STILL OPEN AND UNTOUCHED — the walk of the simulator's OWN reads, engine/medicham2-browser.js DECLARED and grown 215 → 279; that is the one place in this ratchet where a stale-identity confusion would do real damage, and it remains unwalked. VERIFIED BY: node tests/test-effective-identity.js CLOSED 2026-08-23 — THE DECISION IS MOOT BECAUSE THE RULER WAS REPLACED, ON WILL'S OWN INSTRUCTION, AND NONE OF THE THREE OPTIONS WAS TAKEN. Will, asked again: <i>"you do what you think is best just make it a permanent solution."</i> The raw-count ratchet this row is entirely about is RETIRED — not restamped, not waived. tests/test-effective-identity.js now asserts a RUNTIME property instead of counting text: every mon a Board switches in gets a recording accessor and exactly one call site may see the raw field, board.js effective() . The last numbers the count ever asserted are kept under last_count_baseline with the reason, and --update / --propose now REFUSE with exit 2, so options (A) and (B) are no longer executable and (C) is not needed. NOTHING WAS ADOPTED: the 130 matches were not swallowed, the inventory that replaced the ratchet fails nothing, and the gate reads 24 assertions passed, 0 failed. THE ROW'S OWN CENTRAL FINDING SURVIVES AND IS THE REASON THE REPLACEMENT WAS RIGHT — a counter that seven agents read as red, twice below the baseline of the day, and that made people edit comments to satisfy it, was recommending the wrong move by construction. AND THE REPLACEMENT IMMEDIATELY FOUND A REAL ONE THE COUNT WAS GREEN ON, filed as #412. See #411. Account: docs/_reports/2026-08-23-identity-gate-permanent.md . VERIFIED BY: node tests/test-effective-identity.js	
#392	A BODY BUILT AS THE ALREADY-BUSTED MIMIKYU FORME ABSORBED THE HIT ANYWAY, BECAUSE THE ABSORB GATED ON A RUNTIME FLAG AND NEVER ON THE SPECIES — 15 OF THE 56 DAMAGE DIVERGENCES, LANDED 2026-08-23 BY ENGINE, MEASURED AND REGISTERED BY MEASURE. formeOnHitAbsorbs refused only when tg._disguiseBusted was set, and that flag is written only when the breaking hit actually lands — so a body CONSTRUCTED from the observed key mimiky-busted (which buildMon gives ability: 'disguise' and no flag) read as intact and swallowed the move whole: machamp rockslide -> mimikyubusted = 0-0, identical to an intact Mimikyu, against Showdown's 57-68. The authority is the Champions override — data/mods/champions/abilities.ts:14-32 gates Disguise's onEffectiveness on ['mimiky', 'mimikyutotem'].includes(target.species.id) , so the busted forme gets nothing; reading data/abilities.ts here would be reading a different game. THIS REACHED PAST THE DIFFERENTIAL: the live break path was always correct, so an in-battle Mimikyu was fine — the defect fired wherever a body is built from an OBSERVED forme name, which is exactly what the ingest records (mimikyubusted is in move-priors.json) and what any board or rollout constructed from observed state uses. TWO INDEPENDENT MEASUREMENTS, BOTH READ OFF ARTIFACTS RATHER THAN CLAIMED. (1) data/mechanics-census.json (09:02:29Z) carries a live probe formeOnHit — a body that is ALREADY the busted forme absorbs nothing : one click, three bodies — intact Mimikyu + Disguise lost 130 (maxhp/8) and became mimiky-busted ; the ALREADY-BUSTED body lost 170, the same as the ability-blanked control. (2) data/engine-diff.json re-run at 2026-08-23T09:07:44Z, same seed 20260804, same 6,000 comparisons: disagreed 56 → 41, and zero Mimikyu rows remain in the published 40 where there were seven. The drop of exactly 15 matches the reconstructed family size (7 published + 8 hidden). ATTRIBUTION IS BY FAMILY COMPOSITION ACROSS TWO RUNS, NOT BY A PER-ROW CONTROL — stated so nobody upgrades it later. The fix carries a FORMEONHIT_SPECIES_BLIND restore knob	closed 2026-08-23 — probed and measured twice; the engine change is uncommitted at filing time

#	item	note
#393	THE DAMAGE DIFFERENTIAL'S REFERENCE ENTERS AT <code>moveHit</code> AND NEVER RUNS <code>ModifyMove</code> , SO FIVE MEGA ABILITIES ARE INVISIBLE TO THE RULER AND 24 OF THE 40 PUBLISHED ROWS ARE THE INSTRUMENT — MEASURED 2026-08-23 BY MEASURE FROM <code>data/engine-diff.json</code> . <code>tests/test-engine-diff.js:428-429</code> runs only <code>ModifyType</code> , and says so in its own comment at :425 (<code>" useMoveInner runs ModifyMove on the next line and this does NOT ... Recorded as owed rather than smuggled in here"</code>). The authority runs both — <code>sim/battle-actions.ts:431</code> and <code>:439</code> — while the harness enters 931 lines lower at <code>moveHit (:1370)</code>. Mega Sol is the same hole through a different door: <code>Pokemon#effectiveWeather (sim/pokemon.ts:2198)</code> requires <code>battle.activePokemon</code> , which only <code>runMove</code> sets. EVERY ROW IS PROVEN, NOT INFERRED: for each, Showdown's published number equals MEDICHAM's own number with the attacker's ability cleared to <code>none</code> , to the unit, at both corners — <code>cameruptmega flashcannon -> gengar 69-82</code> (ours 90-106, x1.30 Sheer Force), <code>kangaskhanmega fakeout -> pinsir 37-45</code> (ours 44-55, the bonded packet), <code>gyaradosmega earthquake -> rotomwash o-o</code> (ours 136-160, Levitate broken by Mold Breaker), <code>meganiummega weatherball -> licanrocmidnight 21-25</code> (ours 64-75, the private sun), <code>lopunnymega fakeout -> gourggeistsuper o-o</code> (ours 33-40, Scrappy). Knob-cleared both ways. CURRENT COUNT, read off the artifact generated 2026-08-23To9:07:44Z: of the 40 published rows, Sheer Force / Camerupt 11, Parental Bond / Kangaskhan 7, Mold Breaker / Gyarados 3, Mega Sol / Meganium 2, Scrappy / Lopunny 1 = 24, and after #392 these plus Castform are the WHOLE residual. FUR COAT IS NOT A DEFECT DESPITE LOOKING LIKE ONE — measured at x2 Defence, and <code>furfrou</code> diverges only because MEDICHAM correctly lets Mold Breaker through it and the reference cannot. THIS IS ON THE CRITICAL PATH FOR LIFTING QUARANTINE even though it is the ruler: <code>engine/quarantine.js:599</code> requires <code>disagreed === 0</code> , and an instrument defect holds that clause shut exactly as an engine defect does. HOW MANY MORE ROWS OF THIS CLASS EXIST, DERIVED: of the 76 legal megas, 11 carry a slot-o ability with an <code>onModifyMove / onPrepareHit / onModifyPriority</code> handler; across all legal abilities there are 22 such carriers. A related structural blindness, stated so it is not mistaken for agreement: the harness never runs <code>onPrepareHit</code> either, so Protean rows agree for the wrong reason — a green row that asks nothing. INSTRUMENT OWED: run <code>ModifyMove</code> in the harness (or exclude these carriers from the comparison and say so in the artifact), then re-measure at seed 20260804	open — INSTRUMENT DEFECT holding a gate clause shut; the engine is not accused by any of these 24 rows
#394	A CASTFORM-SNOWY BODY IS NEVER RECONCILED TO AN EMPTY SKY, AND WHETHER THAT IS THE HARNESS OR THE ENGINE RESTS ON A READ OF THE SOURCE RATHER THAN ON AN EXECUTION — 16 OF THE 41 REMAINING DAMAGE DIVERGENCES, 39% OF THE RESIDUAL, FILED 2026-08-23 BY MEASURE WITH NO VERDICT. <code>mc_key.js</code> now admits battle-only formes, so the pool contains <code>palafinhero</code> , <code>aegislashblade</code> , <code>mimikyubusted</code> , <code>morpekohangry</code> and <code>castformsnowy</code> ; the three with no switch-in precondition produced zero rows across 116 draws, and the two carrying one produced all 31 (the other is #392). PROOF BY CONTROL, 16/16: rebuilding the body as base <code>castform</code> reproduces Showdown's number to the unit on every published row, and base stats are identical across all four Castform rows in <code>MC.mons</code> , so a types-only revert is numerically complete. THE READING THAT MAKES IT THE INSTRUMENT: MEDICHAM does model this — <code>syncWeatherFormes</code> finds no matching sky and falls to <code>revertsToTypes: ['Normal']</code> , reached through <code>syncFieldTypes</code> , which IS called at battle init (<code>medicham2-browser.js:14522</code>) — while the harness builds bodies and calls <code>dmgRange</code> directly and never calls it, and the Showdown side really switches in so Forecast's <code>onStart</code> fires. <code>applyEntryEffects</code> does NOT revert it (measured: types stay <code>["Ice"]</code>), so only the init-level call would. THE PROBE THAT SETTLES IT, AND IT BELONGS TO WHOEVER HOLDS THE ENGINE: build a lead pair with <code>castform-snowy</code> , run one <code>battle()</code> init, assert the body's types read <code>["Normal"]</code> on turn 1 with no weather, control being the same body under snow reading <code>["Ice"]</code> . If it reads <code>["Ice"]</code> , this is a SECOND engine defect of #392's shape and outranks #393. AND A STALE LOAD-BEARING COMMENT: <code>medicham2-browser.js:12946-12948</code> still says <code>data/engine-data.js</code> holds no row for Castform-Sunny/Rainy/Snowy — that is the whole stated reason Forecast is modelled as a retype with the species label deliberately wrong — and #204 landed those five rows on 2026-08-22; <code>MC.mons</code> holds all four today. The blocker named in the comment is gone and the premise should stop being cited. WHICH SCOREBOARD, DECLARED BEFORE THE RUN: Castform is o.08% of teams (28 of 36,696), so expect the pinned pool NOT to move — 39% of the published damage residual for o.08% of the metagame. INSTRUMENT OWED: the init probe above	open — engine or instrument NOT ADJUDICATED, no verdict filed (MEASURE 2026-08-23)
#395	THE DAMAGE DIFFERENTIAL PUBLISHES 40 ROWS OF HOWEVER MANY IT FOUND, SO A TAIL OF 16 HAD TO BE	open — instrument reporting, NOT A DEFECT: every published row is correctly computed; what is missing is the rest of them (MEASURE 2026-08-23)

#	item	note
	RECONSTRUCTED BY HAND BEFORE ANYONE COULD GROUP IT — MEASURED 2026-08-23 BY MEASURE. tests/test-engine-diff.js:954 writes worst: bad.slice(0, 40) over a list sorted by move usage descending, and every arms list is capped at 40 the same way (:975) — the union of all sixteen arm lists is the same 40 triples as the midpoint's, so the arms do not recover the tail either. At disagreed: 56 that hid 16 rows, all with move uses <= 1507, and recovering them required replaying the harness's LCG (seed 20260804, whose only in-loop consumer is pick) and re-deriving each family by the harness's own rules — a reconstruction that reproduced 39 of the 40 published rows and is therefore near-exact rather than exact. The cost was not academic: the hidden 16 contained 8 of the 15 Mimikyu-Busted rows (#392) and 6 of the 11 Sheer Force rows, so the artifact under-reported the engine defect's size by more than half. Today's re-run (disagreed: 41) hides only 1, so the cap is currently almost harmless — which is exactly when it is cheapest to raise. WHAT WOULD FIX IT: raise or remove the cap, or publish a family tally beside the truncated list so the count is readable without a replay. Individual rows can already be confirmed one at a time with --case att,move,def (:762), and the run is deterministic, so nothing is lost — only hidden. INSTRUMENT OWED: the cap raised, or a per-family count written into the artifact	
#396	THE WHOLE-GAME 82 GROUP INTO 31 MECHANISMS, AND NO MEASUREMENT IN THAT RUN SAYS WHETHER ANY OF THEM IS BOARD-MATERIAL — MEASURED 2026-08-23 BY MEASURE, INDEX ROW. Source: data/game-differential.json , release c36782953dee , census 8c778268919e matches_live: true , pool data\team-pool-frozen digest 0d103fb9fa87 , generated 2026-08-23T06:10:58Z — the run was pinned exactly as claimed. THE ARM MATTERS AND THE HEADLINE IS NOT THE PRIMARY ONE: arms read middle 82 (pins.primary) , top-tie-first 68, bottom-tie-first 77, and the commit body's 77 of 961 = 8.0% is the bottom corner minus nothing; the primary arm is 82 of 961 = 8.5%, or 8.34% over the 959 usable once the 2 mid-void games are removed, which is the only rate the artifact itself computes (see #387 for why the quantity is unlabelled). This row groups the middle arm's 82, because classes[].causes[] is the middle arm's table and nothing else in the artifact is broken out per arm at cause level. THE SINGLE MOST IMPORTANT FACT HERE: state_mode: false , end_state_mode: false , state: null , end_state: null — this was a PROTOCOL run, no board was compared at any turn boundary and no end board was compared, so by Will's standing rule (<i>a divergence is narration-only when something MEASURED it as narration-only</i>) the board-material split cannot be read off it at all, and data/state-ladder.json may not be substituted (16 days old, no engine_release , and it predates the artifact's own declared baseline_reset). Until node engine/game_differential.js --games 961 --end-state --release c36782953dee --team-store data/team-pool-frozen --write runs, "board-material zero" has no denominator — and even a clean end-state run does not clear the -fail family, which needs engine/move_result_state.js (#382) because board_state.js compares no move-result field. COVERAGE, HONESTLY: 82 of 82 attributed at cause-string level; 64 of 82 (78%) backed by evidence (61 cards in first_divergences , +2 mega detailschange , +2 order_probe); 18 read from the cause STRING alone and every claim about those is inferred. **The two rows most worth carding are \	-fail\ <> \

#	item	note
#397	SIX SWITCH-IN / ON-HIT EFFECTS DO NOT FIRE AT ALL IN A REAL GAME, EVERY ONE BOARD-MATERIAL BY STRUCTURE, AND EVERY ONE ALREADY CARRYING A POPULATED TAG ROW – 7 OF THE WHOLE-GAME 82, FILED 2026-08-23 BY MEASURE. Read off data/game-differential.json (release c36782953dee , 2026-08-23T06:10:58Z); none of the six has a Champions override, so the mainline handler is the authority. Forecast (2 games, 15 uses): the authority writes ~-formechange ... Castform-Rainy \	[msg]\
		[from] ability: Forecast in – board-material witho holds the full byWeatl missing bodies (see al – the highest here**):

#	item	note	
#398	A CLOSED ROW SITS OVER A LIVE FIVE-GAME FAMILY: #289 FIXED THE ZERO-MAGNITUDE EMITTER AND FIVE GAMES STILL DIVERGE ON A ZERO-MAGNITUDE BOOST — FILED 2026-08-23 BY MEASURE WITH NO VERDICT. data/game-differential.json (release c36782953dee , 2026-08-23T06:10:58Z) carries 5 of the 82 in the shape ` \	-boost\	...\
#399	THE ONE TOOL THAT RENDERS DIVERGENCE CARDS FOR A HUMAN READS A TWO-DAY-OLD DUMP AND SAYS SO NOWHERE ON THE PAGE — MEASURED 2026-08-23 BY MEASURE. engine/divergence_cards.js reads data/divergence-turns.json , which is stamped 2026-08- 22T03:05:53Z, release 6a05dd9ad60d , of_diverged: 378 , arms {middle:133, top-tie-first:120, bottom-tie-first:125} — the 39.6%-era run. The current differential is release c36782953dee at 2026-08-23T06:10Z with middle 82. Running the tool today produces a readable, well-formed HTML page of the WRONG RUN with no staleness warning anywhere on it; its causes include -unboost: a different body and an ordering case involving	open — instrument DEFECT (a stale render presented as current); no gate reads this file's age today	

#	item	note
	[from]drought / hospitality , neither of which is in today's 82. This is precisely the shape CLAUDE.md warns about — a rendering that computes nothing, over a file nobody regenerated, indistinguishable from a fresh result — and it is the same class as #386 (a tool reporting confidently off one aged input) and the WEB rule that a rendered figure must trace to an artifact or read NOT MEASURED. Will's own card review runs through this page, so a stale render is not cosmetic: it sends a human reading cards at mechanisms that no longer diverge. WHY IT GOES STALE SILENTLY: the dump is only written alongside <code>--write</code> and only when <code>--dump-games</code> is given a COUNT, so an ordinary differential run leaves it untouched while updating everything around it. WHAT WOULD FIX IT: the renderer refuses — or banners — when the dump's <code>engine_release</code> differs from <code>data/game-differential.json</code>'s, and the next differential run is given <code>--dump-games 200 --dump-out data/divergence-turns.json --write</code>. INSTRUMENT OWED: that release comparison, at the top of the page and in the exit code	
#400	A SEMI-INVULNERABLE BODY IS HIT ANYWAY, AND A CLOSED ROW SITS OVER IT — 2 OF THE WHOLE-GAME 82, BOARD-MATERIAL BY STRUCTURE, FILED 2026-08-23 BY MEASURE. <code>data/game-differential.json</code> (release <code>c36782953dee</code> , 2026-08-23To6:10:58Z) carries two: Decorate into a Dragapult in the Phantom Force turn — the authority writes <code>-miss</code> and we write <code>-boost atk 2</code> , so a stat exists on the board here that does not exist there — and a locked Phantom Force that announces and deals nothing. #123 IS CLOSED AND DOES NOT COVER THESE: it settled the semi-invulnerability EXCEPTION LIST (which moves reach an invulnerable body anyway), not whether the invulnerable turn is respected by an ordinary move or whether the second turn of the lock resolves. Anyone grepping the register for semi-invulnerability lands on a row marked closed, which is why this is filed separately. Phantom Force is 698 uses. Likely sites: the invulnerability check on the target, and the locked-move second turn. NO VERDICT ON THE MECHANISM — both are read from cause strings plus their cards, and neither has been staged. INSTRUMENT OWED: a staged pair — Decorate into a body in the Phantom Force turn (control: the same Decorate into the same body out of it), and a two-turn Phantom Force asserting damage on turn 2 HALF CLOSED 2026-08-23 BY ENGINE; THE ROW STAYS OPEN ON ITS SECOND MEMBER. (a) CLOSED — a STATUS move now misses a semi-invulnerable body. <code>hitStepInvulnerabilityEvent</code> is <code>step ZERO of trySpreadMoveHit (sim/battle-actions.ts:556, 621)</code> and that path is taken by EVERY move; this engine implemented it as <code>_stepInvuln</code> inside the ATTACK branch only, so roughly forty non-attack kinds resolved in branches below it. Staged before a line changed, Decorate / Charm / Will-O-Wisp / Taunt into a mid-Phantom-Force Dragapult were byte-identical across the knob on every member — which is what an unwired gate looks like. ONE gate at the common commit site, <code>a.kind === 'attack'</code> deliberately skipped so nothing about the damaging path moves, with the authority's own three exemptions (Helping Hand, a Poison-type's Toxic, <code>guaranteedAgainst</code>) as probe arms that must NOT miss. Both events emitted (<code>[miss]</code> attribute AND the <code>-miss</code> line), because emitting only one trades a board divergence for a narration one. Probe: <code>tests/test-mechanics.js</code> <code>move/semiInvulnerable</code> ; counter <code>MEDSEEN.statusMissedInvuln</code> .Account: <code>docs/_reports/2026-08-23-board-material-remainder.md</code> §5. (b) STILL OPEN — the locked Phantom Force that announces and deals nothing on its second turn. Untouched by the above, never staged, no instrument. INSTRUMENT OWED: a two-turn Phantom Force asserting damage on turn 2, control being the same pair with the lock broken	open — engine DEFECT SUSPECTED on the SECOND-TURN LOCK only; the status-move-into-invulnerability half is CLOSED 2026-08-23 (probed). Site not confirmed, INSTRUMENT OWED (MEASURE 2026-08-24)
#401	FUTURE SIGHT'S DELAYED DAMAGE IS OFF BY 1-2 HP AND THE HP NUMBER ITSELF DIFFERS, SO IT IS BOARD-MATERIAL WITH NO ARGUMENT — 2 OF THE WHOLE-GAME 82, FILED 2026-08-23 BY MEASURE. <code>data/game-differential.json</code> (release <code>c36782953dee</code> , 2026-08-23To6:10:58Z): both <code>-damage field 3</code> divergences in the run follow <code>-end ... move: Future Sight</code> — Hippowdon 35 against 34, Sylveon 95 against 97. Ours is high on one and low on the other, which is what a snapshot difference looks like rather than a multiplier: the delayed slot stores the attacker's stats and boosts at the moment the move was CLICKED, and anything the two engines disagree about there (a stat stage, a weather, an item that has since moved) reappears two turns later as a small HP gap. RANK-SUPPRESSED ON USAGE AND THAT IS THE TRAP: Future Sight reads 18 uses in <code>data/tags.json</code> , so every usage-ordered work list buries it — but the quarantine bar is board-material zero, where an 18-use move that lands a different HP number counts exactly as much as a 20,000-use one. DO NOT LET THE CLASSIFIER'S NAME OUTLIVE THE FACT: the grouping matched on the CLASS <code>-damage field 3</code> , not on Future Sight, and it is right today only because both members happen to follow the same <code>-end</code> line; a third <code>-damage field 3</code> from any other source would be mislabelled here (see #303, which was a shared die read backwards, and #363 for the sibling <code>-boost field 3</code> class). INSTRUMENT OWED: a staged two-turn Future Sight with the attacker's Special Attack changed between click and arrival, asserting	closed 2026-08-23 — engine DEFECT, fixed by ENGINE; the row's proposed mechanism (a click-time snapshot) is WITHDRAWN — it was the span-vs-index draw

#	item	note
	the arrival is priced off the CLICK-time snapshot, control being the same pair with nothing changed CLOSED 2026-08-23 BY ENGINE — AND THE MECHANISM THIS ROW PROPOSED WAS THE WRONG ONE, WHICH MATTERS MORE THAN THE CLOSE. The row's hypothesis was a CLICK-TIME SNAPSHOT disagreement. It is not: the delayed hit SELECTED a position out of a span where every other damage path had been converted to the sixteen-index roll — i.e. it is ROADMAP #304 surviving in the one path #304's fix did not reach. Both members are gone from <code>by_cause</code> on release <code>3e00ea2575a9</code> (Hippowdon 35/34 and Sylveon 95/97), with no other cause moving by a single game across the five-mechanism batch. The row's own warning about the classifier holds and is why this is stated: the grouping matched the CLASS <code>-damage field 3</code>, not Future Sight. A THIRD Future Sight game was EXPOSED by the fix and closed in the same pass (the delayed hit was not writing its <code>-supereffective / -resisted</code> line) — narration, not a regression. Account: <code>docs/_reports/2026-08-23-board-material-remainder.md §3–§4</code>. A REMAINDER IS FILED SEPARATELY: the delayed hit takes no crit draw at all, so it can never emit <code>-crit</code> and can never crit	
#402	TWO PUBLISHED SEED-PREVALENCE FIGURES ARE WITHHELD — THE GENERATOR'S SPECIES LOOKUP DROPPED EVERY FORME — 2026-08-23, MEASURE. <code>engine/rollout_seed_prevalence.js:95</code> read <code>MC.mons[base(sp)]</code>, and <code>base()</code> ends in <code>norm()</code>, which strips the hyphen <code>MC.mons</code> keys every forme with. So Rotom-Wash asked the table for <code>rotomwash</code>, got nothing, and was handed an EMPTY dataset moveset — which the caller compares against the sheet's declared four and scores as <i>"the moves differ"</i>. A miss can only push <code>movesDiffer</code> UP, so the bias is one-directional. Measured over <code>data/games.bo3.json1</code> : 47 of 256 store species names, 10,980 of 144,260 brought bodies (7.61%); at the sheet-body level 19,236 of 231,924 bodies hit it and 7,177 (3.095% of all bodies) flip true → false once the row is found — top flippers Floette-Eternal 3,022, Arcanine-Hisui 1,919, Lycanroc-Dusk 1,059. WITHHELD, NOT ANNOTATED (CLAUDE.md: <i>"A CAPTION IS NOT A QUARANTINE"</i>), from <code>docs/SEARCH.md R14</code> (the section headline, the one-line verdict, the prevalence table's #248 moveset and ANY rows, and the two R14 cells in the argmax-paired reach column), from <code>docs/MILTANK.md §3.9</code>, from this row's #248 closure text, and from <code>docs/MEDICHAM-SPRINT-NOTES.md</code> : <code>pct_248_bench_moves_differ</code> and <code>pct_any</code> of <code>data/rollout-seed-prevalence.json</code>. SCOPE IS BOUNDED AND MEASURED, NOT ASSUMED: <code>datasetMoves</code> is called at exactly ONE site in the generator and feeds only <code>movesDiffer</code>; <code>pct_any</code> is an OR containing it. The other four rates — 17.655, 17.053, 13.089, 0.061 — are computed independently and STAND. NO CORRECTED VALUE IS ESTIMATED: the direction is known, the magnitude is not, and a body-level flip rate does not translate one-for-one to a decision point that ORs over the whole live bench. THE FOUR ROWS CLOSED ON THIS BATCH (#247, #248, #249, #250) DO NOT REST ON THE WITHHELD FIGURE and are NOT reopened — each closed on <code>tests/test-rollout-seed.js</code> shown RED first with its own control, the prevalence figures were published as ceilings on reach beside them (<code>docs/SEARCH.md</code> : <i>"a ceiling on reach, not a count of flipped argmaxes"</i>), and the actual effect was measured separately and independently at 61/131 paired argmax flips under common random numbers (#278). #247's and #249's quoted shares are among the four unaffected. RESTORED BY A PINNED RE-RUN, NOT AN UNPINNED ONE: <code>head -n 14102 data/games.bo3.json1 > data/_seed-prev-pin.json1 && node engine/rollout_seed_prevalence.js --store data/_seed-prev-pin.json1</code>. The store has grown 14,102 → 19,401 games since 2026-08-14, so an unpinned run is a different question; a correct pin must return <code>decisionPoints: 190378</code> exactly, which the bug cannot have touched. The lookup itself is already fixed and goes through <code>engine/mc_key.js</code> — VERIFIED BY: <code>node tests/test-mc-key.js</code> SCOPE NARROWED 2026-08-24 BY MEASURE — THE VERIFIED BY ON THIS ROW DOES NOT DECIDE ITS OPEN HALF, AND THE REGISTER WAS OVERSTATING BECAUSE OF IT. <code>engine/register_reality.js</code> ran <code>node tests/test-mc-key.js</code>, read exit 0, and filed this row as the only STALE ROW in the file — an OPEN row whose instrument is GREEN. That is correct and it is not evidence the row should close: the marker decides the LOOKUP FIX, which is landed and gated, and it is structurally incapable of deciding the WITHHELD FIGURES, which are restored only by the pinned re-run this row already specifies (<code>head -n 14102 data/games.bo3.json1 > data/_seed-prev-pin.json1</code>, and a correct pin must return <code>decisionPoints: 190378</code> exactly). THE TWO FIGURES REMAIN WITHHELD, NOT ANNOTATED, and the re-run has NOT been done. INSTRUMENT OWED for the open half: nothing in the repository fails while those two figures are absent	open — MEASURE. The LOOKUP FIX is landed and gated (VERIFIED BY <code>node tests/test-mc-key.js</code>, which cannot decide this row's open half); the pinned RE-RUN and the restoration of the two WITHHELD figures are owed and INSTRUMENT OWED (MEASURE 2026-08-24)
#403	SUCKER PUNCH LANDED ON A TARGET THAT HAD ALREADY MOVED, AND THE THIRD CLAUSE OF AN IF THAT HAD BEEN FIXED PIECEMEAL TWICE BEFORE IS	-fail\ p1a: Kingambit . **Four v in all three arms with Account: <code>docs/_reports/</code>

#	item	note	
	NOW A SHARED READER — CLOSED 2026-08-23 BY ENGINE. <code>data/moves.ts:18399</code> opens <code>this.queue.willMove(target)</code> and <code>sim/battle-queue.ts:319-327</code> walks the REMAINING queue; <code>engine/medicham2-browser.js</code> asked <code>acts.find(...)</code> over the whole turn's action list, so a target that had already acted still read as <i>about to attack</i>. <code>queueWillMove</code> now mirrors <code>BattleQueue.willMove</code> beside <code>unresolved</code> in <code>battleTurn</code>, and every clause of that if reads it — #60 (priority), #180 (<code>mustrecharge</code>) and this one. Both members of <code>failsIfTargetNotAttacking</code> call the same authority function and the tag's own membership rule is the regex <code>/willMove\\(\\s*target\\s*)/</code>, so a third member printed later is covered without an edit. Shown RED with a knob-cleared control: 85 / 85 across the arms before, 0 / 85 after, and <code>MEDI_SUCKER_QUEUE_BLIND=1</code> puts it back. The authority was asked the exact board and prints ``		CLOSURE WAS DEMONSTRATED BY <code>move/failsIfTargetNotAttacking</code> that has ALREADY MOVED, still green and stages no red. THE QUEUE CLAUSE IS CLOSED, closed OVER FOUR LIVE, <code>queueWillMove</code> reader 1, divergences became zero, still green today. What HAPPENS AFTERWARDS, **THE REFUSAL IS EVALUATED, <code>if(TAGS.has('move',a.move))</code> the queued aim — while <code>until redirectDrawnTo...</code> orders those two the <code>pokemon.getMoveTargets(n...</code> event at <code>sim/pokemon.t...</code> :590 fire <code>singleEvent("T...</code> suckerpunch's <code>onTry</code> (<code>this.queue.willMove(target)</code>) move at +2 and has already redirected's PARTNER, land a 70 BP priority, argued*: release 8ad9446a684709d, pool of games 14 / 46 / 69 / 70, one shape in all four, move <A> suckerpunch <T> 91 HP board difference, untouched for the authority mechanism.md section 1 — upperhand reads the membership rule is the printed later gets the EXECUTABLE MARKER ON THE CITATION IS KEPT WORD, the QUEUE clause and so reopened row, <code>engine/r...</code> STALE ROW (<code>open + gr...</code> nor verified — the exact / #283. <code>data/register-read</code> row and will clear on UNOWNED: a probe that states Powder user under pinning not exist.
#404	STRUGGLE WAS PRICED AT ITS PRINTED TYPE BY THE DAMAGE PATH, SO IT TOOK STAB IT HAS NOT EARNED AND WAS REFUSED BY A GHOST — CLOSED 2026-08-23 BY ENGINE. <code>effMoveType</code> had honoured <code>setOwnTypeAlways</code> since the tag was built; <code>dmgRangeOneHit</code> opened <code>let mvT = mv.t</code> and never asked — the duplication <code>formeMoveType</code>'s own header names. It is ALL SEVEN residual rows of <code>tests/test-engine-diff.js --n 6000 --seed 20260804</code>, in two shapes: six unearned $\times 1.5$ STAB (ditto struggle -> clawitzer, Showdown 11-14 against our 16-21) and one Normal->Ghost immunity the authority does not have (gallade struggle -> gengar, Showdown 51-60 against our 0-0). <code>data/engine-data.js</code> carries <code>t:"Normal"</code> and is CORRECT — that is the dex's printed type, and <code>???</code> arrives from the move's own <code>onModifyMove</code> at use time, so this was never a data fix. Differential 7 of 6000 -> 0 of 6000 across all seventeen arms; knob <code>MEDI_DMG_OWNTYPE_BLIND=1</code>. MEMBERSHIP MEASURED OVER THE WHOLE DEX RATHER THAN ASSUMED: of ten moves assigning a literal <code>move.type</code>, five are Past and four of the five legal ones are conditional with their own tags — exactly one is unconditional and it is Struggle. Account: <code>docs/_reports/2026-08-23-suckerpunch-ragefist-struggle.md</code>. VERIFIED BY: <code>node tests/test-mechanics.js --move/setOwnTypeAlways</code>	closed 2026-08-23 — engine DEFECT in the damage path, fixed and probed by ENGINE	
#405	UNNERVE REACHED TWO OF THE FIVE PLACES A BERRY IS EATEN, AND THE RESIST BERRY IT LET THROUGH WAS THE DIFFERENCE BETWEEN A KO AND A SURVIVOR — CLOSED 2026-08-23 BY ENGINE. The authority raises <code>TryEatItem</code> in exactly ONE place — <code>Pokemon#eatItem</code>, <code>sim/pokemon.ts:1785-1787</code> — so every un-forced berry consumption is refused while an Unnerve body stands opposite. This engine had the refusal written out twice as an inline expression on the two <code>onUpdate</code> berries and nowhere else, so the CURE berry, the instantaneous confusion cure and the RESIST berry were all eaten under Unnerve. The resist berry is board-material with no argument: Close Combat into a Chople Berry Tyranitar reads ``	-damage\	p2a: Tyranitar\

#	item	note	
#406	THE AUTHORITY RUNS ITS Update PASS AT THREE POSITIONS PER ACTION AND THIS ENGINE RAN IT AT ONE, SO EVERY onUpdate SETTLED AN ACTION LATE — CLOSED 2026-08-23 BY ENGINE. The machinery already existed (<code>_updateAll</code> , #81 WIRE 7); what was missing was the POSITION, not the mechanism — <code>eachEvent('Update')</code> at <code>sim/battle-actions.ts:967</code> , INSIDE the hit loop and one statement above <code>faintMessages()</code> at :976. Landing it cost a SPLIT of <code>_updateAll</code> into <code>_updateEvent</code> (the event) and <code>_updateAll = _updateEvent + restoreStatsAll</code> (the schedule), because the White Herb rides a different event that merely shares the schedule; plus one step in the existing per-move step list, one position above <code>_stepFaint</code> . THE SIXTH DIVERGENCE WAS NEVER THIS EVENT AND WAS NOT SMUGGLED IN BEHIND IT: Stone Axe's ``	-sidestart\	p2:\
#407	FOUR MORE NARRATION MECHANICS CLOSED BY REACH IN ONE BATCH-OF-ONE SWEEP — 2026-08-23 BY ENGINE. Ranked by reach, each shown RED with its control cleared before anything was fixed, each followed by its own release and a full re-run: the mechanics clause went 29 → 23 played-and-uncleared and the census 646 → 651 live, 0 missing. The other two members of the batch closed existing rows and are recorded there — Poltergeist on #359, Regenerator on #223 and on #397. CURSED BODY (2,177 teams): the carrier writes nothing and the CONDITION writes everything. TOXIC DEBRIS (1,840 teams): a punish handler that announces itself. DISABLE (1,799 clicks): the <code>-start</code> names the move it sealed. MENTAL HERB (967 teams): the item is spent BEFORE the volatile it frees — <code>Pokemon#useItem</code> writes the <code>-enditem</code> and <code>removeVolatile</code> runs the condition's <code>onEnd</code> , which writes the <code>-end</code> ; this engine had them the other way round, and two members were staged (ENCORE and TAUNT) whose <code>-end</code> lines come from two different conditions with two different labels, so an ordering fix that happened to work for one shape is not enough. EVERY ONE IS NARRATION AND EVERY PROBE ASSERTS THE BOARD AS A CONTROL THAT MUST NOT MOVE — the HP, the volatile, the layer count and the item disposition were already right in all six, which is why the board-material whole-game count did not fall. THE INSTRUMENT WAS WRONG TWICE IN THIS PASS AND BOTH TIMES IT WAS THE PROBE, NOT THE ENGINE: a normaliser stricter than <code>game_differential.js</code> 's own <code>effect-namespace</code> equivalence reported ten volatile <code>-end</code> labels as divergent (measured over all 57 volatiles: ten differ, every one in the NAMESPACE and in nothing else); and a failed self-heal's <code>[still]</code> is a REAL difference the differ does not count (<code>move-target-field</code>) — named here UNFIXED rather than normalised away. NOT CLAIMED: the authority's herb loop removes ALL SIX of its conditions and this engine removes the one that just landed; on a legal board they agree, but that is an argument and no arm tests it. Account: <code>docs/_reports/2026-08-23-mechanics-by-reach.md</code> . VERIFIED BY: <code>node tests/probe_mental_herb_order.js</code> (RED 4 / GREEN 0, knob <code>MEDI_HERB_END_FIRST=1</code>) and the census probes for the other three	closed 2026-08-23 — four engine DEFECTs (narration), each fixed in its own batch with its own release, probed by ENGINE	

#	item	note
#408	THE FAINT QUEUE: FOURTEEN CLASSES DERIVED FROM THE CALL SITES, TWO CONVERTED, AND THE PASS STOPPED THERE ON PURPOSE — 2026-08-23 BY ENGINE. <code>faintMessages()</code> is called at eight places in the authority; the classes were derived from the TRIGGER rather than listed. Two were converted. Measured on three releases, arm middle , 961 games, pinned pool and pinned census: the gate's headline undeclared 53 of 961 (5.5%) → 52 of 961 (5.4%), raw parted 58 → 57, board-material 24 → 24 (did not move), narration 34 → 33 (did not rise), DIFFERENT-END-STATE 18 → 17, census 658 → 660 live / 0 missing. THE ONE DIVERGENCE REMOVED IS EXACTLY THE ONE IT WAS AIMED AT — the whole first-divergence cause table is identical either side except <code>`ordering :: \</code>	-damage\ p2a\
#409	THE SILENT-CATCH RATCHET WAS A GOOD INSTRUMENT WIRED TO NOTHING, SO THE COUNT CLIMBED 67 → 95 IN FOUR DAYS — CLOSED 2026-08-23 BY MEASURE AND ENGINE, AND THE GATE IS GREEN. <code>tests/test-no-silent-failure.js</code> runs in 0.68 s, is keyed by a hash of each catch BODY so moving code does not fool it, and was connected to neither <code>.githubhooks/pre-commit</code> nor <code>tests/run-all.js</code>. It printed into the void, so writing a silent catch carried no consequence at the moment it was written, which is the only moment that matters. Three things landed and no new gate and no new test was created. (1) EIGHTEEN REAL BLOCKS FIXED, 80 → 62, nineteen blocks over seventeen edits, every one re-derived at the line rather than applied from a table — including <code>engine/gate_fail_and_silent.js</code> (an unreadable release now WITHHOLDS the count instead of publishing it), <code>engine/medicham2-browser.js</code> (a failed <code>require('./mc_key.js')</code> writes <code>MEDFAILS.mcKeyModuleUnloadable</code>; demonstrated by intercepting that one require), <code>engine/feature_shift.js</code> (a total <code>weatherTurns</code> loss now REFUSES instead of ageing every board by one turn) and <code>engine/fixture_preflight.js</code> (a PARTIAL authority-source load refuses and names the missing files; only the all-empty case did before). Nothing on a success path moved and the four tests among them report identical pass counts. It did not go 80 → 62 in one step — the first pass landed on 64 with MANUFACTURE going UP by one, which is in the report. (2) THE RATCHET IS WIRED TO THE COMMIT. One new flag, <code>--only <file>...</code>, narrows the VERDICT and not the DETECTION — the scan stays repo-wide and there is exactly one detector. Corrected once before it shipped: comparing only against the baseline refused the next commit over 9 pre-existing blocks the author never touched, so <i>pre-existing</i> means HEAD. (3) --accept IS PER BLOCK, NOT PER FILE. It now takes the block hash the detector already computes and takes one block per run; the hash prints beside every reported block so the command is a copy-paste. This mattered because three files hold both kinds at once (<code>engine/tag_dex.js</code>, <code>engine/medicham2-browser.js</code>, <code>tests/test-web-quarantine.js</code>), so accepting by file would have blanket-excused blocks nobody judged — which is why <code>accepted</code> had sat empty for ten days. THE STATE TODAY, READ OUT OF data/silent-catch-baseline.json : 201 baselined + 62 accepted after review + 0 NEW, and every one of the 62 carries a human-written reason. Demonstrated four ways in order: red at 62 → 58 accept runs → green at 0 → one accepted body edited (a comment) goes RED again by file, line and new hash, then reverted. Accounts: <code>docs/_reports/2026-08-23-eighty-silent-catches-triage.md</code>, <code>...-thirteen-silent-catches.md</code>, <code>...-eighteen-real-fixed.md</code>, <code>...-silent-catch-enforced.md</code>, <code>...-accept-per-block.md</code>. VERIFIED BY: <code>node tests/test-no-silent-failure.js</code>	closed 2026-08-23 — MEASURE and ENGINE; the instrument is green, is enforced at the commit, and goes red again the moment any accepted block is edited
#410	TURNING A SPECIES NAME INTO A KEY OF MC.mons HAD BROKEN FOUR TIMES, TWICE AFTER BEING “FIXED AND GATED”, AND THE TABLE IS SEALED NOW RATHER THAN WATCHED — CLOSED 2026-08-23 BY ENGINE. Will: <i>"i want no more problems not finding certain names cause of spelling differences make a bulletproof solution and close it out."</i> THE COMMON PROPERTY OF THE FOUR IS NOT THE SPELLING — IT IS THE SILENCE: every one returned <code>undefined</code> or <code>null</code>, which reads as <i>the engine has never seen this Pokemon</i> (a real and common	closed 2026-08-23 — engine DEFECT class closed structurally by ENGINE; a miss now throws by name instead of reading as an unknown Pokemon

#	item	note
	condition), so every caller carried on and nothing complained. Every previous fix was A LIST OF WRONG FORMS, and a list cannot catch a form nobody thought of — two of the four got through a list that already existed. Two changes in opposite directions, both needed: THE SEAL — <code>MC.mons</code> is a <code>Proxy</code> and reading a key the table does not have THROWS <code>LookupMiss</code> naming the key the caller meant (<code>engine/mc_key.js</code> <code>seal()</code>, installed on require, no call needed); and ONE TOTAL LOOKUP routed through 96 call sites. THE DEBT FILE IS DELETED, NOT RE-BASELINED. Routing the sites found six megas being priced as their base forme, which no counter had ever reported. Account: <code>docs/_reports/2026-08-23-one-door-species-keys.md</code>. VERIFIED BY: <code>node tests/test-mc-key.js</code>	
#411	THE EFFECTIVE-IDENTITY GATE STOPPED COUNTING TEXT AND STARTED MEASURING RISK, AND ITS FIRST RUN FOUND A DEFECT THE COUNT HAD BEEN GREEN ON FOR THREE WEEKS — CLOSED 2026-08-23 BY MEASURE. Will: <i>"you do what you think is best just make it a permanent solution."</i> <code>tests/test-effective-identity.js</code> is now a RUNTIME tripwire: every mon a <code>Board</code> switches in gets a recording accessor on <code>ability</code>, <code>baseStats</code>, <code>weighthg</code> and <code>weightkg</code>; every active on the test board holds a mega stone whose forme ability differs from every ability its base forme can have — swept out of the dex and the damage table, nothing named — so on that board the declared ability is wrong for all four actives and a raw read is a defect by construction. The board is then driven through the live decision path and every read is recorded with its stack. Exactly one call site may see the raw field: <code>board.js</code> <code>effective()</code> — Fowler's SELF ENCAPSULATE FIELD stated as an executable assertion instead of as a count. 24 assertions passed, 0 failed; the 18 behavioural pins are byte-identical and 6 are new. SHOWN RED SEVEN WAYS BEFORE BEING TRUSTED: six planted shapes compiled through <code>vm.compileFunction</code> with synthetic filenames so no scanner in this repository can ever count them as debt — <code>mon.ability</code>, <code>const { ability } = mon</code>, <code>(({ ability }) => ability, { ...mon })</code>, <code>Object.assign({}, mon)</code> and <code>mon[k]</code> with <code>k</code> built at runtime — the last three of which the old header conceded were undetectable by any text scan — plus one real edit to <code>engine/board.js:3520</code>. THE OLD COUNT BASELINE IS RETIRED, NOT RESTAMPED: its numbers are kept under <code>last_count_baseline</code> and <code>--update</code> / <code>--propose</code> refuse with exit 2. WHAT IT PROVABLY CANNOT DO IS PRINTED ON EVERY RUN: EXECUTION coverage only, blind to a sheet value copied before a <code>Board</code> existed — permanent for the four fields on the driven path, merely better everywhere else, and the gate says which is which. Account: <code>docs/_reports/2026-08-23-identity-gate-permanent.md</code>. VERIFIED BY: <code>node tests/test-effective-identity.js</code>	closed 2026-08-23 — MEASURE; the ruler was replaced rather than restamped, and the live read it found is filed separately
#412	<code>engine/position_features.js:249</code> READS A LIVE STONE-HOLDER'S DECLARED ABILITY WHERE <code>engine/board.js:3520</code> RESOLVES IT — ONE FACT, TWO IMPLEMENTATIONS. FOUND 2026-08-23 BY THE RUNTIME TRIPWIRE, WHICH THE RETIRED COUNT RATCHET WAS GREEN ON. <code>priorityRefusedAbove()</code> in <code>position_features.js</code> builds its defender list with <code>`B.norm(f.mon.ability \</code>	<code>\</code> <code>e.ability \</code>
#413	A RED-DEMO ROW WENT HOLLOW: WIRE 7 the <i>Sitrus</i> is eaten between the two attackers, not at the residual READS <code>reverted-arm=true</code> — OBSERVED 2026-08-23 BY ENGINE. The reversal applies and the reverted engine plays, and it still satisfies the assertion — so the demonstration cannot fail for the reason it claims, and the wire it defends is unwatched. This is a DIFFERENT shape from the twelve stale reversals on #273: a stale reversal has stopped running, a hollow one runs and asks nothing, which is <i>CLAUDE.md's a green test can be asking nothing</i> inside the harness built to prevent exactly that. The same file read 14 failures at 01:05Z and 15 at 02:40Z with only engine bytes between them, so it appeared under the 2026-08-23 engine work. WHAT WOULD DECIDE IT: whether a later wire made the berry timing correct by a	open — instrument DEFECT (a demonstration that cannot fail); ENGINE owns the file, filed by MEASURE 2026-08-24. INSTRUMENT OWED

#	item	note	
	SECOND route (in which case the reversal needs widening) or the assertion stopped reading the timing at all. Account: docs/_reports/2026-08-23-spread-rounding-life-orb.md §7. INSTRUMENT OWED: the assertion re-aimed so that the reverted arm must fail, with the arm printed on every run		
#414	GROWL DOES NOTHING IN THIS ENGINE — OBSERVED 2026-08-23 BY ENGINE, NOT INVESTIGATED. Found incidentally while staging another rule: ``\	move\	p1a:scizor\
#415	THROAT CHOP'S SILENCE LASTS ONE TURN TOO LONG AND ITS -end IS MISSING — FILED 2026-08-23 BY ENGINE, DEFERRED WITH THE FINDING BESIDE IT. _noSound = turns + 1 against the authority's duration: 2 . That is STATE, not narration: a body is refused a sound move on a turn where the authority allows it. The missing -end is the narration half of the same volatile. 2 games of the whole-game pool. It was deferred inside the narration pass because the pass's stop condition was <i>board-material must not rise</i> , and this one moves state. WHAT WOULD DECIDE IT: stage Throat Chop, click a sound move on each of the following three turns, and assert the refusal on exactly the authority's two — control being the same board with no Throat Chop. Account: docs/_reports/2026-08-23-narration-timing.md §6.4 and §10. INSTRUMENT OWED CLOSED 2026-08-24 BY ENGINE, CHECKED AGAINST THE ARTIFACT BY MEASURE — AND THE CLOCK WAS FIXED, NOT JUST THE LINE. throatchop.condition carries duration: 2 and residualEvent spends one on the turn the volatile lands, so the authority silences the turn it lands and ONE more; this engine's turns + 1 silenced a third, on 5,071 corpus uses . OBSERVED in the official simulator rather than reasoned about: a Weavile chops a Primarina that clicks Hyper Voice on three consecutive turns — cant on t1, the volatile at duration 1 with the sound move disabled on t2 plus -end at that residual, and a FREE Hyper Voice on t3. Both halves landed: the clock, and the missing ``\	-end\	BODY\

#	item	note	
#416	A BROKEN SUBSTITUTE'S lastDamage IS THE UNCLAMPED HIT WHERE THE AUTHORITY CLAMPS IT TO THE DOLL'S REMAINING HP — FILED 2026-08-23 BY ENGINE. The authority clamps before RECOIL and DRAIN read it, so an overkill into a doll pays the attacker back on the overkill here and on the doll's last HP there. This is the real STATE defect found behind #358, which turned out to be narration — the two must not be confused, and #358 is closed while this is not. WHAT WOULD DECIDE IT: a substitute at a known HP, one draining move and one recoil move of known damage well above it, asserting the attacker's HP change on both engines; control is the same pair with the doll at full width so no clamp applies. Account: docs/_reports/2026-08-23-narration-timing.md §3 and §10. INSTRUMENT OWED	CLOSED 2026-09-03 BY ENGINE. CENSUS 828 -> 829 LIVE / 829 PROBED / o MISSING, o HOLLOW, o THREW, o UNARMED. THE PREMISE HELD. AUTHORITY: Champions does not override Substitute — grep substitute data/mods/champions/moves.ts returns o, derived rather than remembered — so data/moves.ts is the line and the clamp sits ABOVE both readers: :18341-18343 clamps damage to target.volatiles['substitute'].hp , :18345 assigns it to source.lastDamage , :18352 pays applyRecoilDamage(damage, ...) and :18356 heals ceil(damage * drain[0]/drain[1]) . AND NOTHING DOUBLE-PAYS IT: spreadMoveHit turns a HIT_SUBSTITUTE result into damage[i] = true (data/mods/champions/scripts.ts:351-353), which the hit loop folds to o before move.totalDamage += damage[i] (sim/battle-actions.ts:961-965), so the outer applyRecoilDamage at :982 sees nothing from a doll. THE DEFECT: this engine banked dealt = min(dmg, tg.curHP) sixty lines ABOVE the substitute branch, so the ceiling was the BODY's HP and never the DOLL's. MEASURED: a Double-Edge into a 35-HP doll cost the authority 12 and this engine 25; a Bitter Blade into a 41-HP doll healed the authority 21 and this engine 62. THE FIX gives _reDealt an explicit ceiling — default tg.curHP , so the two existing callers are byte-identical — and the substitute branch passes _s0 , the doll's HP read one line above the subtraction. One helper, three callers, because "how much did this actually deal" is one fact. THE CONTROL IS THE ROW'S OWN AND WAS GREEN BEFORE THE FIX: the identical pair with the target's max HP x12 so floor(H/4) is above the hit and no clamp applies — both engines read -25 and +75 there, before and after, which is what says the fix touches the overkill and nothing else. DECLARED REMAINDER, NOT BUNDLED: the authority's drain on this road is Math.ceil (data/moves.ts:18356) where _payDrainRow is Math.round (sim/battle.ts:2168 , the ordinary road's rule). They coincide for every 1/2-fraction drain move in this format and part on a 3/4 one (Draining Kiss). It is a DIFFERENT line, it was already on the 2026-08-23 hand list, and folding it in would have destroyed this pass's attribution. NO POOL FIGURE IS CLAIMED — light mode, game_differential.js was not run; predicted BEFORE the pass as census +1 and board-parted unmoved (possible -1 or -2). VERIFIED BY: node tests/probe_sub_clamp.js — both engines, one real turn with the target faster so the doll is up when the hit lands, the fixture DERIVED from the format (recoil and drain families, a Substitute learner not immune to either), bodies built identically on both sides so the control arm can agree number for number; shown RED first on exactly the two overkill arms (-12 vs -25, +21 vs +62) with both wide-doll controls already agreeing, and re-run RED under MEDI_SUB_DEALT_UNCLAMPED=1 . tests/probe_recoil_after_clamp.js , the sibling clamp on the BODY, is still green. Plus the census row move/substitute "an overkill into a doll pays recoil and drain on the DOLL'S last HP, not on the swing", which goes MISSING under the same knob. Account: docs/_reports/2026-09-03-crit-draw-and-substitute-clamp.md	
#417	THE PERISH SONG DEATH WRITES A -damage LINE THE AUTHORITY DOES NOT WRITE, BECAUSE IT CALLS damage() WHERE THE AUTHORITY CALLS faint() — FILED 2026-08-23 BY ENGINE. 1 game of the whole-game pool. The two routes differ in more than the line: faint() queues, damage() pays, so anything reading HP or a faint queue between the two can part as well. NO VERDICT ON BOARD-MATERIALITY — the body dies either way and nothing measured whether the intervening state agrees. WHAT WOULD DECIDE IT: a Perish count expiring beside a residual that reads HP, asserting the protocol AND the end state on both engines. Account: docs/_reports/2026-08-23-narration-timing.md §6.2 and §10. INSTRUMENT OWED CLOSED 2026-08-24 ON ITS OWN CLAIM, WITH THE REMAINDER FILED SEPARATELY RATHER THAN LEFT INSIDE IT. perishsong.condition.onEnd is two statements — this.add('-start', target, 'perish0') then target.faint() — and Pokemon#faint() writes no log line, so the \	-damage\	...\
#418	A SCREEN-BREAKING MOVE CLEARS SCREENS THROUGH A TYPE IMMUNITY, A PROTECT AND A MISS — FILED 2026-08-23 BY ENGINE, AND IT IS ARCHITECTURAL RATHER THAN A NARROW PATCH. Board-material, 1 game of the whole-game pool. The clear belongs in the per-target hit loop, where the authority puts it, so that a target that was never legally hit never loses its screens; this engine performs it above that loop. DO NOT PATCH IT NARROWLY — the report says so explicitly: adding an immunity check at the one site leaves the Protect and the miss, which is the piecemeal- if shape that cost three passes on Sucker Punch (#403). WHAT WOULD DECIDE IT: Psychic Fangs into a screened Dark type, into a Protect, and into a miss, asserting the screens survive all three; control is the same move connecting. Account: docs/_reports/2026-08-23-board-material-remainder.md §9.1 and §10. INSTRUMENT OWED	closed 2026-09-04 — FALSE AT HEAD, and BOTH arms of the row are live census rows: "Psychic Fangs leaves the screen up when the target is immune to it" and "Brick Break and Psychic Fangs leave the screens standing when the hit never lands". Closed by MEASURE 2026-09-04 as register hygiene against HEAD 73cca9f8 ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account docs/_reports/2026-09-04-register-closures.md . docs/MEASURE.md THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (board-material); architectural, site identified and deliberately not patched narrowly (MEASURE 2026-08-24). INSTRUMENT OWED	
#419	THE DELAYED HIT TAKES NO CRIT DRAW AT ALL, SO IT CAN NEVER EMIT -crit AND CAN NEVER CRIT — FOUND 2026-08-23 BY ENGINE WHILE CLOSING #401, NAMED IN THE ENGINE AT THE SITE. condition:futuremove 's payout goes through trySpreadMoveHit in the authority and therefore gets the FULL step list; this engine's payout does not. State plus narration. It is the remainder of #401, which is closed — the span-vs-index draw was fixed and this was not, and it is filed separately so a closed row does not read as coverage (the failure #398 and #400 were both filed against). Future Sight is 18 uses, so every usage-ordered work list buries it and the board-material bar does not — a move that can never crit is wrong at any usage. WHAT WOULD DECIDE IT: a two-turn	CLOSED 2026-09-03 BY ENGINE. CENSUS 827 -> 828 LIVE / 828 PROBED / o MISSING, o HOLLOW, o THREW, o UNARMED. THE PREMISE HELD — this was not one of the rows that had already been fixed. AUTHORITY: data/conditions.ts:415 hands the payout to trySpreadMoveHit , so it takes the FULL step list; the die is mainline's inside the per-hit call (sim/battle-actions.ts:1156 -> :1636-1642 , critMult = [0,24,8,2,1] at :1633), the 1.5 lands at data/mods/champions/scripts.ts:222 and the LINE at :285 . Champions overrides neither getDamage nor getSpreadDamage and has NO futuremove key at all — derived (grep futuremove data/mods/champions/*.ts returns nothing), not assumed. THE MEASUREMENT IS THE UNWIRED-KNOB SIGNATURE RATHER THAN A MISSING LINE: handed a crit-CERTAIN die and then a crit-IMPOSSIBLE one on the same board, the authority	

#	item	note
	Future Sight over N seeds with the crit die pinned high, asserting <code>-crit</code> appears on both engines; control is the same attacker's direct move, which does crit. Account: <code>docs/_reports/2026-08-23-board-material-remainder.md</code> §10. INSTRUMENT OWED	answered 72 damage with <code>/-crit/</code> and 48 without, and this engine answered 69 both times with no line either time — <code>MEDSEEN.delayedHitCritDrawn</code> was 0 where the authority drew 1. THE FIX is one draw and one re-price in <code>engine/medicham2-browser.js</code> only, addressed like the <code>dmg</code> draw beside it (the BOOKED move, the BOOKED source, the collecting body, ROADMAP #262) and drawn UNCONDITIONALLY per WIRE 35 so a Shell Armor arm and a bare one spend the same stream. The crit is RE-PRICED through <code>dmgRange(..., isCrit=true, ...)</code> and not multiplied — the authority applies 1.5 above the randomizer, STAB, the type chart and burn — and a rate of exactly 1 is skipped because <code>dmgRange</code> has already folded it in. A ROW THAT WENT RED FOR A FIXTURE REASON AND IS RECORDED RATHER THAN QUIETLY REPAIRED: the existing <code>move/delayedHit</code> band row handed ONE plain function to seven streams, so bucket o's <code>u = 0.03125</code> fell under 1/24 the moment the payout started drawing a crit and the row read 177 against its own band's 118. Its <code>rng</code> is now a split struct pinning <code>crit</code> to no-crit; the other fifteen buckets are byte-identical. NO POOL FIGURE IS CLAIMED — light mode, <code>game_differential.js</code> was not run; predicted BEFORE the pass as census +1 and board-parted unmoved (possible -1), the command in <code>docs/_reports/2026-09-03-crit-draw-and-substitute-clamp.md</code> OWED. DECLARED REMAINDER: none for this row. VERIFIED BY: <code>node tests/probe_delayed_crit.js</code> — both engines, three real turns, the crit die pinned high then low, asserting the <code>-crit</code> LINE and that the DAMAGE MOVES, with a DERIVED direct-click control on the same attacker that must crit on both engines in both arms; shown RED first (4 failing clauses, all on the delayed row, the control green) and re-run RED under <code>MEDI_DELAYED_HIT_NO_CRIT=1</code>, which reproduces the identical four. Plus the census row <code>move/delayedHit</code> <i>"the delayed payout rolls the ordinary crit, prices it and announces it"</i>, which goes MISSING under the same knob. Account: <code>docs/_reports/2026-09-03-crit-draw-and-substitute-clamp.md</code>
#420	STANCE CHANGE'S <code>-formechange</code> MAY BE MISSING [from] ability: Stance Change ,AND THE TWO SOURCES OF TRUTH DISAGREE — FILED 2026-08-23 BY ENGINE WITH NO VERDICT. The authority's <code>attacker.formeChange(targetForme)</code> takes <code>this.battle.effect</code> as its source, which during <code>onModifyMove</code> IS the ability — which argues the field should be there. But this engine's own comment at the site records a REAL battle log WITHOUT the field. The line was left byte-identical on purpose and named at the site rather than changed on the stronger-sounding of two arguments. SETTLE IT WITH A MEASUREMENT, NOT A READING — that is the whole content of this row. WHAT WOULD DECIDE IT: stage an Aegislash clicking a damaging move in the official simulator and read the <code>-formechange</code> line it emits. Account: <code>docs/_reports/2026-08-23-board-material-remainder.md</code> §10. INSTRUMENT OWED	open — UNADJUDICATED; no defect is claimed in either direction and the engine was deliberately not edited (MEASURE 2026-08-24). INSTRUMENT OWED
#421	<code>tests/roster.js --stage all</code> AND THE THREE PER-STAGE RUNS DISAGREE ON TWO ROWS — OBSERVED 2026-08-23 BY ENGINE. <code>axekick</code> and <code>electrify</code> read DEFERRED-BY-OWNER per stage and FIRED-AND-BOARDS-DIFFER under <code>--stage all</code> . This is an instrument inconsistency, not an engine defect, and it matters because the MEDICHAM gate reads the roster stages: two ways of invoking one ruler give two verdicts, and only one of them can be what the gate means. It is the same shape as the nine days in which 162 of 169 roster accusations were the ruler rather than the engine. WHAT WOULD DECIDE IT: run both invocations on one frozen release and diff the per-row verdicts, then fix whichever path carries the wrong default. Account: <code>docs/_reports/2026-08-23-board-material-remainder.md</code> §7 and §10. INSTRUMENT OWED	open — instrument DEFECT in <code>tests/roster.js</code> (two invocations, two verdicts); ENGINE owns the file, observed and filed by MEASURE 2026-08-24. INSTRUMENT OWED
#422	TWO MORE SPEED SORTS STILL USE <code>Array.prototype.sort</code> WHERE THE AUTHORITY USES A SELECTION SORT — THE MEGA PHASE AND THE RESIDUAL. FILED 2026-08-24 BY ENGINE. <code>Battle#speedSort</code> (<code>sim/battle.ts:429-460</code>) resolves a tie with a selection sort whose swaps move UNTIED bodies past the tied pair, and a plain stable sort cannot produce that order. There are four places this engine sorts on speed: the move queue was fixed in 3.74.0, the entry pass was fixed 2026-08-23, and <code>engine/medicham2-browser.js:16787</code> (<code>_run.sort(...)</code> , the mega phase) and <code>:5487</code> (<code>residualOrder</code>) still do not. THE MEGA SITE IS BOARD-MATERIAL — which body evolves first can decide which side spends its one mega — and the residual site is narration. Both mega rows in the pinned pool are exact ties (Starmie Adamant 167 against Charizard Timid 167; two Adamant/Brave Mawile at 112), so the class is already showing them; counts are small either way, 2 mega rows and 2 Tailwind rows of 961. DECISION NEEDED FROM WILL, AND IT IS CARD 3 OF <code>docs/_reports/2026-08-24-ordering-cards.md</code> : both remaining sites in one batch, or one at a time — they share a mechanism but land in completely different parts of the turn, and the standing rule is small batches. Sec #220 for the residual's earlier slot-order-versus-speed history, which this does not restate RE-SCOPED 2026-08-24 BY MEASURE TO ITS REMAINDER — THE MEGA HALF LANDED THE SAME NIGHT AND THE RESIDUAL HALF DID NOT. ENGINE took the mega phase on its own rather than waiting on the batching ruling; its account is <code>docs/_reports/2026-08-24-mega-order.md</code> . THE MECHANISM THIS ROW PROPOSED WAS ONLY HALF OF IT, AND THAT IS THE PART WORTH KEEPING. The authority's two <code>megaEvo</code> actions sit at queue order 104, so they are the FIRST group <code>speedSort</code> places and no swap made	closed 2026-09-04 — THE NARROWED REMAINDER IS FALSE AT HEAD. <code>residualOrder()</code> in <code>engine/medicham2-browser.js</code> is the authority's selection sort, with <code>RESIDUAL_STABLE_SORT</code> restoring the plain <code>Array.prototype.sort</code> , and the live census row on tag <code>move/sideBuff</code> reads that two Tailwinds ending on one turn come out in the order the authority's selection sort leaves them. The mega half was already recorded landed and committed in this row's own cell and is not reopened. Closed by MEASURE 2026-09-04 as register hygiene against HEAD <code>73cca9f8</code> ; this edit moves no engine byte and the evidence is restated from committed artifacts, not from a run. Account <code>docs/_reports/2026-09-04-register-closures.md</code> . <code>docs/MEASURE.md</code> THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT, NARROWED to the RESIDUAL sort alone; the mega half is landed AND COMMITTED (<code>0a78640</code> , release <code>b35e96a0e7c7</code>) with a live census probe, and its close is ENGINE's to write. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT, NARROWED 2026-08-24 to the RESIDUAL sort alone; the mega half was landed by ENGINE the same night with a live census probe, and that close is ENGINE's to write on a settled tree. The residual half is unprobed and shares a blocker with #355 and #440. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (wrong sort at two sites, one board-material); DECISION NEEDED FROM WILL on batching — card 3 of <code>docs/_reports/2026-08-24-ordering-cards.md</code> (filed by MEASURE 2026-08-24)

#	item	note
	for a faster MOVE can reach them — they keep the push order <code>addChoice</code> gave them (<code>sim/battle-queue.ts:306</code> pushes, it does not insert). This engine has no megaEvo action at all and read the order off <code>acts</code>, which <code>sortTurnOrder</code> had ALREADY sorted for the moves, so the MOVE sort's swaps had reversed the tied pair before the mega phase ever looked at it. So it was not only the algorithm, it was the LIST being sorted. <code>data/mechanics-census.json</code> (2026-08-24T20:59Z) carries <code>item/megaStone</code> <i>"a mega-phase SPEED TIE keeps the QUEUE order, not the move order"</i> live. MEASURE DID NOT RE-VERIFY THAT PASS'S GAME COUNTS AND SAYS SO: the ENGINE run was still in flight at the time of this edit (<code>data/all-mechanics-fire.json</code> was being rewritten), so its pool figures are ENGINE's own and the close belongs to ENGINE once the tree settles. WHAT REMAINS IN THIS ROW IS THE RESIDUAL SORT ALONE (<code>engine/medicham2-browser.js:5487</code>, <code>residualOrder</code>) — narration, 2 Tailwind rows of 961, and the same handler-list gap #355 and #440 both sit on. CARD 3 IS THEREFORE NARROWED RATHER THAN ANSWERED: the batching question is moot for the mega site, and the question left is only whether the residual sort is worth taking now or waits for the handler list AND THE MEGA HALF IS COMMITTED — MEASURE, 2026-08-24, an hour after the paragraph above. ENGINE committed it as <code>0a78640</code> (<i>the mega phase reads the authority's queue, and a tied pair had been reversed by the move sort</i>), <code>970a1c4</code> and <code>87d3a17</code>, and the tree settled at release <code>b35e96a0e7c7</code>. What MEASURE has verified on the settled artifacts is the PROBE and not the pool arithmetic: the census carries the mega-phase tie probe live, and <code>data/game-differential.json</code> at that release reads 961 games, 37 diverged. The pool BEFORE/AFTER attribution in that pass is still ENGINE's own, because re-running the whole-game differential was not this pass's to do.	
#423	A ROLLOUT HAS NO COIN TO FLIP ON AN ENTRY TIE, SO IT TAKES ONE BRANCH DETERMINISTICALLY — FILED 2026-08-24 BY ENGINE, AND IT IS LOUD RATHER THAN SILENT. The entry tie is now decided by the shared <code>tie</code> die exactly as the move queue's is. But a caller that hands <code>battleInit</code> no rng — every rollout, every census probe, MILTANK's search — has no die, so the tied group keeps the order the selection sort handed it. <code>MEDFAILS.entryOrderTieNoDie</code> counts every occurrence, which is why this is a row rather than a hole. IT IS NOT A CORRECTNESS DEFECT AGAINST THE AUTHORITY: the differential pins the die to a constant on both sides anyway, so the measured whole-game number does not move either way. It is a SEARCH-QUALITY question. DECISION NEEDED FROM WILL, AND IT IS CARD 4 OF <code>docs/_reports/2026-08-24-ordering-cards.md</code>: leave rollouts deterministic on a tie (cheap, reproducible, biased to one branch), or give <code>battleInit</code> a die and accept that every existing seeded baseline in the repository shifts WILL RULED 2026-08-24, AND THE RULING SPLITS THE ROW IN TWO. No die for rollouts — but a rollout must RECOGNISE that a tie is a tie and PLAN BOTH BRANCHES rather than silently taking one and calling it the future. That is two pieces of work with two owners. THE ENGINE HALF IS LIVE: make the tie VISIBLE where it resolves, so a caller can see that the order it received was one branch of a coin rather than a fact. THE SEARCH HALF IS PAUSED AND IS FILED ONLY, NOT QUEUED — MILTANK is not being invested in while MEDICHAM is being corrected, so "plan both branches" is recorded here so it is not lost, and no work is scheduled against it. THE COST THE ROW NAMED DOES NOT ARISE: no existing seeded baseline shifts, because no die is being added.	open — a search-quality choice, not a correctness gap against the authority. RULED BY WILL 2026-08-24: no die; the tie must be VISIBLE where it resolves (ENGINE half, live) and the search must plan both branches (SEARCH half, PAUSED — filed, not queued). THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — NOT a correctness defect against the authority; a search-quality choice with a measured cost to every seeded baseline. DECISION NEEDED FROM WILL — card 4 of <code>docs/_reports/2026-08-24-ordering-cards.md</code> (filed by MEASURE 2026-08-24)
#424	THE WIRE LADDER'S FIFTEEN ARMS ALL PIN STRANDED RELEASES, SO THE INSTRUMENT CANNOT PRODUCE A FIGURE AT ALL — MEASURED 2026-08-23 BY MEASURE. <code>engine/wire_ladder.js</code> exits 4 on arm <code>a01-baseline-run1</code> and writes nothing, which is the correct behaviour. All fourteen distinct pinned releases (<code>cf6a68fa412c</code> ... <code>26f96c7894d7</code>) were frozen before <code>engine/medicham2-browser.js</code> exported <code>natureL50</code>, <code>rngStreams</code> and <code>spreadL50</code>; <code>node</code> <code>engine/engine_release.js</code> <code>compat</code> says 168 of 351 releases predate an export. <code>data/wire-ladder.json</code> is dated 2026-08-07, is UNSAFE in provenance, and <code>engine/status.js</code> already WITHHOLDS the release-ladder figure. THIS IS LESSONS §12 AT LADDER SCALE: a stranded artifact is withheld and re-measured, never resurrected — the snapshot still verifies and simply stops being openable. THE UNIT OF WORK IS NOT A RE-RUN: it is re-pinning the arms onto releases that can still serve the caller, or rebuilding them. Account: <code>docs/_reports/2026-08-23-gate-refresh.md</code> §7. VERIFIED BY: <code>node</code> <code>engine/wire_ladder.js</code>	open — MEASURE; the instrument is correct and cannot answer, and the figure it feeds is already withheld rather than captioned (filed 2026-08-24)
#425	<code>engine/all_mechanics_fire.js</code> DEFAULTS TO --kind moves, AND ITS CONSUMER SILENTLY DISABLES ITS OWN FILTER WHEN THE OTHER TWO POPULATIONS ARE ABSENT — MEASURED 2026-08-23 BY MEASURE. <code>data/all-mechanics-fire.json</code> was published with <code>rows.moves</code> only. <code>engine/quarantine.js:718</code> requires per-entity rows for moves, abilities AND items, and falls back to counting every divergence unfiltered when any is missing — printing <i>"THE REACH FILTER</i>	open — instrument DEFECT (a default that disables a gate's filter); MEASURE owns the clause, ENGINE owns the generator (filed 2026-08-24). INSTRUMENT OWED

#	item	note	
	CANNOT BE APPLIED". Re-run with <code>--kind all</code> on 2026-08-23 and the clause answers: 29 of 36 played and unclear. THE DEFAULT IS THE DEFECT: an instrument whose default output silently disables its own consumer's filter is this project's founding failure shape — a capability absent and everything reporting success — arriving through a flag default. THE FIX IS ONE OF TWO AND BOTH ARE ACCEPTABLE: default to <code>all</code>, or have the writer REFUSE to publish a partial artifact to the canonical path. Account: <code>docs/_reports/2026-08-23-gate-refresh.md §7</code>. INSTRUMENT OWED: a check that the canonical artifact carries all three populations before the clause reads it		
#426	ROLE PLAY WAS A WHOLE MOVE RESOLVING TO A SPENT TURN, AND THE BOARD WAS THE ONLY WITNESS — CLOSED 2026-08-24 BY ENGINE, CHECKED AGAINST THE ARTIFACTS BY MEASURE. <code>playerAction</code> classifies a click by walking ~40 tag branches and returning the first that matches. Role Play carried <code>[pp, targetClass, neverMisses, ignoresProtect, statusCategory]</code> — an accuracy, a category and a Protect flag, and no effect at all — so it fell past every branch and came out <code>{kind: 'pass'}</code>. 40 corpus uses of a turn that changed nothing. The artifact had nothing to route on and that is the actual defect: <code>tag_dex.js</code> derived <code>rewritesTargetAbility</code> by reading <code>target.setAbility(...)</code> out of the handler, and Role Play is the only legal move that writes to <code>source</code>, so the existing pattern could never match it. Membership was PRINTED over the whole legal move table before anything was wired — exactly four legal moves call <code>setAbility</code>: Entrainment, Role Play, Simple Beam, Worry Seed — so the new tag <code>copiesTargetAbility</code> is Role Play alone today and a fifth member arriving next regulation is caught by shape rather than by name. Its three refusals are the handler's own in the handler's order (<code>data/moves.ts:15327</code>), both flags answered from the same <code>refusesCopy</code> row Trace consults. NO PROTOCOL LINE EVER SAID SO — <code>`\`</code>	<code>-ability\`</code>	<code>is dropped as cosmeti agreed and the two gam authority's \`</code>
#427	A BODY DRAGGED OFF THE FIELD STILL TOOK ITS OWN TURN, FROM THE BENCH — CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. <code>Battle#runAction</code> opens <code>case 'move'</code>: with TWO refusals and this engine had one: <code>if (!action.pokemon.isActive) return false;</code> was missing and <code>if (action.pokemon.fainted) return false;</code> was present. So a body phased out by Roar, Whirlwind, Dragon Tail or Circle Throw came back later in the same turn and used its move from the bench — and that phantom Roar then dragged the other side's lead out, so the two engines finished the turn with different Pokemon standing. IT PRINTED ITSELF IN THIS ENGINE'S OWN EMITTER AND NOBODY READ IT: <code>`\`</code>	<code>move\`</code>	<code>?: farigiraf\`</code>
#428	A SINGLE-TARGET STATUS MOVE WAS NEVER REDIRECTED AT ALL — CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. <code>Pokemon#getMoveTargets</code> runs the <code>RedirectTarget</code> event in its <code>default</code>: <code>case (sim/pokemon.ts:829)</code> for every single-target move, damaging or not, and <code>useMoveInner</code> calls it once for every move it resolves. This engine asked only inside the ATTACK branch, so a Glare, a Will-O-Wisp, a Taunt or a Thunder Wave walked straight past a Follow Me and landed on the body behind it. Measured: an Arbok's Glare crossed a Maushold's Follow Me and paralysed the Blaziken behind it — <code>`\`</code>	<code>-status\`</code>	<code>p1b: Maushold\`</code>

#	item	note	
#429	BETWEEN TWO REDIRECTORS ON ONE SIDE THIS ENGINE TOOK THE ONE IN THE LOWER SLOT; THE GAME TAKES THE FASTER — CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. <code>priorityEvent</code> passes <code>fastExit</code>, so <code>runEvent</code> sorts the handlers with <code>Battle.compareRedirectOrder</code> (<code>sim/battle.ts:413</code>) — handler priority DESC, then holder SPEED desc, then <code>effectOrder</code> ascending between two ABILITY holders — and takes the first that returns a body. Follow Me and Rage Powder both declare <code>onFoeRedirectTargetPriority: 1</code>; Lightning Rod and Storm Drain declare none, which is 0. That is where "the volatile outranks the ability" comes from — it is the handler priority and not a rule this engine invented, and it is why the two families stay in two passes rather than being merged into one sorted list. Within a family only SPEED separates them, and this engine used <code>Array.find</code> over the foe array, i.e. slot order every time. In the diverging game a Maushold's Follow Me and a Sinistcha's Rage Powder were both up: the authority sent the Gunk Shot into the Maushold (base 111 Speed) and killed it, and this engine sent it into the Sinistcha (base 70). Two engines, two different Pokemon dead. It goes through <code>compareTurnOrder</code> because the sort key is Trick-Room-inverted UPSTREAM — <code>pokemon.speed</code> is <code>getActionSpeed()</code>, which is <code>10000 - speed</code> inside a room — so the SLOWER redirector draws in a room. A REMAINING TIE FALLS BACK TO SLOT ORDER AND THAT IS STATED RATHER THAN CLAIMED CORRECT: the authority breaks an ability-versus-ability tie on <code>abilityState.effectOrder</code>, which is not modelled and needs two rods of the same type on one side at identical Speed, which the differential has never produced. 1 game of 961 cleared, BOARD-MATERIAL. VERIFIED BY: <code>node tests/test-mechanics.js — move/redirects "between two redirectors on one side, the FASTER one draws"</code>, live in the census; the knob is the Speed and nothing else, plus a Trick Room arm, and pre-fix all three arms read the same <code>[85,0]</code>. Account: <code>docs/_reports/2026-08-24-board-material.md §3b</code>	closed 2026-08-24 — engine DEFECT (slot order where the authority sorts on speed), fixed and probed by ENGINE; census probe read out of the artifact by MEASURE	
#430	CLANGOROUS SOUL PAID ITS HP BEFORE THE STAGES IT BUYS — 513 CORPUS USES, THE LARGEST SINGLE DIVERGING MECHANIC ROW. CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. The authority splits the move across two handlers and the split IS the order: <code>onTryHit</code> runs <code>this.boost(move.boosts)</code> and <code>onHit</code> runs <code>this.directDamage(maxhp * 33/100)</code>, and <code>onTryHit</code> sits above <code>onHit</code>, so the five stages are on the board before the HP comes off. This engine paid in the generic <code>costsUserHP</code> block, which sits ABOVE the kind dispatch, so ``\	-damage\	came first on every c. IS THE AUTHORITY'S OWN inside ONE <code>onHit</code> — di comes off first. The e carries a DECLARED boo which is what the auth <code>statChangeInCode</code> instea when the dispatch is g landing site pays at t MEDFAILS.<code>hpCostBoostFirs</code> <code>tests/test-mechanics.js — buys</code> land BEFORE the H the census; red on bot and green as <code>["boost","d</code> <code>docs/_reports/2026-08-24-</code>
#431	A MAXED-OUT STAT IS ANNOUNCED WITH -setboost , NOT -boost , AND THIS ENGINE HAD ALL THREE FIELDS WRONG — BELLY DRUM 124 USES AND ANGER POINT 20, ONE RULE. CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. It lives in the authority's own boost emitter (<code>dist/sim/battle.js:1674-1679</code>): for <code>bellydrum</code> and <code>angerpoint</code> it writes <code>this.add("-setboost", target, "atk", target.boosts["atk"], "[from] " + effect.fullname)</code>. Three things differ from a ``\	-boost\	— the event name, fie. step it took) and a [f all three wrong. Confi

#	item	note	
#432	AN ABSORBING ABILITY ANNOUNCED <code>-immune</code> BESIDE ITS GIFT INSTEAD OF INSTEAD OF IT — SAP SIPPER 76 USES AND SEVEN RELATIVES ON ONE RULE. CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. The authority is the same shape eight times, read member by member out of <code>data/abilities.ts</code> with no Champions override on any of them: <code>if (!this.boost({atk: 1})) this.add('-immune', ...)</code> for Sap Sipper, and the identical <code>if (!...)</code> for Lightning Rod, Motor Drive, Volt Absorb, Water Absorb, Dry Skin, Earth Eater and Flash Fire. <code>boost</code> returns false when no stage moved, <code>heal</code> false at full HP, <code>addVolatile</code> false when it is already up. The <code>-immune</code> is the gift's ELSE, never its companion — it means <i>absorbed</i>, and <i>there was nothing in it for me</i>. This engine announced BOTH, on every absorb. Nothing is named in the engine: the gift comes off the tag's own <code>gain</code>, so one rule covers all eight and a ninth arrives working, and two counters (<code>MEDSEEN.absorbGiftLanded</code>, <code>MEDSEEN.absorbImmuneAnnounced</code>) are both non-zero so the branch is provably a branch. STATED, NOT FIXED: Flash Fire's gift is a VOLATILE this engine does not grant, so with nothing for the else to test it keeps announcing <code>-immune</code> on every Fire hit where the authority announces it only on the second — <code>MEDFAILS.absorbGiftUnmodelled</code> counts it, behaviour unchanged and now loud. VERIFIED BY: <code>node tests/test-mechanics.js</code> — <code>ability/typeImmunity</code> <i>"an absorbing ability announces <code>-immune</code> only when its gift had nowhere to land"</i>, live in the census, whose control is the authority's own other branch: a CAPPED Sap Sipper and a FULL-HP Volt Absorb must STILL write the bare <code>-immune</code>, so a fix that merely deleted the line fails there. The whole family re-measured o diverged. Account: <code>docs/_reports/2026-08-24-mechanics-by-reach.md §3</code>	closed 2026-08-24 — engine DEFECT (an else announced as a companion), fixed and probed by ENGINE; census probe read out of the artifact by MEASURE. Flash Fire's ungranted volatile is a STATED remainder, counted rather than hidden	
#433	A BOUNCED MOVE IS USED BY THE BOUNCER AND SAYS SO, AND THIS ENGINE SAID NOTHING — MAGIC BOUNCE 286 USES. CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. The state was already right and the ACCOUNT of it was missing. <code>bounceOff</code> retargets, and #241 had already fixed who ends up trapped; what this engine never said is that a SECOND move happened — <code>magicbounce.onTryHit</code> builds a fresh active move and calls <code>this.actions.useMove(newMove, target, {target: source})</code>, and a use writes a <code>`</code>	move\	line like any other. SILENT, and the reason and one caller is not consulted by the Prote ask <i>*does this click r</i> put a second <code>`</code>
#434	A RE-BANKED CHARGE IS RE-ANNOUNCED AND THIS ENGINE WAS SILENT ON THE SECOND ONE — ELECTROMORPHOSIS 127 USES. CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. The authority's <code>charge</code> condition carries <code>onStart</code> AND <code>onRestart</code> and the two are the same six lines (<code>data/moves.ts</code>), so the second bank announces exactly like the first and names the move that just landed, because field 4 is <code>this.activeMove.name</code>. The FIRST bank was already right. The engine's own comment at the bank site already said "onRestart re-announces and does NOT stack" and the line under it read <code>if (TR && !_had)</code> — the comment was right and the code disagreed with it. <code>_had</code> is now COUNTED (<code>MEDSEEN.chargeReBanked</code>, the subset of <code>chargeBanked</code> landing on a body already holding the charge) instead of being used to suppress, so a silent re-bank and a re-bank that never happened stop reading identically. THIS IS THE FIX WHOSE ENGINE EDIT WAS LOST IN PUBLICATION AND RE-APPLIED THE SAME DAY — see #442. VERIFIED BY: <code>node tests/test-mechanics.js</code> — <code>ability/buffsHolderOnHit</code> <i>"a second hit RE-ANNOUNCES the Charge it re-banks, naming the move that landed it"</i>, live in the census; the two hits use DIFFERENT moves on purpose, so an engine that merely duplicated the first line prints the wrong move name. Account: <code>docs/_reports/2026-08-24-mechanics-by-reach.md §5</code> and <code>docs/_reports/2026-08-24-narration-timing.md §4</code>	closed 2026-08-24 — engine DEFECT (a suppressed re-announcement), fixed and probed by ENGINE; census probe read out of the artifact by MEASURE	
#435	A SPREAD DRAIN HEALED ONCE, LUMPED, AFTER EVERY TARGET'S DAMAGE — THE AUTHORITY HEALS AT EACH TARGET'S OWN DAMAGE LINE AND NAMES THAT TARGET. CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. The heal is INSIDE <code>spreadDamage</code>'s per-target loop, four statements below the <code>-damage</code> line the same iteration wrote (<code>sim/battle.ts:2159-2171</code>). OBSERVED: a chipped Sinistcha's Matcha Gotcha into a Garchomp and a Swampert it kills reads <code>-damage / -heal ...[of] p2a / -damage / -heal ...[of] p2b / faint</code>, while this engine wrote <code>-damage / -damage / -heal</code> — one lumped line with no <code>[of]</code>. THE ARITHMETIC HALF WAS ALREADY RIGHT AND THIS IS THE OTHER HALF ITS OWN COMMENT NAMED: #339 landed one rounding per body on 2026-08-23 and wrote in as many words that the lumped line was still there. NO NUMBER MOVED, AND THAT IS CHECKABLE RATHER THAN CLAIMED: every row is paid <code>Math.min(dmg, tg.curHP)</code> through the identical four steps (round, clamp-to-1, trunc, the Big Root fixed-point modifier), and clamping per row against the running	closed 2026-08-24 — engine DEFECT (narration position and a missing attribution; no state change, asserted rather than assumed), fixed and probed by ENGINE; census probe read out of the artifact by MEASURE	

#	item	note	
	<code>curHP</code> reaches the same final HP as one clamp against <code>_hpPreReact</code> because nothing else moves the user's HP between the rows; the two census probes that guard the NUMBER were green either side and were not edited. 2 narration games cleared. VERIFIED BY: <code>node tests/test-mechanics.js --move/drain "a SPREAD drain heals at each target's own damage line, naming that target"</code>, live in the census, shown RED with the existing knob <code>MEDI_DRAIN_LUMP_ROUND=1</code> , with TWO controls that defeat two different wrong fixes: a single-target Draining Kiss must still read one damage and one heal, and a spread move with NO drain must emit no heal at all. Account: <code>docs/_reports/2026-08-24-narration-timing.md §2</code>		
#436	MIRROR ARMOR SENT THE STAT DROP BACK AND SAID NOTHING ABOUT IT — CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. The reflection itself was already correct. The authority prints <code>`\</code>	-ability\<code>`\</code>	BODY\<code>`\</code>
#437	THE ENTRY PASS RANKED ARRIVING BODIES BY INPUT ORDER ON AN EXACT SPEED TIE, AND THAT IS WHAT "INTIMIDATE BEFORE DRIZZLE" WAS — CLOSED 2026-08-24 BY ENGINE, CHECKED BY MEASURE. <code>runSwitch</code> coalesces the queued switches and <code>speedSort(getAllActive(true))</code> ranks every active body, including ones that are not entering, and only then are the handlers sorted by (<code>onSwitchInPriority</code> DESC, rank ASC). <code>speedSort</code> is a SELECTION SORT whose swaps move UNTIED bodies past a tied pair — a permutation <code>Array.prototype.sort</code> cannot produce from any comparator. <code>Pokemon</code> carries no <code>effectOrder</code> field, so <code>comparePriority</code> on a bare body reduces to speed alone; that was checked rather than assumed, because a fifth sort key would have made the tie unreachable. MEASURED IN THE OFFICIAL SIMULATOR FIRST AND READ AS AN OUTCOME RATHER THAN AS A MESSAGE: four leads at 137 / X / 112 / 123 with Drizzle second and Drought third — X = 113 (no tie) ends SUN, X = 112 (an exact tie) ends RAIN. The tie flips the sky, and the sky multiplies every attack. A 625-board sweep over every combination of four speeds with the tie sitting across the two sides: 0 of 625 disagree now, 30 of 625 under <code>MEDI_ENTRY_STABLE_SORT=1</code> , and those 30 are exactly the pool's <code>`\</code>	-unboost\<code>`\</code>	pia: Archaludon\<code>`\</code>
#438	A MOVE TRAP SURVIVES ON OUR BOARD AFTER THE AUTHORITY HAS RELEASED IT, AND BOTH NARRATION STREAMS AGREE — FOUND 2026-08-24 BY ENGINE WHILE CLOSING THE BOUNCED-MOVE LINE, FILED BY MEASURE. The Magic Bounce scenario parts on the BOARD at turn 4: <code>p2a feraligatr vol.trapped</code> reads showdown 0 / us 1, with the two protocol streams in agreement — so it is a STATE defect that the protocol arm structurally cannot see and only the board arm reports. It was already carried in the published artifact as <code>verdict: STATE</code> , so it is not a regression; it simply had no register row. WHAT IT COSTS IS NOT COSMETIC: a body we believe is trapped cannot switch in this engine and can in the game, so it changes what a search believes is LEGAL, not merely what the log says. NOT INVESTIGATED — the duration rule, the release point, and whether the trap is held by the move or by the bouncer are all unread. MEASURE DID NOT RE-VERIFY THE ARTIFACT READING TONIGHT AND SAYS SO: <code>data/all-mechanics-fire.json</code> was being rewritten by a live ENGINE run at the time, and reading an artifact mid-write produces a plausible, well-formed, completely fictitious answer. WHAT WOULD DECIDE IT: stage a Block or a Mean Look at a Magic Bounce body, read <code>vol.trapped</code> on both engines at the end of every turn and find the turn they part, with the bouncer's other legal ability as the control. VERIFIED BY: <code>node engine/all_mechanics_fire.js --kind abilities --the reflectsStatusMoves row's board arm RE-READ 2026-08-24 AFTER THE TREE SETTLED, AND THE FINDING IS CONFIRMED BY MEASURE ON THE COMMITTED ARTIFACT.</code> The caveat above stood for about an hour. <code>data/all-mechanics-fire.json</code> (release <code>b35e96a0e7c7</code> , arm <code>bottom-tie-first</code> , generated 2026-08-24T22:01:48Z) carries the <code>magicbounce</code> row at <code>rows/abilities</code> , carrier espeon, control	open — engine DEFECT (a trap outliving the authority's), STATE only and invisible to the protocol arm; CONFIRMED BY MEASURE 2026-08-24 on the committed artifact at release <code>b35e96a0e7c7</code> , one diff, <code>vol.trapped us 1 / showdown 0</code> at turn 4. Unfixed and uninvestigated. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: open — engine DEFECT (a trap outliving the authority's), STATE only and invisible to the protocol arm; reported by ENGINE 2026-08-24 and filed by MEASURE, whose read of the artifact was NOT taken because a live run was rewriting it	

#	item	note	
	Synchronize, verdict: FIRED , diverged: false — the protocol streams agree — and its board arm reads verdict: "STATE" , state_parted_without_a_line: true , state_parted_on_turn: 4 , with one diff: p2.active[0].vol.trapped , body feraligatr , us 1 against showdown o, bucket off-by-one . So the row's claim is not ENGINE's say-so any more: it is read out of the settled artifact by the division that owns whether a number is true.		
#439	**A KO'D SPREAD TARGET'S ``\	faint\	IS ANNOUNCED BELOW TH IT ABOVE — FOUND 2026- Matcha Gotcha spread t
#440	**A PERISH DEATH'S ``\	faint\	IS ONE LINE EARLY WHEI RESIDUAL — FOUND 2026- REMAINDER OF #417 AND gone and the faint is
#441	A SINGLE-GAME REPLAY IS NOT REPRODUCIBLE IN ISOLATION, SO EVERY ISOLATED DEBUGGING SESSION IN THIS REPO IS QUIETLY UNRELIABLE — FOUND 2026-08-24 BY ENGINE, FILED BY MEASURE. Replaying one diverging pair on its own gives a DIFFERENT GAME from the same pair played in sequence inside the 961-game run, and the difference is not small: it changes which turn the game diverges on, and sometimes whether it diverges at all. Measured twice inside one batch — the phazing game diverged at turn 3 as the 11th game in a process and at turn 1 played alone, on a mega-order speed tie between two identical Aerodactyls; the redirect game diverged at turn 11 in sequence and at turn 10 on a different cause alone. THE CAUSE IS NAMED: the tie key and the mid-battle address log are PROCESS-SCOPED in a way the per-game reset does not fully clear, so a game's dice depend on how many games ran before it. IT WAS SHOWN NOT TO BE ANY ONE BATCH'S DOING — reproduced with engine/medicham2-browser.js reverted to HEAD in the same isolated position. THE CONSEQUENCE IS THE PART THAT MATTERS: a single-game replay is a fine way to READ a divergence and a BAD way to prove one was fixed; only the full run played in the same order on both arms is an attribution. Nothing published has been shown wrong because of it —	open — instrument DEFECT (process-scoped state leaking between games in one process); found by ENGINE 2026-08-24 and filed by MEASURE. Blocks nothing today and makes every isolated repro untrustworthy. INSTRUMENT OWED	

#	item	note
	the standing figures all come from the sequenced run — but every <code>replay_one.js</code> session is affected. WHAT WOULD DECIDE IT: play one pair alone and at position N of a run with both engines' per-game reset instrumented and assert the first divergence is identical; then reset the tie key and the address log per game and assert it again. INSTRUMENT OWED	
#442	A CENSUS WAS COMMITTED CLAIMING 674 WORKING MECHANICS WHILE THE COMMITTED ENGINE GAVE 673 — A PROBE WAS PUBLISHED WITHOUT THE ONE-LINE ENGINE EDIT IT DESCRIBES. FOUND AND REPAIRED 2026-08-24 BY ENGINE; FILED OPEN BY MEASURE BECAUSE NOTHING IN THE REPOSITORY WOULD HAVE CAUGHT IT. <code>data/mechanics-census.json</code> was committed at HEAD reading 674 live / 674 probed / 0 missing and HEAD's own tree measured 673 live / 1 missing; the missing row was the Charge re-bank (#434). The probe was committed and the engine edit was not. It was found by stashing the following pass's test edits and re-running BEFORE anything else was written, and it was re-applied the same day. THIS IS NOT A GAME DEFECT AND IT IS NOT COSMETIC: the census IS the mechanics-live number, it is one of the MEDICHAM gate's clauses, and a published census that its own tree cannot reproduce is the exact shape this project keeps paying for — a figure that looks measured and is really a record of an earlier state of the world. A SECOND INSTANCE THE SAME DAY, FROM THE OTHER DIRECTION: running the census under <code>MEDI_DRAIN_LUMP_ROUND=1</code> to demonstrate a red WRITES the artifact, and <code>status.js</code> then read 674 of 677 with 3 missing until it was re-run clean. A knob run is not a measurement and its artifact must not be left standing. WHAT WOULD DECIDE IT: regenerate the census from the COMMITTED tree and compare it to the COMMITTED artifact, failing on any difference — cheap, because the census runs in seconds, and it is the only thing that can see a probe landing without its engine edit. INSTRUMENT OWED	open — instrument DEFECT (a published artifact its own tree does not reproduce); MEASURE owns it. Twice on 2026-08-24, both repaired the same day, neither by a check. INSTRUMENT OWED
#443	A DELIBERATE-BREAK PLANT WAS ANCHORED ON A LINE. A LATER FIX DELETED, SO A CORRECT INSTRUMENT WENT RED WITHOUT THE ENGINE BEING WRONG — FOUND AND FIXED 2026-08-24 BY ENGINE, RECORDED BY MEASURE. <code>tests/test-resolution-order.js</code> breaks the engine in twenty-six small, named ways and checks that each break makes the right thing go wrong. Its <code>entry-sort-speed-only</code> revert anchored on <code>_arrived.sort((x,y)=>compareEntryOrder(x,y,field));</code> — a line the entry-order fix (#437) deletes — so the plant matched 0 times and the file reported <code>PLANT FAILED ... anchor matched 0 times</code>. The safety net was doing exactly its job, and the interrupted agent whose work caused it never noticed, which is the whole reason that work was re-checked as a stranger's. It was re-anchored onto <code>entryOrder</code>'s priority key, the one comparator both entry sites reach, and the file is green with the arm that must fail still failing and the arm that must not still passing. THE SHAPE IS WHAT IS WORTH KEEPING: a surgical-revert harness is COUPLED to the source text it reverts, so a correct fix can turn a correct instrument red, and that red looks exactly like a broken engine. Nothing here proposes a gate on a gate; it is recorded so the next red of this shape is diagnosed in a minute instead of an hour. Account: <code>docs/_reports/2026-08-24-entry-order.md</code> §1	closed 2026-08-24 — instrument fragility (a revert plant coupled to source text), fixed by ENGINE the same night and recorded by MEASURE. Not an engine defect and never was
#444	AN ENTRY-HANDLER ANNOUNCEMENT IN MEDICHAM READS <code>mon.ability</code> DIRECTLY, SO A SUPPRESSED ABILITY WOULD STILL NARRATE — NAMED BY ENGINE 2026-08-26 WHEN SUPREME OVERLORD'S THREE LINES WERE WIRED. <code>fallenShown()</code> (Supreme Overlord's <code>-activate / -start fallenN / -end fallenN</code>) and the Zero to Hero forme announcement a few lines above it in <code>bringIn</code> both test the ability field with no suppression predicate, where the authority runs <code>onStart</code> through the event system and Neutralizing Gas removes it. NOT A MEASURED DIVERGENCE AND IT IS NOT CLAIMED AS ONE: no probe fails on it and the pinned pool holds zero games where a suppressor and a <code>boostsFromFallen</code> carrier are on the field together, so this is a hole in the reasoning rather than a number. It is filed because it was found while writing the fix, and a gap noticed and not recorded is the shape this register exists for. WHAT WOULD DECIDE IT: stage a Supreme Overlord entering opposite a Neutralizing Gas body in both engines and compare the entry stream, with the same board minus the suppressor as the control; then decide whether ONE shared entry-handler suppression predicate is worth building, which is the real question — two call sites is not yet a table. VERIFIED BY: <code>node tests/test-mechanics.js</code> (<code>ability boostsFromFallen</code>, four rows) once such an arm exists	open — engine GAP (an announcement that ignores ability suppression), narration-only, ZERO measured games; named by ENGINE 2026-08-26 in the pass that landed the narration, deliberately not fixed in it
#445	A PIERCING ABILITY WAS REFUSED BY A WIDE GUARD BECAUSE <code>guardRefusalOf</code> IMPLEMENTED ONLY CLAUSE 1 OF <code>checkMoveBypassesProtect</code> — FOUND 2026-08-25 BY ENGINE AND DELIBERATELY NOT FIXED IN THAT PASS, CLOSED 2026-08-26 BY ENGINE. The authority's function has two live clauses: <code>move.flags['protect']</code> (clause 1, which this engine had as the <code>ignoresProtect</code> tag — the Feint door) and <code>runEvent('HitProtect', attacker, defender, move)</code> (clause 2), which is the hook Champions' Unseen Fist and Piercing Drill both answer with byte-identical <code>onHitProtect</code> handlers. Wide	closed 2026-08-26 — engine DEFECT (a missing clause of <code>checkMoveBypassesProtect</code> at both side-guard doors), fixed and probed by ENGINE. Release <code>7fc604e5bc44</code> ; <code>docs/ENGINE.md</code>

#	item	note
	Guard's and Quick Guard's conditions call the SAME function the mon-level shield calls, so the shield had pierced correctly since WIRE 158 while the SIDE CONDITION refused every pierce — the mechanic fired, at one of its two doors, which is why nothing saw it. MEASURED IN THE OFFICIAL SIMULATOR FIRST: a Gliscor's Breaking Swipe into a Wide Guard deals 5 and 9 HP with -1 Atk on both bodies, writes an <code>-ability + -zbroken</code> pair per target and <code>NO - activate</code>; this engine dealt 0, the same as the no-ability control, which is the unwired-knob signature. THE FIX IS ONE FUNCTION AND BOTH DOORS: the inline <code>_pierceP</code> IIFE became <code>hitProtectBypass(attacker, moveId, count)</code> with three readers (damage path, <code>guardRefusalOf</code>, the chooser's threat scan); <code>guardRefusalOf</code> returns a third answer <code>'pierced'</code> that every existing caller already reads as "not refused"; <code>sideGuardRefuses</code> reports it at the spread door AND the non-spread Quick Guard door, because wiring only the first would have let a pierced Fake Out through at FULL damage. <code>_guardPierced</code> carries the authority's <code>bypassProtect</code> to the quarter, the FINAL damage modifier in <code>champions/scripts.ts:299</code>. Clause <code>3 (isZOrMaxPowered)</code> is named in the code as unreachable in gen 9 rather than left to be found. The breaksProtect Wide Guard probe was ASSERTING the zero and is corrected in the same pass. Census 715 -> 716; gate clause 16 of 961 unmoved and board-material 13 of 961 unmoved, both PREDICTED before the run — the conjunction needs a guard up, one of two piercing abilities (one legal carrier each, both mega formes) and a contact move of the guard's class on the same turn. VERIFIED BY: <code>node tests/test-mechanics.js --ability/piercesProtect</code> <i>"a piercing ability walks through a WIDE GUARD at a quarter — its ally does not"</i>, live in the census, shown RED first and red on demand under <code>MEDI_GUARD_NO_HITPROTECT=1</code>	
#446	<code>tests/test-resolution-order.js</code> DIES OOM AT NODE'S DEFAULT HEAP, AND HAS BEEN RED FOR LONGER THAN ANY ONE BATCH — FOUND 2026-08-26 BY ENGINE. The file exits 134 with <code>FATAL ERROR: Reached heap limit Allocation failed after printing ~17 release store OVERRIDDEN lines. IT IS NOT THIS BATCH'S</code>, proved rather than argued: <code>git show HEAD:engine/medicham2-browser.js</code> was put in place and the file re-run — same exit 134, same message — and it also OOMs with the batch's own behaviour knob switched off. THE ENGINE IS FINE: at <code>--max-old-space-size=6144</code> it PASSES, 26 arms staged, 1 KNOWN-OPEN (the declared multi-hit granularity arm), o failing. THE CAUSE IS NAMED AND NOT FIXED: the file opens a release per arm and each snapshot loads the frozen engine plus its lazily-read data files, so ~26 of them are resident at once. WHY IT MATTERS BEYOND THE RED: every gate in this repo is read as an exit code, <code>tests/run-all.js</code> spawns children without a heap flag, and an instrument that dies on memory is indistinguishable at the exit code from an instrument that found a defect — which is this project's signature failure wearing the other hat. WHAT WOULD DECIDE IT: open ONE release for the whole file (the arms already share an id) or release each snapshot after its arm, then re-run at the default heap and assert exit 0 with the same 26/1/o line. THE RUNNER HALF IS CLOSED 2026-08-27 BY MEASURE AND THE LEAK IS NOT, AND THE TWO ARE DELIBERATELY NOT MERGED. This row named two defects in one paragraph: a per-arm snapshot leak in the check, and <i>"tests/run-all.js spawns children without a heap flag, and an instrument that dies on memory is indistinguishable at the exit code from an instrument that found a defect"</i>. The second is the runner's and is fixed: a check <code>DECLARES ABRA-HEAP: <MB></code> in its own header, <code>plan()</code> derives it and passes <code>--max-old-space-size</code> before the script path, and <code>--list</code> prints the honoured value so the derivation can be seen to have fired. It is derived from the child rather than tabled in the runner for the reason the CI job list was replaced — a table of costs goes stale and the staleness surfaces as exit 134 on a machine nobody is watching. AND AN OOM STILL FAILS: it is annotated <code>OUT OF HEAP</code>, never downgraded to a SKIP, because a crash filed as "did not run" is the same laundering as a crash filed as a verdict. Verified by capturing the <code>REAL</code> argv of all 165 children with <code>spawnSync</code> stubbed — exactly one carries the flag — and by measuring that the flag moves the ceiling 2,240 MB → 6,336 MB in that position and does nothing at all after the script path. THE LEAK IS UNTOUCHED AND THIS ROW STAYS OPEN ON IT: nothing was done to the 26 resident snapshots, and MEASURE did not re-run the file (it plays games and ENGINE held the play layer), so the 26/1/o line is ENGINE's measurement of 2026-08-26 and not a fresh one. INSTRUMENT OWED	open — instrument DEFECT (a per-arm release snapshot leak, not an engine defect); found by ENGINE 2026-08-26 and shown red identically at HEAD. Passes at a raised heap, so nothing about the game is claimed wrong here. INSTRUMENT OWED
#447	A SPREAD MOVE'S SECONDARY WAS ADDRESSED TO THE BODY IT HIT, AND THE AUTHORITY ADDRESSES IT TO THE LAST BODY OF THE SPREAD — FOUND AND CLOSED 2026-08-26 BY ENGINE. THE TWO BOARD-MATERIAL GAMES IT COST WERE NEVER ENGINE DEFECTS; THEY WERE THE RULER. <code>BattleActions#secondaries (sim/battle-actions.ts:1336-1351)</code> loops its targets and NEVER writes <code>this.battle.activeTarget</code>; the last writer is <code>getSpreadDamage</code>'s own per-target loop (<code>:1154</code>), which leaves it on the LAST body that reached the damage step. So the authority addresses	o p21\ 1 against medicham p21

#	item	note
	EVERY roll of a spread secondary to that one body and separates them with <code>nth</code> — and a secondary that FIRES moves it, because <code>moveHit</code> re-enters <code>spreadMoveHit</code>, whose step 1 writes <code>activeTarget = target</code>. The middle arm of <code>engine/game_differential.js</code> shares its dice by ADDRESS, so two engines that both roll a correct 30% per target still get DIFFERENT NUMBERS when their addresses differ, and the differential reports a divergence that is entirely the instrument's. MEASURED IN THE AUTHORITY BEFORE A BYTE MOVED, one staged doubles turn each, two unshielded foes: Rock Slide (30%, neither fires) `showdown p21\`	

#	item	note	
#448	THE STATUS-MOVE SPREAD BRANCH RESOLVED ONE TARGET AT A TIME WHERE THE AUTHORITY RUNS EACH STEP ACROSS EVERY TARGET — CLOSED 2026-08-27 BY ENGINE. FOUR OF THE FIVE ROWS CLOSED; THE MECHANICS CLAUSE WENT 8 OF 16 -> 5 OF 12. <code>Battle.actions.trySpreadMoveHit (sim/battle-actions.ts:550-577)</code> declares <code>moveSteps</code> as DATA and runs each STEP across the whole target array before the next begins, so <code>hitStepTryHitEvent</code> finishes on every body before any effect lands. ROADMAP #81 WIRE 10 gave the DAMAGING branch that shape (<code>for (step) for (row)</code>) and the <code>a.kind==='affect'</code> branch was never converted — its own header said <i>"EVERY TARGET RUNS THE WHOLE GAUNTLET ON ITS OWN"</i>. THE FIX IS A TRANSPOSITION AND NOTHING ELSE: the per-target sequence is byte-for-byte the sequence this branch already ran, cut at the authority's own step boundaries into six closures over a row {<code>tg, out</code>} (<code>_asGone</code> , <code>_asTryHit</code> , <code>_asTypeImm</code> , <code>_asTryImm</code> , <code>_asAccuracy</code> , <code>_asEffects</code>); twelve <code>continue</code> s became <code>R.out = true</code>; <code>return</code>; and six inner effect-loop <code>continue</code> s were deliberately left alone. At one target the two orders are the same permutation, which is why every single-target status move is untouched — asserted by two control arms, not assumed. MEASURED: diverging MOVE rows of <code>data/all-mechanics-fire.json</code> 14 -> 10 on release <code>ee4e537b7255</code> — <code>cottonspore</code> , <code>stringshot</code> , <code>sweetscent</code> and <code>teeterdance</code> all read NO-DIVERGENCE; census unmoved at 754/754/0; the three roster stages o DIFFER / o DID-NOT-FIRE; the pinned 961-game differential unmoved at 9 played divergences and board-material 1, which is exactly what this row predicted (zero games in the pinned pool). THE FIFTH ROW IS STILL OPEN AND IT IS NOT THIS BRANCH: <code>corrosivegas</code> is classified <code>trickitem</code> by <code>playerAction</code> , so it never arrives here at all — 1 click in 64,846 stored games, REACH-excluded from the clause, and it needs its own change at its own site. ALSO NAMED AND NOT DONE: Substitute stays in the TryHit group where this branch has always asked it, not below accuracy where the authority asks it (<code>onTryPrimaryHit</code>) — a second divergence at a different step that nothing measures today. VERIFIED BY: <code>node -r ./tests/_live_release.js tests/probe_spread_status_steps.js</code> — three red arms (Cotton Spore, String Shot, Teeter Dance) shown RED before the fix, now agreeing clean and parting under <code>MEDI_SPREAD_STATUS_PER_TARGET=1</code> , with three controls holding under both and the engine counters asserted as an exact per-game delta so a knob that missed the driver's module cannot read green	CLOSED 2026-08-27 — landed on release <code>ee4e537b7255</code> , CHANGELOG 5.168.0, full account <code>docs/_reports/2026-08-27-spread-status.md</code> . <code>corrosivegas</code> carried forward as its own row of work at the <code>trickitem</code> site	
#449	<code>tests/probe_red_demo.js</code> EXITS 1 WITH 20 OF 200 DEMONSTRATIONS FAILING, 18 OF THEM STALE REVERSALS, AND IT IS RED AT HEAD — MEASURED 2026-08-26 BY ENGINE. A red demo is not a status and this row is what is being done about it instead of filing it. IT IS NOT ANY ONE BATCH'S, proved rather than argued: <code>git stash push --engine/medicham2-browser.js</code> and a re-run returns the IDENTICAL line — <code>200 demonstrations, 20 failed, 18 of those are STALE REVERSALS</code> — so the spread-secondary pass that measured it staled nothing. The file itself already separates the two kinds and prints the anchor each patch could not find: 18 are STALE REVERSALS (<i>"the demonstration is fine, the patch it applies is not"</i> — a later wire rewrote the source text the surgical revert is coupled to), spanning WIRE 118, 123, 129, 130 x2, ROADMAP #31 x4, #81 x2, #81 WIRE 2, WIRE 7 x4, WIRE 12 and #259; 1 is worse than stale — ROADMAP #81 WIRE 9's <i>"a MISS names its targets"</i> revert APPLIES and produces an engine that cannot play a turn (<code>_mvMissed</code> is not defined), so it demonstrates nothing while looking like it ran. THIS IS #443'S SHAPE AT SCALE: that row closed ONE re-anchoring and recorded the lesson that a surgical-revert harness is COUPLED to the source text it reverts, so a correct fix turns a correct instrument red. Eighteen of them have accumulated since. WHY IT MATTERS: every one of those wires is currently defended by a probe that has NOT been shown to watch its own knob, which is precisely the guarantee <code>probe_red_demo.js</code> exists to provide — the file says so itself: <i>"A green-and-stripped pair that did not flip means the probe does NOT watch its knob."</i> WHAT WOULD DECIDE IT: re-aim each reversal at what the engine says today and require <code>node tests/probe_red_demo.js</code> to exit 0, with the WIRE 9 reversal fixed to produce a runnable engine rather than a throw. VERIFIED BY: <code>node tests/probe_red_demo.js</code> — exit 0, <code>200 demonstrations, 0 failed</code>	CLOSED 2026-08-26 BY ENGINE, ON THE INSTRUMENT'S OWN DECLARED EXIT CODE. <code>node tests/probe_red_demo.js</code> reads 200 demonstrations: 0 HOLLOW, 0 COULD NOT BE APPLIED, 2 not in this format, exit 0, and it now DECLARES that code — <code>ABRA-EXIT 0 VERDICT-GREEN</code> on <code>stderr</code>. <code>engine/medicham2-browser.js</code> was not edited at all in the pass, so no game number could move and none did. THE TWO DEFECTS WERE SEPARATE AND BOTH ARE FIXED. (1) THE UNDECLARED EXIT: the file added a stale reversal to <code>failures</code> and exited 1, and exit 1 is the universal <i>"I failed"</i>, so <code>engine/register_reality.js</code> published eighteen expired certificates as a live, MEASURED simulator defect and <code>openDefectClause</code> held the MEDICHAM gate shut on it. The verdict is three states now, following <code>engine/gate_fail_and_silent.js</code> 's idiom rather than inventing a second: exit 1 VERDICT-RED for a HOLLOW row (a probe whose assertion held on the known-bad engine too), exit 2 CANNOT-ANSWER when nothing is hollow but N demonstrations COULD NOT BE APPLIED, exit 0 otherwise — and the count of demonstrations that could not be applied is printed on EVERY run, green included, so this cannot recur silently. A reverted arm that THROWS is counted with the could-not-be-applied rows and a SHIPPED arm that throws stays a red, because those are two different facts. (2) THE 20 ROWS: 17 patterns re-aimed, WIRE 9's throw diagnosed as a BROKEN PATCH rather than engine fragility (it referred to <code>_mvMissed</code> , the pre-WIRE-9 say-it-once latch that WIRE 9 deleted along with its reader; the shipped engine neither declares nor reads that name, and the latch is restored on the action object so the reverted engine writes ONE body-less `	-miss

#	item	note
#450	THE MIDDLE ARM'S CATEGORY WRAPPER WROTE INTO A MODULE INSTANCE THAT HAD BEEN EVICTED, SO EVERY STAGED-BOARD INSTRUMENT AFTER ITS FIRST SOURCE SWAP WAS COMPARING TWO ENGINES ON UNSHARED DICE — FOUND AND CLOSED 2026-08-26 BY ENGINE. <code>midWrapShowdown</code> patches <code>BattleActions.prototype</code>, which comes out of SHOWDOWN's require cache and is never reloaded; <code>engine/game_differential.js</code> comes out of ours, and <code>tests/staged_board.js</code>'s <code>harness()</code> DELETES it on every change of simulator source. On the second load the guard <code>if (BattleActions.__midwrapped) return;</code> fired, the wrapper was not reinstalled, and the standing closure kept writing <code>MID_CAT</code> into the dead instance. THE LIVE INSTANCE READ 'any' FOR THE REST OF THE PROCESS. Two things went at once and the second is damage: every authority draw was addressed <code>`<seed></code>	<turn> any

#	item	note	
#451	TWO REFUSAL RULES EXISTED IN MACHINE-READABLE FORM IN data/tags.json AND NEITHER HAD A CONSUMER — FOUND FROM WILL'S DIVERGENCE CARDS, CLOSED 2026-08-26 BY ENGINE. He read every card and named the same disease twice: a move the authority REFUSES and we APPLY. One family, two doors. (A) THE MOVE'S OWN onTryImmunity . immunityGate has been derived since 2026-08-22 for six moves, carrying the authority's handler source, a machine-readable CONDITION, the announced event (-immune), its attribution (null — a BARE line on the TARGET) and the step (3, above hitStepAccuracy); tests/test-immunity-gate.js has been proving that condition predicts the official simulator across twelve staged arms the whole time. engine/medicham2-browser.js contained the string immunityGate ZERO times — grepped, not assumed. The cost was not uniform, which is why there are six probes and not one, measured one staged turn each before a line changed: Endeavor (133 uses, pokemon.hp < target.hp — RAW HP, exactly as Will guessed) **dealt the damage anyway and wrote `	-damage\	p2a: garchomp\

#	item	note
#452	GRAVITY BROUGHT DOWN EVERY SEMI-INVULNERABLE CHARGE AND THE AUTHORITY BRINGS DOWN TWO — FOUND FROM THE 2026-08-26 DIVERGENCE DUMP, CLOSED 2026-08-26 BY ENGINE. <code>gravity.condition.onFieldStart</code> names what it grounds one body at a time and every name is AIRBORNE: <code>`removeVolatile('bounce')`</code> \	\ <code>removeVolatile('fly')</code> (ther <code>removeVolatile(twoturnmov</code> if (applies) this.add('activat browser.js matched on tl different set and a bi Force all hide the use use the over-match was Fly 8 + Bounce 2, roug line already read <code>**a</code> down and read past. <code>** engine/replay_one.js</code> on j 2655381344 , REPRODUCED Dragapult charges Phan authority leaves the c Metagross, while here and the Metagross live HANDLER, NOT LISTED**: <code>onFieldStart</code> and keeps c is not a move and skyd leaving by the same lo fly], volatiles: [magnetrise], c wired. <code>queue.cancelMove</code> next board: without it <code>**MEASURED, PINNED AND middle , cap 12, data/te baseline used): board-games remaining at the <code>**17 -> 16**, whole-ga tests/test-engine-diff.js</code> unr probes in <code>tests/test-mecl over-fire control in t under MEDL_GRAVITY_(</code> exactly those two rows MISS</code>
#453	TWO VOLATILES WITH DIFFERENT LIFETIMES WERE LIVING IN ONE FLAG — FOUND FROM THE PREVIOUS BATCH'S OWN HANDOVER, CLOSED 2026-08-26 BY ENGINE. The authority keeps a two-turn move in a PAIR. <code>twoturnmove (data/conditions.ts:287)</code> carries <code>duration: 2</code> and no handler of any kind, so <code>Battle#fieldEvent('Residual')</code> collects it on the duration alone and <code>handler.state.duration--</code> ends it at zero; it is what LOCKS the user in. Its <code>onStart</code> then adds a SECOND volatile named for the move itself(<code>attacker.addVolatile(effect.id)</code>) and that is what makes the user semi-invulnerable — and every charge move's <code>onTryMove</code> opens if (<code>attacker.removeVolatile(move.id)</code>) return; (<code>data/moves.ts:17228</code> <code>solarbeam</code>, identical on all ten), so the sub-volatile dies AT EXECUTION and the wrapper does not. <code>engine/medicham2-browser.js</code> had one <code>_charging</code> doing both jobs and cleared it on the release line, so <code>vol.charging</code> read 0 for the rest of every release turn where the authority reads 1. MEASURED ON THE AUTHORITY BEFORE A LINE CHANGED, Dragapult clicking Phantom Force with <code>Object.keys(volatiles)</code> read after every <code>runMove : charge boundary twoturnmove(d1), phantomforce(d1) ;</code> release turn after the FOE moved, both still there; release turn after DRAGAPULT moved, twoturnmove(d1) ALONE; release boundary, none. IT IS INVISIBLE AT A NORMAL BOUNDARY AND THAT IS WHY IT SURVIVED — both engines read 0 once the residual has run, so it shows only when the residual NEVER runs, which is a battle that ended mid-turn: <code>pair-protect-bust gen9championsvgc2026regmbo3-2657789498</code> turn 11 and ...-2660356793 turn 12, both <code>active[0].vol.charging</code> <code>medicham o / showdown 1</code>. THE NAIVE FIX IS WRONG AND THE SECOND PROBE IS THE TRAP IT WALKS INTO: simply deferring <code>_charging</code> would leave the user SEMI-INVULNERABLE for the rest of the turn after it had already struck. So <code>_charging</code> keeps its exact meaning (the sub-volatile — <code>_invuln</code> , the lock, the PP question, the Gravity cancel) and a new <code>_ttmWrap</code> carries the wrapper: applied only where the authority applies it (BELOW the weather short-circuit and BELOW the <code>ChargeMove</code> event, so a skipped charge and a Power Herb get none), spent once per residual beside the Ally Switch and sealing-volatile clocks — a position that is the whole point, because all four break _TURN sites sit above it — and cleared at switch-out, at an abandoned charge (<code>onMoveAborted</code>), at Gravity's own <code>removeVolatile('twoturnmove')</code> and at <code>queueFaint</code> , where <code>clearVolatile</code> takes the authority's. <code>engine/board_state.js</code> READS THE WRAPPER WITH _charging AS AN EXPLICIT OLD-RELEASE FALLBACK: that file is NOT one of the frozen release SOURCES, so a release cut before <code>_ttmWrap</code> existed would otherwise report 0 on every charge turn and part on a leaf that engine got right; on the live engine the wrapper is a strict superset of <code>_charging</code> , so the fallback cannot mask a live defect. <code>game_differential.js</code> 's <code>vol.charging</code> PLANT now writes both fields — a plant that flipped only <code>_charging</code> would be undetectable on any body actually mid-charge and would read as a broken comparator,	closed 2026-08-26 — ENGINE defect in <code>engine/medicham2-browser.js</code> (one flag holding two volatiles with different lifetimes), handed over diagnosed by the previous batch and fixed by ENGINE. Release <code>de0096d8f078 . docs/ENGINE.md</code>

#	item	note
	the same sentence that site already had to write once. MEASURED, PINNED AND PREDICTED BEFORE THE RUN (--games 1200 , arm middle , cap 12, data/team-pool-frozen , the same 643-row census pin the baseline used, --state --end-state): board-material 9 -> 7 of 961, whole-game clause 11 -> 11 UNMOVED as predicted (no protocol line changes), raw diverged 16 -> 16, census 731 -> 733 live, 0 missing. tests/test-engine-diff.js unmoved at 0/6000 at all sixteen corners. The active[].vol.charging family (2 games, 2 leaves) is GONE and every other board family is identical row for row; the seven remaining first-divergence records are byte-identical to the old seven. All three roster stages re-run on this release at 0 FIRED-AND-BOARDS-DIFFER and 0 DID-NOT-FIRE. A COUNTER THAT IS NaN LOOKS EXACTLY LIKE A COUNTER THAT NEVER FIRED — the six new MEDSEEN keys had to be declared in the initialiser, because an undeclared key makes MEDSEEN.x++ produce NaN, which serialises as null ; caught on the first receipt read and written into the block. VERIFIED BY: node tests/test-mechanics.js — the clock probe reads the LEAF engine/board_state.js compares rather than the private field (a probe on _ttmWrap would go green on an engine whose reader never saw it), on three arms — a mid-turn ending where the release wipes the last foe (must read 1), a normal boundary where the residual ran (must read 0, the over-fire control against an engine that never drops the clock), and the same mid-turn ending with a Shadow Ball instead of a charge (must read 0) — shown RED first and red on demand under MEDI_CHARGE_WRAP_CLEARED_AT_EXECUTION=1 (census 733 -> 732 with exactly that row MISSING); and the trap probe is an HP measurement on the same board, the Flare Blitz aimed at the Dragapult dealing 0 on the charge turn and 33 on the release turn after the strike, with Solar Beam (which does not leave the field) at 33/33 as the opposite control	
#454	DISABLE SEALED A MOVE WITH NO PP LEFT AND A STRUGGLE, AND THE AUTHORITY REFUSES BOTH — FOUND FROM WILL'S OWN BOARD-MATERIAL LIST, CLOSED 2026-08-26 BY ENGINE. The authority refuses a Disable twice. disable.onTryHit (data/moves.ts:3659) returns false on `!target.lastMove	target.lastMove.isZOrMaxPo

#	item	note	
#455	FORECAST MOVED CASTFORM'S TYPES AND NOT ITS SPECIES LABEL, AND THE COMMENT SAYING IT COULD NOT WAS FOUR DAYS STALE — FOUND FROM WILL'S OWN BOARD-MATERIAL LIST, CLOSED 2026-08-26 BY ENGINE. <code>data/abilities.ts</code> forecast calls <code>pokemon.formeChange(forme, this.effect, false, '0', '[msg]')</code> and guards its own re-entry on <code>pokemon.species.id !== 'castformrainy'</code> — the authority's question is WHICH SPECIES AM I, and this engine's was WHICH TYPES DO I HAVE. <code>data/mods/champions/abilities.ts</code> carries no <code>forecast</code> key, so mainline's is what this format runs, checked rather than assumed. THE BLOCK WAS DEAD AND THE COMMENT OUTLIVED IT. <code>syncWeatherFormes</code> refused to rename on the written reason "<code>data/engine-data.js</code> holds a row for <code>castform</code> and NONE for <code>Castform-Sunny</code>, <code>Castform-Rainy</code> or <code>Castform-Snowy</code> ... Adding the three rows is a change to <code>data/engine-data.js</code> and belongs to a refit, not here", and it counted <code>formeWeatherNameUnchanged</code> on every application so the shortfall was loud. Commit <code>f15bf80a</code> (2026-08-22) had already added all three rows: <code>9a060821</code> has zero matches for <code>castform-rainy</code> and HEAD has one, and <code>buildMon</code> resolves every spelling today. SO THE TWO CASTFORM GAMES WERE NEVER BLOCKED ON A REGENERATION OF <code>data/engine-data.js</code> , WHICH IS THE FACT WILL'S PENDING DECISION TURNS ON. AND THE HANDED HYPOTHESIS WAS WRONG ABOUT WHICH GAMES IT COVERS. The batch was routed as "<i>the party map is keyed by DISPLAYED species, potentially 3 of the 6</i>". It is not: re-keying <code>board_state.js</code>'s <code>partyMap</code> fixes ONE game (the transformed Ditto colliding with the Garchomp it copied) and neither Castform game, because those carry an independent <code>p2.active[0].species</code> divergence — <code>castform</code> against <code>castformrainy</code> — that no comparator change can close. Fixing the ENGINE closed both Castform games AND their party-key symptom at once. THE PARTY RE-KEY IS HELD, WITH THE CREDIT SHIFT MEASURED RATHER THAN ARGUED: <code>game_differential.js</code> credits coverage from <code>BS.compare(prev, cur)</code> and buckets by <code>BS.family(path)</code> , where a species-keyed party reports a transform as <code>party.MISSING-OR-EXTRA-MEMBER</code> — which <code>BOARD_FAMILY</code> maps to the <code>fainted</code> family — and an identity-keyed one would report <code>boosts</code> and <code>species</code> instead. That is a different credit, the credit steers <code>covWant</code> , and <code>covWant</code> selects the sample, so the corrected count would be a number about a different run. Filed, not smuggled in. TWO HALVES LANDED TOGETHER, DELIBERATELY. The rename in <code>syncWeatherFormes</code> goes through <code>pasteKey</code> (the file's own door into the mon table) and is a RELABEL, not a <code>formeSwap</code> : the tag's own <code>sameStats: true</code> means there is no stat line to rebase, and <code>formeSwap</code> would emit a second protocol line. <code>_ident</code> is untouched, so every later line still names the original, as the authority's `	-formechange	p2a: Castform
#456	TELEPATHY ANNOUNCES -immune WHERE THE AUTHORITY ANNOUNCES -activate , AND NO BOARD COMPARISON CAN SEE IT — STAGED 2026-08-26 BY ENGINE FROM WILL'S OWN FIXTURE LIST, DELIBERATELY NOT FIXED. Divergence card 16. <code>data/abilities.ts</code> telepathy: <code>if (target !== source && target.isAlly(source) && move.category !== 'Status') { this.add('-activate', target, 'ability: Telepathy'); return null; }</code> — <code>data/mods/champions/abilities.ts</code> carries no <code>telepathy</code> key, so mainline's handler is what this format runs. Measured in the official simulator, Milotic's Surf across a Gardevoir partner: ``\	move\	p1a: Milotic\

#	item	note	
#457	PSYCH UP CHANGES THE STATS CORRECTLY AND ANNOUNCES NOTHING — STAGED 2026-08-26 BY ENGINE FROM WILL'S OWN FIXTURE LIST, DELIBERATELY NOT FIXED. Will asked for "<i>a fast delphox clicks nasty plot and a slow psych up user targets it to make sure the stats change works</i>". THE STATE IS RIGHT, measured both ways across a speed gap: Aromatisse (29 base Speed) copying a Delphox (104) that just clicked Nasty Plot takes +2 SpA, and the identical board with the copier moving FIRST copies an empty board — the control that makes the row non-vacuous, since an engine reading the boosts at end of turn agrees with the slow arm and parts on the fast one. What is missing is the LINE: the authority writes `\\	-copyboost\\	p1a: Aromatisse\\

#	item	note
#460	A CHOICE LOCK BOUND A HANDED-IN ACTION AFTER THE SCARF HAD BEEN KNOCKED OFF, BECAUSE THE ITEM RE-READ WAS A SIDE EFFECT OF BUILDING A MENU — STAGED AND FIXED 2026-08-26 BY ENGINE FROM WILL'S OWN FIXTURE. Will: <i>"test when choice scarf is knocked off if we allow a mon to click other moves"</i>. THE ANSWER WAS NO, ON THE ROAD THAT MATTERS. <code>choiceLock.onBeforeMove</code> (<code>data/conditions.ts:324</code>, no Champions override) opens <code>if (!pokemon.getItem().isChoice) {</code> <code>pokemon.removeVolatile('choicelock');</code> <code>return;</code> <code>}</code>, and this engine's <code>lockMenuMove</code> implemented exactly that — but the <code>COLLECT SITE</code> in <code>mk()</code> read <code>mon._lock</code> RAW and never called it. <code>chooseAction</code> builds a menu on the way past and <code>lockMenuMove</code> clears a dead lock as a SIDE EFFECT of that build, so the chooser road was accidentally right; a CALLER-SUPPLIED action builds no menu, and that road is every rollout candidate, every scripted differential game and every MILTANK evaluation. MEASURED, one board, every click handed in, the foe's turn-2 click the only difference: with the Scarf Knocked Off the body played <code>["swordsdance" x8]</code> and with the Scarf UNTOUCHED it played <code>["swordsdance" x8]</code> — BYTE-IDENTICAL ACROSS THE KNOB, the unwired-knob signature — while a body that had never HELD a Scarf played <code>["swordsdance", "swordsdance", "knockoff" x6]</code>. THE SPEED HALF WAS ALREADY RIGHT AND IS SAID SO PLAINLY: <code>choicescarf</code> carries <code>isChoice: true</code> AND an <code>onModifySpe</code>, <code>effSpeed</code> reads the item live off the slot, and the authority's own request shows <code>spe 120 -> 80</code> across the same Knock Off. It is now probed and was shown red by a deliberate break (caching <code>speedMult</code> on the body) that NO OTHER INSTRUMENT IN THE REPOSITORY CAUGHT — not the 6,000-row differential, not the other 745 census rows. THE FIX IS A PREDICATE, CALLED AT BOTH SITES: <code>lockStillBinds</code> carries the item re-read alone and <code>lockMenuMove</code> keeps <code>hasMove</code> on top of it, because <code>onDisableMove</code> tests <code>hasMove</code> and <code>onBeforeMove</code> does not — collapsing the two into one function cost five census rows on the first attempt and the split is what restored them. <code>MEDI_LOCK_STALE_ON_HANDED_ACTION=1</code> restores the raw read. VERIFIED BY: <code>node tests/test-mechanics.js — item / choiceLock</code>, <i>"a Knock Off frees a HANDED-IN action, not only a chosen one"</i>, LIVE, and <code>item / speedMult</code>, <i>"the Choice Scarf x1.5 dies with the item, on the turn after it leaves"</i>, LIVE. Two controls doing different work: the same Scarf holder whose foe clicks Dragon Claw instead must stay BOUND, and a body that never held a Scarf must CLICK — without the second, a red is a broken fixture rather than a lock. The same-turn boundary is asserted inside the row (turn 2 is the locked Swords Dance in BOTH arms, because Knock Off resolves after the hit)	closed 2026-08-26 — ENGINE defect in <code>engine/medicham2-browser.js</code> (the collect-site lock rewrite read the <code>_lock</code> FIELD instead of the predicate that re-reads the item). Census 743/746 -> 746 live / 749 probed / 3 missing. THE PINNED POOL DID NOT MOVE AND THE PREDICTION SAID IT WOULD — 961 games, 15 diverged on both releases, <code>first_divergences</code> and <code>classes</code> identical — measured in the PUBLISHED configuration (<code>--census data/verification/census-pin-9446a684709d.json --team-store data/team-pool-frozen --end-state</code>), with the first two attempts REFUSED by the artifact's own baseline-comparability guard because the live store had grown 8,778 -> 11,493 teams. Release <code>a98b43a5f384</code>. <code>docs/ENGINE.md</code>
#461	A SUPPRESSED CHOICE ITEM DESTROYED THE LOCK WHERE THE AUTHORITY SUSPENDS IT — STAGED AND FIXED 2026-08-26 BY ENGINE. <code>choiceLock.onDisableMove</code> has TWO escapes with OPPOSITE LIFETIMES and this engine had one: <code>`if (!getItem().isChoice</code>	<code>!hasMove(...)) { removeVolat</code> <code>(pokemon.ignoringItem()</code>

#	item	note
#462	~~ itemRoomForget IS DEAD CODE AND A KNOCK OFF LANDING INSIDE MAGIC ROOM IS SWALLOWED — THE ITEM COMES BACK WHEN THE ROOM ENDS. REPORTED 2026-08-26 BY ENGINE, DELIBERATELY NOT FIXED.~~ engine/medicham2-browser.js:15522 states in prose that a Knock Off inside a room removes an item this engine has already parked and that "the parked slot is cleared with it (itemRoomForget)". The function is declared at 15539 and is called from nowhere in the file — a declared mitigation that was never wired, which is CLAUDE.md's silent-default shape exactly. MEASURED on one board, the foe raising Magic Room on t1 and Knocking Off on t2: the AUTHORITY reads item (none) and spe 80 from t3 to the end of the battle; THIS ENGINE nops the strip (the slot is already empty, so _took is falsy), keeps _roomItem , and at t6 restores item choicescarf and spe 120. That is CLAUDE.md's named cost of an untracked Knock Off — <i>"the damage and speed calculations keep applying an Assault Vest, Choice Scarf or Life Orb that is gone"</i> — arriving through a door the item model leaves open. NOT FIXED HERE BECAUSE IT IS NOT A CLAUSE: every strip site in the file (removesItem , stealsItem , Trick, Fling, the herb spends) reads m.item , so seeing through the park is a refactor of a dozen sites and belongs in its own batch beside the standing hand-list item <i>"MAGIC ROOM PARKS THE ITEM BY EMPTYING THE SLOT, so *.item parts on every room board"</i>. THE INTERACTION IS STATED RATHER THAN LEFT TO BE DISCOVERED: with this open, a Knock Off inside a room leaves _roomItem set, so #461's suspension clause keeps the lock alive across it — the item model's divergence propagating, not a new one, because this engine believes the item came back and therefore believes the lock came back. WHAT WOULD DECIDE IT: route every strip through one site that reads `m.item	m._roomItem and calls asserting spe 80 and i **CLOSED 2026-08-26. F SITE, AND THE PREDICTE now have three answers authority's pokemon.ite m.item (what it can US on purpose, because th _hadItem (what it star leaves by, empties the the return is the gate parks on arrival into Corrosive Gas, Bug Bit Magician, Symbiosis, H ** itemRoomForget final not recalled: one real only knob — KNOCK OFF down, PROTECT reads cl IS THE SECOND CAUSE AN one foe click, and **n because the item's onl read BYTE-IDENTICAL he one knob (MEDI_ROOM two red and nothing el MEDSEEN.itemLostThrougl (a Trick inside a room if(!pokemon.item) is the 123/112 empty-handed a 76 off the same Lefttov (MEDI_EMPTY_HAND_I! ASKING THE TWO ENGINES which a room does not park does; it now take RECORDED RATHER THAN S at all** (957/961 befo the board-material gam two quantities. MEASUR data/verification/census-pin end-state , artifact rep 752 probed / 3 missing material **957 -> 958 12449; whole-game clau unmoved at 15, both sa **unmoved at 0 across change; roster 0 DIFFE e04350588dei . STILL OP refusesIfHolding gate rei: emitted). On t8 it cl: the log reads
#463	THE STALL COUNTER WAS SPENT ON A TURN THE AUTHORITY'S RESIDUAL NEVER OPENS, AND EVERY stall FIX BEFORE IT COULD ONLY REMOVE A COUNTER WE WERE WRONGLY HOLDING — STAGED AND FIXED 2026-08-26 BY ENGINE. The last board-material stall game in the pinned pool pointed the OTHER WAY: baseline turn 8 p2.active[1].stall medicham 0 / showdown 3 , seeds gen9championsvgc2026regmbbo3-2662815123 vs -2662930043 , protocol never parted. IT IS NOT AN ARMING FAILURE AND	upkeep

#	item	note
	THE BRIEF SAID IT WAS. Replayed rather than reasoned about: that game's Garchomp arms the counter on t7 and BOTH engines read 3 there (showdown stall = 3, duration 1, `	
#464	ONE DECLARED LIST HAD ONE READER, SO THE SAME DECLARATION EXEMPTED A ROW IN THE WHOLE-GAME CLAUSE AND WAS COUNTED AS A DEFECT BY THE MECHANICS CLAUSE — FOUND AND CLOSED 2026-08-26 BY MEASURE. DECLARED_DIVERGENCE in engine/quarantine.js was consulted at exactly one site, inside wholeGameClause (:2985 / :2994 are a selftest push/pop, not a reader). That clause declared the Supreme Overlord fallenundefined line AUTHORITY-WRONG and subtracted its 5 games — the authority guards that ability's onStart on a counter and does not guard its onEnd , so with nothing fainted it emits the literal string fallenundefined on a [silent] line players never see, and reproducing a typo is not correctness. tests/test-mechanics.js:15722 carried a live probe asserting we refuse it deliberately. And classifyMechanics counted it anyway: it filters on `if (!r	!r.diverged

#	item	note
#465	THE PARTY IS KEYED BY THE DISPLAYED SPECIES, SO A TRANSFORMED BODY OVERWRITES THE ONE IT COPIED -- MEASURED 2026-08-26 BY ENGINE AND HELD, BECAUSE THE RE-KEY MOVES THE SAMPLE. <code>duplicate_species_in_party</code> has read 20 on every pinned 961-game run and nothing acted on it. It is not a cosmetic tally: <code>partyMap</code> 's second row overwrites the first, so one body is compared against the other engine's OTHER body and the survivor is decided by whichever of them <code>sf.team / side.pokemon</code> lists last. ONE OF THE TWO REMAINING BOARD-PARTED GAMES IS THIS -- <code>baseline ...2636042531 vs ...2635567733</code>, <code>turn 3</code>, <code>p2.party.garchomp</code> reading <code>hp/maxhp 123 vs 183</code>, <code>item lifeorb vs choicescarf</code>, <code>boosts.atk -2 vs 0</code>: one engine's row is a transformed Ditto and the other's is the real Garchomp it copied. THE BRIEF AND #463's CLOSING LINE BOTH SAID BOTH REMAINING GAMES WERE THIS CAUSE. THEY ARE NOT -- the other, <code>-damage: a different body at turn 2</code>, is a genuine spread-target divergence and survives the re-key untouched, so closing the party key takes board-material to ONE, not to zero. THE SAMPLE ANSWER, WHICH IS THE POINT OF THE CARD: the ordered GAME LIST did not move (961 of 961, same pairs, same order) and 109 of 961 TRAJECTORIES DID. The list criterion is necessary and NOT sufficient and it gives the comfortable answer here -- under <code>--games N</code> the scheduler takes a fixed prefix of <code>pairsCached</code>, so the pair list cannot move whatever the credit does; only <code>--until-covered</code> moves the stop point. What moves is WHICH CLICKS the census-coverage driver makes, because credit is bucketed by <code>B5.family(path)</code> and a transform reported as <code>party.MISSING-OR-EXTRA-MEMBER</code> is different credit from one reported as <code>party.species + party.types</code>. So <code>board-material 2 -> 4</code> and diverged <code>15 -> 20</code> are two populations, not a before/after, and neither was landed on. THE INSTRUMENT THAT MADE THIS ANSWERABLE IS NEW AND HAS A CONTROL: <code>MEDI_SAMPLE_DUMP=<file></code> writes per game the pair, the turns, the lines, the board result and a DIGEST OF MEDICHAM2'S OWN STREAM; two identical display-key runs differ in o of 961, and the display arm reproduces the published artifact's 15-game diverged set member for member. WHAT IS IN THE TREE: <code>stableKey</code> MOVED from <code>game_differential.js</code> to <code>board_state.js</code> (not copied -- the fifth instance of the species-key class is not getting a sixth implementation), <code>MEDI_PARTY_KEY_IDENTITY=1</code> off by default, and <code>tests/probe_party_key_collision.js</code>, whose derived fixture was WRONG FIRST -- it put the copied body in the foe's slot A, Imposter copies the DIAGONAL, and it went green because the derived filler happened to be what got copied. Under the knob <code>duplicate_species_in_party</code> reads <code>20 -> 0</code> with no fallback of any kind on 961 games.	closed 2026-08-26 -- LANDED, AND THE SCOREBOARD RE-BASELINED WITH IT. THE COMPARATOR CHANGED HERE: NOTHING MEASURED BEFORE 03:28 COMPARES TO ANYTHING AFTER IT, AND THE NUMBER ROSE BECAUSE THE CORRECTED COMPARATOR SEES MORE. Identity keying is the DEFAULT; <code>MEDI_PARTY_KEY_DISPLAY=1</code> is the positive control that restores the old keying, and the retired <code>MEDI_PARTY_KEY_IDENTITY</code> is a LOUD no-op. <code>reader_failures</code> on the pinned run is <code>{}</code> and roster identities read from <code>DISPLAY</code> state read <code>o/o/o</code>. THE NEW FIGURES, PUBLISHED AS NEW RATHER THAN AS A DELTA: whole-game clause 13 of 961 (1.4%), raw diverged 18 of 961, board-material 4 of 961 (957 of 961 boards never parted), narration-only 13 causes / 14 games, census unmoved at 754 live / 754 probed / 0 missing. Pins: <code>release 6272fa445b73</code>, <code>--arm middle --turns 12 --games 1200</code>, <code>--team-store data/team-pool-frozen, census 9446a684709d</code>; <code>data/whole-game-baseline.json</code> re-stamped at 18 of 961 = 1.9%, no <code>--force</code>. <code>party.species</code> AND <code>party.types</code> ARE IN THE END-STATE WORKLIST FOR THE FIRST TIME -- a transform one engine holds and the authority does not, on a row that had no stable name to be reported under before. THE CARD'S OWN PREDICTION HELD: <code>-damage: a different body at turn 2 survived the re-key untouched</code>, so closing the key did not take board-material to zero. TWO THINGS BROKE ON THE RE-KEY AND ONE WAS SILENT. The six BENCH PLANTS built their expected path from <code>id(m.name)</code>, so <code>tests/test-state-differential.js</code> went 12 FAILS -- <code>caught=true</code>, <code>localised=false</code>, row <code>p2.party.lopunny.item</code> against a plant wanting <code>lopunnymega</code> -- attributed to this batch rather than assumed by re-running under <code>MEDI_PARTY_KEY_DISPLAY=1</code>, which was green; they use <code>rosterKey(m)</code> now. <code>isDuplicateOfActive</code> was the silent one: it matched a party key against the DISPLAYED active species, which stops being the same string for exactly the megas and transforms, so a renamed body's party row would have been listed in the wire queue twice -- <code>board_state.js</code> publishes <code>active_keys</code> through the same <code>stableKey</code> and the dedupe reads those. IDENTITY IS ONE DOOR AND THE CHECK DERIVES ITS MEMBERSHIP: <code>engine/identity_audit.js</code>, registered in <code>tests/run-all.js</code> GATES, reads <code>stableKey</code> 's own source at run time rather than carrying a list of known-bad spellings -- the failure mode of all four previous fixes in this class. It found <code>rosterSnapshot</code> still reading <code>m._switchKey</code>, or-ed to <code>null</code>, with that silent or-null default being the defect, in the one dump that says which body each engine holds when the game stops. IT OVER-MATCHED ON ITS FIRST RUN AT 1,282 ACCUSATIONS (branch verdicts combined with AND instead of OR) and that was printed before it was wired; the population is now 2 DOOR / 4 STAMP / 13 DECLARED / 0 UNROUTED, from 14 UNROUTED. <code>--break</code> goes red on demand. No release cut and none owed -- <code>6272fa445b73</code> reads <code>0</code> of 26 files have moved since after every edit. <code>docs/_reports/2026-08-26-rebaseline.md</code> NOTATION REPAIRED 2026-09-04 BY MEASURE -- pipe characters inside inline code were cutting this status cell, so the register read the row OPEN against its own closure. Only the notation changed; no claim was altered or removed. THE CLOSURE IS THE AUTHOR'S, DATED ABOVE, AND WAS NOT RE-VERIFIED BY THIS PASS: cheap check from committed artifacts only -- <code>MEDI_PARTY_KEY_DISPLAY</code>, the positive control the closure names, is present in <code>engine/board_state.js</code> at four sites; none of the published figures was re-measured.
#466	A DERIVED TAG WITH 992 CORPUS USES AND NO READER IN EITHER ENGINE, AND BOTH OF ITS RULES WERE MISSING -- CLOSED 2026-08-26 BY ENGINE. <code>punishesMinimize</code> sat in <code>data/tags.json</code> carrying <code>{neverMisses: true, mult: 2, halves: ['neverMisses', 'damageMultiplier']}</code> and the literal appeared in neither <code>engine/medicham2-browser.js</code> nor <code>engine/board.js</code>; <code>tests/test-tag-consumed.js</code> had been reporting it STILL DEAD rather than REGRESSED, which is the decisive reading -- no line of engine code looked for it. This is the Destiny Bond shape one level up: there a volatile was written and never read, here a FACT was derived off the authority's own handler and never asked for. IT IS AN IMPLEMENTATION AND NOT A DECLARATION, AND THE CARRIER EVIDENCE IS WHY -- filtered by CARRIER rather than by legality, six legal moves carry <code>flags.minimize</code> (Body Slam, Dragon Rush, Flying Press, Heat Crash, Heavy Slam, Supercell Slam) and seven legal species learn Minimize (Starmie, Qwilfish, Chandelure, Sandaconda, Overqwil and the Starmie/Chandelure megas), so this is not the Neutralizing Gas shape, which has none. THE AUTHORITY IS MAINLINE AND CHAMPIONS DOES NOT OVERRIDE IT -- <code>data/moves.ts:11920-11945</code>, and <code>minimize</code> does not appear in <code>data/mods/champions/moves.ts</code> at all. <code>onAccuracy</code> is the Accuracy event, the LAST step of <code>hitStepAccuracy</code> (<code>sim/battle-actions.ts:729-735</code>), which computes the stages and then throws them away -- so <code>Minimize</code>'s own +2 evasion cannot claw the guarantee back, and the clause sits in the No Guard / Lock-On / Toxic cluster for exactly that reason and ABOVE the OHKO branch on the Toxic block's argument. <code>onSourceModifyDamage</code> is the ModifyDamage relay (<code>:1826</code>), so the <code>x2</code> is a member of the SAME <code>MODMUL</code> chain as Life Orb and the resist berries and truncates once with them rather than being a second <code>Math.floor</code>. MEASURED ON THE OFFICIAL SIMULATOR BEFORE AN ENGINE LINE WAS WRITTEN, volatile applied directly so the evasion stage is out of frame: Body Slam 52 -> 104, Heavy Slam 49 -> 98, Ice Beam 78 -> 78; and	CLOSED 2026-08-26. Release <code>6272fa445b73</code>. Census 753 -> 754 live / 754 probed / 0 missing; <code>tests/test-tag-consumed.js</code> RED -> GREEN with <code>dead 13 -> 12</code> and <code>dead_floor 13 -> 12</code>. The tag reads UNREACHED , not LIVE , and that is honest -- the sweep never puts a minimized body on the field, so the consumer's volatile gate is never crossed; making it LIVE would mean asking the tag before the volatile, which is making the TEST pass rather than the mechanic right. PREDICTION WRITTEN FIRST AND HELD ON EVERY LINE: the 6,000-row differential was OWED by this batch because this moves damage and stayed at o/6000 at the midpoint and at all sixteen arms; the pinned pool was predicted not to move (Minimize is 32 of 198,840 sheet entries) and did not -- whole-game 8 of 961 (raw 13), board-material 2 of 961, mechanics clause 8 of 16, all unmoved; roster identical count for count on all three stages with <code>FIRED-AND-BOARDS-DIFFER 0</code> and <code>DID-NOT-FIRE 0</code>. <code>docs/_reports/2026-08-26-dead-tag.md</code>

#	item	note
	hitStepAccuracy at +6 evasion with every roll FORCED TO FAIL returns true for Body Slam with the volatile up and false for Ice Beam beside it. THE PROBE WAS WATCHED FAIL FIRST on the staging it ships with — Body Slam MISSED at roll 0.85 and dealt 15 where it should deal 30 — and MEDI_PUNISH_MINIMIZE_BLIND=1 returns every number to that column and reds exactly this census row (753 live, 1 missing) and nothing else. MY FIRST CARRIER WALK WAS WRONG AND WOULD HAVE PRODUCED THE COMFORTABLE ANSWER: it reported ZERO legal carriers for all six moves and for Minimize, because D.learnsets is undefined on a Dex.forFormat handle and a try/catch { continue } ate the throw for every species; the accessor is D.species.getLearnsetData(id) . Two counters kept apart because halves exists to make a one-sided implementation visible — punishMinimizeNeverMiss and punishMinimizeDamage , 4 and 4 clean, 0 and 0 under the knob: REACHED, not present.	
#467	A DIVERGENCE PROPOSED AS INCOMPARABLE IS A SHARED-STREAM ADDRESS BUG, AND THE VOID GAME IT RIDES ON CANNOT BE SUBTRACTED FROM THE CLAUSE — REFUSED AND FILED 2026-08-27 BY MEASURE. Two findings, one game. (a) THE DECLARATION WAS REFUSED. -damage: a different body ::	-damage p2a
#468	A SECOND THROAT CHOP RESTARTED OUR CLOCK, AND EVERY PIECE OF THE ANSWER WAS ALREADY IN THE TREE - STAGED AND CLOSED 2026-08-27 BY ENGINE. Pokemon#addVolatile (sim/pokemon.ts:1994-1997) refuses a volatile a body already carries when its condition declares no onRestart . throatchop 's does not, the fact is DERIVED into data/tags.json as volatileRestart.throatchop = {restart:false, duration:2} , and a general reader for exactly that rule - volRefusesRestart - has existed since 2026-08-26. Throat Chop walked past it because its state lives in tg._noSound , outside _vol , which is the only thing applyMoveVolatile can see. Worse, _n0 was already read at the application site, to suppress the duplicate -start line: the engine knew the lock was standing and rewrote the counter anyway. On the pinned pool that is baseline t6 of gen9championsvgc2026regmbbo3-2656159439 vs -2656116859 , where the authority writes `	-end\ p1a: Mawile\

#	item	note	
#469	PROTECT IS ANSWERED IN A PRE-PASS ABOVE THE STEP DRIVER, SO STEP 1 RUNS BEFORE STEP 0 - CLOSED 2026-08-27 BY ENGINE, AFTER TWO EARLIER PASSES AIMED AT THE WRONG THING. <code>trySpreadMoveHit</code> names its own order in its own comments (sim/battle-actions.ts:553-577) - <code>0</code> <code>hitStepInvulnerabilityEvent</code> , <code>1</code> <code>hitStepTryHitEvent</code> (Protect, Wide Guard, the absorbing abilities), <code>2</code> <code>hitStepTypeImmunity</code> - and the driver runs STEP OUTSIDE, TARGET INSIDE, so every target's step-0 answer precedes every target's step-1 answer. ROADMAP #81 WIRE 1 hoisted the shield above the DRIVER to get it above the ACCURACY roll, which was right, and it went one stage further, above step 0 as well. Card, <code>pair-protect-bust</code> <code>t10: authority`</code>	-miss\	p1b: Garchomp\
#470	A HAZARD AT ITS CAP FAILS AND WE SAID NOTHING - AND THE CARD WAS HANDED OVER AS A DIFFERENT DEFECT, WHICH WAS SHOWN WRONG BEFORE A BYTE MOVED. CLOSED 2026-08-27 BY ENGINE. The card (<code>baseline</code> <code>t7: authority`</code>	-fail\	p1b:Ariados after a sec as <code>"a</code> move with no <code>le</code> useMoveInner 's early <code>r</code> NOT THAT**: Sticky Web branch of that same if Both shapes were STAGE cap PARTS, a move whos refutation is a measur moveHit : <code>hitResult</code> = targ (didAnything === false) { th Side#addSideCondition <code>re</code> whose condition declar IDENTICAL RULE HAD BEE hazard half was never layer went down and it that decides, not the Spikes at 2, Spikes at caps. Knob <code>MEDI_HAZA</code> below-the-cap over-fir both arms (the same au what says the probe ca MATERIAL, NOT NARRATIO MISSED**: board-materi exactly this card.~~ * FOR THE BOARD, AS ORIG games_board_never_diverg matched artifacts (ide arm, census pin and te the turn-6 board entry p1.party.ditto.species , <code>p1.</code> the authority does not

#	item	note
		null . The reported 3 w games , which is keyed therefore drops a game no board. **This fix r parting**, which is a board that parts with by the coordinator dif report. Account: docs/ node -r ./tests/_live_release
#47	retrograde analysis (Will): with 2v2 solved, a 3v2 reduces to <i>which solved 2v2 do I steer into</i>	this is how chess tablebases are built
#29	THE SACK — a planned sacrifice is invisible to a material-weighted leaf	it is the same idea: the sacrifice is the move that reaches a won position
#41	reach the Python equilibrium solver from the JavaScript bot	a small-case port is verifiable against <code>nash.py</code> ; a full port is not
#30	DUSK as a goal — nothing in the project represents a goal, only positions and actions	

5.5 Built, measured, and never used in a live decision

The recurring failure of this project, kept together so the pattern stays visible.

model	state
DODUO	fitted, refitted, +2.1 top-1 over independent choice, never read by the search (#53)
PORYGON2	built, separation gate passed, never called by a live decision (#24)
GARY prior	wired end to end, default is a coin (#32)
MAG's scores at the candidate list	not attached, so the screen prunes with the rollout instead (#37)
MACHAMP	"half-run and stale — the single largest untested lever in the project"; artifact deleted 2026-08-02
the Nash solver	verified on RPS and a 2×2, in Python, unreachable from the bot (#41)

5.6 Measurement integrity

#	item
#26	refit MAG on bo3 only — the corpus change is in code and unrun. OWNED BY WILL 2026-08-11: he is reworking the weights himself, so nothing here refits or restamps on the way past. The feature-semantics check on <code>data/policy-weights.json</code> is failing (damage table regenerated under the fit, 318→317 species, 405c836793d1 → 1b66b563c229) and STAYS failing until then — a restamp would make the warning vanish without making it untrue. #79 (the 58 features were never designed) and the fit-vs-play environment question land inside this rework, not beside it. See <code>docs/MEDICHAM-SPRINT-NOTES.md</code>
#50	WIRE 124 moved a position feature; models fitted on it predate the correct number
#48	27 of 28 Python generators could write a file JavaScript cannot read — closed, gated
#27	three instruments built that had never written an artifact — truncation curve now run , two left
#44	a move that provably does nothing is a weighted preference, not a rule — needs a refit
#57	SHELVED BY THE OWNER 2026-08-11 — do NOT re-run, and do not put it back on a plan. Will, twice in one session: " <i>i might want to rework a lot of them and dont want to waste time</i> ", then " <i>Dont re run the artifacts weve bene over this</i> ". When the gate opened these ~52 stopped being WITHHELD and became RE-RUNNABLE, which READS like a to-do list and is not one. Most of the list is downstream of MILTANK and MAG — the fits, the exploitability runs, the rollout artifacts, the head-to-heads, the leaf contrasts, the collinearity audits — and Will owns both and is reworking them himself (#194). Re-running now measures a model about to be replaced, so the work is thrown away twice. <code>quarantine.js</code> already prints them as "NOT withheld and NOT current — must be re-run before quoted", which is the correct resting state and needs no action. The row reopens when Will lands his rework, not before. If a number from this list is ever needed, say it is stale and why rather than quietly refreshing it
#108	TWO REGISTERED GATES ASK DIFFERENT QUESTIONS AND NEITHER CONSULTS THE OTHER. <code>engine/status.js</code> prints figures out of artifacts <code>engine/provenance.js</code> calls UNSAFE — the release-ladder block at <code>status.js:353</code> reads <code>data/wire-ladder.json</code> , which provenance flags UNSAFE for computing from a <code>games.bo3.json1</code> that has since moved. Both are gates in <code>tests/run-all.js</code> and both are green about their own question.

#	item	
	#105 removed the excuse: every artifact <code>status.js</code> reads now HAS a row, so the two can be asked the same question about the same file. Needs a decision on the SHAPE first — withhold like the quarantine, or caption — and CLAUDE.md is explicit that a caption is not a quarantine	
#105	~50 ARTIFACTS HAD NO ROW IN THE DEPENDENCY GRAPH — DONE, 3.85.0. <code>engine/provenance.js</code> found a writer only by literal name in <code>engine/</code> and <code>build/</code> , so everything written by <code>tests/</code> (the census, the differential, the interaction matrix, the deliberate roster) or through a computed path had no row at all, and nothing could compare it to its source. It scans <code>tests/</code> now, turns a concatenated or template path into a pattern, and accepts an artifact's own <code>by</code> where no source can say — each labelled in a new <code>via</code> field, ranked, and a template match revoked unless its key shape agrees with something the generator writes by name. 115 → 160 artifacts, 61 → 16 unknown, quarantine 34 of 114 → 40 of 160. <code>data/rollout-r1-explore1.json</code> — the arm <code>engine/miltank.js</code> runs — classifies on its own and is HELD, so the both-files workaround at <code>status.js:665</code> is retired-able. Four false attributions closed on the way, three of them introduced by the change itself and caught before it landed. The UNKNOWN list is printed every run, including at zero	done — <code>docs/MEASURE.md §00a</code>
#58	two moveAccuracy implementations disagreed for 78 moves and nothing compared them. <code>board.js</code> read the dex and was right; <code>medicham2</code> read a hand-typed list and was wrong. Make it one function and assert it	
#110	THE MOVES QUEUE — the roster's moves stage, measured for the first time at 3.79.0 and worked down since. <code>52 → 40 → 32 → 25 → 23 FIRED-AND-BOARDS-DIFFER</code> , plus 24 DID-NOT-FIRE and 91 COULD-NOT-STAGE, against 362 matching. 3.94.0 closed Clanging Scales and Scale Shot: Showdown keeps the user's own stat change in TWO fields (<code>self.boosts</code> AND <code>selfBoost.boosts</code>) and the builder read one — the six moves using <code>self</code> all read MATCH, which is exactly why it looked closed. 3.93.0 closed the partial-trap family — seven rows on one fact. Bind, Fire Spin, Infestation, Sand Tomb, Snap Trap, Whirlpool and Wrap each read <code>showdown 4 / ours 3 then 3 / 2</code> ; identical deltas across seven independent rows is one fact, not seven bugs. <code>partialTrap: {turns: '4-5'}</code> was typed by hand and is the FELT duration, while the compared quantity is Showdown's <code>partiallytrapped</code> counter, which starts at 5 and is decremented in the Residual of the turn the trap lands. The volatile-duration defect a third time (Perish Song, then #111's family) — it survived both because this counter lives in <code>_trap</code> and not <code>_vol</code> . Now derived from the condition itself (duration, the <code>random(5,7)</code> range, the Grip Claw branch, <code>onStart</code> 's <code>boundDivisor</code>) and failing closed . Red shown on the frozen pre-fix release. Whole-game differential unmoved at 65/107. Remaining clusters BY CORPUS WEIGHT, which is not the order they look biggest in — <code>encore</code> alone is 6,102 uses, 55% of what is left, while the whole lock-in family is 101: 13 <code>move/plain-attack</code> (of which <code>outrage/petaldance/ragingfury/thrash/uproar</code> are one lock-in family), 4 remaining <code>move/volatile</code>, 9 <code>move/generic-status</code>, 6 <code>move/boosts-target</code>, 3 <code>move/multihit</code> (Triple Axel, Scale Shot, Dragon Darts — three DIFFERENT mechanics), 3 <code>move/fixed-damage</code>	open — <code>docs/ENGINE.md</code>
#70	TWO USAGE COUNTS EXIST AND NEITHER KNOWS ABOUT THE OTHER — but the engine queue is NOT where it bites. <code>data/meta-usage.json</code> (what CHOMP reads) filters hard: collected 39,792 → after_bot_filter 13,263 → behavioural bots 10,480 → forfeits → min turns → full bring → 7,123 clean. <code>data/tags.json</code> (which every "N uses" in this project is quoted from) reads <code>loadCorpus({scope: 'all'})</code> across <code>games.bo3</code> + <code>games.ots</code> + <code>games.ladder</code> — 11,429 games, 137,148 sheet entries, no bot filter and no quality filter. CORRECTED 2026-08-10 BY WILL, and the correction is the important half: "it doesnt really matter because its not like bots can break the	open — MEASURE, not ENGINE

#	item	
	<i>rules, the moves, mons, and items they use we still need to have in medicham right". Exactly right. A bot cannot click an illegal move, so every entity it uses is one MEDICHAM must handle anyway — and the MEDICHAM gate demands zero DIFFER and zero DID-NOT-FIRE, so the count changes the ORDER of work and never the destination. The unfiltered count is arguably the CORRECT one for an engine queue: it measures exposure. Where it does still bite is the OPPONENT MODEL — what a human will bring and click — which is what the bot filter exists for and what CHOMP consumes. So this is a MEASURE item about <code>meta-usage.json</code>, not an ENGINE item about the queue. (An earlier draft of this row claimed three sources disagreeing 13x, citing a scan of <code>games.ladder.jsonl</code> alone that found 15 Iron Ball rows against <code>tags.json</code>'s 139. That scan was WRONG — ladder alone carries sheets on 1.7% of sides because the sheets live in the <code>bo3</code> and <code>ots</code> files. The 139 is sound. Retracted.)</i>	
#100	THE ROSTER'S CONTROL ARM MEASURED THE CONTROL — DONE, 3.78.o. <code>ability/generic</code> picked its control by taking another live ability off the same species, so Sand Rush's control was Fluffy and Fluffy's control was Sand Rush, and the two deltas were the same numbers swapped. Four false findings across 2,049 uses, every one of them reported as an engine bug. A control is now a NAMED quiet ability, chosen once and recorded in one place, and <code>tests/probe_pair.js</code> refuses a control that is not quiet. This is why #116's pairing class is declarable rather than fatal: forcing a per-species-legal control would reintroduce exactly this	done — <code>docs/ENGINE.md</code>
#114	MEDI_SPREAD WAS PUBLISHED TO NOBODY, SO EVERY SPREAD MOVE WAS PRICED AS SINGLE-TARGET — CLOSED 2026-08-09. <code>medicham2-browser.js</code> assigned <code>root.MEDI_SPREAD = SPREAD</code> and never put it on <code>module.exports</code>, so <code>require(...).MEDI_SPREAD</code> was undefined on every build that ever ran. <code>game-differential.js</code>:2816 reads <code>M.MEDI_SPREAD ? M.MEDI_SPREAD.has(id) : false</code> — a feature-detect that always took the <code>false</code> branch — so the 0.75x spread multiplier was never applied to a staged span. Verified fixed 2026-08-10: the symbol is on <code>module.exports</code>, holds 38 moves, and Earthquake is in it. THIS ROW EXISTED ONLY IN A COMMIT MESSAGE UNTIL NOW (<code>247d26b</code>, which named it while recording the browser-wrapper decision) and was the first thing #149's new commit-message clause caught — a citation with no register entry, and the number free for anyone to take	closed 2026-08-09
#116	new Battle() VALIDATES NOTHING — a probe could measure a mechanic this format does not contain. Will: <i>"why dont we use showdowns teams validator that is universal truth".</i> Now it does. <code>engine/champions_sim.checkLegal</code> asks the <code>TeamValidator</code> — the authority ADR-002 names, through the instance <code>packTeam</code> already builds — and splits its complaints into BANNED (the format does not contain this; always fatal) and PAIRING (this species cannot hold this; a legitimate isolation, declarable in writing). It catches strictly more than an <code>isNonstandard</code> check: <i>"Meganium can't learn Flamethrower"</i> is a legal move on a species that cannot have it, and that set was hand-staged on 2026-08-08. Caught two defects in its own harness on the first run — <code>Tackle</code> is <code>Past</code> and every inert slot carried it, and the padding species were named from memory and included one not in the format. The sweep that followed (3.92.o) settled the blast radius: the <code>.item / .ability</code> assignment surface across 238 files is clean (one hit, <code>noability</code>, the blank sentinel), so the hazard this item was written about never actually fired — but move literals hit five sites, two of them defects. <code>test-priority-block.js</code> silenced three slots with Splash, which has no MC.moves row at all, so the silencing worked through ABSENCE rather than through the move doing nothing; and <code>test-dead-volatile.js</code> guarded on <code>.exists</code>, which is true for a banned move, so the guard could never fire and the case always ran on Thousand Arrows. Tightening that guard would have moved the hole rather than closing	open — <code>docs/ENGINE.md</code>

#	item						
	it, so the subject is derived from the format (Smack Down). Still open: staged_board.js and game_differential.js (buildPair)						
#117	THIRTEEN MOVES EXECUTED AND NEVER RECORDED _lastMove , SO ENCORE COULD NOT REACH MOST OF WHAT IT EXISTS TO PUNISH — CLOSED 2026-08-11. Measured across the whole 500-move table at 3.98.0, not grepped: every heal kind (Recover, Roost, Life Dew, Moonlight, Morning Sun...), both switch kinds, tail , trickroom , wideguard . volNeedsLastMove then CORRECTLY refused a sealer against a body with no recorded move — so a live Encore could never lock anyone into Trick Room, Tailwind, Wide Guard or a recovery move , which is the list Encore is clicked for. The fix is ONE line at ONE site, and the site is the point: _lastMove=_mid rides the shared "the move was committed" position (pokemon.moveUsed() 's) beside the Choice lock (#118), so a fourteenth kind cannot be missed the way thirteen were. Re-measured 2026-08-11 over all 500 moves and all 37 action kinds: o fail to record it. The probe is the PAYOFF, not the field: sealsMoves / "Encore reaches a move kind that never recorded itself — the room comes DOWN". A Trick Room goes up 4 turns; left alone the victim clicks Scald and it ticks to 3; ENCORED it re-clicks Trick Room and the room reads o — a second Trick Room ends it, which is two mechanics deep and cannot be faked by writing a field	closed 2026-08-11					
#118	THE CHOICE LOCK DID NOT ARM ON A STATUS MOVE — CLOSED 2026-08-11. Choice Scarf is legal here on 7,844 uses and Scarf+Trick is a real set. Measured on the frozen release: knockoff -> _lock=knockoff, _lockT=Infinity ; taunt , tailwind , trickroom , swordsdance -> _lock=undefined . The arming line sat below if(a.kind!=='attack')continue , the same guard that hid #119, and ~30 status kinds resolve below it. The close is a POSITION, not a condition: the lock is armed at the one site that means "this move was committed" — everything above it is a BeforeMove refusal that has already continue d out, everything below it is a move that RAN — which is useMoveInner 's position in the authority and is where _lastMove (#117) now rides too. Re-measured 2026-08-11: taunt , tailwind , trickroom , swordsdance , protect , recover , wideguard , quickguard and transform all leave _lock set with _lockT=Infinity ; MEDSEEN.choiceLockArmedOnStatus reads 9. Probe: choiceLock / "the lock arms on a STATUS move, not only on an attack", whose control is the identical pair of clicks with NO item — so the second click must be honoured or the probe proves nothing	closed 2026-08-11					
#119	STRUGGLE WAS A SILENT NO-OP AND THREE CALL SITES RETURNED IT — CLOSED 2026-08-11. Handed {kind:'struggle'} with both foes passing: **o damage to the foe, o to the user, and no `	move	line** — the whole turn was	turn	1	upkeep	. There was no a.kind=== 'struggle' branch and if(a.kind!=='attack')continue ate it. **A retraction is recorded here rather than quietly dropped:** the 3.98.0 diagnosis claimed a locked body Struggled *for 62 damage a turn* and called Encore inverted. That was wrong — 62 was the ALLY acting, because the probe let the engine choose for both bodies. **#118 AND #119 ARE ONE MISSING CONCEPT, WHICH IS WHY THEY CLOSED TOGETHER:** the engine had "its PP is gone" and nothing else, while Showdown Struggles on an empty menu of ENABLED moves (sim/side.ts:697 , sim/pokemon.ts:1022-1044) and a slot leaves that menu when it is **disabled OR at 0 PP**. Will: **U ONLY STRUGGLE

#	item					
						WHEN COMPLETELY OUT OF PP OR ARE LOCKED INTO A MOVE U CANT USE LIKE CHOICE SCARF DISABLED OR ENCORED DISABLED."* Closed as a family: moveDisabledBy (a per-slot refusal that returns ITS SOURCE, not a boolean), selectableMoves , mustStruggle as selectableMoves().length===0 , one struggleAction builder, a typeless ??? damage path and recoil as a fraction of MAX HP. **Re-measured 2026-08-11:** an empty PP bar Struggles for 50 and pays 64 recoil with a real
#120	A GATE CLAUSE REPORTS PASS WITHOUT REPORTING ITS DENOMINATOR. Will, 2026-08-10: " <i>check the abilities clause for the same hole</i> ". The abilities clause reads 84 fired and matched and passes. It is 84 of 316 rows — 26.6%. 217 are COULD-NOT-STAGE and 15 are CONTROL-NOT-QUIET. On the fair denominator (excluding the 115 abilities with NO LEGAL CARRIER in this format, which are correctly out of scope rather than untested) it is 84 of 201 = 42% . The clause is not lying about what it tested; the GATE is silent about what it did not, and that is the same shape as "a caption is not a quarantine". A clause that says PASS must say <i>of how many</i> . Arena Trap and Magnet Pull are in the 115 and are genuinely not applicable; Shadow Tag is the one real gap — its carrier is mega-tier, the forme change WRITES the ability, so the only available control is suppression ITEMS IS THE CONTRAST, AND IT MATTERS — this is NOT a systemic reporting failure. The items clause is 139 of 148 = 93.9% actually tested (8 COULD-NOT-STAGE, 1 deferred by the owner). So a stage CAN reach near-total coverage, and the abilities figure is a property of that stage rather than of the instrument. Printing the denominator will show items looking strong and abilities looking thin, which is the true picture and the reason to print it	DONE 2026-08-10 — the clause states tested / in scope / total and names every unattributable row; the artifact carries a scope block written at the refusal. docs/MEASURE.md \$000				
#121	FIFTEEN ABILITY ROWS ARE INVALID AND ARE CARRIED INSIDE A GREEN CLAUSE. CONTROL -NOT-QUIET means the control arm was ANOTHER LIVE ABILITY — Damp controlled against Electromorphosis, Imposter against Limber, Ice Body against Sturdy, Fluffy against Sand Rush — because those species have no quiet alternative. Two live abilities on one board measures neither. This is #100 surviving in the cases #100 could not fix: #100 stopped the control being <i>another live ability of the same species chosen automatically</i> , and these 15 are where no quiet alternative exists at all. They are not passing and they are not failing; they are unmeasured, and the clause counts them in neither column. Either stage a quiet carrier from another species or declare the row untestable with its reason — the one thing that must stop is a green clause with 15 invalid rows inside it	DONE 2026-08-10 — a SECOND CONTROL varies the live control arm on 98 rows; 9 of the 15 resolved (5 DID-NOT-FIRE, 2 DIFFER, 1 MATCH, 1 unattributable-but-measured), 6 DECLARED UNTESTABLE with the pool printed, and 8 VACUOUS GREENS were caught. docs/MEASURE.md \$000				
#122	THE ROSTER SCRIPT LANGUAGE HAS NO SWITCH ACTION, AND board_state.js DOES NOT COMPARE TRAPPING AT ALL. Will, 2026-08-10: " <i>we need to test the switch blocking like shadow tag, block, and the trapping moves, so we need to be able to switch in the test</i> ". The roster says it in its own source at tests/roster.js:1814 . Two layers, and they must not be confused: the ENGINE switches fine — measured, {kind:'switch', to:body} through battleTurn , and a partial trap correctly REFUSED it while the untrapped control left freely — but the roster drives through the differential's script language, which cannot express the action, and even if it could the comparator has no field for trapping. That is why Block reads INERT: nobody ever tried to leave. Covers the hard-trap moves (Block, Mean Look, Spider Web, Anchor Shot, Thousand Waves), the partial-trap family's switch-prevention half (chip damage was closed at 3.93.0; the trapping half has never been compared), and the trapping abilities	open - the trapping half is not done. THE CELL BELOW IS THE ROW'S EARLIER STATUS, UNCHANGED: PART DONE 2026-08-10 — the ABILITY stage can ask for a switch and the ask is proven in both engines (abilitySwitchWorks()); it closes ZERO trapping rows because Arena Trap and Magnet Pull have no legal carrier and Shadow Tag is mega-tier with suppression measured dead. The MOVE half is the moves stage s . docs/MEASURE.md \$000 **STATUS WORD CORRECTED 2026-09-05 BY MEASURE**: the cell led PART'DONE`, which a human reads as a closure and which the shipping detector reads as neither closed nor open, so this row's verdict rested on prose. The cell's own words settle it - it " <i>closes ZERO</i>				

#	item	
		trapping rows" - so OPEN is what the row states. Nothing else was altered and nothing was re-run.
#123	THE SEMI-INVULNERABILITY EXCEPTION LIST WAS ABSENT — CLOSED 2026-08-11, AND THIS ROW CARRIES TWO RETRACTIONS. Will, 2026-08-10: <i>'dig gets double hit by earthquake? and maybe some others? same with dive and surf?'</i> Right on both counts. RETRACTION ONE: it first read <code>'_invuln</code> is set and never read, a charge move gives ZERO protection', measured with Knock Off dealing 50 to an underground body — that was the PROBE, not the engine (both bodies were Incineroar, a SPEED TIE). RETRACTION TWO, and it was in this row's own title: it said Earthquake <i>"MISSES a digging target"</i>. Earthquake hits Dig for DOUBLE; the engine dealt 0, so the counter-play was impossible rather than free — the opposite sign. IT IS TWO LISTS AND THE ROW CONFLATED THEM. <code>onInvulnerability</code> says who gets THROUGH; <code>onSourceModifyDamage / onSourceBasePower</code> says who gets DOUBLED, and Fly's second list is two of its first list's seven — so an engine implementing "pierces ⇒ doubles" is wrong on Thunder, Hurricane, Sky Uppercut, Smack Down and Thousand Arrows, five moves that hit a flying target for NORMAL damage. Both lists are now derived onto <code>semiInvulnerable</code> (<code>pierces</code> , and <code>doubles{via,mult,moves}</code>) — Fly doubles at final damage, Bounce at BASE POWER, off the same two moves, so <code>via</code> keeps them at the two stages the authority uses). Phantom Force declares <code>onInvulnerability: false</code> and carries <code>pierces: []</code> — an empty list is a statement, not a missing value. Measured 2026-08-11, charger SLOWER so the charge survives the attack: Earthquake into Dig 23 -> 46; Iron Head into Dig 51 -> 0; Surf into Dive 29 -> 57; Hurricane into Fly 94 -> 94; Smack Down into Fly 16 -> 16. <code>invulnPierced</code> 6 against <code>invulnDoubled</code> 4 over that sample — equal counts would BE the wrong rule	closed 2026-08-11
#124	THE WEATHER-CONDITIONAL MOVE FAMILY, SWEEP. Nine moves in this format change behaviour with the sky. Verified CORRECT and now on record rather than assumed: Thunder and Hurricane are never-miss in rain and 50 in sun; Blizzard is never-miss in snow and untouched by sun; Solar Beam and Solar Blade closed under the precondition layer. Broken: <code>growth</code> does not double in sun (sd +2/+2, ours +1/+1) — it read MATCH for months and went red the moment its precondition was real, which is the sharpest argument for the precondition work there is. Open: only Growth. A later sweep at release <code>f727f7fdee4f</code> closed the rest and this line is corrected rather than left standing: <code>weatherScaled</code> has ELEVEN members and five payload shapes, Growth is the sole member carrying a boosts payload. Weather Ball now reads MATCH (#96's fix took), and the other nine — including Electro Shot, which this item wrongly called untested — are all green under <code>move/needs-the-sky-it-names</code>. The remaining untested weather surface is on the ABILITY side: <code>solarpower</code> , <code>raindish</code> and <code>icebody</code> all read CONTROL-NOT-QUIET. <code>dig</code> appears in a naive weather scan and is a FALSE POSITIVE — its weather mention is <code>onImmunity</code> (underground bodies take no sandstorm or hail chip), a different mechanic, recorded here so the next sweep does not re-find it as a bug	open — <code>docs/ENGINE.md</code>
#125	THIRTY-TWO MOVES RESOLVED TO {kind: 'pass'} — A WHOLE NO-OP TURN — AND THE LINE THAT RETURNED IT HAD NO COUNTER. COUNTER LANDED 2026-08-11; THE QUEUE IS NOW FIFTEEN. <code>playerActionPrimary</code> 's terminal fall-through is where every move no branch claimed became a wasted turn. Nine of the thirteen rows the precondition layer newly exposed on 2026-08-10 arrived through this one line, and only ONE of the thirteen was named by any counter (<code>MEDFAILS.healProceduralFirst</code> , <code>first: 'swallow'</code>) — CLAUDE.md's founding failure sitting in the exact place built to catch it. The counter went in FIRST, before any fix, so the fixes had a number to move, and it records the	closed 2026-08-11

#	item	
	MOVE ID rather than a tally: MEDFAILS.unmodelledClick , unmodelledClickFirst , unmodelledClickBy (a per-move histogram). A bare {kind: 'pass'} - a CALLER built carries no mv and is deliberately not counted — counting it read 21 on a sweep with no target supplied and would have made this a counter nobody reads. THE LIST WILL WAS OWED, printed by tests/test-unmodelled-clicks.js and RATCHETED (it may shrink, never grow): transform 89, entrainment 72, worryseed 69, roleplay 35, simplebeam 17, speedswap 12, spite 8, recycle 3, fairylock 1, magneticflux 1, and guardsplit / healbell / lockon / powersplit / teatime at 0 — 15 moves, 307 clicks. The row's own "32 moves / 1,702 uses" was measured at 3.98.0 and is superseded by that sweep, which is the number the artifact now carries. VERIFIED BY: node tests/test-unmodelled-clicks.js — the register row and the gate that decides it, checked against each other by engine/register_reality.js rather than by anyone remembering.	
#126	QUICK GUARD WAS THE ONLY SOURCE OF PRIORITY REFUSAL THAT DID NOT WORK — CLOSED, 927 uses. Will, 2026-08-10: <i>'have quick guard block all prio moves and test it against some prio moves not that hard'</i>, then <i>'its like armor tail'</i> — and that comparison was the whole diagnosis. Measured, a +1 Quick Attack into a Farigiraf: Armor Tail o, Dazzling o, Queenly Majesty o, Psychic Terrain o — all REFUSED. Quick Guard let it through for 25, identical to the no-guard control. The ability and field sources were correct; only the MOVE source was unwired. The tell was that quickguard and wideguard carry BYTE- IDENTICAL tags — so the engine was telling them apart by NAME (id=== 'wideguard'), which this repo forbids, and Quick Guard fell through the classifier to {kind: 'pass'} (#125). Closed the way the tell demanded: oneTurnGuard gained a blocks param derived from each move's own condition ("priority moves" / "spread moves"), the classifier matches the TAG, and kind: 'wideguard' is kept as the name of the side- guard ACTION KIND while mv plus the artifact decide behaviour. Re-measured 2026-08-11: Quick Guard blocks Quick Attack (38 -> o) and Fake Out (38 -> o) and does NOT block Earthquake (55 -> 55); Wide Guard blocks Earthquake (55 -> o) and does NOT block Quick Attack (38 -> 38) — the two arms cross, which is what no name-match can produce. Wide Guard at 3,997 uses rides the same tag and did not regress	closed 2026-08-11
#127	FIVE TAG SIGNATURES ARE CARRIED BY MOVES THE ENGINE TREATS DIFFERENTLY — so something other than the tag is deciding, and 105,709 uses sit on those signatures. Generalised from #126 rather than left as one anecdote: every move in data/tags.json was resolved through playerAction on the frozen release and grouped by its sorted tag list. Five groups split on the resolved kind, and a split can only mean a NAME match (forbidden here) or a tag missing the param that should separate them (#71). Ranked: 96,406 protect + detect -> protect but endure -> affect , the highest-usage signature in the format carrying THREE behaviours — and PROTECTMOVES is an exported NAME LIST, which is the mechanism; 4,924 wideguard -> a branch, quickguard -> nothing (#126); 3,721 roost / recover / slackoff / softboiled -> heal while wish and rest -> nothing, because the tag says <i>heals self</i> and cannot express DELAYED or ALSO-SLEEPS; 524 beatup / hex / storedpower -> attack , spitup -> nothing; 134 bellydrum / tidyup -> statcode , acupressure -> nothing. The last three overlap #125's dead list and groups A/B of the 13-row diagnosis, so the fix is shared. The check itself is the deliverable — it is cheap, it is derived, and it should be a gate: a signature whose members resolve to different kinds is either a name match or an under- specified tag, and both are defects by this repo's own rules	open — docs/ENGINE.md

#	item	
#128	THE BERRY-ABILITY FAMILY WAS UNIMPLEMENTED — SIX OF SEVEN, AND TWO WERE BLOCKED ON A FIELD THE SIMULATOR DID NOT HAVE. CLOSED 2026-08-11. Will, 2026-08-10: <i>'do you have the berry abilities impelmented, like cud chew, harvest, etc'.</i> Measured then: Cud Chew, Harvest, Gluttony, Cheek Pouch and Pickup were <code>untagged</code> and all COULD-NOT-STAGE; Ripen was tagged <code>damageReduce</code> , which catches its resist-berry doubling and NOT that it doubles every berry effect — a Citrus under Ripen healed 25%, so it was modelled in a way that READ AS COVERED, which is harder to find than absent; Unnerve alone worked. THE STRUCTURAL BLOCKER WAS ONE FACT: <code>lastItem</code> and <code>ateBerry</code> appeared ZERO times in <code>medicham2-browser.js</code> . Closed by making berry consumption a SINGLE SITE (<code>consumeBerry</code>) rather than six bookkeeping lines: <code>m.item= ''</code> appears twelve times in that file and only the THREE that are a berry being EATEN route through it — a berry Knocked Off or Tricked away is not one you ate, which is precisely the distinction Harvest and Cheek Pouch turn on. It also unblocks Recycle (#71). Six new derived tags, all from the abilities' own handlers: <code>healsOnBerryEaten</code> , <code>doublesBerryEffect</code> , <code>lowersBerryThreshold</code> , <code>restoresBerryAtResidual</code> , <code>reEatsBerry</code> , <code>picksUpUsedItem</code> , plus <code>healsAtThreshold.triggersBelowWithAbility</code> — because Gluttony's number lives on the BERRY, not on the ability (the handler is one <code>if</code> with two arms). Measured 2026-08-11, a 170 HP body started at 76 with every other item and ability blank (the roadmap's own interaction hazard honoured: a berry at half heals 25%, identical to Life Dew / Hospitality / Aqua Ring): plain Citrus 118; Cheek Pouch 170; Ripen 160 (+84, exactly double); Cud Chew 118 then 160 the next turn — and the second helping IGNORES the threshold, which the first implementation got wrong; Harvest gives the berry back on a winning roll and ALWAYS in sun, and not on a losing roll in rain; Pickup takes the spent Citrus off the body beside it. GLUTTONY IS A MEASURED NULL AND IS RECORDED AS ONE: it is read, and the only two pinch berries above this format's usage floor (Citrus, Oran) already trigger at 1/2 — the fraction Gluttony would raise them to — so <code>MEDSEEN.gluttonyRaisedThreshold</code> is 0 by regulation, not by breakage. Seven census probes, one per member	closed 2026-08-11
#129	'GOOD ENOUGH' IS A USAGE-WEIGHTED THRESHOLD, NOT A ROW COUNT — and by that measure the engine is already at 99.24%. Will, 2026-08-10: <i>'we do have to ask ouselves at what point is the engine good enough and has enough used features that it can perform its end goal'.</i> Measured against the corpus rather than argued: 553,910 total move clicks; 781 (0.14%) land on a move that resolves to a whole no-op turn; 3,888 (0.70%) land on a move the roster says disagrees with the authority; the union is 4,208 = 0.76%. So 99.24% of real clicks land on a move with no known defect, and the game differential is 150/150 clean on real replayed games. THE GATE IS A COVERAGE STANDARD AND HAS BEEN READ AS A SUFFICIENCY STANDARD. Its 45 open move rows are deliberately-staged edge cases, which is precisely why they survived — they barely occur. The honest test of sufficiency is ROADMAP #68: scale the differential from 150 games to 1,000+ real stored games. If it stays clean there, the remaining rows are polish and can run BESIDE search work rather than blocking it, and the #99 quarantine can lift on a measured condition instead of a feeling. Note the standing counter-evidence, which cuts the same way: <code>docs/MEASURE.md</code> \$0 carries a TIGHT NULL at n=8,883 on the hypothesis that correctness fixes move the leaf at all	open — <code>docs/MEASURE.md</code>
#130	THE OPTIMIZATION PASS — hand MEDICHAM to Fable 5 and say make it faster, but only behind a correctness gate a stopwatch cannot fake. Will, 2026-08-10: <i>'once we think medicham is good enough i plan to run some optimization programs to speed it up' / 'handing it over to feeble 5 and saying make this go faster'.</i>	open — <code>docs/ENGINE.md</code>

#	item	
	Dispatchable directly — the model is selectable per agent. THE HAZARD IS THAT 'FASTER' AND 'DIFFERENT' ARE INDISTINGUISHABLE TO A BENCHMARK. An optimizer that shaves 30% by skipping a branch produces a beautiful number and a broken engine, and every existing test can still pass because those tests cover mechanics the branch never touches. So the pass runs THREE instruments, not one: <code>tests/bench-medicham.js</code> for the score (built 2026-08-10, pinned fixture, best-of-5, refuses to record a baseline under load — first reading 0.4341 ms/turn); the 500-move x 4-turn whole-board digest sweep for the verdict (2,000 cells; WIRE 147 used it to show only 11 moved, and that is the shape every optimization must reproduce at ZERO); and <code>tests/test-damage-stages.js</code> at 1728/1728 exact so arithmetic shortcuts cannot drift. SEQUENCING, which decides how much work this is: the classifier cascade is BOTH the most likely optimization target AND the thing the remaining ~20 wires append to. Restructure it first and the queued fixes land in the restructured version; do the wires first and the optimizer rewrites code that was just written	
#131	THE COST OF A FEATURE DEPENDS ENTIRELY ON WHICH PATH IT SITS IN, and the first measurement of that was taken 2026-08-10. Will: <i>'how many of these features will slow down the engine? cause if it costs nothing to have harvest in there, then sure we can, but if there is a time cost to it then maybe we otta reconsider'</i> — the right question, and the answer splits cleanly. HOT PATH — runs on every click of every rollout: the damage chain and the ~40-branch classifier cascade. Tonight's five wires cost +6.2% (0.776 -> 0.825 ms/turn, interleaved A/B across frozen releases under load), and the per-hit damage loop is almost certainly all of it, being the only wire that touched damage. COLD PATH — fires once per turn on a trigger, and is effectively free: Harvest, Cud Chew, Cheek Pouch, Quick Guard, the heal family. So Harvest specifically: yes, free. The thing to actually guard is COMPOUNDING — 6% per five wires, twenty more wires, and the engine is ~25% slower to serve 0.76% of clicks. That trade stops being worth it well before the queue empties, which is the argument for #76's ratchet firing on every wire rather than at the end	open — <code>docs/ENGINE.md</code>
#132	THE SECONDARY-EFFECT CHANCES, CHECKED AGAINST THE AUTHORITY IN TWO SECONDS AND NO GAMES. Will proposed running a few million self-play games to see whether our accuracy and secondary rates are right. Self-play cannot answer that — it samples OUR declared number, so a tag saying 30% where Showdown says 10% would produce a beautiful, confident, wrong 30%. The authority STATES the chance, so it is a derivation check rather than a statistical one. Measured 2026-08-10 over the 128 moves in this format carrying a secondary: 93 match exactly, 1 MISMATCHES, 34 carry no chance at all. The mismatch is Triple Arrows — the authority has TWO secondaries, a 50% defence drop and a 30% flinch, and we carry only the first. The 34 without a recorded chance are mostly 100% secondaries which may be modelled as certainties elsewhere; that needs checking, not assuming, because 'modelled as a certainty' is how the Focus Sash drag became a hard 1.0 (CLAUDE.md). Do this BEFORE #133	CLOSED 2026-08-11 — and the row was asking the wrong file. The real declaration surface is <code>data/tags.json</code>, NOT <code>data/move-effects.js</code>: the engine's secondary loop is <code>rng()*100 >= (_fmt != null ? _fmt : _generic)</code>, so the format-derived TAG wins and the rulebook is only the fallback. Comparing the rulebook alone reports six disagreements and five of them are not real. Asked properly at release <code>7c6dca70ae09</code> : 118 (move,effect) pairs, 117 agree, 1 disagrees. Of the chances, 103 come from the tag, 2 are procedural, and 13 come from the rulebook alone with nothing format-derived guarding them. The row's '34 moves with no chance' was TWO different things: 41 certainties (not dice at all) and 7 closures with no chance on either side. Triple Arrows was already fixed — it carries both secondaries with both tag chances at that release, so this row was stale on its own headline example. The single real disagreement is registered separately
#133	A FEW MILLION SELF-PLAY GAMES TO PROVE THE SAMPLER FIRES AT THE RATE WE DECLARED — Will's idea, and the right tool for the question it actually answers. <i>'once we feel our engine is good we can have it play a few million games and then determine if our rate for accuracy and secondary effects is right'</i>, then, on the distinction: <i>'correct run the million games after we determine'</i>. So: #132 first (is the declared number right — free, exact), then this (does the code fire at it	open — measure. RUNNER BUILT AND PROVEN at n=50,000 (<code>engine/million_run.js -> data/million-run.json</code>), against frozen release <code>7c6dca70ae09</code> and never HEAD, free-running mulberry32, denominators from the target rows. Both red proofs CAUGHT on the published run and both refusals

#	item						
	— statistical). This catches the failure a probe structurally cannot: a probe FORCES the outcome to test the mechanic, so a secondary declared at 30% that a bug makes fire at 0% or 100% passes every probe and every roster row. That is CLAUDE.md's founding failure — a capability absent while everything reports success — in the one place the existing instruments are blind. Cost is not a constraint: at the measured 0.4341 ms/turn and a mean 3.6-turn rollout, a million games is roughly half an hour. Compare observed firing rates against the declared chances with a confidence interval per move, and expect the answer to be boring — the value is the one that is not	shown RED first (<code>MILLIONRUN_SABOTAGE=flagger</code> and <code>=refusal</code>) — exit 1, no artifact. Results: accuracy 89.92% observed against 89.78% declared ($z +1.52$) and full paralysis 13.00% against 12.5% ($z +0.68$) are CORRECT; secondaries diverge at $z -4.60$, and split by effect the residual is one place — major status $z = -5.18$ over 30,052 trials. Two rows survive Bonferroni, both 30% status secondaries: <code>scald->brn</code> 23.87% of 1,818 and <code>bodyslam->par</code> 25.12% of 1,441, split-half spreads 1.71 and 0.38 against gaps of 6.1 and 4.9. NOT ATTRIBUTED between engine and instrument — the same 30% status is correct on <code>poisonjab->psn</code> at 29.54% of 1,970. DO NOT RUN A MILLION GAMES UNTIL THAT IS ATTRIBUTED: a full run costs 1.63–1.75h single-process and moves none of the three, because the busiest row already holds 20,987 trials. COVERAGE IS THE ACTION SET, NOT N — 182,196 switch events of which only 572 are VOLUNTARY, and 106 of the 233 target rows sit at zero; crit, proc, duration and random-choice are declared unobservable with reasons					
#134	THE STORE REACHES FOUR FACTS AND THROWS THEM AWAY — and #68 needs every one of them. Will, reading the replay design: <i>'i think multi hit moves say how many times they hit in the chat', 'life orb in the chat too', 'same with focus sash'.</i> All correct. <code>engine/durable-ingest.js</code> parses 30 protocol lines and drops these four: **`	-hitcount	** (never read — so the differential SKIPS 630 multi-hit rows as incomparable when the real hit count was on the wire); **	cant	** (never read — a flinched, paralysed, asleep, frozen or recharging body is simply an ABSENT m event, indistinguishable from each other, 0 games in 52,089 carry a flinch marker); **the [from] clause on chip damage** — the parser TESTS for it to decide the damage is residual and then discards the text, so burn, sandstorm, Life Orb and Rocky Helmet all land as a bare {t:hp} with no source, and its own comment names them; **	-enditem	**, read only to infer what item a body held, never emitted as a turn event, so 'the Focus Sash triggered this turn' is gone. The store's event vocabulary is <code>bffshp mega swx</code>. This contradicts the project's founding principle — <i>store raw, analyze on top, every fact we might ever want</i> — at the parser. Four regex arms; the raw replays can be re-pulled so no history is lost. OPS work, touches no engine, and it makes #68 able to attribute a divergence instead of reporting an unexplained HP drop

What is safe and what is not, because the distinction decides what has to be re-run. Anything that ranks or predicts through `board.js` never touched the broken simulator — the truncation curve, DODUO’s top-1 figures, MAG’s realism table. Anything that runs a **rollout** went through `medicham2`, where until 2026-08-06 the weather resolved backwards from turn one, 78 moves could not miss, Last Respects reverted, and a type-converted move dealt zero. **That is not a claim the numbers were wrong** — WIRE 123’s error was symmetric across both arms of a paired comparison, so a contrast may survive it while an absolute rate does not. Establishing which is the work.

5.6b Are there too many variables? — audited, and the answer is not the count

RETRACTED 2026-08-06 — EVERY MAGNITUDE BELOW IS FROM A FIT THAT PREDATES THE ENGINE. `data/policy-weights.json` is stamped 2026-08-05T04:00:43Z ; commit 3be3f3b at 16:47 the same day rewrote `movesFirst` in `engine/board.js` (“MAG stops re-deriving turn order”) because the old path carried **no dynamic speed at all**. Nine further engine commits followed, including WIRES 123–132. `data/collinearity-joint.json` shares that provenance. **The SHAPE of the finding stands — a correlated block’s individual weights are not readable — but no coefficient, correlation, firing rate or observation count here may be quoted until the refit.** Registered as **#78** (the feature-hash guard reports all 76 hashes unchanged across the commit that rewrote `movesFirst`, so it is blind and may not clear or block the refit) and **#79** (the 58 features were hand-written in four batches with no admission rule and no duplication check — reconsider the set WITH the post-differential refit, on Will’s invitation).

76 weights on 81,515 joint turns is ~1,000 observations per parameter, and the fit is stable: handed 15,279 further turns it flipped **none** of them. The problem is not size.

`data/collinearity-joint.json` found the real shape: **8 of the 18 coordination features fire on under 1% of decisions** — `boostsPartnerDamage`, `boostMayConvertKill`, `weatherSetupHelpsPartner`, `healsPartner`, `redirectThenSetup`, `doubleK0`, `terrainSetupHelpsPartner`, `screenWhileThreatened` — and three of the live ones are correlated enough that their individual weights are not interpretable (`bothSameTarget` ↔ `overkill` $r=0.53$, `bothSameTarget` ↔ `focusFireKills`

r=0.47).

Its own honest caveat, which must travel with the finding: *"Collinear coefficients can be individually meaningless while the MODEL is well calibrated, because the correlated block still carries the right total."* So this is not evidence the pair model predicts badly. It is evidence about which single weights can be read, and about a third of the block that rarely does anything.

5.6c Self-play cannot find a shared blind spot

Kept in the register because it is the standing reason not to trust a head-to-head on its own. CLAUDE.md: *"a head-to-head, an exploitability search and a prediction score all compare two bots that **share the blind spot**, so a missing capability **cancels out exactly**."*

Every bug found on 2026-08-06 would be invisible in self-play — both sides had the weather backwards, both believed the same 78 moves could not miss. And it is not theoretical: self-play training measured **48.1% [46.5, 49.8]** over 9,728 paired games, an interval entirely below 50. The guard that exists is MACHAMP's — every promoted champion plays **every** previous generation, not only the one it displaced, which is the only thing that would detect a cycle.

5.7 Deferred by decision — not dropped

#	item	why it is here
#52	HYPNO — rates the opponent, sets how hard to exploit	Will, 2026-08-06: <i>"NAH LETS HOLD OFF ON HYPNO BUT I DONT WANT TO FORGET ABOUT IT."</i> Worth remembering that it is the only model on the board whose inputs do not include MEDICHAM , so it is never blocked by §5.1 and never needs re-running when the engine changes.

5.8 The paper

#	item
#54	position against THREE comparables, because the field is not empty and none of them is playing our game. VGC-Bench (AAMAS 2026) — whose archive we already consume, and dropped from the fit for staleness (four days, mid-June, seven weeks old); <code>engine/corpus_shift.js</code> exists to measure the metagame distance and has never written an artifact. Future Sight AI — closed-source Showdown bot, reportedly top 5% of the most popular format, matches revealed attributes to the most common set, ships a win-probability browser extension; also settles a naming question, since "Future Sight" is TAKEN and this project names everything after Pokémon. The PokeAgent Challenge (arXiv 2603.15563, NeurIPS 2025, 100+ teams, two tracks) — by far the strongest reference of the three and not yet read . Add Sarantinos (arXiv 2212.13338, Cambridge) as the closest published method: Gen 7 Random Battles singles, peaked 33rd in the world , regret-minimisation opponent prediction, fills unknown properties from usage statistics, and explicitly rejects Monte Carlo sampling as too sample-inefficient — which is what MILTANK's leaf does. It also names our #29 independently: its first "biggest challenge" is keeping the team balanced, with a worked example that is THE SACK, and it asserts <i>"an AI agent cannot achieve this just by looking ahead with MinMax or Monte Carlo Tree Search."</i> Finally, its ELO caveat is load-bearing for Will's stated goal: comparing human and AI ratings is invalid unless they played each other — laddering with ALAKAZAM is exactly that, so the plan is sound and should say why
—	the candidate contributions are the truncation curve , the represented-clicks curve , the armed-coverage critique , and the negative results — a separation gate that uniform noise passes, a tablebase killed by reach rate, and per-turn metrics that cannot separate a 1043 player from a 1600
—	PHASE 4, and it is a paper either way (3.59.0) . If §5.0's gate says compute buys nothing, the project adopts VGC-Bench's recipe — behaviour cloning plus PPO under self-play, fictitious play and double oracle — and reports that as the result. Will approved this branch in advance. Either way the instrumentation is the honest contribution : no project in <code>docs/PRIOR-ART.md</code> publishes a mechanics census that must be shown RED before it counts, a step-level protocol differential against the official engine, ratchets on silent failure, or a record of what it retracted. If search loses, this is still the only account of what a hand-written VGC simulator gets wrong and how you would know

The honest summary

What is genuinely strong: the data discipline, the validated engine, the measurement culture, and the tooling that now enforces all three. The project's most valuable output to date is not a model — it is the ability to tell which of its own results are real, which was demonstrated by dissolving four of them in a single day.

What is not: every in-battle model. MAG is a competent starting position and a measurably exploitable one. No model in this project has yet been shown to play VGC well against anything other than a worse version of itself.

The gap between those two paragraphs is the whole roadmap.